



coreBuilder

User Guide

coreBuilder — Product Code: 2618-0

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History.....11

Preface.....15

 About this User Guide..... 15

 Related Documents..... 15

 Manual Overview.....16

 Typographical and Symbol Conventions.....17

 Synopsys Common Licensing (SCL).....17

 Synopsys Statement on Inclusivity and Diversity.....17

 Customer Support.....18

Chapter 1 Introduction..... 19

 1.1. The coreTools.....20

 1.1.1. coreBuilder..... 20

 1.1.2. coreConsultant.....20

 1.1.3. coreAssembler.....21

 1.2. coreBuilder Application Inputs and Outputs.....22

 1.3. Key Terms and Concepts..... 24

Chapter 2 The coreBuilder Environment.....27

 2.1. Activity-Based Design Flow.....28

 2.2. Workspaces..... 30

 2.3. User Interfaces.....31

 2.3.1. Command Line Mode..... 31

 2.3.2. Batch Mode..... 31

 2.3.3. GUI Mode.....31

 2.3.4. Generating Reports and Optional Views.....32

 2.4. Online Help Tools.....34

 2.5. Suggested Development Flows..... 35

 2.5.1. coreKit Development Flow.....35

 2.5.2. IP-XACT Component Development Flow..... 37

Chapter 3 Setting Up Your Workspace.....41

3.1. Invoking coreBuilder.....	42
3.1.1. Examples.....	42
3.2. Creating a Workspace.....	43
3.2.1. IP-XACT Flow through coreBuilder.....	44
3.3. Basic Workspace Operations.....	45
3.3.1. Saving a Workspace.....	45
3.3.2. Saving a Workspace As	45
3.3.3. Closing a Workspace.....	45
3.3.4. Re-Opening a Workspace.....	45
3.3.5. Moving a Workspace.....	46
3.4. Quitting and Restarting coreBuilder.....	47
3.5. Optional Setup Activities.....	48
3.5.1. Loading Consultant Plugins.....	48
Chapter 4 Annotating Design Files.....	49
4.1. Specifying Configuration Intent.....	50
4.1.1. Configuration Intent Methodology.....	50
4.1.2. HDL Annotation Syntax.....	50
4.1.3. Basic Parameter Attributes.....	52
4.1.4. The Parameter Specification Interface.....	53
4.1.5. Advanced Value Control and Checking Attributes.....	55
4.1.6. Advanced Dialog Control Attributes.....	60
4.1.7. Attributes for Bitfield and Array Parameters.....	63
4.2. Support for Verilog Compiler Directives.....	65
4.3. Conditional Inclusion of Cells and Ports.....	66
4.3.1. Conditional Cells.....	66
4.3.2. Conditional Ports.....	67
4.3.3. Filtering Objects for Conditional Inclusion.....	68
4.4. HDL Coding Guidelines.....	70
4.4.1. HDL Syntax Restrictions.....	70
4.4.2. Working Around HDL Syntax Restrictions.....	70
Chapter 5 Writing Intent Command Files.....	73
5.1. Overview.....	74
5.1.1. Synthesis Intent Development.....	74
5.2. Synthesis Intent Objectives.....	75
5.3. Naming Intent Command Files.....	76

5.4. Intent Command File Syntax.....	77
5.4.1. Commands.....	77
5.4.2. Subscripted Attributes.....	77
5.4.3. Specifying Attribute Values as Formulas.....	78
5.4.4. Default Attribute Values.....	79
5.5. Creating Clocks.....	80
5.5.1. Creating a Real Clock.....	80
5.5.2. Creating a Virtual Clock.....	81
5.5.3. Creating a Generated Clock.....	81
5.6. Specifying Clock Intent.....	82
5.7. Specifying Synthesis Methodology.....	83
5.7.1. Advanced Synthesis Attributes.....	86
5.8. Specifying Port Constraints.....	88
5.8.1. Selecting Sets of Ports.....	88
5.8.2. Port Constraints in Intent Command File.....	88
5.8.3. Creating Port Groups.....	90
5.9. Intent Command File Example.....	91
5.10. Post-Elaboration Intent.....	92
5.11. Importing Synthesis Intent.....	93
5.11.1. Importing Intent Specified in GUI Mode.....	93
5.11.2. Importing Synopsys Design Constraints.....	93
Chapter 6 Refining Configuration Intent.....	97
6.1. Overview.....	98
6.2. Configuration Intent Specification Methods.....	99
6.3. Design Configuration Intent Activity.....	100
Chapter 7 Loading Your Design.....	103
7.1. Organizing Design and Intent Files.....	104
7.2. Executing the Load Designs Activity.....	105
7.2.1. Setting Up an HDL File List.....	108
7.2.2. Command Line Mode.....	108
Chapter 8 Specifying a Memory Map.....	111
8.1. Memory Map Overview.....	112
8.2. Specifying the Memory Map.....	113
8.2.1. Memory Map GUI Interface.....	113
8.2.2. Define Memory Maps Activity.....	113

8.2.3. Advanced Memory Map Attributes Activity.....	115
8.2.4. Memory Map Documentation Activity.....	116
8.2.5. Using Memory Maps Notes.....	116
8.2.6. Memory Map Command Interface.....	117
8.3. Memory Map Attributes.....	118
8.3.1. Endianness.....	121
8.3.2. Memory Map Compatibility with IP-XACT.....	121
8.4. Memory Map Examples.....	122
8.4.1. Example 1: Simple Memory.....	122
8.4.2. Example 2: Relative Base Address.....	122
8.4.3. Example 3: Memory Banks.....	122
8.4.4. Example 4: Register Definition.....	123
8.4.5. Example 5: Register Field Values.....	123
Chapter 9 Specifying Synthesis Intent.....	125
9.1. Specifying Target Technology.....	126
9.1.1. Specifying Reference Technology in GUI Mode.....	126
9.1.2. Setting Tool Installation Roots.....	127
9.1.3. Specifying Reference Technology in Command Line Mode.....	127
9.2. Specifying Clocks.....	128
9.3. Specifying Synthesis Methodology.....	130
9.3.1. UPF Packaging.....	130
9.4. Specifying Test Methodology.....	131
9.4.1. General Test Methodology Attributes.....	131
9.4.2. Scan Insertion.....	131
9.4.3. Auto Fix.....	132
9.4.4. Test Wrappers.....	132
9.4.5. Wrapper Integration.....	133
9.4.6. Built-in Self Test.....	133
9.4.7. Testpoint Insertion.....	134
9.5. Specifying Timing Exceptions.....	135
9.5.1. Specifying Timing Exceptions in GUI Mode.....	135
9.5.2. Specifying Timing Exceptions in Command Line Mode.....	135
9.6. Specifying Port Constraints.....	139
9.7. Specifying Power Intent	140
9.8. Verifying Intent	142

9.8.1. Performing Intent Verification.....	142
9.8.2. Completeness Check.....	143
9.8.3. Consistency Check.....	143
9.9. Area Estimation Modeling.....	144
9.9.1. Area Estimation Example.....	144
Chapter 10 Checking Your Work in Progress.....	147
10.1. Create Integration Workspace.....	148
10.2. Test the coreKit in coreConsultant.....	150
Chapter 11 Setting Up Configurable Filegroups.....	151
11.1. coreBuilder Filegroups.....	152
11.1.1. Types of Filegroups.....	152
11.1.2. How Filegroups Are Created.....	152
11.2. Configurable Filegroups.....	153
11.3. Selective Installation of Files.....	154
11.3.1. Example – Selective Filegroup Installation Based on an Existing Parameter.....	154
11.3.2. Example – Creating New Activity for Selective File Installation.....	154
11.4. Configuring Contents of Files.....	157
11.4.1. Enabling Text Substitution in Files.....	157
11.4.2. Configuring a Filegroup for Conditional Text Inclusion.....	159
11.5. Setting Up Post-Configuration Commands.....	162
Chapter 12 Setting Up Verification Suites.....	165
12.1. Basic TRE Setup Procedure.....	166
12.2. TRE Requirements.....	167
12.3. Example TRE Structure.....	168
12.4. Setting Up TRE Filegroups.....	169
12.5. Setting Up a Verification Subprocess.....	171
12.5.1. Defining the Subprocess Parameters.....	171
12.5.2. Defining the Subprocess.....	172
12.6. TRE Packaging Example.....	174
12.6.1. Configurable Files.....	174
12.6.2. Loading the TRE Filegroups.....	175
12.6.3. Setting the Filegroup Attributes.....	176
12.6.4. TRE Configuration Intent File.....	176
12.6.5. Completing the Testsuite Setup Activity.....	178
12.6.6. Resulting coreConsultant Verification Activities.....	178

12.7. Complex Verification Environments.....	179
12.7.1. Testing a Slave within coreAssembler.....	179
12.8. Creating an ATPG Vector Generation Script.....	180
Chapter 13 Building the coreKit or IP-XACT Component.....	181
13.1. Creating the Bill of Materials.....	182
13.1.1. Viewing Existing Filegroups.....	182
13.1.2. Adding a Filegroup.....	182
13.1.3. Editing Members of a Filegroup.....	183
13.1.4. Deleting a Filegroup.....	184
13.1.5. Setting Filegroup Attributes.....	184
13.1.6. Setting File Attributes.....	185
13.1.7. Setting Configurable Filegroups.....	186
13.1.8. Checking the BoM.....	188
13.2. Setting Up the Help Menu.....	189
13.3. Building the coreKit	190
13.4. Building the IP-XACT Component.....	193
13.4.1. Defining Component Views.....	193
13.4.2. Build IP-XACT Component.....	193
13.5. Verifying coreKit Contents.....	194
13.6. Working with BoM Templates.....	195
13.6.1. Synopsys BoM Templates.....	195
13.6.2. Creating and Using Custom BoM Templates.....	195
Chapter 14 Defining Interfaces for coreAssembler.....	199
14.1. Subsystem Assembly with coreAssembler.....	200
14.2. Terminology.....	202
14.3. General Procedure for Creating Interfaces.....	203
14.3.1. Step 1: Define Interface - GUI Method.....	203
14.3.2. Step 1 (alternate): Define Interfaces - Command Method.....	205
14.3.3. Step 2: Load Interface Definitions.....	209
14.3.4. Step 3: Add Interfaces.....	210
14.3.5. Step 4: Set Attributes on Interface Instance.....	211
14.3.6. Step 5: Test Interface.....	215
14.3.7. Summary of Interface Related Commands.....	215
14.3.8. NumConsumers Parameter.....	216
14.4. Interface Setup Example.....	217

14.4.1. Required Interfaces.....	217
14.4.2. Defining the SimpleBusM Interface.....	218
14.4.3. Instantiating SimpleBusM for Processor.....	222
14.4.4. Defining the SimpleBusS Interface.....	223
14.4.5. Instantiating Interfaces for SimpleBusIP.....	226
14.4.6. Defining the PeriphBus Interface.....	230
14.4.7. Instantiating Interfaces for PeriphBus IP.....	233
14.4.8. Instantiating PeriphBus for a Consumer.....	236
14.4.9. Example coreAssembler Connections.....	237
14.5. coreAssembler Customization Hooks.....	239
14.5.1. Custom Component Actions (Hook).....	239
14.5.2. Custom Component Dialogs.....	239
Chapter 15 Batch Mode	241
15.1. Command Line Interface.....	242
15.1.1. Accessing the Command Line Interface.....	242
15.1.2. Reporting Activity Parameters.....	243
15.1.3. Setting the Value of an Activity Parameter.....	243
15.1.4. Re-Executing the coreBuilder Design Flow.....	243
15.1.5. Abbreviating Commands.....	244
15.2. Using Batch Mode.....	245
15.2.1. Batch Command File Examples.....	245
15.2.2. Write Batch Script.....	245
15.2.3. Executing a Batch Command File.....	246
Chapter 16 Using IP-XACT Components.....	249
16.1. IP-XACT Overview.....	250
16.2. List of Features.....	251
16.2.1. IP-XACT Packaging Features.....	251
16.3. Importing and Exporting IP-XACT.....	252
16.4. For More Information.....	253
Chapter 17 Advanced Topics.....	255
17.1. Setting Up Advanced Parameter Dialogs.....	256
17.2. Customizing coreTools.....	259
17.2.1. General Customization Procedure.....	259
17.2.2. Activities.....	259
17.2.3. Implementing a Custom Tcl Command Procedure.....	260

17.2.4. Modifying Existing Activities.....	261
17.2.5. Adding Custom Activities.....	261
17.2.6. Custom Subsystem Checking.....	264
17.2.7. Customizing Workspaces.....	264
17.3. Working with Plugins.....	265
17.3.1. Plugin Directories.....	265
17.3.2. Writing Plugins.....	265
17.3.3. Building a Core-Specific Plugin.....	266
17.3.4. Building a General Purpose Plugin.....	268
17.4. Packaging Verification IP.....	269
17.5. Packaging Hard Macros.....	271
17.6. Editing Tool Setup and User Preferences.....	273
17.6.1. Setup Variables.....	273
17.6.2. User Preferences.....	274
17.7. Table Driven Data Import/Export.....	286
17.7.1. General Table Formatting for All coreTools Applications.....	286
17.7.2. Table Commands.....	287

Revision History

The following table provides a summary of changes made to this user guide.

Version	Date	Description
W-2024.09-1	October 2024	Updated the version and date.
W-2024.09	September 2024	Updated: Table 17-4 on page 274 with Include Doc attributes GUI details.
V-2024.06-2	June 2024	Updated version
V-2024.06	June 2024	■ Added interface_to_design_port_width_check parameter in Table 17-4 on page 274 ■ Removed the remote option for Runstyle
V-2024.03-1	May 2024	Updated version and date
V-2024.03	March 2024	Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions.
V-2023.12-1	January 2024	Updated version and date
V-2023.12	December 2023	Added include_defines_in_fileset GUI preference in Table 17-73 on page 246
V-2023.09-1	September 2023	Updated version
V-2023.09	September 2023	Updated version and date
U-2023.06-1	June 2023	Updated version
U-2023.06	June 2023	Updated “HDL Annotation Syntax” on page 46
U-2022.12-2	February 2023	Updated version and date
U-2022.12	December 2022	■ Added the Testable attribute in Table 8-26 on page 108 ■ Updated parameter type in table headings and column in all tables
T-2022.06-SP2	September 2022	Updated version and date
T-2022.06-SP1	July 2022	Updated version and date

Version	Date	Description
T-2022.06	June 2022	Updated version and date
S-2021.09-SP4	April 2022	Updated version and date
S-2021.09-SP3	March 2022	Updated version and date
S-2021.09-SP2	January 2022	Updated TetraMax to TestMax in “Creating an ATPG Vector Generation Script” on page 161
S-2021.09-SP1	October 2021	■ Updated the default supported version of Synopsys Design Constraints (SDC) files to 2.1 in “Supported SDC Versions” on page 84
S-2021.09	September 2021	■ Added include_accesshandle_information preference attribute in Table 17-73 on page 246 ■ Added write_absolute_dependency_path preference attribute in Table 17-73 on page 246
R-2020.12-SP5	July 2021	■ No technical changes
R-2020.12-SP4-1	June 30, 2021	■ No technical changes
R-2020.12-SP4	June 2021	■ No technical changes
Q-2020.03-SP4	September 2020	■ Updated run parameters and descriptions in Table 12-47 on page 153
Q-2020.03-SP2	June 2020	■ Added Fusion Compiler in areas where synthesis tool support is noted ■ Added Fusion Compiler synthesis control attributes in Table 5-17 on page 77 and remove others that are no longer available ■ Added the ‘fc_shell’ entry and adjusted the ‘dc_shell’ entry in the Build coreKit implementation tools list in Table 13-57 on page 173
Q-2020.03	March 2020	■ Added the -used parameter for create_interface_parameter in Table 14-60 on page 186
P-2019.06-SP1	July 2019	■ Added the PreventSVPrefix and SkipPrefixNames parameters to Table 7-22 on page 96 for loading designs ■ Added the PreventSVPrefix parameter to Table 13-56 on page 172 for building coreKits
O-2018.06-SP2	December 2018	■ Updated Table 7-22 on page 96 ■ Enhanced “Editing Tool Setup and User Preferences” on page 244 and added Table 17-73 on page 246 to explain preferences

Version	Date	Description
N-2017.12-SP5	July 2018	■ Adjusted the default supported version of Synopsys Design Constraints (SDC) files to 1.9 in “Supported SDC Versions” on page 84 ■ Removed out-dated examples of the <code>get_supply_voltage</code> command and surrounding text below Table 9-39 on page 127
L-2016.09-SP3	March 2017	Added example of setting register field values; pg 112 Added material for adding files to plugins; pg 241
L-2016.09-SP2	December 2016	Corrected typos in Attribute names for RegisterField and RegisterFieldValues in Table 8-25 on page 107 Updated the description of the RegisterFieldValues attribute in Table 8-26 on page 108
K-2015.06-SP2	October 2015	Corrected Write Batch Script option for the GUI in step 3 (p. 83) and in options table (p. 221)
K-2015.06	June 2015	Version/date update only; technical content unchanged
J-2015.03	March 2015	Added Specify Power Intent activity, p. 127
J-2014.12	December 2014	IP-XACT updates, p. 73, p. 116, p. 226
J-2014.09	September 2014	Version/Date update only; technical content unchanged.
I-2014.03-SP1	April 2014	RALAccessType and VolatileMemory attributes updated, p. 108.
I-2014.03	March 2014	Date/Version updates. Changed “AddressBitUnits” to “BitsInLAU” p. 108.
I-2013.09	September 2013	Platform and version update only.
H-2013.03-SP1	May 23, 2013	Doc update: removed incorrect reference to “Library Button” p. 95.
H-2013.03-SP1	April 2013	Added 'Static Timing Analysis' tab in the Specify Synthesis Methodology window p. 118.
H-2013.03	March 2013	Copyright page and date/version updated.
G-2012.06-SP3	January 2013	Memory Map attribute updates, p. 108.
G-2012.06-SP2	October 2012	Fixed broken link to Licensing Information on www.synopsys.com
G-2012.06	June 2012	Added Table-driven Data Import/Export. p. 255. Packaging of stand-alone System Verilog interface.
F-2011.12	December 2011	Date / Version change only; no content change.
F-2011.06	June 2011	Date / Version change only; no content change.
E-2010.09-SP2	January 2011	Various documentation updates. Add Interface Definitions from directory, p. 189.

Version	Date	Description
E-2010.09	September 2010	GUI for creating Interface Definitions, p. 183; System Verilog “Load Designs” support, p. 94; Area estimation modeling, p. 130;
D-2010.03-SP1	April 2010	Design file prefixing, p. 172; Define Component Views activity supports hierarchical views, p. 24;
D-2010.03	March 2010	Copyright update; Define Views activity for IP-XACT, p. 35
C-2009.06-SP2	January 2010	New GUI tabs and parameters for Load Designs activity, p. 94
C-2009.06-SP1	October 2009	Generate Reports, Report tab, p. 28, Generating Optional Views, p. 28
C-2009.06-3	September 2009	Build IP-XACT Component, p. 40, MemoryAccess values, p. 108
C-2009.06	June 2009	IP-XACT workspace flow, p. 40; UPF packaging, p. 118
B-2008.12	December 2008	This information not available
B-2008.06	June 2008	Creating Interface Definitions, p. 179

Preface

This chapter contains the following sections:

- [“About this User Guide”](#) on page 15
- [“Related Documents”](#) on page 15
- [“Manual Overview”](#) on page 16
- [“Typographical and Symbol Conventions”](#) on page 17
- [“Synopsys Common Licensing \(SCL\)”](#) on page 17
- [“Synopsys Statement on Inclusivity and Diversity”](#) on page 17
- [“Customer Support”](#) on page 18

About this User Guide

This user guide is for core developers who use the coreBuilder tool to develop and package reusable cores. Readers of this user guide should be familiar with:

- IC design methodology including the concept of integrating previously designed blocks into a larger design
- High-level design languages (VHDL, Verilog, SystemVerilog)
- Synthesis with Synopsys Design Compiler
- Design verification, configurable testbenches, and VHDL/Verilog/SystemVerilog simulation tools

Related Documents

This user guide is part of the coreTools document set. Information about the coreTools is available from the following sources and from the online help system on the Help menu of each tool:

- General Documents
 - Guide to coreTools Documentation*
 - coreTools Release Notes*
- Working with Packaged coreKits
 - coreConsultant User Guide*
 - coreAssembler User Guide*

For thorough coverage of the theory and practice of design-reuse, we recommend:

- Keating, Michael and Bricaud, Pierre. Reuse Methodology Manual, 3rd Edition. Kluwer Academic Publishers, 2002.

You may also find useful the documentation for the following related Synopsys products: Synopsys IP, Design Compiler, Fusion Compiler, DC-Topographical, PrimeTime, and Power Compiler. For more information about these products, see:

- Documentation on the Web, which is available through SolvNetPlus on the Synopsys Web page at <http://solvnetplus.synopsys.com> (a Synopsys username and password are required).
- Documentation for Synopsys IP components, located using “Search for IP”:
<https://www.synopsys.com/designware-ip.html>

Manual Overview

This user guide contains the following information:

- [Chapter 1, “Introduction”](#) introduces the coreTools set and defines key terms and concepts.
- [Chapter 2, “The coreBuilder Environment”](#) introduces activity-based design flow and work environment; suggests an activity flow for new users.
- [Chapter 3, “Setting Up Your Workspace”](#) describes how to invoke coreBuilder and set up a workspace.
- [Chapter 4, “Annotating Design Files”](#) describes how to annotate design files to define configuration intent.
- [Chapter 5, “Writing Intent Command Files”](#) describes how to write synthesis intent command files.
- [Chapter 6, “Refining Configuration Intent”](#) describes how to organize your design files and load your design into coreBuilder.
- [Chapter 7, “Loading Your Design”](#) describes how to check and refine your configuration intent specifications for the core.
- [Chapter 8, “Specifying a Memory Map”](#) describes how to create, view and edit Memory Map intent using both the activity flow and command interface.
- [Chapter 9, “Specifying Synthesis Intent”](#) describes how to view and edit your synthesis intent interactively.
- [Chapter 10, “Checking Your Work in Progress”](#) describes how to build a coreConsultant workspace to check the current state of your coreKit.
- [Chapter 11, “Setting Up Configurable Filegroups”](#) describes how to set up files for selective installation and custom file content.
- [Chapter 12, “Setting Up Verification Suites”](#) describes how to package a configurable, automated verification environment for your core.
- [Chapter 13, “Building the coreKit or IP-XACT Component”](#) describes how to specify and load deliverables and build the coreKit.
- [Chapter 14, “Defining Interfaces for coreAssembler”](#) describes how to define interfaces so that coreAssembler can automatically connect your core to other cores that use the same interfaces.
- [Chapter 15, “Batch Mode”](#) describes how to use batch mode features.

- [Chapter 16, “Using IP-XACT Components”](#) covers details of IP-XACT components and designs within coreTools, and how to use IP-XACT generators and generator chains.
- [Chapter 17, “Advanced Topics”](#) describes how to use advanced features including setting up advanced parameter dialogs and customizing coreTools.

Typographical and Symbol Conventions

The following conventions are used throughout this document:

Convention	Description and Example
%	Represents the UNIX prompt.
Bold	User input (text entered by the user). % \$SYNOPSIS/bin/coreConsultant
Monospace	System-generated text (prompts, messages, files, reports). No Mismatches: 66 Vectors processed: 66 Possible"
Italic or Italic	Variables for which you supply a specific value; as a command line example: % setenv SYNOPSIS prod_dir In body text: In the previous example, prod_dir is the directory where your product must be installed.
(Vertical rule)	Choice among alternatives, as in the following syntax example: -effort_level low medium high
[] (Square brackets)	Enclose optional parameters: pin1 [pin2 ... pinN] This notation means that you must enter at least one pin name (pin1), but others are optional ([pin2 ... pinN]).
TopMenu > SubMenu	Pulldown menu paths, such as: File > Save As ...

Synopsys Common Licensing (SCL)

You can find general SCL information on the following page:

<http://www.synopsys.com/keys>

License Information -- Synopsys Common Licensing Quickstart:

<https://www.synopsys.com/support/licensing-installation-computeplatforms/licensing.html>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in

our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

Synopsys provides various methods for contacting Customer Support, as follows:

- For fastest response, enter a case through SolvNetPlus:
 - a. Open the SolvNetPlus web page using the following URL:
<https://solvnetplus.synopsys.com/>
 - b. Click the Cases menu and then click Create a New Case (below the list of cases).
 - c. Complete the mandatory fields that are marked with an asterisk and click Save.
Make sure to include the following:
 - Product L1: coreTools
 - Product L2: coreBuilder
 - d. After creating the case, attach any debug files you created.
For more information about general usage, see the following article in SolvNetPlus:
<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>
- Or, send an e-mail message to support_center@synopsys.com (your e-mail will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
- Include the Product L1, Product L2, process, and version number in your e-mail so it can be routed correctly.
- Attach any debug files you created.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

1

Introduction

The coreBuilder application is part of the Synopsys coreTools for design reuse. Core developers use the coreBuilder tool to package designs for use with the coreConsultant and coreAssembler tools. Core integrators use the coreConsultant and coreAssembler tools to install, configure, verify, and synthesize the reusable cores and subsystems.

This chapter contains the following sections:

- [“The coreTools”](#) on page 20
- [“coreBuilder Application Inputs and Outputs”](#) on page 22
- [“Key Terms and Concepts”](#) on page 24

1.1 The coreTools

The coreTools set of applications includes coreBuilder, coreConsultant, and coreAssembler.

1.1.1 coreBuilder

Core developers use the coreBuilder tool to build coreKits for use with the coreConsultant and coreAssembler tools. By following a guided flow of activities, core developers define the configuration options of the core, and apply attributes to the core's designs and ports to describe the internal design structure and its expected target environment. The coreBuilder tool stores this information in a manner that is independent of the target clock rate and process technology. These attributes, plus additional application and technology-specific information provided by the core integrator, enable the coreConsultant tool to interact with Synopsys synthesis tools to synthesize an implementation of the core that it is optimized for the target application.

The coreBuilder tool also enables the core developer to (optionally):

- Package a configurable verification environment with the core. The coreConsultant tool then automates the configuration and execution of the verification environment for the core integrator.
- Define interfaces so that coreAssembler users can automatically connect cores that share common interfaces.

1.1.2 coreConsultant

Core integrators use the coreConsultant tool to work with coreKits created by core developers. The coreConsultant tool provides an automated, guided flow that core integrators use to:

- Install the coreKit.
- Configure the core to fit the application. Configuration information specified by the core developer enables the coreConsultant tool to: guide the core integrator through the selection of configuration options, generate configured HDL code, and prevent the generation of invalid configurations.
- Synthesize the core. The coreConsultant tool interfaces with Synopsys synthesis tools to synthesize the core using the integrator's target technology and environmental constraints. The coreConsultant tool provides several synthesis strategies that use Synopsys Design Compiler, Fusion Compiler, DC-Topographical, and/or Synplify-FPGA. The coreConsultant tool also extracts reports from the synthesis tools and organizes the data into an easy-to-understand hierarchical set of hyperlinked reports.
- Verify the core. If the core developer includes a configurable verification environment with the coreKit, the coreConsultant tool automatically configures the verification environment as needed, and invokes the core integrator's selected simulation tool to execute selected tests and report the results.
- Generate timing models. The coreConsultant tool interfaces with Synopsys PrimeTime to (optionally) generate a timing model of the configured, synthesized core for chip-level testing.
- Formally verify the gate-level implementation. The coreConsultant tool interfaces with Synopsys Formality to formally verify the synthesized gate-level implementation of the core, using either the RTL version or another gate-level implementation as the reference design.

1.1.3 coreAssembler

The coreAssembler tool is a subsystem design tool that is similar to the coreConsultant tool but works at the subsystem level, where a subsystem is an interconnected set of cores. The coreAssembler tool enables core integrators to connect a group of cores with common interfaces together to form a subsystem and synthesize it.

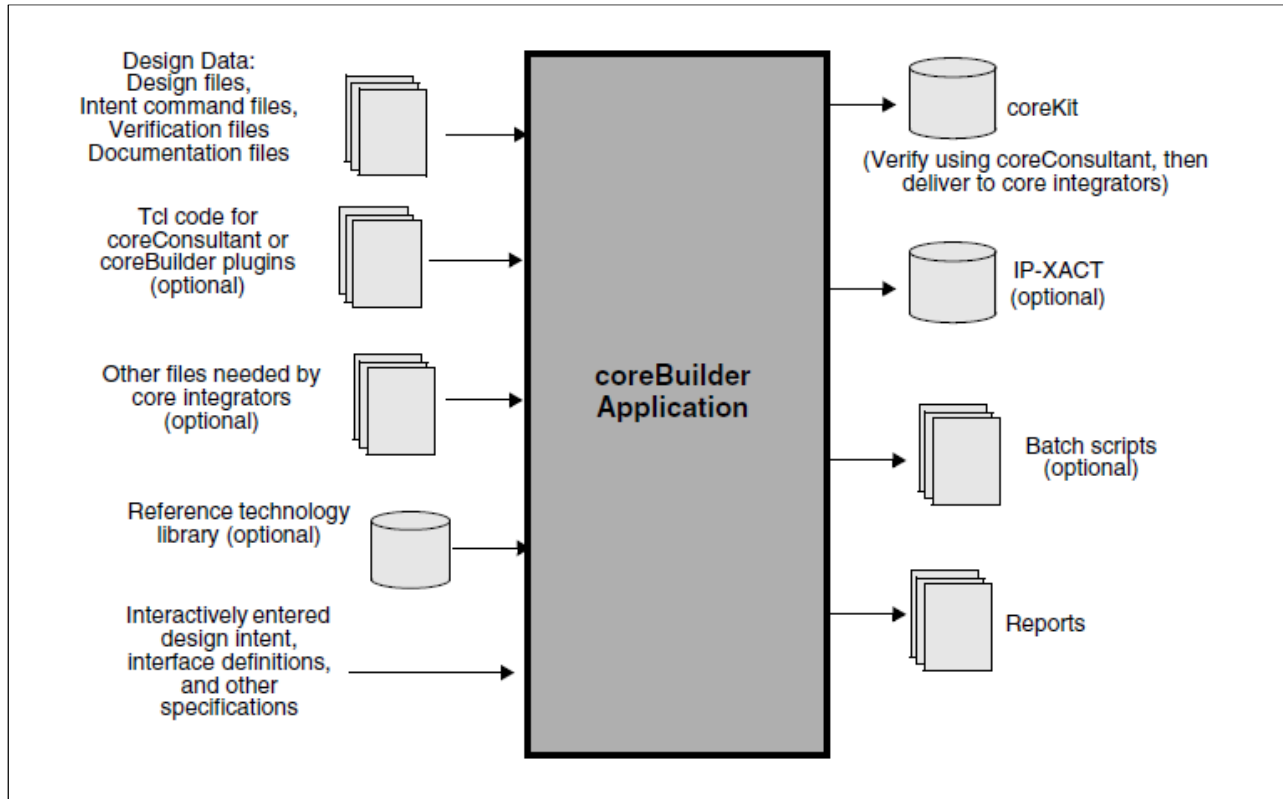
The coreAssembler tool allows a core integrator to:

- Select which cores to include in the subsystem
- Configure the individual cores and their interfaces
- Verify individual cores in verification environments packaged using the coreBuilder tool
- Automatically connect pins between cores and generate the HDL for the subsystem
- Synthesize the subsystem

1.2 coreBuilder Application Inputs and Outputs

The coreBuilder tool packages both your design and its associated design intent information into a coreKit, as shown in [Figure 1-1](#).

Figure 1-1 coreBuilder Application Inputs and Outputs



[Table 1-1](#) lists the coreBuilder tool input data for coreKit creation.

Table 1-1 coreBuilder Input Data

Input Data	Description
Design files	HDL files annotated with design configuration intent information.
Intent command files	Synthesis intent information contained in files separate from the HDL source. In addition to synthesis intent specified in coreBuilder syntax, you can also import design constraints in Synopsys Design Constraints (SDC) format.
Interactively entered design intent	After the coreBuilder tool builds a model of a design, you can interactively specify additional design intent information. Interactively specified intent overrides the design intent information inherited from the HDL source files and intent command files.
Reference technology library	The coreBuilder tool uses a default reference technology library to validate synthesis constraint information that the core developer has entered in a technology-independent format. If you plan to specify any of your design intent in a technology-dependent format, you must load your reference technology library into coreBuilder.

Table 1-1 coreBuilder Input Data (continued)

Input Data	Description
Plugins	You can customize the behavior of coreBuilder by building plugins. First, you code the desired functionality in Tcl files, then coreBuilder builds the plugin automatically. For core-specific plugins, coreBuilder automatically adds the plugins to the coreKit.
Documentation files	You can specify documentation files for inclusion in the coreKit and link them to the coreConsultant help interface.
Verification files	You can supply a configurable verification environment written specifically for your design. The coreBuilder tool provides utilities to automatically package your verification environment into the coreKit so that core integrators can configure and operate the verification environment directly through the coreConsultant interface.
Interface definitions	If your core shares a common interface with other cores, you can define the interfaces in coreBuilder so that coreAssembler users can automatically connect sets of cores with common interfaces to build a subsystem.
Other optional files	Any other files that you want to include in the coreKit. For example, you may want to supply a design example that shows an example implementation of the core.
Packaging Information	Information that coreBuilder needs to create a bill of materials and build the coreKit.

[Table 1-2](#) lists the coreBuilder tool output data.

Table 1-2 coreBuilder Output Data

Output Data	Description
coreKit	The file delivered to the core integrator containing all information required to implement the reusable core.
Batch Scripts	You can use the coreBuilder <code>write_batch_script</code> command to automatically generate a coreBuilder batch script that creates a mirror image of your workspace. You can use this batch script to set up a similar workspace or use it as a baseline script that you modify, so as to continue your work in batch mode.
Design Constraints	The coreBuilder tool can export attributes on design objects as design constraints in SDC format for use by other tools such as PrimeTime.

1.3 Key Terms and Concepts

Discussion of the packaging and reuse of cores using the coreTools revolves around the key terms and concepts defined in [Table 1-3](#).

Table 1-3 Key Terms in coreTools

Term	Definition
Core	A reusable design developed by a core developer and incorporated into chip-level applications by one or more core integrators.
Intellectual Property (IP)	In the broad context of design reuse, IP refers to any reusable design. Many features of the coreTools focus on the reuse of soft cores (user-configurable, technology-independent blocks of synthesizable RTL code). However, the coreTools also support packaging verification IP blocks and hard macros into coreKits.
Core Developer	The creator of a reusable core; using the coreBuilder tool, the core developer applies attributes to a core design and packages the core so that core integrators can easily implement it in system-on-a-chip applications.
coreKit	The design package created using the coreBuilder tool to facilitate IP reuse. Each coreKit includes the core itself, the design intent specified by the core developer, and optional items, such as verification environment files and core-specific documentation.
Core Integrator	The person who incorporates a reusable core into a larger design. Core integrators use the coreConsultant tool for implementation of individual cores and coreAssembler to build subsystems from multiple individual cores.
Application Design	The core integrator's overall, chip-level design, into which the reusable core is integrated. Typically, an application includes one or more reusable cores, plus custom, application-specific logic.
Intent	In broad terms, the core developer's knowledge of a design's functionality and characteristics. Intent also refers to the core developer's assumptions as to the context in which the core integrator implements the core.
Configuration Intent	At the most general level, the range of values allowed for each parameter associated with a reusable core. The core developer specifies the exact ways in which the core may be configured for reuse in chip-level applications. The core developer also identifies parameter dependencies so that the coreConsultant tool accepts only legal combinations of parameter values from the core integrator.
Synthesis Intent	Attributes that the core developer applies to the core's top-level design, ports, and subdesigns. The coreConsultant tool uses these attributes to drive the synthesis process. Synthesis intent determines how the coreConsultant tool generates synthesis constraints, strategies, and directives.
Design Context	Aspects of the core's target environment that affect the synthesis of the core. The core integrator specifies the design context before the coreConsultant tool generates a synthesis strategy for the core. Design context includes information such as wire load model and operating conditions.

Table 1-3 Key Terms in coreTools (continued)

Term	Definition
Workspace	A file system in which the coreBuilder tool automatically maintains your coreKit files during development. Your source files are inputs to the workspace and the coreKit, but are not dependent on the workspace/coreKit. That is, you maintain your source files independently of the workspace and (re)load them into the workspace as needed.

2

The coreBuilder Environment

Before you begin coreKit development, you should understand the aspects of the coreBuilder tool work environment.

This chapter contains the following sections:

- [“Activity-Based Design Flow”](#) on page 28
- [“Workspaces”](#) on page 30
- [“User Interfaces”](#) on page 31
- [“Online Help Tools”](#) on page 34
- [“Suggested Development Flows”](#) on page 35

2.1 Activity-Based Design Flow

The coreBuilder tool uses an activity-based design flow to guide you through the coreKit development and packaging tasks. Each activity implements a logical step in the coreKit development process. The order of activity execution is flexible. However, the coreBuilder tool does enforce some logical dependencies by disabling activities that depend on data from other activities until those activities have been completed.

[Table 2-1](#) lists the default set of coreBuilder activities. To support custom design flows, you can also define your own custom activities and create coreBuilder plugins to insert them into the activity flow.

Table 2-1 Default Activity Set

Activity	Function
Setup – Load Consultant Plugins	Optional activity to create core-specific plugins for the coreConsultant tool.
Setup – Load Designs	Loads your design and design intent files into the coreBuilder tool.
Setup – Design Configuration Intent	Enables you to view, modify, and check your design configuration intent (attributes that determine the core's configuration options).
Memory Maps – Define Memory Maps	Activity to create and view memory maps.
Memory Maps – Advanced Memory Map Attributes	Activity to define advanced attributes on memory maps.
Memory Maps – Memory Map Documentation	Activity to specify all memory maps attributes related to documentation.
Verification – Testsuite Regression Environment	Defines configurable testsuites and includes them in the coreKit.
Synthesis – Specify Reference Technology	Optional activity for designs that require technology-specific intent.
Synthesis – Specify Clocks	Defines the design clocks and default clock rates.
Synthesis – Specify Synthesis Methodology	Displays the current values for attributes that describe design characteristics and control the synthesis process, and enables you to modify those values.
Synthesis – Specify Test Methodology	Allows you to set and/or review the Design for Test (DFT) intent attributes on your components and designs.
Synthesis – Specify Timing Exceptions	Displays the currently specified timing exceptions in the design and enables you to modify them and/or add new timing exceptions.
Synthesis – Specify Port Constraints	Displays the current values for port attributes, most of which control constraint generation, and enables you to modify those values.
Synthesis – Verify Intent	Checks the currently specified port attributes for completeness and consistency.
Packaging – Create Integration Workspaces	Creates a coreConsultant workspace for the current state of the coreKit so that you can use the coreConsultant tool to check your work from the core integrator's perspective.

Table 2-1 Default Activity Set (continued)

Activity	Function
Packaging – Soc Assembly Intent – Load Interface Definitions	Optional activity to load interface definitions into the coreBuilder tool. This activity applies only to cores that are intended for use with the coreAssembler tool.
Packaging – Soc Assembly Intent – Add Interfaces	Optional activity to instantiate interfaces (from previously loaded interface definitions) for the core and set instance-specific attributes on the interfaces. This activity applies only to cores that are intended for use with the coreAssembler tool.
Packaging – Create Bill of Materials	Specifies the complete set of files to be included in the coreKit and how the files are to be installed in the core integrator's environment.
Packaging – Set Up Help Menus	Links core-specific documentation to the coreConsultant Help menu.
Packaging – Define Component Views	(IP-XACT flow) Allows you to define the IP-XACT views to be included in the generated IP-XACT component.
Packaging – Build coreKit	Builds the coreKit using the data specified in all previous activities.
Packaging – Build IP-XACT Component	(IP-XACT flow) Builds the IP-XACT component to the “version” specified in Edit > Preferences > IP-XACT using data specified in previous activities.

2.2 Workspaces

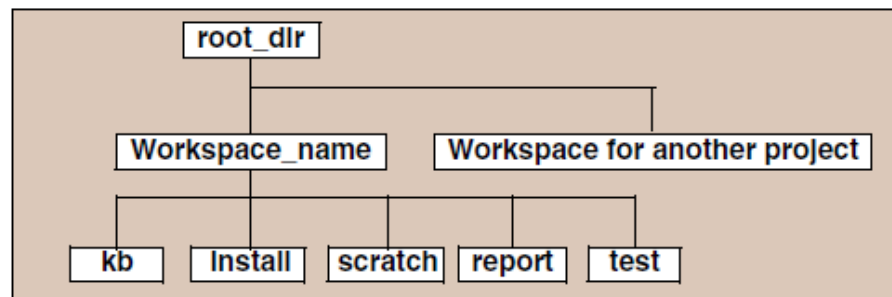
The coreBuilder tool uses a workspace to store the data associated with the creation of a coreKit. A workspace is essentially a directory structure in which coreBuilder manages the files that it creates as part of the coreKit development. Your own design data (HDL source files, test suite files, and documentation files) remain independent of the workspace; that is, you can create more than one workspace from the same set of source files.

To work with coreBuilder, the first step is to create a workspace. You then step through the design activities in your workspace, importing data as needed, and saving, closing, and re-opening your workspace as needed until your final coreKit is ready for distribution.

Your workspace uses the directory structure shown in [Figure 2-1](#). The coreBuilder tool uses the following subdirectories:

- kb The directory where coreBuilder stores the knowledge databases (.kb files) for your project.
- scratch The directory where coreBuilder stores temporary files.
- install The directory where coreBuilder stores your final coreKit files.
- report The directory where coreBuilder stores report files.
- test The directory where coreBuilder builds creates coreConsultant workspaces for preliminary coreKit testing.

Figure 2-1 Workspace Directory Structure



The coreBuilder tool uses knowledge databases (.kb files) as a binary file storage format for data associated with a coreKit, including the coreBuilder working model of your design. Every coreBuilder workspace contains a workspace .kb file that stores the current state of the workspace. The coreKit that you build at the end of the coreBuilder design flow (*name.coreKit*) is a special form of knowledge database.

2.3 User Interfaces

The coreBuilder tool provides the following user interfaces:

- [“Command Line Mode”](#)
- [“Batch Mode”](#)
- [“GUI Mode”](#)
- [“Generating Reports and Optional Views”](#)

2.3.1 Command Line Mode

Command line execution is available through the coreBuilder console in GUI mode or by invoking it in shell mode. In command line mode, you execute activities by using the following commands:

```
set_activity_parameter  
autocomplete_activity
```

2.3.2 Batch Mode

Batch mode enables you to execute a series of coreBuilder commands from a batch file. The coreBuilder tool provides a utility to write a batch file that contains the complete set of commands to re-create the current state of your workspace. [“Batch Mode”](#) on page 241 describes how to use batch mode.

2.3.3 GUI Mode

GUI mode is recommended for new users. In GUI mode, you interact with the coreBuilder tool through the console pane, the activity list pane, and activity view pane.

■ Console Pane

Contains three tabs. The Log tab contains the sequential log of all coreBuilder commands and their respective output. The History tab contains a sequential list of all coreBuilder commands executed directly in the console. The Errors/Warnings tab contains a sequential list of all errors messages and warnings and supports linking from a specific message into the Log tab.

■ Activity List Pane

Shows all activities available in a hierarchical tree. Completion status of each activity is indicated by a check mark in the box associated with each activity. Clicking on a label opens the Activity View Pane for that activity.

■ Activity View Pane

This pane shows the current activity and contains three tabs: a Dialog tab allowing data entry relevant to the current activity, a Report tab which allows you to generate reports on the status of the current activity and generate optional views, and a Help tab which shows Help information for the current activity.

2.3.3.1 Spreadsheets

In GUI mode, the coreBuilder tool presents spreadsheets with the current values of your synthesis intent attributes on design elements. These resemble the spreadsheets used in popular database and financial applications. Left-click to select a row, column, or cell. Right-click to access a menu of options including copy, cut, paste, help, and column-fill operations. Common spreadsheet operations are listed in [Table 2-2](#).

Table 2-2 Common Spreadsheet Operations

Operation	How To
Resize a column	Use your left mouse button to grab the column separator in the column title bar and move it left or right.
Enter a value in a cell	Click in a cell and type in text or select a value from the combo box menu. The coreBuilder tool does not save the value until you click the OK button. If you click the Cancel button, your edits are not saved.
Evaluate formula	Click your left mouse button on a cell containing a formula, then observe the value in the Current Cell field.
Select multiple cells in a column	To select all the cells in a column, click your left mouse button on the title bar of the column. To select several (but not all) cells, use your left mouse button to select the first cell and drag down to the last cell you wish to select. Alternately, press Shift, and use the left mouse button to select the last cell. CTRL-L is the toggle control for selecting/deselecting cells.
Cut	To cut the contents of one or more cells, select the cells, click the right mouse button, and select Cut. The keyboard shortcut for Cut is CTRL-X.
Copy	To copy the contents of one or more cells, select the cells, click the right mouse button, and select Copy. The keyboard shortcut for Copy is CTRL-C.
Paste	To paste the content of a previous cut or copy operation into one or more cells, select the cells into which you want to paste the data, click the right mouse button and select Paste. The size of the paste must match the size of the corresponding cut/copy. The keyboard shortcut for Paste is CTRL-V.
Fill a column	To fill one or more consecutive cells in a column with the current contents of a cell, select the cell containing the content, plus one or more cells below it. Click the right mouse button, and select Fill Down.
Find text in a column	To find text in a column, select a cell in the column, click the right mouse button, select Find, then enter the text you for which you are searching.
Replace text	To replace text in a column, select a cell in the column, click the right mouse button, select Replace, then enter the text you want to find and the text to insert in its place.

2.3.4 Generating Reports and Optional Views

When you complete an activity, the Report tab at the bottom of the View Pane shows you the reports and views that are available for that activity. To enhance performance, these reports and views are not automatically generated, but complete control is provided through the links in the activity summary page.

Table 2-3 The Activity Summary Page Controls

Control name	Description
generate report	Click on the link text to generate the specific report and view the report.
generate view	Click on the link text to generate specific optional views (such as IP-XACT)

Table 2-3 The Activity Summary Page Controls (continued)

Control name	Description
regenerate report	Discards the previous report and regenerates the report, and views the report
<report_name>	Click on the report name (previously generated) to view the specific report
File-->Generate Reports	Performs the same behavior as the File > Generate Reports menu item
File-->Generate Optional Views	Performs the same behavior as the File > Generate Optional Views menu item
View-->Reports	Performs the same behavior as the View > Reports menu item; presents a dialog box containing an “index” of all of the generated reports

2.3.4.1 Generating Multiple Reports

The **File > Generate Reports** menu item presents a dialog box that allows you to generate multiple activity reports from a single action. The “Delete Old Reports” option removes existing reports before regenerating the reports that you choose. There is a check box for each report you can generate, but if the item is grayed out, that report cannot yet be generated (the corresponding activity may not be complete). Click OK in the Generate Reports dialog box to generate the selected reports.



Note

1. Old reports are not automatically removed when you make design changes, so you may want to “Delete Old Reports” first to avoid viewing stale data.
2. Reports for synthesis and verification activities are generated by these tools, and not handled by Generate Reports dialog.

2.3.4.2 Generating Multiple Views

The **File > Generate Optional Views** menu items allows you to generate multiple activity design views from a single action. There is a check box for each view you can generate, but if the item is grayed out, that view cannot yet be generated (the corresponding activity may not be complete). Click OK in the Generate Optional Views dialog box to generate the selected views.

2.4 Online Help Tools

The coreBuilder tool provides several types of help:

■ “What’s This?” Quick Help

When you position your mouse pointer over a GUI item and right-click, the coreBuilder tool displays a menu with the “What’s This?” option. Clicking “What’s This?” displays a brief help message for the selected item (if available). The “What’s This?” information can also be accessed by left-clicking the Question Pointer and then left-clicking a GUI item.

■ Toolbar Help Menu

Click the toolbar Help button to display a list of help topics. When you click a topic, the corresponding help information appears in the activity pane (if defined in HTML format) or in your web browser (if specified in PDF).

■ Activity View Help Tab

The Activity View pane features a Help tab that displays a context-specific help page with additional links to the online command reference and other appropriate references. The Activity View Help tab contains several toolbar buttons at the top. You can use the buttons on the upper left to go to the tool startup page, the previous page, the next page, or to refresh the current page (from left to right). You can use the buttons on the upper right to open a separate window with the same page in it, or to open the page in a browser for printing. Opening the page in a separate window can be useful because it allows you to see the information in the Help tab and in the Dialog tab at the same time.

■ Help on coreBuilder Commands

The Synopsys coreTools Online Command Reference Index is available through the Help menu. It contains a collection of man pages for all coreBuilder commands, attributes, variables, and item types. You can also access coreBuilder command help by entering one of the following commands at the coreBuilder prompt in the console pane:

```
coreBuilder> help [cmd] [-verbose]
coreBuilder> cmd -help
coreBuilder> man [cmd]
```

■ Other GUI Help Procedures

Useful trouble-shooting information is available through the Report tab of the Activity View pane and through the Errors/Warnings tab of the Console pane.

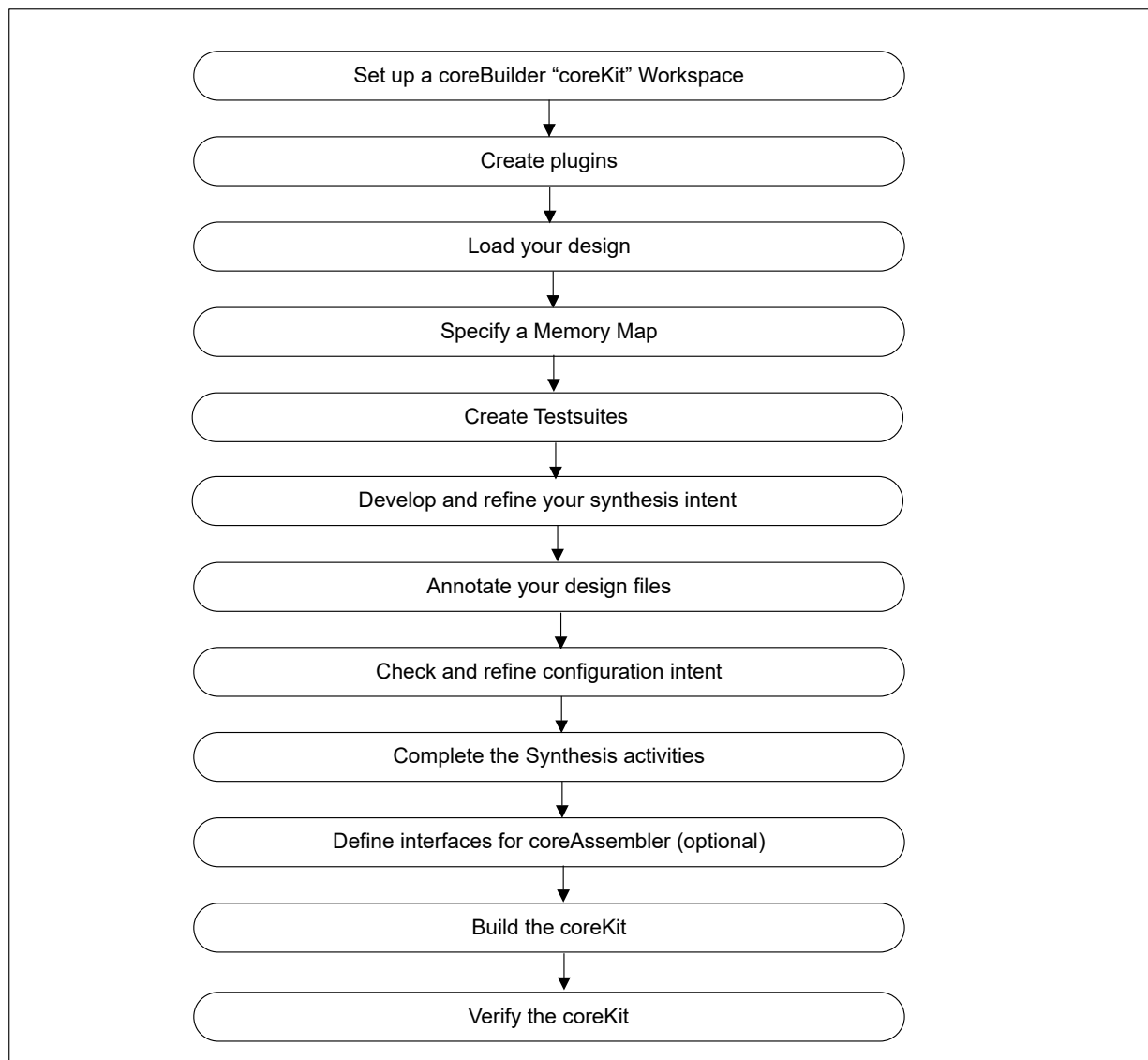
2.5 Suggested Development Flows

The coreBuilder tool provides two flexible workflows for developing IP: one for a coreKit and one for an IP-XACT component. You can perform non-dependent tasks in any order. You can also partially complete tasks and return to them later.

2.5.1 coreKit Development Flow

Figure 2-2 shows the recommended steps to develop and verify a coreKit for a synthesizable core, using the coreBuilder tool for packaging and the coreConsultant tool to verify the coreKit from the user's perspective.

Figure 2-2 Suggested coreKit Development Flow



The steps in [Figure 2-2](#) are:

Step 1: Set up a coreBuilder “coreKit” Workspace

Create a workspace of type “coreKit” in which to develop and refine the synthesis intent for the core.

Step 2: Create plugins

This optional step allows you to add custom, core-specific activities to the coreConsultant activity flow. You can return to this step at any time in the coreKit development flow if you discover the need to include plugins in the coreKit.

Step 3: Load your design

Load the HDL source files for your design into the coreBuilder tool.

Step 4: Specify a Memory Map

The memory map specification for a component allows you to specify the memory resources needed for each slave interface of your component. A memory map is a container holding address regions that are described by adding address blocks and address banks to the memory map.

Step 5: Create testsuites

Create one or more user-configurable test suite regression environments and prepare them for packaging in the coreKit.

Step 6: Develop and refine your synthesis intent

Use the coreBuilder tool in either GUI or batch mode to specify your synthesis intent. Check your synthesis intent specifications by synthesizing the design with various configurations, target environments, and target technologies. Refine the synthesis intent to get consistently acceptable QoR.

Save your synthesis intent specifications as Tcl intent command files. If you specify your synthesis intent in GUI mode, use the **File > Write Script** command to generate a Tcl intent command file that implements your specifications.

Step 7: Annotate your design files

To set up the coreConsultant tool to support automatic configuration of your core, you must insert coreTools reuse pragmas into your HDL source files. These special lines of text (which Design Compiler interprets as comment lines, and therefore ignores) hold special meaning for the coreTools. Using these pragmas, you can specify configuration intent (for example, minimum and maximum values for design parameters).

Step 8: Check and refine configuration intent

Once you have your design loaded into the coreBuilder tool, preview the coreConsultant configuration dialogs and modify any features that need correction or enhancement.

Step 9: Complete the Synthesis activities

Refine your synthesis intent and complete the synthesis activities.

Step 10: Define interfaces for coreAssembler (optional)

For cores with common interfaces, define the core’s interfaces for use with the coreAssembler tool.

Step 11: Build the coreKit

Working from either a default or a custom template, you create a Bill of Materials, which specifies all the files to be included in your coreKit. Once you have verified the contents of the coreKit, you are ready to ship your reusable design.

Step 12: Verify the coreKit

Use the coreConsultant tool to install the coreKit and create one or more workspaces in which you can verify that all the features of your coreKit work as intended.

You can also verify the state of the coreKit at any time in the coreBuilder development process by performing the Create Integration Workspaces activity to create a coreConsultant workspace.

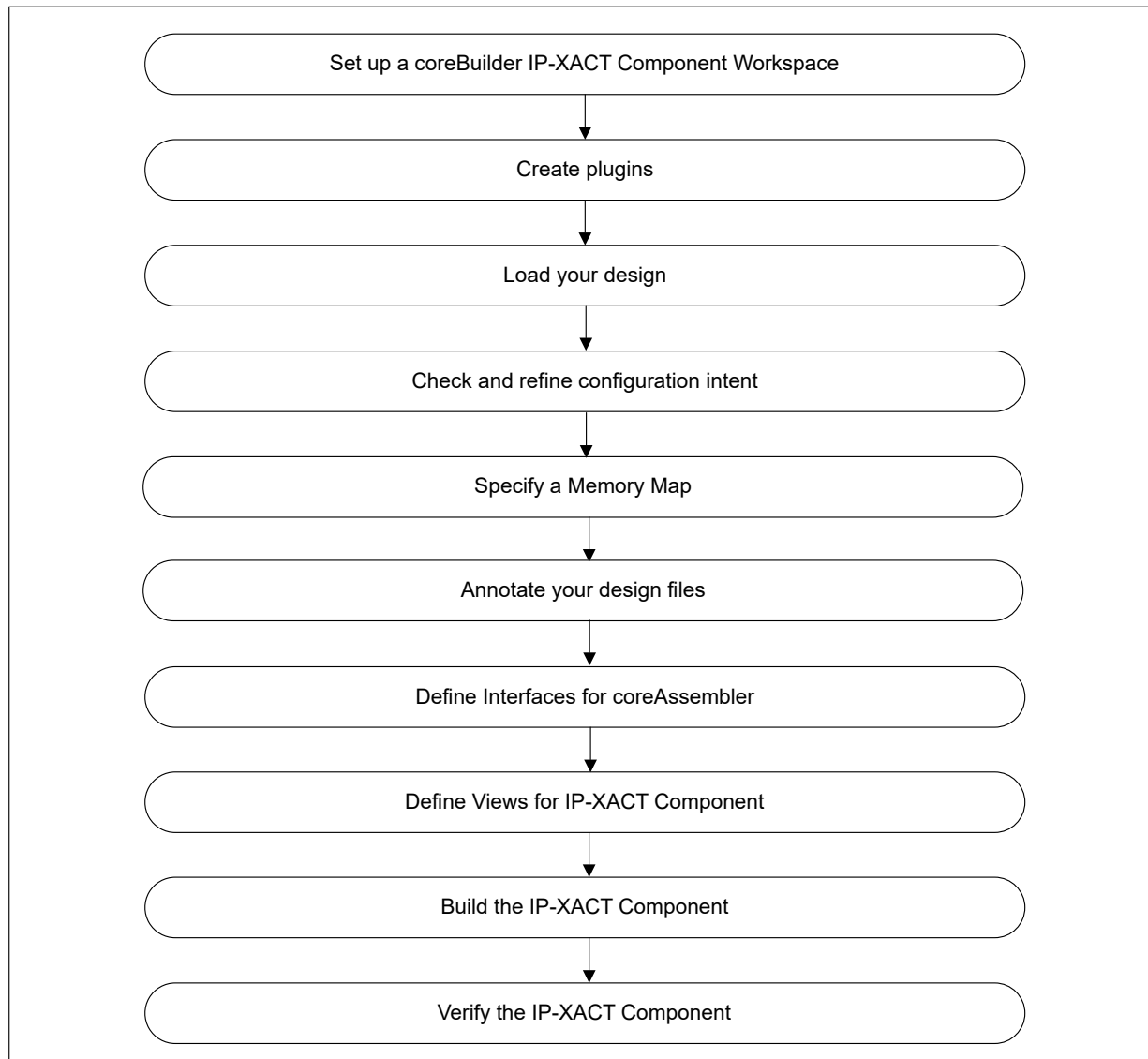


Note

In addition to packaging soft IP, as above, you can also use the coreBuilder tool to package other types of IP such as verification IP and hard macros. For more, see [“Advanced Topics”](#).

2.5.2 IP-XACT Component Development Flow

[Figure 3](#) on page [38](#) shows the steps to develop and verify an IP-XACT component for a synthesizable core, using the coreBuilder tool for packaging and the coreConsultant tool to verify the coreKit from the user’s perspective.

Figure 2-3 Suggested IP-XACT Component Development Flow

The steps in [Figure 2-3](#) are:

Step 1: Set up a coreBuilder IP-XACT Component Workspace

Create a workspace in which to develop and refine your synthesis intent for the core. For more details, see [“Creating a Workspace”](#) on page 43.

Step 2: Create plugins

This optional step allows you to add custom core-specific activities to the coreConsultant activity flow. You can return to this step at any time in the coreKit development flow if you discover the need to include plugins in the coreKit. For more details, see [“Optional Setup Activities”](#) on page 48.

Step 3: Load your design

Load the HDL source files for your design into the coreBuilder tool. For more details, see [Loading Your Design](#) on page 103.

Step 4: Check and refine configuration intent

Once you have your design loaded into the coreBuilder tool, preview the coreConsultant configuration dialogs and modify any features that need correction or enhancement. For more details, see [Refining Configuration Intent](#) on page 97.

Step 5: Specify a Memory Map

The memory map specification for a component allows you to specify the memory resources needed for each slave interface of your component. A memory map is a container holding address regions that are described by adding address blocks and address banks to the memory map. For more details, see [Specifying a Memory Map](#) on page 111.

Step 6: Annotate your design files

To set up the coreConsultant tool to support automatic configuration of your IP-XACT component, you must insert coreTools reuse pragmas into the XML source files for your design. These special lines of text (which Design Compiler interprets as comment lines, and therefore ignores) hold special meaning for the coreTools applications. Using these pragmas, you can specify configuration intent (for example, minimum and maximum values for design parameters). For more details, see “[Subsystem Assembly with coreAssembler](#)” on page 200.

Step 7: Define Interfaces for coreAssembler

This activity is optional. For more details, see [Defining Interfaces for coreAssembler](#).

Step 8: Define Views for IP-XACT Component

Allows you to define the IP-XACT views to be included in the generated IP-XACT component.

Step 9: Build the IP-XACT Component

Working from either a default or a custom template, you create a Bill of Materials, which specifies all the files to be included in your IP-XACT component. Once you have verified the contents of the IP#XACT component, you are ready to ship your reusable design. For more details, see “[Building the IP-XACT Component](#)” on page 193.

Step 10: Verify the IP-XACT Component

Use the coreConsultant tool to load the IP-XACT component and create one or more workspaces in which you can verify that all the features of your component work as intended.

3

Setting Up Your Workspace

To do work with the coreBuilder tool, you need to set up a workspace.

This chapter contains the following sections:

- [“Invoking coreBuilder”](#) on page 42
- [“Creating a Workspace”](#) on page 43
- [“Basic Workspace Operations”](#) on page 45
- [“Quitting and Restarting coreBuilder”](#) on page 47
- [“Optional Setup Activities”](#) on page 48

3.1 Invoking coreBuilder

To invoke the coreBuilder tool in GUI mode, enter the following command at the UNIX prompt:

```
% <coreTools_root>/bin/coreBuilder
```

To invoke the coreBuilder tool in command line mode, enter the following command at the UNIX prompt:

```
% <coreTools_root>/bin/coreBuilder -shell
```

See *coreTools Release Notes* for supported platforms and operating system versions.

The complete syntax for the command that invokes the coreBuilder tool is:

```
coreBuilder [Workspace_name] [-shell] [-x "command string"] [-f cmd_file]
```

Table 3-1 Command Line Options when Invoking coreBuilder

Argument	Description
Workspace_name	Loads the workspace Workspace_name upon startup.
-shell	Selects command line mode instead of GUI mode. For more information about command line mode, see “Batch Mode”.
-x “ command_string”	Executes command_string upon startup. The command_string is one or more coreBuilder commands, separated by semicolons (;).
-f cmd_file	Executes commands in a specified command file (cmd_file) containing coreBuilder commands in batch mode on startup. For information about batch mode operation, see “Batch Mode”.

3.1.1 Examples

To invoke the coreBuilder tool and load the Builder1 workspace, enter the following command:

```
% coreBuilder path/Builder1
```

To invoke the coreBuilder tool in command line mode and execute the batch command file named buildcoreKit.tcl, enter the following command:

```
% coreBuilder -shell -f buildcoreKit.tcl
```

3.2 Creating a Workspace

To create a new workspace in GUI mode, select File > New Workspace from the tool menu. A dialog box appears for the information for your new workspace. Enter values for the options listed in [Table 3-2](#), then click OK to create the workspace.

Table 3-2 Create Workspace Options

Option	Description
Workspace Name	Enter a name for your new workspace (the default is Builder1).
Workspace Root Directory	Enter the full path to the root directory where you want to store your workspace, or use the Browse button to navigate to your desired root directory. The coreBuilder tool creates your workspace under the selected root directory as path/Workspace (for example, path/Builder1). If you do not specify a root directory, the coreBuilder tool creates the workspace under your current directory.
BoM Template	Select a Bill of Materials (BoM) Template. A BoM Template creates pre-defined sets of deliverables (filegroups) to be included in the coreKit. You populate these filegroups when you execute the Create Bill of Materials activity. You can select either of the following BoM Templates, or you can create and select your own custom BoM Template as described in “Building the coreKit or IP-XACT Component” on page 181: Default_Template – Contains optional filegroups for test suites and documentation. Select this template when you plan to add all materials to the coreKit manually, and you don't need the coreBuilder tool to verify that the required contents are present. RMM_Template – Contains the deliverables suggested by the Reuse Methodology Manual. The list of BoM Templates available for selection includes all the files with the extension .tcl that the coreBuilder tool finds in the directories specified by \$rt_bom_template_dirs (see “Building the coreKit or IP-XACT Component” on page 181).
Output Format	Specify whether the final component will be a coreKit or an IP-XACT component. The IP-XACT choice is equivalent to using the -ipxact option with the create_workspace command, and the activity list is reduced to just what is needed for IP-XACT component generation. For additional information, see “IP-XACT Flow through coreBuilder” on page 44.

When the coreBuilder tool finishes creating and setting up your workspace, it automatically displays the Builder Activity List. By default, the first activity is checked off (completed). If you need to execute Load Consultant Plugins, select the activity from the Activity List and enter your specifications in the Activity View to re-specify the activity. See [“Optional Setup Activities”](#) on page 48 for details. You can also return to this activity at any time before building your final coreKit.

At this point, the next step is to load your design files. However, Synopsys recommends that you annotate your design files and write your intent command files before you load your design. Therefore, the recommended sequence for continuing is:

1. Save and close your workspace. See [“Closing a Workspace”](#) on page 45.
2. Read [“Annotating Design Files”](#) on page 49 and annotate your HDL design files as needed.
3. Read [“Writing Intent Command Files”](#) on page 73 and write your intent command files.

4. Re-start the coreBuilder tool and reload your workspace. See “[Quitting and Restarting coreBuilder](#)” on page 47.
5. Go to “[Loading Your Design](#)” on page 103 and follow the procedure to load your design.

You can also create a workspace in command line mode. To do so, use the `create_workspace` command. For details, see the coreBuilder command reference.

3.2.1 IP-XACT Flow through coreBuilder

When a workspace is created with the IP-XACT option, it implies that a coreKit is not required and that the ending deliverable is an IP-XACT component file. If an IP-XACT workspace is created, the following activities are not available in the coreBuilder flow:

- Verification
- Set-up Help Menu
- Create Integration Workspace
- Verify Intent
- Build IP-XACT Component (appears in place of Packaging/Build coreKit)

The Build IP-XACT Component activity allows some customization of the generated component file, allowing specification of the VLNV, and some options about what content to include in the XML file:

- Include (Synopsys) Vendor Extensions (default = true)
- Include Synthesis Intent - Constraints within component (default = false). This only applies if constraints are defined and specified in a technology independent format.
- Include Non-HDL File Sets (default = true)
- Common Prefix Replacement - String to use as a replacement for the common file prefix when generating the IP-XACT component. This is used to make file names relative or environment variable dependent.
- Common File Prefix - Common prefix on files to be replaced by the value of the `CommonPrefixReplacement` parameter.

In addition, certain existing activities are modified:

- Specify Configuration Intent -- an additional entry field is available to specify the value of the existing `ResolveType` attribute on parameters. This attribute corresponds to the `spirit:resolve` attribute in the IP-XACT parameter element.

3.3 Basic Workspace Operations

The following sections describe how to perform basic workspace operations: saving, closing, opening, and moving.

3.3.1 Saving a Workspace

The coreBuilder tool maintains the current state of your workspace in a knowledge database in the workspace. You can save this information at any time in the design flow. Once you've done so, you can close the workspace and quit the coreBuilder tool. You can then restart the coreBuilder tool, re-open your workspace, and continue where you left off.

To save your current workspace in GUI mode, select **File > Save Workspace** from the coreBuilder toolbar menu. (coreBuilder automatically saves to disk after a successfully completed activity).

In command line mode, use the coreBuilder `save_workspace` command to save your workspace, for example:

```
coreBuilder> save_workspace
```

3.3.2 Saving a Workspace As

You can also save your current workspace to a different workspace name by selecting **File > Save Workspace As...**. When you are using the **Save Workspace As...** command, the dialog box contains a checkbox which allows you to require that linked installation files are copied to the duplicate workspace.

3.3.3 Closing a Workspace

To close a workspace in GUI mode, select **File > Close Workspace** from the coreBuilder toolbar menu. If you have made any changes since the last time you saved the currently loaded workspace, coreBuilder prompts you to save the workspace.

In command line mode, use the coreBuilder `close_workspace` command to close your workspace, for example:

```
coreBuilder> close_workspace (Close without saving.)
```

```
coreBuilder> close_workspace -save (Save, then close.)
```

See the coreBuilder online reference for details.

3.3.4 Re-Opening a Workspace

Typically, you open an existing workspace by specifying the workspace name on the command line when you invoke the coreBuilder tool.

For example, to open the workspace Builder1, enter the following command:

```
% coreBuilder path/Builder1
```

You can also open an existing workspace by selecting **File > Open Workspace** from the coreBuilder toolbar menu.

In command line mode, use the coreBuilder `open_workspace` command (see the coreBuilder online reference for details) to open your workspace, for example:

```
coreBuilder> open_workspace path/Builder1
```

3.3.5 Moving a Workspace

You can move a coreBuilder workspace to a new location as long as you do not change the workspace name. To do so, use the UNIX mv command to move the workspace to a new directory, while keeping the same workspace name. For example:

```
% mv old_path/Builder1 new_path/Builder1
```

If you move the \$DESIGNWARE_HOME location, you must run the update_workspace_links -moved_install command on the workspace to update links to the new install area.

To replace the links in a workspace with the actual objects (for portability), you run the update_workspace_links -copy command. This is useful for backup and archive purposes, but creates a much larger workspace tree.

The update_workspace_links command is used to modify the symbolic links within a workspace.

```
update_workspace_links [-copy] [-moved_install <dir>] [workspace]
```

Parameters:

-copy	Copy all files linked to the workspace. This option removes all workspace symbolic links into the installation area. All links from the workspace to the installation area are replaced with copies of the actual files/directories.
-moved_install <dir>	New location of the installation. This option is used to indicate that the installation area for the given workspace has been moved from its original location to the directory specified by <dir>. Workspace links to the prior installation area are modified to point into <dir> instead.
workspace	Workspace to change.

For the complete description, enter “man <command_name>” from the coreAssembler command line.

3.4 Quitting and Restarting coreBuilder

You can quit the coreBuilder tool at any time, then restart it and reload your workspace to continue your work.

To quit the coreBuilder tool from GUI mode, select **File > Exit**. If you have executed commands that modify the workspace, coreBuilder prompts you to save your workspace before exiting. You can choose to save the workspace and quit, quit without saving the workspace, or cancel the quit operation

To quit the coreBuilder tool in command line mode, enter `exit` or `quit` at the coreBuilder prompt. For example:

```
coreBuilder> quit
```

If you execute the `exit` or `quit` command in GUI mode after executing commands that modify the workspace, the coreBuilder tool prompts you to save your workspace before exiting. However, in shell mode (`coreBuilder -shell`), coreBuilder does not prompt you to save your workspace. To save your work before quitting, enter the `save_workspace` command at the coreBuilder prompt before executing `quit` or `exit`.

To restart the coreBuilder tool and reload your workspace, specify the workspace with the coreBuilder, command:

```
% coreBuilder path/Builder1
```

3.5 Optional Setup Activities

The following setup activity is initially checked off (completed) because it is optional. You can execute this activity at any time in the coreBuilder flow. However, you need to re-execute any dependent activities that you have already completed.

3.5.1 Loading Consultant Plugins

You need to execute the Load Consultant Plugins activity to create core-specific plugins and include them in the coreKit if any of the following issues applies to your coreKit:

- You want to include custom Tcl command procedures in the coreKit (for example, to perform advanced checking of parameter dependencies).
- You want to add core-specific activities to the coreConsultant Activity List for your coreKit.
- You want to add core-specific hooks to existing coreConsultant activities for your coreKit.

Including core-specific plugins in your coreKit is a two-step process:

1. Write the Tcl code to implement the functionality of the plugin.
2. Execute the Load Consultant Plugins activity in the Setup tab to build the plugin(s) and automatically include the plugins in the coreKit.

For detailed information on this topic, see “[Advanced Topics](#)” on page 255.

4

Annotating Design Files

This chapter describes how to annotate your HDL design files to define your core's configuration options and to work around any problems you may encounter when coreBuilder parses your HDL source code. Annotating design files is typically an iterative process in which you load your design into the coreBuilder tool, observe the results of your HDL annotations, refine the annotations, and reload the design.

This chapter contains the following sections:

- [“Specifying Configuration Intent”](#) on page 50
- [“Support for Verilog Compiler Directives”](#) on page 65
- [“Conditional Inclusion of Cells and Ports”](#) on page 66
- [“HDL Coding Guidelines”](#) on page 70

4.1 Specifying Configuration Intent

Configurability is an important feature of most reusable cores. As a core developer, you specify design configuration intent to achieve the following objectives:

- Indicate which of the designs VHDL constant declarations and/or Verilog 'define statements are top-level design configuration parameters. (coreBuilder automatically recognizes VHDL generics and Verilog parameters on the top-level design as configuration parameters.)
- Define the legal range of values for each parameter.
- Define parameter inter-dependencies.
- Control the layout and appearance of the Specify Configuration dialog that the coreConsultant tool presents to the core integrator.

You specify design configuration intent by setting attributes on the core's top-level configuration parameters. By doing so, you enable the coreConsultant tool to:

- Guide the core integration through the process of configuring the core
- Generate configured HDL code. The coreConsultant tool automatically replaces the default parameter values with the user-specified values.

4.1.1 Configuration Intent Methodology

To specify your configuration intent:

1. Set attributes on your top-level design parameters, as described in the following sections. You can start by setting only the basic attributes, as described in [“Basic Parameter Attributes”](#) on page 52.
2. Load your design files into the coreBuilder tool by executing the Load Designs activity (see [“Loading Your Design”](#) on page 103).
3. Check your parameter attribute specifications by executing the Design Configuration Intent activity. In GUI mode, Design Configuration Intent enables you to preview the coreConsultant Specify Configuration dialog and test the value ranges and dependencies that you set on the parameters.
4. Refine your parameter specifications as needed. You can specify new attribute values directly in the Design Configuration Intent activity dialog. However, it is best to save your new attribute specifications as annotations in the HDL source files so that the configuration intent remains with the design instead of with the workspace.

4.1.2 HDL Annotation Syntax

To set an attribute on a parameter in an HDL file, use one of the syntaxes listed below. For each of the following syntaxes, the pragmas apply to the next parameter coreBuilder encounters in the file.

- To set an attribute on the next VHDL generic or constant:

```
-- reuse-pragma attr <attr> <value>
```

For example:

```
-- reuse-pragma attr MaxValue 16
constant addr_width : integer := 4;
```

- To set an attribute on the next Verilog parameter or constant (define):

```
// reuse-pragma attr <attr> <value>
```

For example:

```
// reuse-pragma attr MaxValue 16
`define addr_width 4;
```

- To set an attribute with a multi-line value on the next VHDL generic or constant:

```
-- reuse-pragma beginAttr <attr> <value>
-- <value continued>
-- <value continued>
-- reuse-pragma endAttr
```

For example:

```
-- reuse-pragma beginAttr Description Width of address bus
-- input from application logic
-- reuse-pragma endAttr
constant addr_width : integer := 4;
```

- To set an attribute only if an expression is valid (Verilog syntax shown):

```
// reuse-pragma attr if (<expr>) <attr> <value>
```

For example:

```
// reuse-pragma attr if (data_width=16) MaxValue 16
`define addr_width 4;
```

This conditional use applies within the port declaration section and applies to all ports regardless of where in the port declaration section the annotation appears.

Verilog `defines without a value are treated as Boolean parameters. For example:

```
// reuse-pragma attr Label Include FPU
`define use_FPU
```

defines a Boolean parameter. If the value is false in the configured core, the `define line is automatically commented out.

Setting an attribute on a VHDL constant or Verilog `define identifies the constant or `define as a top-level design parameter, which triggers coreBuilder to:

- Enable text substitution for the value of the parameter
- Display the parameter in the Design Configuration Intent dialog

coreBuilder automatically does both of the above for VHDL generic and Verilog parameter declarations.

4.1.3 Basic Parameter Attributes

For most parameters on most designs, you must set the basic attributes listed in [Table 4-1](#). These attributes define legal parameter values and assign meaningful labels to the parameter names and values. For complete descriptions and examples, see the man pages in the coreBuilder online reference. If you do not specify values for MinValue and MaxValue, coreBuilder automatically sets MinValue and MaxValue based on the parameter type and range (if defined in the HDL).

Table 4-1 Basic Attributes for Parameters

Attribute	Short Description
MinValue	Specifies a minimum legal value for a parameter.
MaxValue	Specifies a maximum legal value for a parameter.
EnumValues	Specifies a set of discrete legal values for a parameter, instead of using MinValue and MaxValue.
Label	Specifies a label to use for the parameter in the coreConsultant Specify Configuration GUI.
SymbolicNames	Specifies symbolic names for the parameter’s legal values.
Description	Specifies a brief description of the parameter to be used as part of the help text message for the parameter.

[Example 4-1](#) shows how to set basic attributes on a VHDL generic. [Example 4-2](#) shows how to set basic attributes on a VHDL constant. [Example 4-3](#) shows how to set basic attributes on a Verilog parameter. [Example 4-4](#) shows how to set basic attributes on a Verilog ‘define.

Example 4-1 Setting Basic Attributes on a VHDL Generic

```
entity TOP is
generic (
-- reuse-pragma attr MinValue 4
-- reuse-pragma attr MaxValue 16
-- reuse-pragma attr Label Address Bus Width
-- reuse-pragma beginAttr Description Width of address bus input
-- from application logic
-- reuse-pragma endAttr
  addr_width : integer := 4);
port (
  ...
```

Example 4-2 Setting Basic Attributes on a VHDL Constant

```
-- reuse-pragma attr MinValue 4
-- reuse-pragma attr MaxValue 16
-- reuse-pragma attr Label Address Bus Width
-- reuse-pragma beginAttr Description Width of address bus input
-- from application logic
```

```
-- reuse-pragma endAttr
constant addr_width : integer := 4;
```

Example 4-3 Setting Basic Attributes on a Verilog Parameter

```
module TOP
  (port list);
// reuse-pragma attr MinValue 4
// reuse-pragma attr MaxValue 16
// reuse-pragma attr Label Address Bus Width
// reuse-pragma beginAttr Description Width of address bus input
// from application logic
// reuse-pragma endAttr
  parameter addr_width = 4;
  ...
```

Example 4-4 Setting Basic Attributes on a Verilog Constant

```
// reuse-pragma attr MinValue 4
// reuse-pragma attr MaxValue 16
// reuse-pragma attr Label Address Bus Width
// reuse-pragma beginAttr Description Width of address bus input
// from application logic
// reuse-pragma endAttr
`define addr_width 4
```

4.1.4 The Parameter Specification Interface

When the coreConsultant tool builds the parameter specification dialog, it selects the appropriate mechanism for the user to specify the parameter value, depending on how you specify the parameter’s legal values ([Table 4-2](#)).

Table 4-2 Parameter Selection Mechanism Controls

Mechanism	For a Parameter with These Attributes
Checkbox	A boolean-type parameter with no SymbolicNames attribute. An integer parameter that has legal values 0 and 1 and either has no SymbolicNames or has SymbolicNames true and false.
Radio button	An enumerated parameter with 2 possible values. An integer parameter with legal values 0 and 1 and SymbolicNames other than true and false.
Combo box	An enumerated parameter with more than two possible values. A numeric parameter that has SymbolicNames for each of its legal values.
Text entry field	A numerical parameter with a legal range greater than 0 to 1 and that is also not restricted to a set of enumerated values and also does not have SymbolicNames for its legal values. A string parameter that does not have a restricted set of enumerated values.

[Example 4-5](#) shows a set of parameter attribute assignments for several constants declared in a VHDL package file.

Example 4-5 Setting Basic Attributes on Design Parameters (VHDL)

```

package TOP_parameter is

-- Use Checkbox
-- reuse-pragma beginAttr Description Implements the power management
-- feature, power management controlled by host device.
-- reuse-pragma endAttr
-- reuse-pragma attr Label Support Power Management
constant pwr_mgmt : boolean := false;

-- Use Checkbox
-- reuse-pragma attr Description Implements the optional 16-bit timer.
-- reuse-pragma attr SymbolicNames false true
-- reuse-pragma attr Label Secondary Timer
constant timer2 : natural range 0 to 1 := 0;

-- Use Radio Buttons
-- reuse-pragma attr Description Builds interface to RAM of selected size.
-- reuse-pragma attr SymbolicNames {128 Bytes} {256 Bytes}
-- reuse-pragma attr Label RAM size
constant ram_256 : natural range 0 to 1 := 0;

-- Use Radio Buttons
-- reuse-pragma attr Description Selects width of address/data bus.
-- reuse-pragma attr EnumValues 32 64
-- reuse-pragma attr Label Address/Data Bus Width
constant ad_width : integer := 64;

-- Use Combo Box
-- reuse-pragma beginAttr Description Selects the bit-width of the
-- peripheral device data bus.
-- reuse-pragma endAttr
-- reuse-pragma attr EnumValues 8 16 32 64
-- reuse-pragma attr Label Peripheral Data Width
constant p_width : integer := 32;

-- Use Combo Box
-- reuse-pragma attr Description Select the number of interrupt sources.
-- reuse-pragma attr MinValue 0
-- reuse-pragma attr MaxValue 6
-- reuse-pragma attr SymbolicNames Zero One Two Three Four Five Six
-- reuse-pragma attr Label Number of Interrupts
constant intrs : integer := 2;

-- Use Text Entry Field
-- reuse-pragma beginAttr Description Select the number of delay cycles
-- to insert when accessing a slow peripheral device.
-- reuse-pragma endAttr
-- reuse-pragma attr MinValue 0

```

```
-- reuse-pragma attr MaxValue 16
-- reuse-pragma attr Label Delay Cycles
constant delay_cycles : integer := 4;

end TOP_parameter;
```

[Example 4-6](#) shows parameter attribute assignments for check boxes declared in a Verilog package file.

Example 4-6 Setting Basic Attributes on a Design Parameter (Verilog)

* An existence parameter (for use with ``ifdef`):

```
// reuse-pragma attr Label Support Power Management
`define PW_MGNT
```

* An integer a parameter that can only take the values 0 and 1

```
// reuse-pragma attr Label Support Power Management
// reuse-pragma attr EnumValues 0 1
`define PW_MGNT 0
```

```
// Or use MinValue/MaxValue if you like that better
// reuse-pragma attr Label Support Power Management
// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
`define PW_MGNT
```

```
// Or let coreBuilder figure out Min/Max by giving a bit width
// reuse-pragma attr Label Support Power Management
`define PW_MGNT 1'b0
```

4.1.5 Advanced Value Control and Checking Attributes

The attributes listed in [Table 4-3](#) provide more advanced value control and checking features by setting up default parameter values, enabling or disabling user modification of parameter values, and implementing parameter value dependencies.

Table 4-3 Attributes for Parameter Value Control and Checking

Attribute	Description
Value	The current value of the parameter. Use Value to get or set the current value of the parameter. The coreBuilder tool automatically sets Value to the value set for the parameter in the HDL code.
DefaultValue	The coreBuilder tool automatically sets DefaultValue the default value specified in the HDL code. If no value has been specified, or if you want to calculate the default value as a function of other parameters, set DefaultValue to a specific value or to a Tcl parameter expression that the coreBuilder/coreConsultant tool evaluates.

Table 4-3 Attributes for Parameter Value Control and Checking (continued)

Attribute	Description
Value Required	Indicates the parameter requires a value. If ValueRequired is true, and no default value exists, the user must explicitly specify a value.
IsHexVal	Indicates that the parameter value is in hexadecimal format.
SetInHex	Indicates that the user can, must, or cannot specify the parameter value in hexadecimal format.
ReadOnlyParam	Specifies that the user cannot modify the parameter value. If Visible and ReadOnlyParam are both true, the parameter appears in the Specify Configuration dialog, but its value cannot be changed.
Enabled	Enables modification of the parameter value under specified conditions.
CheckCmd	Specifies the name of a Tcl command procedure that checks the user-specified value for the parameter. If you are creating a new Tcl command procedure to implement the checking, you must build the command procedure into a plugin by executing the Load Consultant Plugins activity.
CheckExpr	Specifies a Tcl expression to check the value of a parameter. This is typically used to set up dependencies between parameter values.
CheckExpr Message	Specifies the error message to display when a parameter cannot be set to a specified value because CheckExpr evaluates to false.
CrossParam Group	Sets up a group of parameters whose values must be mutually exclusive.
Parameter CheckCmd	Specifies a Tcl command procedure to check parameter values for a design. (ParameterCheckCmd is actually a design attribute, not a parameter attribute.) If you are creating a new Tcl command procedure to implement the checking, you must build the command procedure into a plugin by executing the Load Consultant Plugins activity.

4.1.5.1 Setting Up Simple Parameter Dependencies

The following methods exist to setting up simple parameter dependencies:

- Set CrossParamGroup to the same value on a set of parameters that must have mutually exclusive values.
- Use a formula to specify an attribute value for one parameter as a function of another. For example, a design may require that register B be at least as wide as register A.

The following example shows how to set the MinValue for b_reg_width equal to the user-specified value a_reg_width so that coreBuilder only accepts a b_reg_width value that is greater than or equal to a_reg_width:

```
-- reuse-pragma attr MinValue 0
-- reuse-pragma attr MaxValue 8
constant a_reg_width : integer := 4;
-- reuse-pragma attr MinValue {@a_reg_width}
-- reuse-pragma attr MaxValue 8
constant b_reg_width : integer := 8;
```

- Use the CheckExpr attribute to implement the required dependency. For example, you may want to accept different values for parameter B depending on the value of parameter A. CheckExpr is a case-subscripted attribute in which the subscript is a boolean expression evaluated by coreBuilder. If the subscript expression evaluates to true, then coreBuilder also evaluates the value of the attribute as a boolean expression. If expression evaluates to false, then coreBuilder returns the value “out of range.” You can use the CheckExprMessage attribute to generate a more specific error message.

If you use a subscript with CheckExpr, use the same subscript with the CheckExprMessage to generate an error message for the specific case. [Example 4-7](#) shows how to apply CheckExpr and CheckExprMessage to a design in which the range of maximum value of max_burst depends on the value of num_masters:

- If the num_masters value is four or less, the maximum value for max_burst is 1024 (as defined by MaxValue on max_burst).
- If the num_masters value is greater than 4, the maximum value for max_burst is 512.
- If the num_masters value is greater than 8, the maximum value for max_burst is 256.

Example 4-7 Checking Parameter Values with CheckExpr

```
-- reuse-pragma attr MinValue 1
-- reuse-pragma attr MaxValue 16
constant num_masters : integer := 8;

-- reuse-pragma attr CheckExpr[@num_masters>4] @max_burst<=512
-- reuse-pragma beginAttr
-- CheckExprMessage[@num_masters>4]
-- "If more than 4 masters, the max burst is 512."
-- reuse-pragma endAttr
-- reuse-pragma attr CheckExpr[@num_masters>8] @max_burst<=256
-- reuse-pragma beginAttr
-- CheckExprMessage[@num_masters>8]
-- "If more than 8 masters, the max burst is 256."
-- reuse-pragma endAttr
-- reuse-pragma attr MinValue 64
-- reuse-pragma attr MaxValue 1024
constant max_burst : integer := 512;
```

4.1.5.2 Setting Up Complex Parameter Dependencies

[Example 4-8](#) on page 58 shows how to use the attributes listed in [Table 4-3](#) on page 55 to implement more complex value control and checking for a design with the following parameter checking requirements:

- Memory 1 end address is fixed at Memory 1 start address plus 0x1FFF. Memory 1 start address is required and is checked by a custom command procedure named CheckMem1.
- The status3 parameter is enabled only if imp_func3 is true.
- Start and end addresses are required for memory spaces 2 and 3. Each end address must be greater than its respective start address.

- No memory address ranges can overlap (checking implemented by custom command procedure CheckAddresses).
- The values for parameters status1, status2, and status3 must be mutually exclusive.

[Example 4-8](#) also requires a coreConsultant plugin that creates the command procedures CheckMem1 and CheckAddresses.

Example 4-8 Setting Attributes for Parameter Value Control and Checking

```
-- reuse-pragma attr ParameterCheckCmd CheckAddresses
entity TOP is
generic (
-- reuse-pragma beginAttr Description Implements Function 3 and enables
-- Status Register 3 Address
-- reuse-pragma endAttr
-- reuse-pragma attr MinValue 0
-- reuse-pragma attr MaxValue 1
-- reuse-pragma attr Label Support Function 3
  imp_func3 : integer := 0;

-- reuse-pragma attr Description Start address of memory space 1
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xE000
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 1 Start
-- reuse-pragma attr ValueRequired true
-- reuse-pragma attr CheckCmd CheckMem1
  mem1_start : integer := 16#0000#;

-- reuse-pragma attr Description End address of memory space 1
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 1 End
-- reuse-pragma attr DefaultValue =(@mem1_start+0x1FFF)
-- reuse-pragma attr ReadOnlyParam true
  mem1_end : integer;

-- reuse-pragma attr Description Start address of memory space 2
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 2 Start
-- reuse-pragma attr ValueRequired true
  mem2_start : integer := 16#2000#;

-- reuse-pragma attr Description End address of memory space 2
```

```
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 2 End
-- reuse-pragma attr ValueRequired true
-- reuse-pragma attr CheckExpr @mem2_end>@mem2_start
-- reuse-pragma beginAttr CheckExprMessage "End address must be greater
-- than start address"
-- reuse-pragma endAttr
mem2_end : integer := 16#3FFF#;

-- reuse-pragma attr Description Start address of memory space 3
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 3 Start
-- reuse-pragma attr ValueRequired true
mem3_start : integer := 16#4000#;

-- reuse-pragma attr Description End address of memory space 3
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Memory Space 3 End
-- reuse-pragma attr ValueRequired true
-- reuse-pragma attr CheckExpr @mem3_end>@mem3_start
-- reuse-pragma beginAttr CheckExprMessage "End address must be greater
-- than start address"
-- reuse-pragma endAttr
mem3_end : integer := 16#5FFF#;

-- reuse-pragma attr Description Address of function 1 status register
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Status Register 1 Address
-- reuse-pragma attr CrossParamGroup status_regs
status1 : integer := 16#7AFF#;

-- reuse-pragma attr Description Address of function 2 status register
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Status Register 2 Address
```

```
-- reuse-pragma attr CrossParamGroup status_regs
   status2 : integer := 16#8AFF#;

-- reuse-pragma attr Description Address of function 3 status register
-- reuse-pragma attr MinValue 0x0000
-- reuse-pragma attr MaxValue 0xFFFF
-- reuse-pragma attr IsHexVal true
-- reuse-pragma attr SetInHex Required
-- reuse-pragma attr Label Status Register 3 Address
-- reuse-pragma attr Enabled @imp_func3==1
-- reuse-pragma attr CrossParamGroup status_regs
   status3 : integer := 16#9AFF# );
```

4.1.6 Advanced Dialog Control Attributes

The advanced dialog control attributes in [Table 4-4](#) control the appearance, grouping, and sequence of your design parameters in the coreConsultant/coreAssembler Specify Configuration dialog.

Parameter groups are useful for designs with large numbers of parameters that can be in logical groups.

Table 4-4 Attributes for Advanced Dialog Control

Attribute	Short Description
GroupName	Assigns a parameter to a parameter group. For a single level of groups, the coreConsultant tool places each parameter group on a separate tab. For multiple levels, the coreConsultant tool creates a hierarchy of pages.
Sequence	Specifies the order in which parameters appear in the Specify Configuration dialog, or within a parameter group.
Visible	Determines whether the parameter appears in the coreBuilder Design Configuration Intent dialog and coreConsultant Specify Configuration dialog. You can use parameter expressions to display the parameter under specified conditions. Visible is true by default on design parameters.
HelpUrl	Specifies the URL of the help text for the parameter, either a local URL (typically a file you supply with the coreKit) or a file on another web site.
ParameterInfo	Provides enhanced control over parameter grouping and layout in the Specify Configuration dialog and is particularly useful for designs with large numbers of parameters. ParameterInfo is not a parameter attribute. Instead, set ParameterInfo on the design. For detailed information about how to use ParameterInfo, see “Advanced Topics” .
ResolveType	Specify the value of existing “Resolve Type” attribute on parameters. This corresponds to the spirit:resolve attribute in the IP-XACT parameter element.

[Example 4-9](#) shows the HDL annotations that group design parameters into three groups: Optional Features, I-O Address Ranges, and Memory Address Ranges. You can set the ParameterInfo attribute on the design to control how the parameter groups are arranged in the coreConsultant Specify Configuration dialog:

You can set ParameterInfo on the design with the following annotation:

```
`include parameters.inc
```

```
// reuse-pragma attr ParameterInfo {ContainerType tree}
module TOP
(
    ...

```

Example 4-9 Setting Attributes for Single-Level Parameter Grouping

```
// reuse-pragma beginAttr Description Implements the power management
// feature, power management controlled by host device.
// reuse-pragma endAttr
// reuse-pragma attr GroupName Optional Features
// reuse-pragma attr Sequence 5
// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
// reuse-pragma attr SymbolicNames false true
// reuse-pragma attr Label Support Power Management
`define pwr_mgmt 0

// reuse-pragma attr Description Implements the optional 16-bit timer.
// reuse-pragma attr SymbolicNames false true
// reuse-pragma attr GroupName Optional Features
// reuse-pragma attr Sequence 6
// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
// reuse-pragma attr SymbolicNames false true
// reuse-pragma attr Label Secondary Timer
`define timer2 0

// reuse-pragma attr Description Builds interface to RAM of selected size.
// reuse-pragma attr GroupName Optional Features
// reuse-pragma attr Sequence 2
// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
// reuse-pragma attr SymbolicNames {128 Bytes} {256 Bytes}
// reuse-pragma attr Label RAM size
`define ram_256 0

// reuse-pragma attr Description Start address of memory space 1
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label Memory Space 1 Start
// reuse-pragma attr GroupName Memory Address Ranges
// reuse-pragma attr Sequence 1
`define mem1_start 16'h0000

// reuse-pragma attr Description End address of memory space 1
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label Memory Space 1 End
// reuse-pragma attr GroupName Memory Address Ranges

```

```
// reuse-pragma attr Sequence 2
`define mem1_end 16'h0FFF

// reuse-pragma attr Description Start address of memory space 2
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label Memory Space 2 Start
// reuse-pragma attr GroupName Memory Address Ranges
// reuse-pragma attr Sequence 3
`define mem2_start 16'h1000

// reuse-pragma attr Description End address of memory space 2
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label Memory Space 2 End
// reuse-pragma attr GroupName Memory Address Ranges
// reuse-pragma attr Sequence 4
`define mem2_end 16'h1FFF

// reuse-pragma attr Description Start address of I-0 space 1
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label I-0 Space 1 Start
// reuse-pragma attr GroupName I-0 Address Ranges
// reuse-pragma attr Sequence 1
`define io1_start 16'h3000

// reuse-pragma attr Description End address of I-0 space 1
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label I-0 Space 1 End
// reuse-pragma attr GroupName I-0 Address Ranges
// reuse-pragma attr Sequence 2
`define io1_end 16'h30FF

// reuse-pragma attr Description Start address of I-0 space 2
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label I-0 Space 2 Start
// reuse-pragma attr GroupName I-0 Address Ranges
// reuse-pragma attr Sequence 3
`define io2_start 16'h3100

// reuse-pragma attr Description End address of I-0 space 2
// reuse-pragma attr IsHexVal true
// reuse-pragma attr SetInHex Required
// reuse-pragma attr Label I-0 Space 2 End
// reuse-pragma attr GroupName I-0 Address Ranges
```

```
// reuse-pragma attr Sequence 4
`define io2_end 16'h31FF
```

4.1.7 Attributes for Bitfield and Array Parameters

The coreBuilder tool automatically uses a special parameter type (the bitfield parameter) to model the following types of HDL parameters:

- VHDL parameters of type `bit_vector`, `std_logic_vector`, `std_ulogic_vector`, `signed`, or `unsigned`.
- Verilog parameters that have a bitwidth designator, for example:

```
`define preg_addr 8'b11110000
```

Array parameters are bitfield parameters made up of smaller elements (fields). For example, a design may have a parameter named `trap_table` that concatenates a set of individual trap addresses. In such a case, you want the core integrator to select values for the individual trap addresses, rather than specify a single value for the entire `trap_table` parameter.

[Example 4-10](#) shows how the `trap_table` parameter might be coded in a VHDL design, in which parameter fields are coded as arrays. The coreBuilder tool automatically creates the required array field parameters and assigns default attribute values for array field parameters.

[Example 4-11](#) illustrates how the `trap_table` parameter might be coded in a Verilog design, where you must explicitly identify the array parameter as such by setting array parameter attributes (because Verilog does not provide built-in support for defining a parameter as an array).

Example 4-10 Array Parameter Definition in VHDL

```
subtype table_entry is std_logic_vector (7 downto 0);
type table_type is array (2 downto 0) of table_entry;

-- reuse-pragma beginAttr Description Trap table built from a set of three
-- 8-bit fields, where each field represents a jump address for a trap
-- reuse-pragma endAttr
constant trap_table : table_type := (x"1a", x"2b", x"3c");
```

Example 4-11 Array Parameter Definition in Verilog

```
// reuse-pragma beginAttr Description Trap table built from a set of three
// 8-bit fields, where each field represents a jump address for a trap
// reuse-pragma endAttr
// reuse-pragma attr ArrayFieldSize 8
// reuse-pragma attr ArrayEnd 2
// reuse-pragma attr ArrayStart 0
`define trap_table {8'h1a, 8'h2b, 8'h3c}
```

Table 4-5 lists the valid attributes for array-type parameters.

Table 4-5 Attributes for Array Parameters

Attribute	Short Description
IsArray	Indicates that the parameter is an array. The coreBuilder tool automatically sets IsArray to true on VHDL array parameters and Verilog parameters that have array parameter attributes set on them.
ArrayStart	Index of the first array element.
ArrayEnd	Index of the last array element.
ArrayFieldSize	Number of bits of each element in the array.

When the coreBuilder tool creates an array field parameter, it automatically sets values for the Description, HelpUrl, MinValue, and MaxValue attributes on the field parameters. You can use the `foreach_array_field_parameter` command to iterate through the set of field parameters to set additional attributes or override the values for the default attributes.

Example 4-12 sets the Label attribute on each array parameter created for the `trap_table` example.

Example 4-12 Intent Command File for Setting Array Parameter Attributes

```
foreach_array_field_parameter param idx trap_table {
    set_parameter_attribute $param Label "Table value $idx"
}
```

You do not have to explicitly create array parameters because the coreBuilder tool automatically creates them for HDL array parameters as described above.If you are creating an array parameter for a purpose other than HDL design configuration (for example, for a TRE), then you need to use the `define_array_field_parameters` command. You can then use the `foreach_array_field_parameter` command to set attributes on the individual field parameters. For more information, see the `define_array_field_parameters` man page.

4.2 Support for Verilog Compiler Directives

In coreBuilder, parameters are dynamic. That is, when you change the value of any parameter, any expressions that reference that parameter automatically update their values immediately.

In Verilog, the ``define` statement defines a macro that is evaluated statically every time it is seen. This is mandated by the Verilog standard. In fact, most Verilog parsers have two phases of analysis. The first phase streams the files and evaluates all preprocessor directives. The next phase actually parses the code and makes sure that it is syntactically correct.

This means that there is an inherent mismatch between these two mechanisms. We are using the static Verilog macros in a dynamic way within the coreBuilder tool. This necessitates breaking strict compatibility with the Verilog standard when you make a macro into a parameter. You make a macro into a parameter by placing one or more attributes on it with either the `add_hdl_pragma` command, or with a `reuse-pragma` annotation. (Note that if the macro stays a macro, we follow the standard completely, which means no reconfigurability).

To help get around this mismatch, the coreBuilder tool has two modes for dealing with conditional compiler directives when they refer to parameters. The two modes are called “all_branches” and “standard.” The default at the start of every file is “all_branches.” In this mode, the coreBuilder tool analyzes all branches of the conditional section of code. In certain situations, this can cause syntax errors. If all_branches causes an error during analysis, then the mode can be changed to “standard.” In standard mode the branches of the conditional statement are handled like a traditional ``ifdef`. This means that coreBuilder will only see the branch that corresponds to the `DefaultValue` of the parameter.

You can change the preprocessor mode in either of two ways:

- Use the command `'add_hdl_pragma -process_ifdef <mode> line_selector'`
- Use one of the `reuse-pragma` annotations in the source file
 - `// reuse-pragma process_ifdef standard, or`
 - `// reuse-pragma process_ifdef all_branches`

The mode selected when an ``ifdef` (or ``ifndef`) statement is encountered applies to matching ``elsif`, ``else`, and ``endif` statements. It is possible to have nested ``ifdef` statements with different preprocessing modes.

4.3 Conditional Inclusion of Cells and Ports

In addition to standard configuration through parameters supported by VHDL and Verilog, coreBuilder supports the conditional inclusion of both cells and ports within VHDL or Verilog designs. VHDL-based designs sometimes use the generate statement to conditionally include cells and other functionality.

Verilog 2001 now supports a generate statement, but also supports the concept of a pre-processor, which conditionally includes and excludes arbitrary sections of code.

To synthesize a design properly, the coreConsultant tool must have a model that accurately represents the design hierarchy. This means that the coreBuilder and coreConsultant tools must understand what is being conditionally included in the design. The coreBuilder tool currently supports the conditional inclusion of cell instantiations and port definitions.

For designs that have conditionally generated cells, the coreBuilder tool automatically implements the conditional inclusion/exclusion of cells because coreBuilder interprets VHDL and Verilog generate statements. However, extra work is necessary to enable conditional inclusion for ports in VHDL and Verilog, and for conditional cells in Verilog if they are defined using pre-processor constructs such as ``ifdef`.

4.3.1 Conditional Cells

You can tell the coreBuilder tool about a conditional cell in Verilog in the following ways.

- Use the new 'generate' statement

If you use this method, no extra action is required, as the coreBuilder tool understands the conditionality by interpreting the generate statement expressions.

- Use pre-processor statements that refer to user-modifiable parameters

A user-modifiable parameter is a ``define` which contains at least one reuse-pragma annotation. If you use this method, no extra action is required. [Example 4-13](#) demonstrates this method.

- Use pre-processor statements which refer to constants that are user-modifiable, but which always have a value.

In this case, the core developer must explicitly indicate the conditionality expression for the coreBuilder tool by setting the `GenerateIf` attribute on the cell and by removing the ``ifdef` around the cell instantiation. Strictly speaking, the ``ifdef` statements do not need to be removed, but if they are left in the code, they must evaluate to true by default, or the coreBuilder tool will not see the cell instance when it analyzes the incoming RTL source code. [Example 4-14](#) demonstrates this method. When `include_FPU` is not 1, the entire cell instantiation is automatically removed when the configured code is written to disk by the coreConsultant or coreAssembler tool.

Example 4-13 Conditional Cells with Modifiable Parameters

```
// reuse-pragma attr Label Include FPU

// reuse-pragma attr DefaultValue false
`define include_FPU

module top(input in1, output out1);
...
`ifdef include_FPU
fpu i_FPU(...);
```

```
`endif
...

endmodule
```

Example 4-14 Conditional Cells with Constants

```
// reuse-pragma attr Label Include FPU

// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
`define include_FPU 1

module top(input in1, output out1);
    ...
    // reuse-pragma attr GenerateIf @include_FPU==1
    fpu i_FPU(...);
    ...
endmodule
```

4.3.2 Conditional Ports

To implement conditional ports in VHDL, you must apply the GenerateIf attribute to the affected ports, as shown in [Example 4-15](#). The declaration of input 'fpu_data' will automatically be removed from the configured code if include_FPU is not set to 1.

There are two ways to implement conditional ports in Verilog:

- Use pre-processor statements which refer to user-modifiable parameters. A user-modifiable parameter is a `define which contains at least one reuse-pragma annotation. If you use this method, no extra action is required. [Conditional Ports with Modifiable Parameters](#) on page 68 demonstrates this method.
- Use pre-processor statements that refer to constants that are not user-modifiable. Here, the core developer must provide the conditionality expression for the coreBuilder tool by setting the GenerateIf attribute on the port and removing the `ifdef. The removal of `ifdef statements is not required but, if they remain, they must evaluate to true by default or the coreBuilder tool will not see the port declaration when it analyzes the incoming RTL. [Conditional Ports with Constants](#) on page 68 demonstrates this. In this case, when include_FPU is 1, the fpu_data port is removed when the tool saves the configured code.

Example 4-15 Conditional Ports in VHDL

```
entity TOP is
    port (clk : in std_logic;
          -- reuse-pragma attr GenerateIf @include_FPU==1
          fpu_data : in std_logic_vector(15 downto 0);
          alu_data : in std_logic_vector(15 downto 0)
          ...);

end TOP;
```

Example 4-16 Conditional Ports with Modifiable Parameters

```
// reuse-pragma attr Label Include FPU

// reuse-pragma attr DefaultValue false
`define include_FPU

module top(input clk,
`ifdef include_FPU
    input fpu_data,
`endif
    input alu_data,
    ...);
...
endmodule
```

Example 4-17 Conditional Ports with Constants

```
// reuse-pragma attr Label Include FPU

// reuse-pragma attr MinValue 0
// reuse-pragma attr MaxValue 1
`define include_FPU 1

module top(input clk,
    // reuse-pragma attr GenerateIf @include_FPU==1
    input fpu_data,
    input alu_data,
    ...);
...
endmodule
```

4.3.3 Filtering Objects for Conditional Inclusion

For cores that have many conditional ports, you can use a filter expression to apply the GenerateIf attribute to certain ports (for example, ports with names that match a specified name string). [Example 4-18](#) shows a VHDL example that applies different GenerateIf values to ports that end with `_1`, `_2`, or `_3`.

Example 4-18 Applying GenerateIf to a Filtered Set of Ports

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
-----ENTITY
DECLARATION
entity TOP is
    generic (
-- reuse-pragma attr MinValue 1
-- reuse-pragma attr MaxValue 4
        num_masters : integer := 1);
    port (
```

```
-- reuse-pragma attr if(Name =~.*_1) GenerateIf @num_masters>1
-- reuse-pragma attr if(Name =~.*_2) GenerateIf @num_masters>2
-- reuse-pragma attr if(Name =~.*_3) GenerateIf @num_masters>3
    clk                : in  std_logic;
    addr_0              : in  std_logic_vector (15 downto 0);
    data_0              : in  std_logic_vector (15 downto 0);
    addr_1              : in  std_logic_vector (15 downto 0);
    data_1              : in  std_logic_vector (15 downto 0);
    addr_2              : in  std_logic_vector (15 downto 0);
    data_2              : in  std_logic_vector (15 downto 0);
    addr_3              : in  std_logic_vector (15 downto 0);
    data_3              : in  std_logic_vector (15 downto 0);
    rst_n               : in  std_logic
);
end TOP;
```

If you build a coreKit with the design in [Example 4-18](#), the coreConsultant tool generates configured code for the core as follows:

- If the core integrator sets num_masters to 1, ports clk, addr_0, data_0, and rst_n exist.
- If the core integrator sets num_masters to 2, ports clk, addr_0, data_0, addr_1, data_1, and rst_n exist.
- If the core integrator sets num_masters to 3, ports clk, addr_0, data_0, addr_1, data_1, addr_2, data_2, and rst_n exist.

4.4 HDL Coding Guidelines

The coreBuilder tool does not enforce coding style guidelines. It is up to the core developer to follow good design-for-reuse coding practices. For detailed information and guidelines about design-for-reuse coding, Synopsys recommends the *Reuse Methodology Manual*, 3rd Edition, Kluwer Academic Publishers, 2002, by Michael Keating and Pierre Bricaud.

The coreBuilder tool does impose some HDL coding restrictions. The following sections describe those restrictions and how to deal with them.

4.4.1 HDL Syntax Restrictions

Relative to the Design Compiler (DC) synthesis subset, the coreBuilder tool imposes the following additional restrictions on the set of HDL constructs that can occur in the source code that the coreBuilder tool analyzes to determine the structure of your design (entity, parameter, port, pin, and instance declarations):

- Provides only limited support for function calls. You must supply the function implementation to coreBuilder by setting the FunctionDefinition attribute on the function (see the coreBuilder online reference for details). This applies only to functions used to initialize constraints or port widths.
- Does not support VHDL record types.
- Does not support multiple VHDL architectures. Design Compiler also does not support multiple VHDL architectures.
- Does not support the use of the VHDL others construct to initialize user-modifiable constants.
- Does not support more than one for-generate statement in a VHDL architecture for a design that is on or above the implementation frontier.

4.4.2 Working Around HDL Syntax Restrictions

The coreBuilder tool only needs to analyze those HDL code blocks that describe the structure of the subblocks that are designated for compilation as individual units. (In other words, the coreBuilder tool only needs to analyze designs on or above the implementation frontier). So, in most cases, the unsupported HDL constructs described above occur in sections of source code that the coreBuilder tool does not need to analyze. Therefore, if you encounter a coreBuilder HDL parser error due to an unsupported HDL construct, you can, in most cases, disable analysis of the code containing the unsupported construct.

The Load Designs activity works in two steps:

1. Analyzes all HDL code
2. Builds a structural data model

To prevent parsing of selected sections of code, the coreBuilder tool uses a `translate_off/translate_on` pragma pair, similar to the pair used by Design Compiler. The coreBuilder tool does not analyze the HDL code between the `reuse-pragma translate_off` and `reuse-pragma translate_on`. Note that while coreBuilder does ignore this code, it must be syntactically and semantically correct because the code is still read by the front-end HDL analyzer.

For VHDL, the coreBuilder `translate_off/translate_on` syntax is:

```
-- reuse-pragma startSub [IncludeIF 0 %subText]
-- synopsys translate_off coreBuilder does not analyze this code
-- reuse-pragma startSub [IncludeIF 0 %subText]
```

```
-- synopsys translate_on
```

For Verilog, the coreBuilder `translate_off/translate_on` syntax is:

```
// reuse-pragma startSub [IncludeIF 0 %subText]
// synopsys translate_off coreBuilder does not analyze this code
// reuse-pragma startSub [IncludeIF 0 %subText]
// synopsys translate_on
```

Only coreTools applications recognize and interpret `reuse-pragma`. Although embedded within your HDL code, these pragmas do not affect the analysis performed by Design Compiler (or simulation tools). The coreTools applications also honor Design Compiler `translate_off/translate_on` blocks.

[Example 4-19](#) illustrates how to apply `reuse-pragma translate_off` and `reuse-pragma translate_on` to disable analysis of one architecture in a multiple-architecture VHDL design. In this case, the coreBuilder tool ignores the second architecture and the configurations.

Example 4-19 Disabling Parsing for an Unsupported HDL Construct

```
entity TOP is
  generic/port lists

end TOP;
architecture fast of TOP is
begin
  RTL for fast
end fast;
-- Use only fast architecture for this core
-- reuse-pragma translate_off
architecture small of TOP is
begin
  RTL for small
end small;
-- reuse-pragma translate_on
--pragma translate_off
configuration TOP_cfg_fast of TOP is
  for fast
  end for;
end TOP_cfg_fast;
configuration TOP_cfg_small of TOP is
  for small
  end for;
end TOP_cfg_small;
--pragma translate_on
```


5

Writing Intent Command Files

Creating and refining synthesis intent is one of the main processes involved in building a coreKit. As a core developer, your goal should be to capture your synthesis intent in intent command files for portability to future projects. This chapter describes how to write intent command files for coreBuilder.

This chapter contains the following sections:

- [“Overview”](#) on page 74
- [“Synthesis Intent Objectives”](#) on page 75
- [“Naming Intent Command Files”](#) on page 76
- [“Intent Command File Syntax”](#) on page 77
- [“Creating Clocks”](#) on page 80
- [“Specifying Clock Intent”](#) on page 82
- [“Specifying Synthesis Methodology”](#) on page 83
- [“Specifying Port Constraints”](#) on page 88
- [“Intent Command File Example”](#) on page 91
- [“Post-Elaboration Intent”](#) on page 92
- [“Importing Synthesis Intent”](#) on page 93

5.1 Overview

Intent command files are the primary mechanism to specify synthesis-related intent for your design. When you load your design into the coreBuilder tool, it automatically executes your intent command files and applies the specified synthesis-related attributes to the specified design objects.

5.1.1 Synthesis Intent Development

To specify synthesis intent in the coreBuilder tool, perform the following steps:

- Write intent command files for the coreBuilder tool as described in this chapter. You must create coreConsultant workspaces in which to synthesize the core to test your synthesis intent.
- Specify synthesis-related intent interactively through the coreBuilder GUI by executing the Synthesis Intent activities. You can specify your synthesis intent through the GUI; however, it is generally better to use intent command files for synthesis intent because of the following:
 - The coreBuilder tool automatically re-applies your synthesis intent from intent command files if you make a design change and repeat the Load Designs activity. By contrast, all synthesis intent you specify interactively is lost when you repeat the Load Designs activity.
 - Your synthesis intent is directly portable to future projects because it is independent from your current workspace and coreKit.

If you prefer to get started by specifying synthesis intent in GUI mode, you can defer writing your intent command files by loading your design into the coreBuilder tool and specifying synthesis intent through the GUI. You can then automatically generate an intent command file from your interactively specified intent. For details, see [“Importing Intent Specified in GUI Mode”](#) on page 93.

5.2 Synthesis Intent Objectives

You need to specify synthesis intent to:

1. Create clock objects for each of the design clocks; for more, see [“Creating Clocks”](#) on page 80.
2. Set attributes on clocks to describe the default cycle time and waveform; for more, see [“Specifying Clock Intent”](#) on page 82.
3. Set attributes on designs to describe design characteristics and control the synthesis process; for more, see [“Specifying Synthesis Methodology”](#) on page 83.
4. Set attributes on ports to control generation of port constraints; for more, see [“Specifying Port Constraints”](#) on page 88.

You do not need to write complete intent command files before loading your design into the coreBuilder tool for the first time. In fact, you generally need to experiment with synthesis and edit your intent command files to refine your synthesis intent specifications.

5.3 Naming Intent Command Files

Use the same base filename for the intent command files that you use for the corresponding HDL files. For example, if the HDL source file for the top-level design is `top.vhd`, then use `top.tcl` for the associated intent command file.

When you specify `top.vhd` as a design file, the coreBuilder tool automatically loads and processes `top.tcl` (if such a file exists in the Tcl intent search path). If there is also an SDC file named `top.sdc` in your Tcl search path, the coreBuilder tool automatically executes the commands in the SDC file when you execute the Load Designs activity. For more information about reading and writing SDC files, see [“Importing Synopsys Design Constraints”](#) on page 93.

5.4 Intent Command File Syntax

Intent command files are scripts that specify your synthesis intent using coreBuilder Tcl commands. To write intent command files, you need to understand coreBuilder intent command syntax, including:

- [“Commands”](#)
- [“Intent Command File for Simple Design”](#)
- [“Specifying Attribute Values as Formulas”](#)

5.4.1 Commands

Table 5-1 lists the commands most often used to set attributes on designs, clocks, ports, and cells. For complete information about individual commands and attributes, see the coreBuilder online reference.

Table 5-1 Basic Commands for Setting Synthesis Intent

Command	Description
set_design_attribute	Set an attribute on one or more designs.
set_port_attribute	Set an attribute on one or more ports.
set_clock_attribute	Set an attribute on one or more clocks.
create_virtual_clock	Create a virtual clock.
current_design	Set or get the current design.
all_designs	Create a collection of designs.
all_subdesigns	Create a collection of subdesigns within the current design.
all_inputs	Create a collection of input ports.
all_outputs	Create a collection of output ports.

5.4.2 Subscripted Attributes

Several of the attributes use subscripts. For example, you specify the MinInputDelay attribute as a value (or formula) with respect to a particular clock. In this case, the name of the clock is the subscript to the MinInputDelay attribute. The syntax is:

```
{AttributeName[subscript]} value
```

To specify the minimum input delay for input port data_in relative to clk, use the following syntax:

```
set_port_attribute data_in {MinInputDelay[clk]} \
    {=percent_of_period 15.0 clk}
```

In this example:

- The [clk] subscript specifies that the minimum input delay value is relative to the clock named clk. (For clock attributes, you can omit the subscript if there is only one clock in the design.)
- The {=percent_of_period 15.0 clk} formula specifies the minimum input delay value as 15% of the clk cycle time.

**Note**

You must use braces ({ }) in the subscript attribute specification because Tcl uses brackets ([]) to specify nested commands.

5.4.3 Specifying Attribute Values as Formulas

For many attributes, you can specify the value as a formula instead of a value. Formulas (which begin with the equal (=) character), provide the following benefits:

- Your intent specifications remain independent of specific technologies. (When the coreConsultant tool generates synthesis commands, it evaluates the formula and translates the result into appropriate technology-specific units).
- Formulas enable you to specify an attribute value as a function of another attribute. For example, by specifying delay values as a function of the clock cycle time, a change to the CycleTime attribute automatically propagates to the delay attributes.

The coreBuilder tool provides built-in formulas to specify values for several attributes, as described in the following sections.

5.4.3.1 Built-in Formula for Delay Attributes

Use the following formula for delay attributes, such as MinInputDelay, to specify the value as a percentage of the CycleTime (period) of a specified clock:

```
{=percent_of_period percent [clock_name]}
```

- If the design has only one clock, you can omit *clock_name*.
- *percent* represents the percent of the clock cycle time and must be a positive value of type float.

The following example sets the value of MinInputDelay on port in1 to 25% of the period of the only clock in the design:

```
set_port_attribute in1 MinInputDelay {=percent_of_period 25}
```

5.4.3.2 Built-in Formulas for Library Cell Selection Attributes

The coreBuilder tool provides the following formulas for library cell selection attributes, such as DrivingCell:

- {=select_by_class <class> [low | median | high]}

Selects a representative library cell of the specified drive strength for the specified class of cells (combinational, sequential, or tristate). Drive strength is optional and defaults to median if not specified.

For example, to select the median drive strength combinational cell:

```
set_port_attribute in1 DrivingCell {=select_by_class "combinational"}
```

- {=select_by_function <function> [low | median | high]}

Selects a representative library cell of the specified drive strength for the specified cell function (nand2, mux, xor, buf, inv, ff, latch, or tristate). Drive strength is optional and defaults to median if not specified.

For example, to select a high drive strength nand2 cell:

```
set_port_attribute in1 DrivingCell \
```

```
{=select_by_function "nand2" "high"}
```

■ `{=select_by_name <library_cell_name> [<pin_name>]}`

Selects a library cell by name and, optionally, pin name. This is useful if you know that your core is always targeting a set of libraries with a well-defined naming convention. For example to select library cell `ssc_core_slow/or2c15`:

```
set_port_attribute in1 DrivingCell \
    {=select_by_name "ssc_core_slow/or2c15"}
```

5.4.3.3 Built-in Formula for Capacitance Specification

Use the following formula to specify an explicit value for capacitance:

```
{=explicit_capacitance <cap_val>}
```

For example:

```
set_port_attribute myOutPort PinLoadType {=explicit_capacitance 10}
```

The default capacitance unit is the unit used by the currently loaded technology library.

Use the following formula to set the DrivingCell attribute on a clock's input port to infinite drive:

```
{=infinite_drive}
```

For example:

```
set_port_attribute myInPort DrivingCell {=infinite_drive}
```

5.4.3.4 Useful Building Blocks for Formulas

The following items, called building blocks, are useful when working with formulas or other coreBuilder Tcl commands:

- `%item` – The item on which this attribute is set.
- `%attr` – The attribute on which this formula is set.
- `%attrVal` – The current value of the attribute on which this formula is set.
- `%subs` – The attribute subscript currently being used.

5.4.4 Default Attribute Values

Most attributes have default values, either explicit or inherited. For example, the default value for the PreserveHierarchy attribute is false (explicit).

Inherited default values involve a formula mechanism that calculates the default value for the attribute from the attribute values on designs either above or below the current design in the design hierarchy. For example, the default AreaEstimate value for a design is the sum of the AreaEstimate values of its subblocks.

The online man page for each coreBuilder attribute lists that attribute's default value, either as an explicit value or as one of the following formulas:

- `=InheritValue options`
- `=CombineInheritValues options`

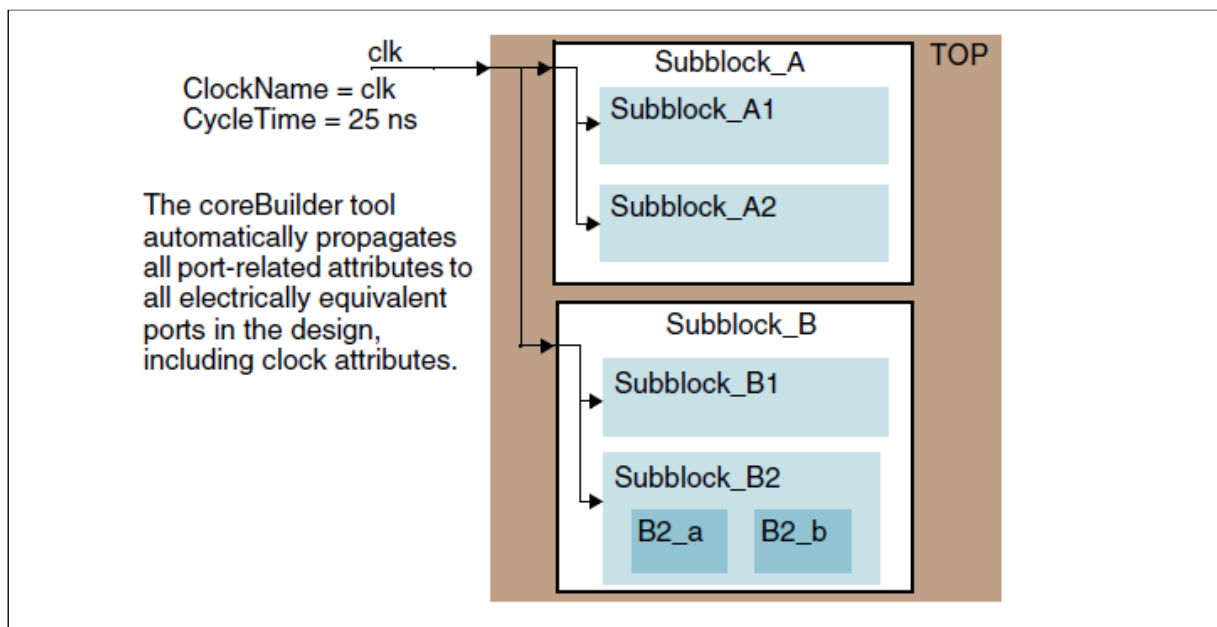
For details, see the man pages for InheritValue and CombineInheritValues.

5.5 Creating Clocks

The coreBuilder tool supports real clocks, virtual clocks, and generated clocks. If every module in your design is driven by the same clock, which is a top-level input port, you only need to create and specify attributes for the top-level clock because the coreBuilder tool automatically propagates a clock port's attributes to all electrically equivalent ports in the design hierarchy. Under special conditions, however, you may need to create additional clocks. For example:

- An additional clock (such as a derived clock) exists in the design and you want to specify internal design port-timing attributes relative to that clock.
- You want to create a virtual clock for internal port-timing attributes. For example, a lower-level combinational design may require individual compilation, and you want to specify the port-timing attributes relative to a virtual clock.

Figure 5-1 Automatic Propagation of Port-Related Intent



5.5.1 Creating a Real Clock

To create a real clock, set the ClockName attribute on the clock port, as in the following example:

```
current_design TOP
set_port_attribute clk ClockName "clk"
```

Setting the ClockName attribute on a port creates a clock object with the specified name and associates the clock object with the specified port. The example above creates a real clock named clk and associates it with the clk port.

The ClockName attribute can be set on a bit of a bus or on a single port, but not on a port that contains multiple bits. The Add Clocks dialog lists individual bits of a port as potential clock sources.

**Attention**

It is not possible to represent clocks on bits of a bus in IP-XACT versions P1685-2009 and earlier. Bit-level clock definitions, and timing constraints associated with bit-level clocks are not included in the generated IP-XACT component file if writing a schema version older than P1685-2014.

5.5.2 Creating a Virtual Clock

To create a virtual clock that inherits the clock attributes from an existing clock, use the `create_virtual_clock` command, as shown in the following example:

```
#Create a virtual clock in combo_block, inherit attributes
#from the real clock named clk.
current_design combo_block
create_virtual_clock clk
```

To create a virtual clock and assign clock attributes to it, use both the `create_virtual_clock` and `set_clock_attribute` commands. For example:

```
#Create a virtual clock in Subblock_A1
current_design Subblock_A1
create_virtual_clock vclk
set_clock_attribute vclk CycleTime 10ns
```

5.5.3 Creating a Generated Clock

To create a generated clock object as a frequency divider or frequency multiplier, use the `create_generated_clock` command as shown in the following example:

```
#Create a generated frequency-divided clock on port genClkPort
create_generated_clock -name genClk -divide_by 2 \
-sourceclkPort genClkPort
```

To create a generated clock whose edges are 1, 3, and 5 of the master clock, use the “edges” parameter, as shown in the following example:

```
create_generated_clock -edges (1 3 5) -source clkPort \
-name genClk genClkPort
```

5.6 Specifying Clock Intent

To set attributes on a clock, use the `set_clock_attribute` command. As a core developer, you only need to set the `CycleTime` attribute, as shown in the following example:

```
#Design TOP has single input clock with default cycle time of 25ns.
```

```
current_design TOP
```

```
set_port_attribute clk ClockName "clk"
```

```
set_clock_attribute clk CycleTime 25ns
```

The core integrator then overrides the default `CycleTime` with a context-specific value, and specify values for additional context-specific attributes. For more information about clock attributes, see the [coreBuilder online reference](#).

5.7 Specifying Synthesis Methodology

Synthesis methodology attributes fall into the general categories listed in [Table 5-2](#). To set design attributes in intent command files, follow the examples listed in [Table 5-2](#). Like other synthesis intent attributes, you do not have to completely specify values for all design attributes before synthesizing the core. For descriptions of the individual design attributes and guidelines on how to set them, see the coreBuilder online reference.

Table 5-2 Synthesis Methodology Attribute Groups

Group Name	Description	Intent Command File Example
Strategy Control	Determine the design's implementation frontier (which subdesigns are to be compiled individually) and estimated area.	Example 5-1
Design Description	Describe characteristics of designs that determine what optimization algorithms to apply.	Example 5-2
Synthesis Directives	Control the generation of synthesis directives.	Example 5-3
Synthesis Controls	Provide direct control to the synthesis tool.	Example 5-4

Example 5-1 Setting Strategy Control Attributes

```
#TOP.tcl - synthesis intent for TOP

#Set up the clock
set_port_attribute clk ClockName "clk"
set_clock_attribute clk CycleTime 25ns

#TOP design attributes
current_design TOP
set_design_attribute AreaEstimate 9000
set_design_attribute MapSubblocksIndividually
#Subblock_A design attributes
current_design Subblock_A set_design_attribute MapSubblocksIndividually
#Subblock_B design attributes
current_design Subblock_B set_design_attribute MapSubblocksIndividually
#No attributes for B2_a and B2_b (below implementation frontier)
```

Example 5-2 Setting Design Description Attributes

```
#TOP design attributes

current_design TOP
set_design_attribute AreaEstimate 9000
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_A design attributes
```

```
current_design Subblock_A
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural
```

```
#Subblock_A1 design attributes
current_design Subblock_A1
set_design_attribute HasArithmetic
```

```
#Subblock_B design attributes
current_design Subblock_B
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural
```

```
#Subblock_B1 design attributes
current_design Subblock_B1
set_design_attribute HasSequential
```

```
#No attributes for B2_a and B2_b (below implementation frontier)
```

In [Example 5-3](#), the optimization priority for the top-level design is `timing_area` (meet timing then try to reduce area). However, for B1, which is known to have critical timing, the optimization priority is `timing`. The coreBuilder tool automatically propagates the `CanFlatten` attribute on B2 down to B2_a and B2_b.

Example 5-3 Setting Synthesis Directives Attributes

```
#TOP design attributes

current_design TOP
set_design_attribute AreaEstimate 9000
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural
set_design_attribute OptimizationPriorities timing_area

#Subblock_A design attributes
current_design Subblock_A
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_A1 design attributes
current_design Subblock_A1
set_design_attribute HasArithmetic

#Subblock_A2 design attributes
current_design Subblock_A1
#Subblock_A2 has regular structure that should be preserved.
set_design_attribute PreserveStructure

#Subblock_B design attributes
current_design Subblock_B
```

```
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_B1 design attributes
current_design Subblock_B1
set_design_attribute HasSequential
#Subblock_B1 has critical timing.
set_design_attribute OptimizationPriorities timing

#Subblock_B2 design attributes
current_design Subblock_B2
#Subblock_B2 will benefit from flattening
set_design_attribute CanFlatten

#No attributes for B2_a and B2_b (below implementation frontier)
```

Example 5-4 Setting Synthesis Controls Attributes

```
#TOP design attributes

current_design TOP
set_design_attribute AreaEstimate 9000
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural
set_design_attribute OptimizationPriorities timing_area

#Subblock_A design attributes
current_design Subblock_A
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_A1 design attributes
current_design Subblock_A1
set_design_attribute HasArithmetic
#before the initial compile
set_design_attribute {TclAuxSynthesisScript[initial]} A1_script.scr

#Subblock_A2 design attributes
current_design Subblock_A1
#Subblock_A2 has regular structure that should be preserved.
set_design_attribute PreserveStructure

#Subblock_B design attributes
current_design Subblock_B
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_B1 design attributes
current_design Subblock_B1
set_design_attribute HasSequential
```

```
#Subblock_B1 has critical timing.
set_design_attribute OptimizationPriorities timing
set_design_attribute {CompileMapEffort[initial]} high

#Subblock_B2 design attributes
current_design Subblock_B2
#Subblock_B2 will benefit from flattening
set_design_attribute CanFlatten

#No attributes for B2_a and B2_b (below implementation frontier)
```

5.7.1 Advanced Synthesis Attributes

These are more advanced synthesis attributes that do not generally require modification. The most common circumstances for modifying these attributes is to add to an auxiliary script. [Table 5-3](#) describes these attributes.

Table 5-3 Advanced Synthesis Attributes

Attribute	Description
TclAuxSynthesis Script[setup]	Executes the Design Compiler Tcl-mode script before any other processing
TclAuxSynthesis Script[elab]	Executes the Design Compiler Tcl-mode script before elaboration
TclAuxSynthesis Script[constrain]	Executes the Design Compiler Tcl-mode script before setting constraints
TclAuxSynthesis Script[dcrmlInitial]	Executes the Design Compiler Tcl-mode script immediately after SDC constraints are applied (used in the Design_Compiler_Reference_Methodology synthesis strategy)
TclAuxSynthesis Script [dcrmlPathGroupScript]	Executes the Design Compiler Tcl-mode script after the existing group_path commands (used in the Design_Compiler_Reference_Methodology and DCTCL_quick_map_strategy)
TclAuxSynthesis Script[cleanup]	Executes the Design Compiler Tcl-mode script after all processing is completed, just prior to quitting
TclAuxSynthesis Script[report]	Executes the Design Compiler Tcl-mode script, which applies to report generation
TclAuxSynthesis Script[dftInsertion]	Executes the Design Compiler Tcl-mode script, which applies to DFT insertion
TclAuxSynthesis Script[dftCleanup]	Executes the Design Compiler Tcl-mode script, which applies DFT cleanup
TclAuxSynthesis Script[qmap]	Executes a Design Compiler Tcl-mode script prior to a specific optimization phase
FcAuxSynthesis Script[post_init_design]	Executes a Fusion Compiler script at the very end of init_design stage before save_block

Table 5-3 Advanced Synthesis Attributes (continued)

Attribute	Description
FcAuxSynthesis Script[pre_read_rtl]	Executes a Fusion Compiler script before reading RTL
FcAuxSynthesis Script[post_read_rtl]	Executes a Fusion Compiler script immediately after reading RTL
FcAuxSynthesis Script[pre_compile]	Executes a Fusion Compiler script before compile
FcAuxSynthesis Script[post_compile]	Executes a Fusion Compiler script after compile
FcAuxSynthesis Script[pre_compile_unified]]	Executes a Fusion Compiler script in the UNIFIED_FLOW before compile_fusion -from final_place
FcAuxSynthesis Script[pre_place_opt]	Executes a Fusion Compiler script before placement optimization (place_opt)
FcAuxSynthesis Script[post_place_opt]	Executes a Fusion Compiler script immediately after placement optimization (place_opt)
FcAuxSynthesis Script[write_data]	Executes a Fusion Compiler script before data is written out
FcAuxSynthesis Script[report]	Executes a Fusion Compiler script after data is written out; can be used to include report generation commands
FcAuxSynthesis Script[cleanup]	Executes a Fusion Compiler script after all processing is completed, just prior to quitting

5.8 Specifying Port Constraints

Use the `set_port_attribute` command in intent command files to specify values for port attributes based on your expectation of a typical target environment for the core. For descriptions of port attributes and guidelines on setting their values, see the coreBuilder online reference.

The basic syntax for `set_port_attribute` is:

```
set_port_attribute port attr value
```

The following sections describe how to:

- Automatically select sets of ports on which to execute the `set_port_attribute` command
- Set port attributes in intent command files
- Create groups of ports to be listed separately in the Specify Port Constraints GUI

5.8.1 Selecting Sets of Ports

To automatically select sets of ports on which to execute the `set_port_attribute` command, use the output of a coreBuilder port selection command as the *port* argument to `set_port_attribute`, as follows:

- To set an attribute-value pair on all input ports of the current design, use the syntax:

```
set_port_attribute [all_inputs] attr_name attr_value
```
- To set an attribute-value pair on all input ports of the current design, except clock inputs, use the syntax:

```
set_port_attribute [all_inputs -no_clocks] attr_name attr_value
```
- To set an attribute-value pair on all output ports of the current design, except ports that are connected to the core's primary output ports, use the syntax:

```
set_port_attribute [all_outputs -no_primary] attr_name attr_value
```
- To set an attribute-value pair on all ports with names that match a pattern (for example, names that contain the string "addr"):

```
set_port_attribute *addr* attr_name attr_value
```

5.8.2 Port Constraints in Intent Command File

[Example 5-5](#) shows how to set port attributes for top-level input and output ports in an intent command file, using formulas for `InputDelay` and `DrivingCell`. `InputDelay` is a special attribute that you can use in intent command files to specify the same value for both `MinInputDelay` and `MaxInputDelay`. Similarly, you can use `OutputDelay` to specify the same value for both `MinInputDelay` and `MaxInputDelay`. The `set_port_attribute` commands in [Example 5-5](#) also uses the commands `all_inputs` and `all_outputs` to select groups of ports on which to set attribute values.

Example 5-5 Specifying Port Constraints

```
current_design TOP
```

```
set_port_attribute clk      ClockName "clk"
set_clock_attribute clk     CycleTime 25ns
set_port_attribute rst_n    {InputDelay[clk]} {=percent_of_period 10.0}
set_port_attribute test_n   {InputDelay[clk]} {=percent_of_period 10.0}
set_port_attribute data_in  {InputDelay[clk]} {=percent_of_period 25.0}
```

```

set_port_attribute in1      {InputDelay[clk]} {=percent_of_period 25.0}
set_port_attribute in2      {InputDelay[clk]} {=percent_of_period 10.0}
set_port_attribute in3      {InputDelay[clk]} {=percent_of_period 15.0}
set_port_attribute in4      {InputDelay[clk]} {=percent_of_period 15.0}
set_port_attribute in5      {InputDelay[clk]} {=percent_of_period 15.0}
set_port_attribute in6      {InputDelay[clk]} {=percent_of_period 15.0}
set_port_attribute in7      {InputDelay[clk]} {=percent_of_period 15.0}
set_port_attribute in8      {InputDelay[clk]} {=percent_of_period 15.0}

#-- all inputs except clocks assume there's some combinational driver
set_port_attribute [all_inputs -no_clocks] DrivingCell \
    {=select_by_class "combinational" "median"}

#-- these inputs assume they're coming from a register
#-- (they override prior the settings)
set_port_attribute rst_n    DrivingCell {=select_by_class "sequential" "low"}
set_port_attribute test_n   DrivingCell {=select_by_class "sequential" "low"}
set_port_attribute data_in  DrivingCell {=select_by_class "sequential" "low"}

#--asynchronous reset
set_port_attribute rst_n    IdealPort
set_port_attribute [all_inputs] HelpUrl "../doc/inputs.html"

#-- all outputs have the same basic %
set_port_attribute [all_outputs] {OutputDelay[clk]} \
    {=percent_of_period 20.0}

#-- all outputs drive the same type of load
set_port_attribute [all_outputs] PinLoadType {=select_by_class
    "combinational"}

#-- the outputs are expected to drive different numbers of loads
set_port_attribute out1      PinLoadCount 10
set_port_attribute out2      PinLoadCount 5
set_port_attribute out3      PinLoadCount 5
set_port_attribute out4      PinLoadCount 10
set_port_attribute out5      PinLoadCount 10
set_port_attribute out6      PinLoadCount 10
set_port_attribute out7      PinLoadCount 10
set_port_attribute out8      PinLoadCount 5
set_port_attribute out9      PinLoadCount 5
set_port_attribute out10     PinLoadCount 5
set_port_attribute rxd0_out   PinLoadCount 5
set_port_attribute rxd1_out   PinLoadCount 5
set_port_attribute [all_inputs] HelpUrl "../doc/outputs.html"

```

5.8.3 Creating Port Groups

You can create groups of ports by setting the `GroupName` attribute on ports. Ports with the same `GroupName` value are grouped together in the Specify Port Constraints spreadsheet and in the HTML synthesis reports that the coreConsultant tool generates.

All input ports with the same `GroupName` value appear in one group in the Input port list. All output ports with the same `GroupName` value appear in one group in the Output port list.

[Example 5-6](#) shows a sample intent command file that groups ports by function (Address, Data, Interrupt, and Control).

Example 5-6 Creating Port Groups

```
set_port_attribute clk ClockName clk

set_clock_attribute clk CycleTime 20

set_port_attribute [all_inputs -no_clocks] MinInputDelay 2
set_port_attribute [all_inputs -no_clocks] MaxInputDelay 2
set_port_attribute [all_outputs] MinOutputDelay 2
set_port_attribute [all_outputs] MaxOutputDelay 2
set_port_attribute *addr* GroupName Address
set_port_attribute *data* GroupName Data
set_port_attribute *int* GroupName Interrupt
set_port_attribute *idle* GroupName Control
```

5.9 Intent Command File Example

[Example 5-7](#) shows an intent command file that applies the previously discussed design and port constraint commands on a simple design.

Example 5-7 Intent Command File for Simple Design

```
#TOP design attributes

current_design TOP
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_A design attributes
current_design Subblock_A
set_design_attribute MapSubblocksIndividually false
set_design_attribute HasArithmetic

#Subblock_B design attributes
current_design Subblock_B
set_design_attribute MapSubblocksIndividually
set_design_attribute PurelyStructural

#Subblock_B1 design attributes
set_design_attribute -design Subblock_B1 OptimizationPriorities timing

#Subblock_B2 design attributes
current_design Subblock_B2
#Subblock_B2 will benefit from flattening
set_design_attribute CanFlatten

#Top-level port constraints
#Subblock port constraints is in subblock_name.tcl

#Input ports all have 2ns input delay
current_design TOP
set_port_attribute clk      ClockName "clk"
set_clock_attribute clk     CycleTime 25ns
set_port_attribute [all_inputs -no_clocks] InputDelay 2ns

#Output ports
#All outputs have 3ns output delay
set_port_attribute [all_outputs] OutputDelay 3ns
```

5.10 Post-Elaboration Intent

Certain intent information cannot be applied until after the coreConsultant tool elaborates your design. For example, you may want to set configuration-specific intent which can only be applied after the core integrator chooses a configuration.

The coreBuilder tool uses post-elaboration intent command files to apply intent (attributes) to designs after elaboration. Post-elaboration intent command files are identical to regular intent command files with the following exceptions:

- The coreBuilder tool reads the post-elaboration intent command file with the other intent command files, but stores the post-elaboration intent in the design .kb file instead of applying it immediately. The coreConsultant tool applies the post-elaboration intent after the core integrator performs the Specify Configuration activity.
- Post-elaboration intent command files use the filename convention *design.elab.tcl*.

The coreConsultant tool enables the core integrator to specify a design name prefix before configuring the core. Therefore, after configuration, design names may include a prefix assigned by the core integrator. In post-elaboration intent command files, use the following commands to determine the post-configuration design names:

- `get_top_design_name` – Get the name of the top-level design
- `get_design_prefix` – Get the design name prefix specified by the core integrator

Also, when writing post-elaboration intent command files, use guard statements to indicate which configurations of the design are targeted by the post-elaboration intent.

The following example shows a post-elaboration intent command file that applies a design attribute conditionally (only when the value of the width parameter exceeds 6):

```
#Post-elaboration intent for MyDesign
current_design "[get_design_prefix]MyDesign"
if {[get_configuration_parameter width] > 6} {
    set_design_attribute MapSubblocksIndividually
}
```

5.11 Importing Synthesis Intent

Instead of (or in addition to) writing intent command files, you can import design constraints and other synthesis intent in the following ways:

- Specify synthesis intent in GUI mode, then automatically generate an intent command file that applies the same intent specifications
- Read in existing design constraint commands in Synopsys Design Constraint (SDC) format

5.11.1 Importing Intent Specified in GUI Mode

If you prefer to get started by specifying synthesis intent in GUI mode, you can automatically generate an intent command file from your interactively specified intent. To do so, follow this procedure:

1. Execute the Load Designs activity (see [“Loading Your Design”](#) on page 103), and (optionally) the Design Configuration Intent activity (see [“Refining Configuration Intent”](#) on page 97).
2. Execute the Synthesis Intent activities (see [“Specifying Synthesis Intent”](#) on page 125).
3. Use the **File > Write Batch Script** command to generate a batch mode script. Select only the “Include Design Constraints” option. Select the name *design.tcl* as the “Output File Name” option.

The corresponding command line mode command is:

```
write_batch_script -include -design_intent design.tcl
```

4. Use *design.tcl* as your intent command file.
5. As you edit *design.tcl*, repeat the Load Designs activity to reload the design and apply your revised synthesis methodology.

The above procedure combines the ease-of-use of specifying synthesis intent in GUI mode with the manageability of intent command files. The **File > Write Script** menu writes out the synthesis intent in the exact format required for the intent command files.

5.11.2 Importing Synopsys Design Constraints

The coreBuilder tool supports the import and export of design constraints in SDC format, which is commonly used to exchange synthesis methodology among Synopsys tools such as Design Compiler, PrimeTime, and coreBuilder, as well as other vendors’ tools licensed under the Synopsys Tap-in program.

An SDC file is simply a Tcl script that sets constraints on a design. Synopsys tools that support SDC can:

- Read SDC files and apply those constraints to design objects. If you include the directory containing your SDC files in your intent command file search path, coreBuilder automatically reads the SDC files when you load your design into coreBuilder. You can also use the coreBuilder `read_sdc` command to read SDC files explicitly.
- Record existing constraints in SDC format. Use the coreBuilder `write_sdc` command to write out the currently specified constraints (attributes currently set on designs and ports) in SDC format for direct use by other tools.

You do not have to translate the Design Compiler or PrimeTime constraints into coreBuilder attribute specifications, since all three tools understand SDC format.

5.11.2.1 Supported SDC Versions

By default, the coreBuilder tool imports and exports version 2.1 SDC files.

5.11.2.2 Restrictions

Not all tools support the same set of SDC constraints. The coreBuilder SDC syntax represents a subset of the constraints supported by all tools. When you import an SDC file into the coreBuilder tool, either automatically or by using the `read_sdc` command, it executes only commands that map directly to attributes it supports. Commands that are not fully supported are handled as the following:

- **Pass-through commands:** commands are passed through from the SDC file that is read to the SDC file that is written out using `write_sdc`. Pass-through commands do not alter the state of the design. They are written out verbatim.
- **Pass-through switches:** options to a command that will cause the command to be passed-through (similar to a pass-through command). These commands are not passed through if none of these special switches are invoked with the command.
- **Unsupported switches:** Some switches are not supported by the coreTools implementation of the SDC command. These switches are supported by DC, but not by the coreTools applications. Unsupported-switches cause the coreTools applications to stop parsing the SDC file and report an error.

Table 5-4 lists the commands and options in the SDC version 2.1 syntax that the coreBuilder tool passes through, or does not currently support. If the SDC file specifies a constraint with a technology-dependent value, the coreBuilder tool does not automatically convert it to a technology-independent value.

Table 5-4 Restrictions to SDC 2.1 Commands

SDC Command	Support Type	Restriction
<code>all_inputs</code>	Unsupported switches	<code>-level_sensitive</code> , <code>-edge_triggered</code>
<code>all_outputs</code>	Unsupported switches	<code>-level_sensitive</code> , <code>-edge_triggered</code>
<code>create_clock</code>	Unsupported switches	<code>-add</code>
<code>get_ports</code>	Unsupported switches	<code>-of_objects</code>
<code>get_pins</code>	Unsupported switches	<code>-of_objects</code> , <code>-leaf</code>
<code>get_nets</code>	Unsupported switches	<code>-of_objects</code>
<code>get_libs</code>	Unsupported switches	<code>-of_objects</code>
<code>get_lib_cells</code>	Unsupported switches	<code>-of_objects</code>
<code>get_lib_pins</code>	Unsupported switches	<code>-of_objects</code>
<code>group_path</code>	Passed through	Not supported.
<code>set_annotated_check</code>	Passed through	Not supported.
<code>set_annotated_delay</code>	Passed through	Not supported.
<code>set_annotated_transition</code>	Passed through	Not supported.
<code>set_clock_gating_check</code>	Pass-through switches	<code>-high</code> , <code>-low</code>
<code>set_clock_latency</code>	Pass-through switches	<code>-min</code> , <code>-max</code> , <code>-early</code> , <code>-late</code> , <code>-clock</code>

Table 5-4 Restrictions to SDC 2.1 Commands (continued)

SDC Command	Support Type	Restriction
set_disable_clock_gating_check	Passed through	Not supported.
set_drive	Pass-through switches	-rise, -fall, -min, -max
set_driving_cell	Unsupported switches	-min, -max
set_fanout_load	Passed through	Not supported.
set_input_delay	Pass-through switches	-network_latency_included, -source_latency_included -level_sensitive
set_input_transition	Passed through	Not supported.
set_load	Pass-through switches	-min, -max, -subtract_pin_load, -wire_load
set_max_dynamic_power	Passed through	Not supported.
set_max_leakage_power	Passed through	Not supported.
set_max_time_borrow	Passed through	Not supported.
set_operating_conditions	Pass-through switches	-max_phys, -min_phys, -object_list, -analysis_type
set_output_delay	Pass-through switches	-network_latency_included, -source_latency_included -level_sensitive
set_propagated_clock	Passed through	Not supported.
set_resistance	Passed through	Not supported.
set_timing_derate	Passed through	Not supported.
set_wire_load_model	Pass-through switches	-min, -max
set_wire_load_selection_group	Pass-through switches	-min, -max

For a list of the fully supported commands, see the `read_sdc` man page in the coreTools applications.

6

Refining Configuration Intent

The Design Configuration Intent activity gives you the opportunity to check the configuration intent specifications, if any, that the coreBuilder tool extracted from reuse-pragmas in your HDL source code. In GUI mode, you view and modify configuration intent specifications and check your configuration from the perspective of a core integrator. This chapter describes the purpose of the Design Configuration Intent activity in the context of coreKit development and describes how to execute the Design Configuration Intent activity.

This chapter contains the following sections:

- [“Overview”](#) on page 98
- [“Configuration Intent Specification Methods”](#) on page 99
- [“Design Configuration Intent Activity”](#) on page 100

6.1 Overview

Configuration intent defines the ways in which a design may be configured when integrated into a system-on-a-chip application. For instance, a core may support an 8, 16, or 32 bit-wide data bus. It may also contain optional peripheral components.

Configuration intent also defines the presentation of the configuration parameters seen by the core integrator in the configuration dialog. Specifying configuration intent accurately guarantees that the core integrator cannot generate an invalid configuration of the core.

6.2 Configuration Intent Specification Methods

You can specify configuration intent for a core by using the following methods singly or in combination:

- Annotate your HDL source files – Insert reuse-pragmas in the HDL source code files to set attributes on design parameters. When you execute the Load Designs activity, coreBuilder applies the attributes specified by the reuse-pragmas to their respective parameters.
- Write set_parameter_attribute commands in intent files – Synopsys recommends using intent command files only for synthesis intent. However, it is possible to set parameter attributes in intent command files. When you execute the Load Designs activity, coreBuilder executes the intent command files it finds in the search path. Parameter attributes specified in intent command files override parameter attributes specified through the HDL files.
- Set the ParameterInfo attribute on the design – The ParameterInfo attribute provides greater control over how parameters are grouped and presented in the coreConsultant Specify Configuration dialog. ParameterInfo is especially useful for designs that have a large numbers of parameters. See [“Setting Up Advanced Parameter Dialogs”](#) on page 256 for more information.
- Execute the Design Configuration Intent activity – The Design Configuration Intent activity is a required step in the coreKit development flow. In GUI mode, you can interactively view and modify parameter attributes. You can also preview the parameter dialog that the coreConsultant tool presents to the core integrator.

In practice, you should specify configuration intent using the first method (annotating the design HDL source files), because:

- It is convenient to enter the intent specifications in the HDL files where the parameters are declared. If you make any changes to the parameter declarations in the HDL code, you can update your configuration intent at the same time in the same text file
- The configuration intent contains much of the same explanatory information usually found as comments in well written code. Thus, the coreConsultant configuration intent specification syntax in your HDL files serves two purposes: it explains the parameters and it drives the reuse process.
- The configuration intent remains with your design and remains independent of your coreBuilder workspace. If you reuse the core in a new coreBuilder workspace, your configuration intent specifications are already complete.

For design configuration parameters implemented as Verilog ``define` or VHDL constant statements, you must specify at least one attribute to the parameter in the HDL code to identify the parameter to the coreConsultant tool as a design configuration parameter. This makes the parameter visible in the Design Configuration Intent GUI and prompts coreBuilder to enable automatic text substitution of the parameter's value.

6.3 Design Configuration Intent Activity

To execute the Design Configuration Intent activity in GUI mode:

1. Select the Design Configuration Intent activity from the Activity List. The corresponding Activity View dialog appears.
2. Click the Preview User Configuration Dialog button in the Configuration Intent dialog.
The coreBuilder tool displays a parameter dialog that looks exactly like the one the coreConsultant tool builds for the core integrator, except the coreBuilder dialog contains an additional Check button, which allows core developer to verify the parameter configuration space.
3. Check the labeling, grouping, sequence, and selection mechanisms for the design parameters.
4. Enter legal values for parameters, then click Check. The coreBuilder tool should return the message "Parameter checks OK."
5. Enter illegal values for parameters, then click Check. Verify that coreBuilder rejects the illegal parameter values and returns the appropriate error message.
6. Click Cancel to return to the Design Configuration dialog.
7. If you are not satisfied with the results of the checks, modify your parameter attributes. You can modify the parameter attributes directly in the Design Configuration Intent dialog. However, if you repeat the Load Designs activity, your changes are lost. After experimenting with attribute values through the Design Configuration Intent dialog, it is best to implement your final choices as HDL annotations so that the configuration intent remains with the design.

Table 6-1 shows where to find the parameter attributes in the Design Configuration Intent dialog. For more information about the parameter attributes, see "[Annotating Design Files](#)" on page 49 and the coreBuilder online reference.

Re-check your specifications by repeating steps 2 through 5.

8. Click Apply to complete the Design Configuration Intent activity.

Table 6-1 Parameter Attributes for Configuration Intent

Location	Label	Attribute	Short Description
Main Page	Default Value	DefaultValue	Default value of the parameter
Main Page	Default Value Message	DefaultValue Message	Specifies message associated with default value
Main Page	ReadOnlyParam	ReadOnlyParam	Specifies that the parameter value cannot be modified by the user
Main Page	Sequence	Sequence	Specifies the order in which parameters appear in the Specify Configuration dialog, or within a parameter group
Main Page	GroupName	GroupName	Assigns parameter to a parameter group
Main Page	CrossParamGroup	CrossParam Group	Identifies groups of parameters that must have different values

Location	Label	Attribute	Short Description
Properties / Range	Minimum Value	MinValue	Minimum legal value for the parameter
Properties / Range	Maximum Value	MaxValue	Maximum legal value for the parameter
Properties / Range	Enumerated Values	EnumValues	Specifies a set of discrete legal values for the parameter
Properties / Range	Symbolic Names	SymbolicNames	Symbolic names for the parameter's legal values
Properties / Label and Help	Label	Label	Label to use for the parameter in the coreConsultant Specify Configuration GUI
Properties / Label and Help	Help URL	HelpUrl	The URL of the help text for the parameter
Properties / Label and Help	Description	Description	A brief help-text description of the parameter
Properties / Checking	Check Expression	CheckExpr	Specifies a Tcl expression to check the value of a parameter The If box is for the value of the CheckExpr subscript (enter 1 to check the parameter value by default). The Then box is for the Tcl check expression. Click Apply to apply CheckExpr to the parameter and add it to the CheckExpr list for the parameter.
Properties / Checking	Check Command	CheckCmd	Specifies the name of a Tcl command procedure that you want to use to check the user-specified value of the parameter
Properties / Enabled	Enabled Expression	Enabled	Specifies a Tcl expression that, when true, the parameter should be enabled for modification The If box is for the value of the Enabled subscript. Enter 1 to evaluate the Then expression by default or enter an expression that, when true, the Then expression should be evaluated. The Then box is for the enabling Tcl expression. Click Apply to apply Enabled expression to the parameter and add it to the Enabled expression list for the parameter.
Properties / Alternate Value	Alternate Value	AltValue[actual]	An alternate legal value for a parameter, in addition to the legal values specified by MinValue/MaxValue or EnumValues.

Location	Label	Attribute	Short Description
Properties / Alternate Value	Value Label	AltValue [valueLabel]	Label to use for the non-alternate values.
Properties / Alternate Value	Alternate Value Label	AltValue [altValueLabel]	Label to use for the alternate value.
Properties / Alternate Value	Alt Value Symbolic	AltValue [symbolicName]	Symbolic name to use for the alternate value.

7

Loading Your Design

After you set up your workspace and prepare your design for the coreBuilder tool (by annotating your source files and writing intent command files), you are ready to load your design into the coreBuilder tool and continue building your coreKit.

This chapter contains the following sections:

- [“Organizing Design and Intent Files”](#) on page 104
- [“Executing the Load Designs Activity”](#) on page 105

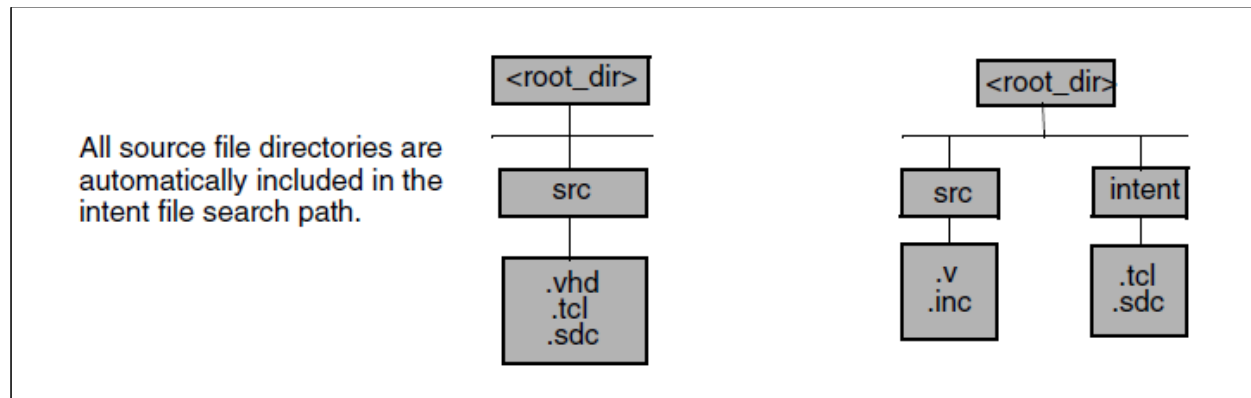
7.1 Organizing Design and Intent Files

The following types of files provide input for the coreBuilder Load Designs activity:

- HDL Files – You must specify a proper sequence of Verilog (.v), VHDL (.vhd), or System Verilog (.sv) files for analysis. The coreBuilder tool requires the same file order that Design Compiler requires.
- Verilog/SV Include Files – The coreBuilder tool analyzes all include files referenced in the Verilog source that it finds in the specified Verilog/SV Includes search path.
- Intent Command Files – The coreBuilder tool automatically executes all the intent command files (*HDL_File_basename.tcl*) that it finds in the specified intent file search path.
- Synopsys Design Constraint Files – The coreBuilder tool automatically executes all SDC files (*HDL_File_basename.sdc*) that it finds in the specified intent file search path.

You can specify search paths for each of the above file types. Therefore, you do not have to use a specific directory structure for your design and intent files. However, the suggested directory structures shown in [Figure 7-1](#) may ease your file management tasks.

Figure 7-1 Suggested Directory Structures for Source Files



7.2 Executing the Load Designs Activity

When you execute the Load Designs activity, coreBuilder loads and analyzes the specified HDL source files, then builds an internal model of your design. The coreBuilder tool uses this model to store your synthesis methodology.

After building the design model, the coreBuilder tool executes the associated intent command files, applies the intent attributes to the various elements of your design (cells, parameters, and ports), and stores the intent annotations in the respective design knowledge databases. Some intent command files can be executed only after the core has been elaborated. (See “[Post-Elaboration Intent](#)” on page 92).

To execute the Load Designs activity in GUI mode, click Load Designs in the coreBuilder Activity List. In the Load Designs dialog which appears in the Activity View pane, you can specify values for the parameters listed in [Table 7-1](#). When you have finished specifying values for the parameters, click Apply.

The **Options** tab allows you to set the language, language version and default libraries

Table 7-1 Load Designs Activity “Options” Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Input Language (InputLanguage)	Specify the language in which your design is coded (VHDL, Verilog or System Verilog)
VHDL Options	
VHDL Version (VhdlVersion)	The VHDL version used to analyze the input files. Values: VHDL-87 or VHDL-93. Default: VHDL-93
Default Library (VhdlLibrary)	For VHDL designs, specify the library into which the VHDL files are to be analyzed. To specify more than one library, enter a line into the input file list using the format “library <i>name</i> ” in HDL File Analysis Order. The coreBuilder tool analyzes all files between adjacent separators into the most recently specified library.
Verilog Options	
Verilog Version (VerilogVersion)	The Verilog version used to analyze the input files. Values: Verilog-95 or Verilog-2001. Default: Verilog-2001
Include File Directories (IncludePath)	For Verilog designs, specify a list of directories for the coreBuilder tool to search for include files You do not need to specify directories that are referenced by a relative pathname in your .v files. For example, you do not need to specify the directories for include files referenced by statements such as <code>`include TOP_parameter.inc</code> .

The **Source Files** tab allows setting the source files that are to be loaded.

Table 7-2 Load Designs Activity “Source Files” Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Append To ‘HDL Files’ From File... (None)	Enables you to automatically append files to the HDL file list from a file File – Create a file listing the HDL source files (see “Setting Up an HDL File List” on page 108), then select the file in the file browser and click Open. The coreBuilder tool automatically extracts the file names and updates the Tcl Intent Search Path to include the directory containing your list files and any new directories in your list file. You can then add, delete, or rearrange the files and search paths, as needed.
Append To ‘HDL Files’ From Directory... (None)	Enables you to automatically append files to the HDL file list from a directory Directory – Select the directory in the file browser and click Open to extract a list of all HDL source files in a selected directory. The coreBuilder tool automatically updates the Tcl Intent Search Path to include the selected directory. You can then add, delete, or rearrange the files and search paths, as needed.
HDL Files <input field> (InputFileNames)	Specifies the name of your top-level design The LoadDesigns dialog does not prompt you for the name of the top-level design, because in most cases, the coreBuilder tool can determine the name through the HDL file analysis. If, for some reason, the coreBuilder tool cannot determine the top design name, it posts a special dialog to prompt you for the name of the top-level design.
Analyze Subset of Files (AnalyzeSubsetOfFiles)	Enables the HDL Files To Be Analyzed field
HDL Files To Be Analyzed (FilesToBeAnalyzed)	List all the HDL files to be analyzed during Load Designs By default, this parameter has the same value as the InputFileNames parameter. If some of the files should not be analyzed, check the “Analyze Subset of Files” checkbox and specify the list of files to be analyzed in this field.

Table 7-3 Load Designs Activity “Advanced” Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Strict Language Semantics (StrictSemantics)	Enforce strict language semantics When false (Default), commonly-supported non-compliant language semantics are accepted without error.
PreventDesignPrefix (PreventDesignPrefix)	Prevent insertion of design and cell prefixes in the source code Default: 0 (false)

Table 7-3 Load Designs Activity “Advanced” Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
PreventSVPrefix (PreventSVPrefix)	Prevent insertion of the design prefix for SystemVerilog interface and package names in the configured source code
Autoread Supported? (AutoReadDCSupported)	Allows usage of autoread mode in DC to analyze the RTL files, given that the RTL code is autoread-compatible The files should be structured to be under one top directory. Default: 0 (false)
Skip Designs (SkipDesigns)	List of patterns that indicate which RTL designs should be skipped A pattern is a glob-style expression. If a pattern starts with '!', then a design must match that pattern, otherwise it is skipped. You can use multiple patterns.
Skip Prefix Names (SkipPrefixNames)	List of names and patterns to indicate modules and SystemVerilog interfaces and packages to not prefix A pattern is a glob-style expression. If a pattern starts with '!', then only names that match the pattern are prefixed.
Source License (SourceLicenseString)	License string that allows you to protect your source code with encryption Unless the user has the specified license, the coreConsultant tool encrypts the source code for the core when installing the coreKit. If you do not specify a license string, the coreConsultant tool does not encrypt the source code for the core.
Tcl Intent Search Path (IntentSearchPath)	The complete path to one or more directories that you want the coreBuilder tool to search for intent command files Arrange your entries in the order in which you want them searched. The coreBuilder tool executes the first intent command file it finds whose base name matches that of an HDL file it has analyzed. For example, after the coreBuilder tool analyzes top.vhd, it executes the first “top.tcl” file it finds in the Tcl Intent Search Path. When this parameter is specified as a relative path, the non-absolute path is expanded relative to the location of the file for which intent is being specified. If you choose one of the alternate file loading strategies, the coreBuilder tool automatically includes all HDL source file directories in the Tcl Intent Search Path.
Extra Link Libraries (ExtraLinkLibs)	Specifies additional libraries required to link the design in Design Compiler or Fusion Compiler Examples include library files that define hard macros and/or timing models. The coreBuilder tool includes the specified files in the coreKit; the coreConsultant tool adds the specified files to the Design Compiler link_library variable.

7.2.1 Setting Up an HDL File List

To use the File input strategy with the Load Designs activity, set up the file list as shown in [Example 7-1](#) (Verilog) or [Example 7-2](#) (VHDL). The list in [Example 7-2](#) uses multiple design libraries.

Example 7-1 List of HDL Files for Verilog Core

```
/rcs/designs/cores/MyCore/Subblock_A1.v
/rcs/designs/cores/MyCore/Subblock_A2.v
/rcs/designs/cores/MyCore/Subblock_A3.v
/rcs/designs/cores/MyCore/Subblock_B1.v
/rcs/designs/cores/MyCore/Subblock_B2.v
/rcs/designs/cores/MyCore/Subblock_A.v
/rcs/designs/cores/MyCore/Subblock_B.v
/rcs/designs/cores/MyCore/TOP.v
```

Example 7-2 List of HDL Files for Multiple-Library VHDL Core

```
library Common
/rcs/designs/commonparts/Subblock_A1.vhd
/rcs/designs/commonparts/Subblock_A2.vhd
library MyCore
/rcs/designs/cores/MyCore/TOP_parameter.vhd
/rcs/designs/cores/MyCore/Subblock_A3.vhd
/rcs/designs/cores/MyCore/Subblock_A4.vhd
/rcs/designs/cores/MyCore/Subblock_B1.vhd
/rcs/designs/cores/MyCore/Subblock_B2.vhd
/rcs/designs/cores/MyCore/Subblock_A.vhd
/rcs/designs/cores/MyCore/Subblock_B.vhd
/rcs/designs/cores/MyCore/TOP.vhd
```



Note

Do not include curly braces {} around the library name statements in the file list for the Load from File strategy.

7.2.2 Command Line Mode

To complete the LoadDesigns activity in command line mode or from a batch command file, use the commands shown in [Example 7-3](#) (loading from File List), or [Example 7-4](#) (specifying files and libraries directly through the InputFileNames parameter).

Example 7-3 Batch Script for LoadDesigns Using File Strategy

```
set_activity_incomplete LoadDesigns
set_activity_parameter LoadDesigns InputStrategyArguments /usr/fred/list
set_activity_parameter LoadDesigns InputLanguage VHDL
set_activity_parameter LoadDesigns InputStrategy File
autocomplete_activity LoadDesigns
```

Example 7-4 Batch Script for Load Designs using InputFileNames

```
set_activity_incomplete LoadDesigns
set_activity_parameter LoadDesigns InputLanguage VHDL
```

```
set_activity_parameter LoadDesigns IntentSearchPath \  
{/rcs/designs/commonparts /rcs/designs/cores/MyCore}  
  
set_activity_parameter LoadDesigns InputFileNames {{library Common} \  
/rcs/designs/commonparts/Subblock_A1.vhd \  
/rcs/designs/commonparts/Subblock_A2.vhd {library MyCore} \  
/rcs/designs/cores/MyCore/TOP_parameter.vhd \  
/rcs/designs/cores/MyCore/Subblock_A3.vhd \  
/rcs/designs/cores/MyCore/Subblock_A4.vhd \  
/rcs/designs/cores/MyCore/Subblock_B1.vhd \  
/rcs/designs/cores/MyCore/Subblock_B2.vhd \  
/rcs/designs/cores/MyCore/Subblock_A.vhd \  
/rcs/designs/cores/MyCore/Subblock_B.vhd \  
/rcs/designs/cores/MyCore/TOP.vhd}  
  
autocomplete_activity LoadDesigns
```


8

Specifying a Memory Map

The memory map specification for a component allows you to specify the memory resources needed for each slave interface of your component. A memory map is a container used to hold different address regions. The address regions are described by address blocks and address banks. The memory map is then associated with an interface instance with the `MemoryMap` attribute to describe the memory for that interface.

This chapter contains the following sections:

- [“Memory Map Overview”](#) on page 112
- [“Specifying the Memory Map”](#) on page 113
- [“Memory Map Attributes”](#) on page 118
- [“Memory Map Examples”](#) on page 122

8.1 Memory Map Overview

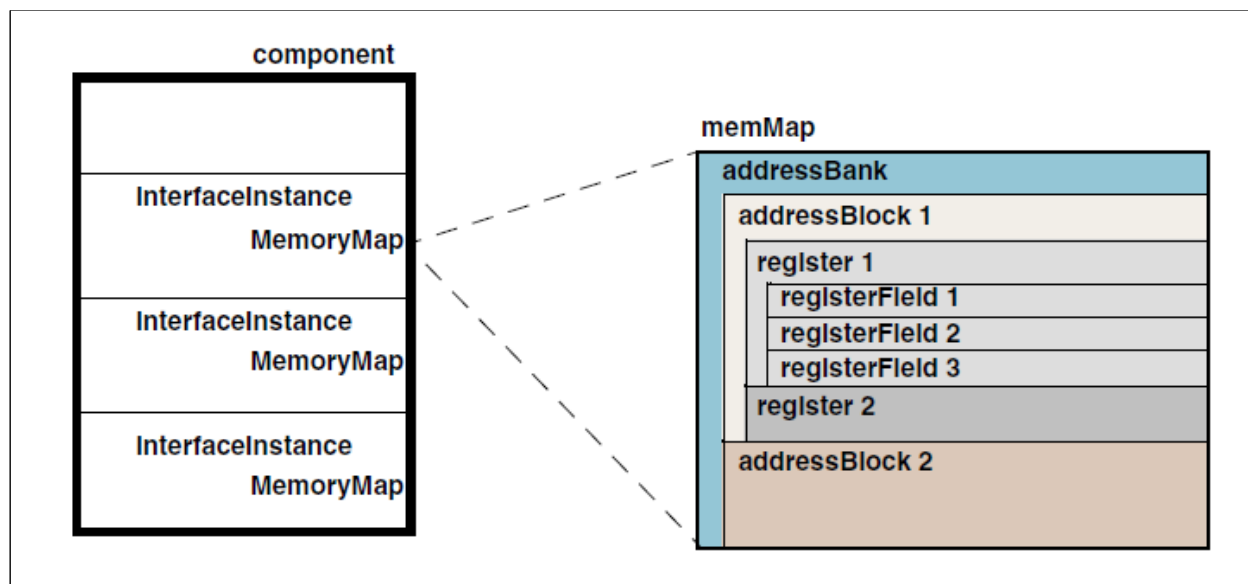
Because memory is usually shared between multiple interfaces on a component, and multiple components in your subsystem, these memory allocations are usually specified relative to a base address whose position in the real memory space can vary.

The memory map is divided into the following nested segments:

- **Address bank** - An address bank is a container to hold address blocks or address banks in a logical grouping. An address bank defines an alignment that determines how the address blocks and banks are organized in the memory map. Most memory maps will contain a single address block and no address banks.
- **Address blocks** - An address block represents a continuous block of memory within a memory map. The address block also acts as a container for a set of registers in a memory region.
- **Registers** - Describes a register within an address block. The register can also hold other items to describe the internal structure (fields) of the register.
- **Register Fields** - The register field is used to describe the structure of a register in further detail. Field items describe the individual bits and their uses.

Figure 8-1 shows the items that you specify for a typical component memory map.

Figure 8-1 Items in a Memory Map



Hint

You should also set the “MemoryMap” attribute on the corresponding slave bus interface from which the memory map is visible. This is required in order to enable proper address map initialization and generation of a complete IP-XACT XML file.

8.2 Specifying the Memory Map

You can specify memory map intent in your component by the following methods:

- [Memory Map GUI Interface](#) on page 113
- [Memory Map Command Interface](#) on page 117

**Note**

You can also import a memory map from an IP-XACT component using the `import_ipxact_data` command, or the **File > Import IP-XACT Data...** menu.

8.2.1 Memory Map GUI Interface

The coreBuilder tool provides a GUI interface to create or modify memory map information. The Memory Maps activity group provides two activities and a set of dialog fields where you specify memory definitions. The Memory Maps activities are:

- Define Memory Maps -- Allows you to create memory maps and add address blocks, registers, and register fields.
- Advanced Memory Map Attributes -- Allows you to add memory map attributes, address block attributes, and register attributes.
- Memory Map Documentation -- Allows you to document all parts of a memory map.

8.2.2 Define Memory Maps Activity

This activity, and accompanying dialog is divided into two parts, as described in the following sections:

- [Memory Maps, Address Banks, Address Blocks](#) on page 113
- [Registers, Register Fields Dialog Section](#) on page 114

8.2.2.1 Memory Maps, Address Banks, Address Blocks

The Memory Maps, Address Banks, Address Blocks section is listed in hierarchical format to show the relationship. The columns in the spreadsheet view are editable (except for the Name column) to allow modifying structure information. This dialog section has the buttons in [Table 8-1](#).

Table 8-1 Memory Map GUI Interface Buttons

Button Name	Field ID	Corresponding Command	Description
Add Memory Map...	MP	create_memory_map	Creates/removes a memory map with the given name A memory map is a container that holds address regions.
Add Address Bank...	BN	create_address_bank	Creates/removes an address bank Specify either -map or -bank. If -map, then -base_address must also be provided; otherwise, the BaseAddress is automatically computed from the rules of the address bank.
Add Address Block...	BL	create_address_block	Creates/removes and address block in the specified memory map or address bank

Table 8-1 Memory Map GUI Interface Buttons (continued)

Button Name	Field ID	Corresponding Command	Description
Add Register...	RG	create_register	Creates/removes a register in the given address block at the specified offset (in bytes) The -size argument specifies the register size in bits.
Delete Item		remove_memory_map remove_address_bank remove_address_block remove_register remove_register_field	Removes the specified memory item
Add Register Field...	RF	create_register_field	Creates/removes a register field in the given register at a bit offset of size specified in the -size argument

The columns in this dialog that you use to specify the memory map details are as follows:

- Name
- BankAlignment
- BaseAddress
- MemoryRange
- MemoryWidth
- Description

8.2.2.2 Registers, Register Fields Dialog Section

The Registers, Register Fields section is listed in hierarchical format showing field relationship to each register. The column views in the spreadsheet view are editable (except for the Name column), to allow modifying structure information. This dialog section has the following buttons to help you create these objects.

Table 8-2 Memory Map Register and Register Field Buttons

Button Name	Field ID	Corresponding Command	Description
Add Register Field...	RF	create_register_field	Creates/removes a register field in the given register at a bit offset of size specified in the -size argument
Delete Item		remove_memory_map remove_address_bank remove_address_block remove_register remove_register_field	Depending on the selected item, the specific memory item is removed

The fields that you use to specify the Register and Register Field details are as follows:

- Name
- MemoryAccess
- AddressOffset
- BitAddressOffset
- RegisterSize
- RegisterResetValue
- Description

8.2.2.3 Creating Memory Maps

To create a Memory Map, click on the buttons above the spreadsheet fields of the dialog box, and fill in the column information for each of the row entries.

Use the **Add Memory Map** button to create an initial map (MP). A dialog box appears to supply the map name and map description (documentation).

To add an Address Bank, select the “MP” memory map row, and click **Add Address Bank**. Supply a bank name, base address, alignment, and description. When added, the new address bank (BN) appears hierarchically below the map.

You can add an Address Block to an Address bank: First select the Bank (BN row) and then click **Add Address Block**. In the dialog, supply the name, range, width, and an optional description.

Use the **Add Register** button to add a register to an Address Bank or Address Block. The “Create a new register” dialog asks for specific register information: name, offset, size, and optional description.

If the register does not immediately appear in the lower section, first select the object in the upper view that the register is attached to, and then click **Refresh Views**. You should see the register object (RG) appear in the bottom view.

Use the Add Register Field button above the lower view to create register fields. First select the register (RG) row, and then click **Add Register Field**. The “Create a new register field” dialog ask for name, offset, size and description.

8.2.3 Advanced Memory Map Attributes Activity

The Advanced Memory Map Attributes Activity give you an interface to set attributes for a memory map, address block, address bank, register, and register field. A row of tabs (one for each of the object types) allows you to view and assign attributes to a single object type at a time, making specifying attributes a simpler process. The tabs are:

- Memory Map Attributes
 - BitsInLAU
 - Volatile Memory
- Address Block Attributes
 - Vendor Extensions
 - MemoryUsage

- ❑ VolatileMemory
 - ❑ Endian -- big or little endian
- Address Bank Attributes
 - ❑ Vendor Extensions
 - ❑ MemoryUsage
 - ❑ VolatileMemory
- Register and Register Field Attributes
 - ❑ Visible
 - ❑ SideEffects
 - ❑ VolatileMemory
 - ❑ Vendor Extensions
 - ❑ RegisterResetMask
 - ❑ HeaderNameFormat[RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset, MemoryAccess]
 - ❑ VerilogHeaderValue[RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset, MemoryAccess]
 - ❑ CHeaderValue[RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset, MemoryAccess]

For a detailed description of these attributes, see [Memory Map Attributes](#) on page 118.

8.2.4 Memory Map Documentation Activity

The report_memory_maps command or the Reports tab within the Memory Map Activity lets you generate a report on memory maps in a component.

8.2.5 Using Memory Maps Notes

- Because of the hierarchical relationship, some buttons are grayed out until you have created or selected the appropriate “parent” item. For example, you must create a Memory Map before you can create a Memory Block.
- You must select a Memory Block before you can add or view the registers associated with that block in the register view.
- Values are specified in default units, which may be decimal or hex. If entry is decimal, it is converted to and displayed as hex for ease of viewing.
- Memory validity checks are not performed until the Define Memory Maps activity is applied.
- Advance Memory Map Attributes are not checked until this activity is applied.

When you apply the Memory Maps activity, the memory structures are checked for range and overlap values, and unique naming. A successful memory map displays the Report tab, where you can generate the Memory Map Report, which contains the structural details of your map.

If you need to change your memory map, click on the Memory Maps activity again and add, delete or change any of the map structure.

8.2.6 Memory Map Command Interface

Table 8-3 describes the commands you use to create and manipulate the memory map. For command details, syntax, and allowed values, use the command help from within the coreBuilder tool.

Table 8-3 Memory Map Commands

Command	Associated Attribute	Description
create_memory_map remove_memory_map get_memory_map_attribute set_memory_map_attribute	MemoryMap	Creates/removes a memory map with the given name A memory map is a container used to hold different address regions.
create_address_bank remove_address_bank get_address_bank_attribute set_address_bank_attribute	AddressBank	Creates/removes an address bank You must specify either -map or -bank. If you specify -map, then also specify the -base_address option; otherwise, the BaseAddress is automatically computed from the rules of the address bank.
create_address_block remove_address_block get_address_block_attribute set_address_block_attribute	AddressBlock	Creates/removes an address block in the specified memory map or address bank
create_register remove_register get_register_attribute set_register_attribute	Registers RegisterSize	Creates/removes a register in the given address block at the specified offset (in bytes) The size in bits of the register is specified by the -size argument.
create_register_field remove_register_field get_register_field_attribute set_register_field_attribute	RegisterField	Creates/removes a register field in the given register at a bit offset of size specified in the -size argument
create_register_field_value remove_register_field_value get_register_field_value_attribute set_register_field_value_attribute	RegisterFieldValues	Creates/removes a register field value These “values” describe the value that a register field may take.
report_memory_maps		Generates an HTML or text report of the currently defined memory maps, memory registers, and register fields Use the -type option to determine whether you are generating HTML or text.

8.3 Memory Map Attributes

The preceding register commands create attributes in your design that are read and used by subsequent tools to set up the required memory. [Table 8-4](#) describes the Memory Map attributes.

Table 8-4 Memory Map Attributes

Attribute	Valid On	Description
AddressOffset	register	Specifies the address offset from the base address of the memory map in bytes for registers
BankAlignment	addressBank	Describes how the blocks and banks within this bank are organized in memory If the value is “serial”, then any contained blocks or banks are all organized sequentially in memory. If the value is “parallel”, then any contained blocks or banks are all organized in parallel.
BaseAddress	addressBank addressBlock	Specifies the base address of an address block or address bank
BitAddressOffset	addressBank addressBlock registerField	Specifies the offset from the base address of the address block of the first register in the address block It is also used to specify the offset in bits for this register field from the start of the register.
BitsInLAU	memMap	Specifies the concept of a Least Addressable Unit (LAU) or “word” size The default is byte addressable (8).
CHeaderValue[]	register registerField	Allows developers to specify how the value associated with a register/register field is displayed in the generated header file Subscripted attributes are: RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset and MemoryAccess.
Endian	addressBlock	Defines the memory layout for this address block Are the elements in this address block big or little endian. Valid values are “big”, and “little”. The default endian value is “big”.
HeaderNameFormat[]	register registerField	Controls how the name of a register (or register field) is displayed in the header files that are created in the ./export/headers directory by the coreConsultant tool This is a subscripted attribute that includes: RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset and MemoryAccess.
MemoryAccess	addressBank addressBlock memMap register	Specifies the access to this memory map or register Valid values are read-write, read-only, write-only, writeOnce, and read-writeOnce.

Table 8-4 Memory Map Attributes (continued)

Attribute	Valid On	Description
	registerField	
MemoryMap	interfaceInstance	Holds the name of the memory map that this slave interface refers to
MemoryRange	addressBank addressBlock	Specifies the address range of the memory element This value is expressed as the number of addressable units of size BitsInLAU for the containing memory map.
MemoryUsage	addressBank addressBlock	Specifies the usage of the memory map Possible values are memory, register, and reserved.
MemoryWidth	addressBank addressBlock	Specifies the bit width of the address block or bank
RALAccessType	register registerField	Defines the VMM RAL access type for this register field In most cases, this is calculated through the MemoryAccess, WriteBehavior, and ReadAction values for the field. This is typically only set explicitly when the a* or user* types are desired.
ReadAction	register registerField	Indicates that reading the given field causes the field value to be modified as described by the value of this attribute Valid values are: clear, set, and modify.
Registers	addressBlock	A read-only attribute that returns a collection holding all the visible registers defined for this memory map
RegisterFields	register	The visible fields for this register
RegisterFieldValues	registerField	Returns the visible values from this register field, which can depend on the configuration
RegisterResetMask	register	Specifies the value to use as a mask when comparing against the RegisterResetValue The default value is all ones.
RegisterResetValue	register	Specifies the value of this register in the reset state
RegisterSize	register registerField	Specifies the size of the register or register field in bits
SideEffects	register registerField	Indicates whether or not writing/reading this field or register has any side effects Values are 'true' and 'false'.
VerilogHeaderValue[]	register registerField	Allows developers to specify how the value associated with a register/register field is displayed in the generated header file

Table 8-4 Memory Map Attributes (continued)

Attribute	Valid On	Description
		The subscribed attributes include: RegisterSize, RegisterResetMask, RegisterResetValue, AddressOffset, BitAddressOffset and MemoryAccess.
Visible	register registerField	Sets this parameter to be visible in the GUI
VolatileMemory	memoryMap addressBlock addressBank register registerField	Allows for some additional mechanism by which registers can acquire new values other than reads/writes/resets and other access methods.
WriteBehavior	register registerField	Indicates that writing the given field causes a write of a value other than the one written The written value will be modified as described by the value of this attribute. Valid values are: oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle, clear, set, and modify.
Testable	registerField	The testable attribute indicates if the field can be tested with automated register routine. The presumed value of testable is true if not specified. If the testable attribute is present, the following is the IP-XACT description: ■ untestable: This maps to IP-XACT testable as set to false. The field will skip all the tests ■ unconstrained: Read and write all legal values ■ restore: Read and write legal values but the value must be restored to the initially read value before accessing another register ■ writeAsRead: Value read before a write may be written to the field ■ readOnly: Values may only be read from the field

8.3.1 Endianness

Endianness is defined by the Endian attribute. There are two legal values ('big' and 'little') to specify your endianness:

- Big endian -- the most significant byte of any multi-byte data field is stored at the lowest memory address, which is also the address of the larger field.
- Little endian -- the least significant byte of any multi-byte data field is stored at the lowest memory address, which is also the address of the larger field. Two ways exist for big-endianness, byte invariant and word invariant:
 - Word invariant -- a word access to address 0x0 is on D[31:0]. The MS byte will be on D[31:24], the LS byte will be on D[7:0].
 - Byte invariant -- a word access to address 0x0 is on D[31:0]. The MS byte will be on D[7:0], the LS byte will be on D[31:24].

The interpretation of big-endian is the byte invariant style. The above discussion is byte centric as that is the most common BitsInLAU. However, the discussion in general applies to any size BitsInLAU.

8.3.2 Memory Map Compatibility with IP-XACT

When writing out a coreBuilder-packaged component in IP-XACT format from a coreTools application, memory map intent is preserved in the IP-XACT XML description.

When importing an IP-XACT compliant component containing memory map descriptions into the coreBuilder tool, the memory map information is preserved and populates the Memory Map activity with data.

When importing an IP-XACT compliant component containing memory map descriptions into the coreAssembler tool, the memory map information is preserved in the workspace, whether or not it is specifically used by the tool. In other words, the coreAssembler tool does not block IP-XACT memory map content, but preserves it.

8.4 Memory Map Examples

The following examples show how to create memory maps using the commands in [Table 8-3](#) on page 117 and how to modify these maps using memory map attributes described in [Table 8-4](#) on page 118.

8.4.1 Example 1: Simple Memory

Here is a sample of a MemoryMap element definition for a simple memory (with address map 0x0000 to 0x0FFF). Only the name (`my_memory`), the `baseAddress` element (0x0000) and the range element (`memory size`) are mandatory. Other parameters are optional.

```
create_memory_map my_memory
create_address_block block1 -map my_memory -base_address 0x0000 -range 4096
-width 8
...
```

8.4.2 Example 2: Relative Base Address

The base address in a multi-block definition is dependent on the `base_address` and range of the first block in the MemoryMap. This dependency is specified by not assigning it a value. In the example, the `base_address` and range of the previous memory map is used to compute the second `base_address`.

```
create_memory_map -name mmap
set_memory_map_attribute mmap MemoryAccess read-write
create_address_block block1 -map mmap -base_address 0 -range 786432 -width 32
create_address_block -map mmap -range 4096 -width 32 block2
```

8.4.3 Example 3: Memory Banks

Banks allow multiple address groups. If it is a parallel bank, each address block under the bank is understood to be located at the same address with different bit offsets to accommodate the widths of the preceding blocks.

If it is a serial bank, the first block under the bank is understood to be located at the bank's address, the next block is at the bank's address plus length of the first block, and so forth.

In the following example, the only address specified is 0x10000, but it causes memory blocks `ram0`, `ram1`, `ram2` and `ram3` to be mapped to addresses 0x10000, 0x14000, 0x18000 and 0x1C000 respectively.

```
create_address_bank -alignment serial base_address 0x10000 Bank0
create_address_block -bank mmap/Bank0 -range 0x1000 -width 32 ram0
create_address_block -bank mmap/Bank0 -range 0x1000 -width 32 ram1
create_address_block -bank mmap/Bank0 -range 0x1000 -width 32 ram2
create_address_block -bank mmap/Bank0 -range 0x1000 -width 32 ram3
```

In the following example, memories `ramA`, `ramB`, `ramC` and `ramD` are all mapped to address 0x10000, but at bit offsets 0, 8, 16 and 24 respectively using parallel offsets.

```
create_address_bank -alignment parallel -base_address 0x10000 Bank1
create_address_block -bank mmap/Bank1 -range 0x1000 -width 8 ramA
```

```
create_address_block -bank mmap/Bank1 -range 0x1000 -width 8 ramB
create_address_block -bank mmap/Bank1 -range 0x1000 -width 8 ramC
create_address_block -bank mmap/Bank1 -range>0x1000 -width 8 ramD
```

In some cases the parallel example could be described by a single memory block, but there are a number of cases where tools need to know more details about how a memory bank is setup. For instance, if the memory bank is ROM, and the tools need to understand the setup to understand how to setup the ROM image.

8.4.4 Example 4: Register Definition

The following register definition demonstrates many of the register commands and attributes.

```
create_register -block mmap/block1 -name VectCnt -offset 0x200 -size 32
set_register_attribute mmap/block1/VectCnt MemoryAccess read-write
set_register_attribute mmap/block1/VectCnt RegisterResetValue 0x0
set_register_attribute mmap/block1/VectCnt Description
(This is the interrupt vector control)

create_register_field -register mmap/block1/VectCnt -name E -width 1 -offset 5
set_register_field_attribute mmap/block1VectCnt/E MemoryAccess read-only
set_register_field_attribute mmap/block1VectCnt/E Description >
(Set if any other bits are set)

create_register_field -register mmap/block1VectCnt -name IntSource -offset 0
-width 5
set_register_field_attribute mmap/block1VectCnt/IntSource MemoryAccess
read-write
set_register_field_attribute mmap/block1VectCnt/IntSource Description >
(Set if interrupt occurred)
```

So although the register is 32 bits wide, only the first 6 bits have been defined with bit fields.

8.4.5 Example 5: Register Field Values

The following register field contains explicitly defined field values:

```
set f [create_register_field -register mmap/block1/VectCnt -name E -width 1
-offset 5]
create_register_field_value OFF -field $f -value 0x0 -description "In the off
state"
create_register_field_value ON -field $f -value 0x1 -description "In the on
state"
```


9

Specifying Synthesis Intent

The synthesis intent activities enable you to interactively view and modify your synthesis intent specifications. You must complete the synthesis intent activities even if you use intent command files to specify all of your synthesis intent. If you do use intent command files for all of your synthesis intent (which Synopsys recommends), you can simply click Apply in the dialogs for each of the synthesis intent activities, or use `autocomplete_activity` to complete them in batch mode. This chapter describes how to execute each of the synthesis intent activities.

This chapter contains the following sections:

- [“Specifying Target Technology”](#) on page 126
- [“Specifying Clocks”](#) on page 128
- [“Specifying Synthesis Methodology”](#) on page 130
- [“Specifying Test Methodology”](#) on page 131
- [“Specifying Timing Exceptions”](#) on page 135
- [“Specifying Port Constraints”](#) on page 139
- [“Specifying Power Intent ”](#) on page 140
- [“Verifying Intent ”](#) on page 142
- [“Area Estimation Modeling”](#) on page 144

9.1 Specifying Target Technology

The coreBuilder tool uses a default reference technology to determine the units used for spreadsheet display and to validate the technology-independent formulas that you use to specify synthesis intent.

For most coreKits, you can skip the Specify Reference Technology activity and let the coreBuilder tool use the default technology library. As long as you specify synthesis intent in a technology-independent manner, your coreKit remains technology-independent.

If you plan to specify synthesis intent in a technology-dependent manner (for example, if you use a specific cell name as the value for the DrivingCell attribute), you need to specify the technology library that contains that cell as your reference technology. In addition, you must ensure that users of the coreKit also use the same technology library.

9.1.1 Specifying Reference Technology in GUI Mode

When you select the Specify Target Technology item in the Activity List pane, the Library Setup tab appears in the Activity View pane. Prior to showing this pane for the first time, your setup file (.synopsys_dc.setup) is read to determine initial values for the fields.

The Dialog tab of the Specify Target Technology activity contains the search path and libraries associated with dc_shell-t and fpga_shell. These parameters are defined in [Table 9-1](#).

Table 9-1 Library Setup Parameters

Activity Parameters	Description
search_path	Include the path to your technology library and any other libraries required for your design
target_library	Specify the names of your technology library files If your target library uses a separate file to characterize the library cells for best-case operating conditions (minimum timing), specify this file in the Min Timing Library field rather than in the target_library variable definition.
link_library	Include the names of your technology library files and any other libraries required to link your design For the names of libraries that must be linked to synthesize the core, see the documentation supplied by the core developer.
Min Timing Library	If your technology library uses a separate file to characterize the library's cells for best-case (minimum timing) operating conditions, enter the name of this .db file in the Min Timing Library field. This causes the generation of a Design Compiler set_min_library command, thus enabling Design Compiler to evaluate minimum and maximum timing simultaneously. You can also specify this parameter as a list of the form {{<worst case library A> <best case library A>} {<worst case library B> <best case library B>} ...}. List specification of this parameter supports multiple min-timing libraries. A min-timing library can be associated with any library currently specified in the target or link library lists.

9.1.2 Setting Tool Installation Roots

If the entry fields are not enabled, it means that you do not have a path defined for access to dc_shell-t. To resolve this problem, specify the tool installation areas in the Tool Installation Roots dialog, which is accessed from the toolbar menu through **Edit > Tool Installation Roots**. Enter values directly in the data fields or use the buttons to rearrange, delete, or add values.

9.1.3 Specifying Reference Technology in Command Line Mode

To complete the Specify Reference Technology activity in command line mode, use the following commands, as shown in [Example 9-1](#):

- `set_activity_incomplete` – Set the state of the activity to incomplete
- `set_activity_parameter` – Set values for activity parameters
- `autocomplete_activity` – Complete the activity

Example 9-1 Completing TechSetup in Command Line Mode

```
set_activity_incomplete TechSetup
set_activity_parameter TechSetup LinkLibrary \
{* core_slow.db cells_slow.db dw_foundation.sldb}
set_activity_parameter TechSetup dc_shellEnabled 1
set_activity_parameter TechSetup dc_shellRoot /tools/synopsys/2000.11
set_activity_parameter TechSetup SearchPath \
{. /tools/synopsys/2000.11/libraries/syn /tools/libraries/tech_lib}
set_activity_parameter TechSetup TargetLibrary {core_slow.db cells_slow.db}
autocomplete_activity TechSetup
```

The following commands are also helpful when executing activities in command-line mode:

- `report_activities` – List the names and current status of all coreBuilder activities
- `report_activity_parameters` – List the names and descriptions of activity parameters

9.2 Specifying Clocks

To specify clock intent in GUI mode, select the Specify Clocks activity in the coreBuilder Activity List. In the Specify Clocks dialog which appears in the Activity View pane, select a design from the Design pulldown menu, then enter values for the clock attributes directly in the appropriate row of the spreadsheet.



Note

When you initiate the Specify Clocks activity, the coreBuilder tool automatically selects the top-level design as `current_design`. If all clocks in your design are driven from top-level ports, then you only need to specify clock intent for the top-level design.

The Specify Clocks dialog only lists clocks that you have previously created, either through the GUI or through clock intent commands in the design intent command file.

To add a clock to a design through the Specify Clocks dialog, select the design, then click the Add... button to bring up the Add Clock dialog. To create a real clock select Real Clock and click the Next button to select an input or output port from the port list and enter a name (ClockName) for the clock. To create a virtual clock, select Virtual Clock and click the Next button to enter a name for the virtual clock. After you click Finish, the new clock appears in the Specify Clocks dialog, where you can then enter values for the clock attributes.

The ClockName attribute can be set on a bit of a bus, or single port, but not on a port that contains multiple bits. The Add Clocks dialog lists individual bits of a port as potential clock sources.



Note

It is not possible to represent clocks on bits of a bus in IP-XACT versions P1685-2009 and earlier. Bit-level clock definitions and timing constraints associated with bit-level clocks are not included in the generated IP-XACT component file if writing a schema version older than 1685-2014.

On the Real and Virtual Clocks tab, specify values for each of the attributes listed in [Table 9-2](#). Do this for each clock in the design. The coreBuilder online reference provides detailed descriptions of these attributes.

Table 9-2 Clock Attributes

Attribute	Description
CycleTime	Clock period time
Waveform	Rise and fall edge times of the clock over an entire clock period Timing units default to library time units, and the default value for Waveform is: {0 CycleTime/2}.
TestClock	Clock to be used in test mode
TestClockCycleTime	Cycle time to be used for a clock in test mode
TestClockWaveform	Waveform for this clock in test mode
FixHold	Whether to fix hold-time violations for the selected clock

Table 9-2 Clock Attributes (continued)

Attribute	Description
	If FixHold is true, a set_fix_hold command is generated, which causes Design Compiler to insert a delay, if necessary, to fix hold time violations at registers driven by the selected clock.
ClockRiseLatency/ ClockFallLatency	Clock signal propagation from the clock definition point to the clock pin of a register, for a rising or falling transition, respectively, at the clock definition point
ClockSourceRiseLatency/ ClockSourceFallLatency	Clock signal propagation from its actual ideal waveform origin point to the clock definition point, for a rising or falling transition (respectively) at the waveform origin point
ClockSetupUncertainty/ ClockHoldUncertainty	Setup and hold uncertainties, respectively, applied to all endpoints of a specified clock
MinTransitionDelay/ MaxTransitionDelay	Minimum and maximum transition times, respectively, for an ideal clock

Numeric values can be entered as fixed numeric values, fixed numeric values with a time unit, or as a formula related to the cycle time. When time units are not specified, the units of the main target library are used. For example, assuming the technology library units are 'ns' and assuming a 10 ns cycle time, then each of the following entries would specify the same 10 nano-second value: 10, 10 ns, 10000 ps, =percent_of_period 10.

When you have created the necessary clocks and specified values for the clock attributes, click Apply to complete the Specify Clocks activity.

To specify clock intent in command line or batch mode:

1. Insert clock intent commands in the intent command file for the design as described in "[Writing Intent Command Files](#)".
2. Execute the Load Designs activity to apply the clock intent.
3. Complete the Specify Clocks activity (and any other prerequisite activities) by using the autocomplete_activity command:

```
coreBuilder> autocomplete_activity SpecifyClocks
```

9.3 Specifying Synthesis Methodology

To specify synthesis methodology in GUI mode, select the Specify Synthesis Methodology in the coreBuilder Activity List to display the Specify Synthesis Methodology dialog in the Activity View pane. The Specify Synthesis Methodology dialog displays the current values of synthesis-related design attributes for the top-level design and subdesigns. The top-level design is selected by default.

The Specify Synthesis Methodology dialog groups the design attributes onto the separate tabs listed in [Table 9-3](#). On each tab, use the Design drop-down menu to select a design, then view and, if necessary, modify the attribute values that appear in the spreadsheets.

Be aware that attribute values that you specify through the GUI override the corresponding attribute values in your intent command files. If you repeat the Load Designs activity, coreBuilder re-applies the attribute specifications from your intent command files; effectively removing your interactively specified intent. (To implement your interactively specified intent in an intent command file, see [“Importing Intent Specified in GUI Mode”](#) on page 93)

To complete the Specify Synthesis Methodology activity in command line or batch mode:

1. Insert `set_design_attribute` commands in the intent command file for the design as described in [“Writing Intent Command Files”](#).
2. Execute the Load Designs activity to apply the synthesis methodology.
3. Complete the Specify Synthesis Methodology activity (and any other prerequisite activities) by using the `autocomplete_activity` command:

```
coreBuilder> autocomplete_activity SpecifyDesignIntent
```

Table 9-3 Specify Synthesis Methodology Tabs

Tab Name	Description
Strategy Control	Attributes that determine the design implementation frontier (the subdesigns to be compiled individually) and estimated area
Description	Attributes that describe characteristics of designs that determine what optimization algorithms to apply
Synthesis Directives	Attributes that control the generation of synthesis directives
Synthesis Controls	Attributes for script customization and other synthesis controls
Formal Verification	Specify attributes associated with Formal Verification
Static Timing Analysis	Specify attributes associated with Static Timing Analysis (StaticTimingAuxScript)
Generate Test Vectors	Attributes that provide data for synthesis of FPGA designs

9.3.1 UPF Packaging

The attribute `UpfFile`, found in the “Synthesis Controls” tab, lets you associate a UPF file with a design in coreBuilder. Setting the `UpfFile` attribute will cause coreBuilder to automatically include the referenced file in the coreKit so that it is available in the coreConsultant and coreAssembler tools. As with other text files, the `UpfFile` can be defined to be configurable.

9.4 Specifying Test Methodology

In this activity you set and review Design for Test attributes.

When you select the Specify Test Methodology activity on the Activity List, the corresponding Activity View pane appears. The Test Methodology Activity View contains the following tabs:

- [“General Test Methodology Attributes”](#)
- [“Scan Insertion”](#)
- [“Auto Fix”](#)
- [“Test Wrappers”](#)
- [“Wrapper Integration”](#)
- [“Built-in Self Test”](#)
- [“Testpoint Insertion”](#)

9.4.1 General Test Methodology Attributes

[Table 9-4](#) provides information on common scan test attributes.

Table 9-4 General Test Attributes

Attribute	Description
DftAuxSynthesis Script	Specifies the name of the script to use to perform insert_dft The script may do all of the setup for DFT and run all of the commands, in which case InsertDft should be set to false. The script may also be used to issue more setup commands in addition to those selected in the DFT options below. If InsertDft is set to true and a script is specified, the script will run after all of the setup commands have been issued, but just before running insert_dft.
ScanSubblocksIndividually	Causes scan insertion to be performed on the subdesigns of this design Default Value = 1 for top design, 0 elsewhere
ScanBlockIndividually	Causes scan insertion to be performed on this design Default Value = expr {[InheritValue up -attr ScanSubblocksIndividually 0] [get_attribute %item -attr WrapBlockIndividually] [get_attribute %item -attr BistBlockIndividually]}

9.4.2 Scan Insertion

[Table 9-5](#) provides information on scan insertion attributes.

Table 9-5 Scan Insertion Attributes

Attribute	Description
ScanMethodology	Scan style to be used Enum values = full_scan, none

Table 9-5 Scan Insertion Attributes (continued)

Attribute	Description
	Default Value = InheritValue up full_scan
Scan Style	Sets the test scan implementation style for this design Enum values = multiplexed_flip_flop, clocked_scan, lssd, aux_clock_lssd, combinational, none. Default Value = multiplexed_flip_flop
Scan Exclude	List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells

9.4.3 Auto Fix

Table 9-6 provides information on auto fix test attributes.

Table 9-6 Auto Fix Attributes

Attribute	Description
EnableDftAutoFix	Enables the DFT Compiler Auto Fix utility Default Value = InheritValue up FALSE
AutoFixClocks	Enables/disables automatic fixing of uncontrollable clock violations Default Value = InheritValue up TRUE
AutoFixAsync	Enables/disables automatic fixing of asynchronous preset/clear violations Default Value = InheritValue up TRUE
AutoFixAsyncWithScanEnable	When TRUE, the test scan enable signal will be used to autofix asynchronous set and reset violations Default Value = InheritValue up FALSE
AutoFixAsyncLogicGate	When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal Enum Values = mux, gate Default Value = InheritValue up MUX
EnableDftShadowLogic	Enables the DFT Compiler Shadow LogicDFT for this design Default Value = InheritValue up FALSE

9.4.4 Test Wrappers

Table 9-7 provides information on test wrapper attributes.

Table 9-7 Test Wrapper Attributes

Attribute	Description
WrapBlockIndividually	Causes test wrapper insertion to be performed on this design Default Value = xdbist
WrapSubblocksIndividually	Causes test wrapper insertion to be performed on the subdesigns of this design Default Value = 0
WrapperDefaultSafeState	Specifies the default safe state for the wrapper cells Enum Values = 0, 1, none Default Value = none

9.4.5 Wrapper Integration

[Table 9-8](#) provides information on wrapper integration attributes.

Table 9-8 Wrapper Integration Attributes

Attribute	Description
IntegrateTestWrappers	Specifies that all wrapped subdesigns of this design should be integrated using SOC methodology If set to TRUE, all wrapper control signals of integration-ready subdesigns are hooked up to the top-level ports during the dft insertion process. Also, a test controller is implemented to allow testing of integration-ready subdesigns in various test modes. Default Value = 0
IntegrationModeDecodingStyle	Specifies the type of port decoding to be used for the core integration test controller Enum Values = default, one-hot Default Value = default

9.4.6 Built-in Self Test

[Table 9-9](#) provides information on built-in self test attributes.

Table 9-9 Built-in Self Test Attributes

Attribute	Description
BistBlockIndividually	Causes insertion of BIST logic into this design. Default Value = InheritValue up -attr BistSubblocksIndividually
BistSubblocksIndividually	Causes BIST logic to be inserted into the subdesigns of this design. Default Value = 0

Table 9-9 Built-in Self Test Attributes (continued)

Attribute	Description
BistMode	BIST operations to be performed. Choosing “all” will cause the design to be made bist-ready and the code to be inserted and integrated with the controller. For hierarchical BIST insertion, choose codec_insertion for the lower levels of hierarchy and integration at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design. Enum Values = all, bist_ready, codec_insertion, integration Default Value = all
BistType	Type of BIST to implement. Enum Values = dbist, xbist, sbist Default Value = dbist

9.4.7 Testpoint Insertion

[Table 9-10](#) provides information on testpoint insertion attributes.

Table 9-10 Testpoint Insertion Attributes

Attribute	Description
InsertTestPoints	Controls whether observe and control points are inserted into this design. Default Value = 0
TestabilityMethod	BIST testability to be used. Enum Values = bist, tdvr Default Value = bist

9.5 Specifying Timing Exceptions

In some designs, certain timing paths cannot be characterized by single-cycle timing specifications. Such timing paths are commonly referred to as timing exceptions. As a general design practice, you should avoid point-to-point timing exceptions. However, if your design does contain timing exceptions, you should identify all timing exceptions so that core integrators can achieve optimal synthesis results without having to identify and account for timing exceptions themselves.

The coreBuilder tool allows you to capture timing exceptions as synthesis methodology, either through the Specify Timing Exceptions GUI or in command line mode by using the same timing exception commands used by Design Compiler and PrimeTime.

As a core developer, you can use the coreBuilder timing exception capability to add new timing exceptions, report the timing exceptions currently set on a design, and Remove timing exceptions. The coreConsultant tool provides the same capabilities to the core integrator.

Even if your design has no timing exceptions, you must complete the Specify Timing Exceptions activity to enable the subsequent activities.

9.5.1 Specifying Timing Exceptions in GUI Mode

In GUI mode, use select the Specify Timing Exceptions activity from the Activity List. The Specify Timing Exceptions dialog appears in the Activity View pane, where you can view, edit, add, or delete timing exceptions. The currently existing timing exceptions appear in the spreadsheet at the top of the dialog.

To edit a timing exception:

1. Select the timing exception command in the spreadsheet.
2. Modify the options for the command in the Edit Timing Exception area.
3. Click the Update button at the bottom of the dialog to complete the modification.

To add a timing exception:

1. In the Edit Timing Exception area, select a timing exception command from the Timing Exception drop-down menu.
2. Enter the required To, From, and Through information.
3. Select other options for the timing exception command.
4. Click New to add the timing exception.

To delete a timing exception:

1. Select the timing exception command in the spreadsheet.
2. Click Delete in the Edit Timing Exception area.

9.5.2 Specifying Timing Exceptions in Command Line Mode

If you are familiar with Design Compiler timing exception command syntax, you may prefer to use the commands listed in [Table 9-11](#) to report and edit timing exceptions. (For detailed command syntax, see the online command reference.)

You can enter the commands through the command line interface or include them in an intent command file. If you want to use the Design Compiler find command to find design objects to specify as arguments to the timing exceptions, you must use the SDC syntax and include your timing exceptions command in an intent command file with the filename extension .sdc. You can automatically include the SDC file when

you execute the Load Designs activity or explicitly read the SDC file by using the coreBuilder `read_sdc` command.

After you specify your timing exceptions using the commands described in the following sections, use the `autocomplete_activity` command to complete the Specify Timing Exceptions activity:

```
coreBuilder> autocomplete_activity SpecifyTimingExceptions
```

Table 9-11 Timing Exception Commands

Command	Description
<code>report_timing_exceptions</code>	Lists currently specified timing exceptions
<code>set_false_path</code>	Removes timing constraints from particular paths
<code>set_multicycle_path</code>	Modifies the single-cycle timing relationship of a constrained path
<code>set_max_delay</code>	Specifies a maximum delay target for paths in the current design
<code>set_min_delay</code>	Specifies a minimum delay target for paths in the current design
<code>reset_path</code>	Resets the specified paths to single-cycle timing
<code>set_disable_timing</code>	Disables timing arcs between two defined pins in the current cell

9.5.2.1 Reporting Timing Exceptions

To generate a timing exceptions report, use the following command:

```
report_timing_exceptions [-designs designs ] [-recursive] [-brief]
```

Table 9-12 Options for `report_timing_exceptions` Command

Argument	Description
<code>-designs</code> <i>designs</i>	Lists the designs for which you want to report timing exceptions The default is <code>current_design</code> .
<code>-recursive</code>	Reports timing exceptions recursively in subdesigns
<code>-brief</code>	Reports timing exception objects by name only For each timing exception set, the coreBuilder tool automatically assigns a sequence number, which it uses for tracking. The coreBuilder tool also names each timing exception according to the following system: FP<n>, MCP<n>, MaxD<n>, MinD<n>, RP<n>, or DT<n> Where <n> is the sequence number and the prefix depends on the type of timing exception command.

As [Example 9-2](#) illustrates, if you do not include the `-brief` option, `report_timing_exceptions` returns the current set of timing exceptions in the form of the commands that created those timing exceptions.

The `report_timing_exceptions` output also shows the automatically assigned name for the timing exception (FP0, for example). Sequence numbers are consecutive regardless of the type of timing exception. Thus, a new `set_multicycle_path` exception added to the list in [Example 9-2](#) is named MCP4.

Example 9-2 Output of `report_timing_exceptions` Command

```
coreBuilder> report_timing_exceptions
Design: top
Timing Exceptions:
  set_false_path -name FP0 -from clk
  set_multicycle_path -name MCP1 -to dout -from din
  set_max_delay -name MaxD2 -to dout -from din 100
  set_disable_timing -name DT3 u1
```

9.5.2.2 Adding Timing Exceptions

To add timing exceptions, use the commands listed in [Table 9-11](#) on page 136. For example, to specify a false path from input port `int_1` to output port `idle_out`:

```
set_false_path -from int_1 -to idle_out
```

9.5.2.3 Removing Timing Exceptions

The coreBuilder tool provides two ways to remove a previously set timing exception:

- Use the `remove_item` command to remove the timing exception object.
- To undo, or partially undo, a timing exception command, use `reset_path` (which is the same as the Design Compiler and PrimeTime `reset_path` command).

9.5.2.3.1 Removing a Timing Exception Object

The coreBuilder `remove_item` command deletes a specified item from a design knowledge database. You can use `remove_item` to eliminate a timing exception object that you created with `set_false_path`, `set_multicycle_path`, `set_max_delay`, `set_min_delay`, or `set_disable_timing`. The removal a timing exception object restores to all affected paths the timing that existed prior to the timing exception command.

To remove a timing exception object:

1. Use the `report_timing_exceptions` command to determine the name of the timing object. For example:

```
coreBuilder> report_timing_exceptions
Design: top
Path Exceptions:
  set_false_path -name FP0 -from clk
  set_multicycle_path -name MCP1 -to dout -from din
  set_max_delay -name MaxD2 -to dout -from din 100
  set_disable_timing -name DT3 u1
```

2. Remove the timing exception object. For example:

```
coreBuilder> remove_item [find_item MCP1]
```

9.5.2.3.2 Resetting a Timing Path

The coreBuilder `reset_path` command resets a timing exception created by `set_false_path`, `set_multicycle_path`, `set_max_delay`, or `set_min_delay`. The `reset_path` command creates a new timing exception object in the design knowledge database, and assigns it the next available sequence number. During synthesis, the timing exception commands are executed in sequential order so that the `reset_path` command restores the default single-cycle timing to the specified path.

You can also use `reset_path` to narrow the scope of a previous timing exception command. For example, if you executed a `set_false_path` command with multiple through objects specified by a single `-through` option, the through objects are combined as a logical OR. If you execute `reset_path` with the same options, but fewer through objects, you are, in essence, removing paths through objects from the set of paths marked false.

9.6 Specifying Port Constraints

To specify port constraints in GUI mode, select the Specify Port Constraints activity from the coreBuilder Activity List. The Specify Port Constraints dialog appears in the Activity View pane. The Specify Port Constraints dialog displays the current values of port attributes for the core's top-level design and subdesigns. The top-level design is selected by default.

The Specify Port Constraints dialog has separate tabs for Input and Output ports. Within each tab, you can select sub-groups if you have previously created sub-groups. (See [“Creating Port Groups”](#) on page 90.)

On each tab of the Specify Port Constraints dialog, use the Design drop-down menu to select a design, then view and, if necessary, modify the attribute values that appear in the spreadsheets. For descriptions of individual attributes, see the coreBuilder online reference.

To view (and possibly modify) attribute values for individual bits of a bus, select a bus port and click Set Bit-level Intent. Then, edit the attribute values in the Bit-level Intent spreadsheet.

Be aware that attribute values that you specify through the GUI override the corresponding attribute values in your intent command files. If you repeat the Load Designs activity, coreBuilder re-applies the attribute specifications from your intent command files; effectively removing your interactively specified intent. (To implement your interactively specified intent in an intent command file, see [“Importing Intent Specified in GUI Mode”](#) on page 93.)

When you are finished specifying your port constraints, click Apply to complete the Specify Port Constraints activity.

To complete the Specify Port Constraints activity in command line or batch mode:

1. Insert `set_port_attribute` commands in the intent command file for the design as described in [“Writing Intent Command Files”](#).
2. Execute the Load Designs activity to apply the port constraints.
3. Complete the Specify Port Constraints activity (and any other prerequisite activities) by using the `autocomplete_activity` command:

```
coreBuilder> autocomplete_activity SpecifyPortIntent
```

9.7 Specifying Power Intent

The Specify Power Intent activity enables you to capture power intent for your design. The power intent is specified in one or more files containing Unified Power Format (UPF) commands. The coreBuilder tool extracts information from select UPF commands in the power intent files to allow the user to customize the power intent in the Specify Power Intent activities in the coreConsultant and coreAssembler tools (for more information, see the *coreConsultant User Guide* and *coreAssembler User Guide*). The Specify Power Intent activity adds the power intent files to a filegroup called “UPF”, which is unloaded into a top-level “upf” directory when the coreKit is used in the coreConsultant tool.

When you select the Specify Power Intent item in the Activity List pane, the Specify Power Intent dialog appears in the Activity View pane. The power intent files are added in the Power Intent Files entry of the dialog. The dialog section labeled UPF Cells contains check boxes to indicate that either a single isolation cell, single level shifter cell, or single retention cell specification is desired across the entire UPF files. The activity parameters associated with the Specify Power Intent activity are described in [Table 9-13](#).

Table 9-13 Specify Power Intent Activity Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Power Intent Files (IntentFileNames)	List of power intent files
Use One Isolation Cell (UseOneIsolationCell)	Use only one isolation cell in the UPF files Default is true.
Use One Level Shifter Cell (UseOneLevelShifterCell)	Use only one level shifter cell in the UPF files Default is true.
Use One Retention Cell (UseOneRetentionCell)	Use only one retention cell in the UPF files Default is true.

Upon completion of the Specify Power Intent activity, the power intent files are processed to extract information from select UPF commands and to record information about any IncludeIf reuse-pragmas present in the files. This information is used in the Specify Power Intent activity in the coreConsultant and coreAssembler tools. The UPF commands and information stored by the coreBuilder tool are described in [Table 9-14](#).

Table 9-14 Commands Analyzed by Specify Power Intent

UPF Command	Description	Values
add_port_state	The voltage level for each of -state options (except for voltage level "off") is stored for the specified port.	supply_port_name state_name state_value
add_power_state	The voltage level for the given supply set name and state name are stored for net state of FULL_ON in the supply expression.	supply_set_name state_name supply_expr

Table 9-14 Commands Analyzed by Specify Power Intent (continued)

UPF Command	Description	Values
map_isolation_cell	The isolation cell list is stored for the indicated domain and isolation strategy.	isolation_strategy domain lib_cells
map_level_shifter_cell	The list of level shifter cells for the given domain and strategy are stored.	level_shifter_strategy domain lib_cells
map_retention_strategy	The list of retention cells for the specified domain and retention strategy is saved.	retention_strategy domain lib_cells

9.8 Verifying Intent

The Verify Intent activity performs two types of checks to determine whether your synthesis intent specifications are complete and consistent. This enables you to detect and correct errors in constraint specifications before you attempt to synthesize the design.

Do not confuse the Verify Intent activity with functional verification of the core or with timing verification.

9.8.1 Performing Intent Verification

To execute the Verify Intent activity:

1. Select the Verify Intent activity from the Activity List.
2. In the Verify Intent dialog which appears in the Activity View pane:
 - a. Select which checks you want to perform (see the descriptions of the checks in the following sections). Both the completeness and consistency test are enabled by default.
 - b. Click Choose Configuration to select a core configuration on which to perform intent verification. If your core includes conditionally generated ports or cells, select a configuration that includes all conditionally generated ports and/or cells so that Verify Intent checks attribute settings on all possible ports in the design.



Note

The Default button in the Verify Intent dialog sets default values for the selected checks (both completeness and consistency checks selected). The Default button in the Choose Configuration dialog sets the default values for the core's configuration parameters.

- c. Click Apply.

The coreBuilder tool performs the selected checks on your selected configuration.

3. The Reports tab of the Verify Intent Activity View automatically appears when verification is completed.
4. If either the completeness or the consistency check fails, correct the port attribute settings as needed, then repeat the Verify Intent activity.

To complete the Verify Intent activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands as shown in the following example:

```
set_activity_incomplete VerifyIntent
set_activity_parameter VerifyIntent DoConsistencyCheck true
set_activity_parameter VerifyIntent DoCompletenessCheck true
set_activity_parameter VerifyIntent SettingsForElaboration {{ad_width 4}}
autocomplete_activity VerifyIntent
```

9.8.2 Completeness Check

The completeness check verifies that you have specified all the required synthesis intent attributes for all designs on or above the implementation frontier:

- The CycleTime attribute is set for all clocks in the design.
- All ports have values specified for MaxInputDelay, MinInputDelay, MaxOutputDelay, and MinOutputDelay (depending on the port's directionality).
- All top-level inputs have a DrivingCell value set on them.
- All top-level outputs have PinLoadCount and PinLoadType values set on them.

9.8.3 Consistency Check

The consistency check verifies that all electrically-equivalent ports (ports connected to each other with no intervening logic) have equivalent values for the following attributes:

- MinInputDelay
- MinOutputDelay
- MaxInputDelay
- MaxOutputDelay
- DrivingCell
- PinLoadType

In addition, the consistency check verifies the following conditions for each input or output port:

- $\text{MinInputDelay} \leq \text{MaxInputDelay} \leq \text{CycleTime}$
- $\text{MinOutputDelay} \leq \text{MaxOutputDelay} \leq \text{CycleTime}$

9.9 Area Estimation Modeling

The coreBuilder tool supports storage of area estimation numbers based on configuration and technology. This information is accessible within the coreConsultant and coreAssembler tools for reporting.

Each area estimation value provided as part of the IP packaging is associated with a specific technology and configuration. The technology can be specified using an arbitrary string, to allow IP consumers to use this without any indication of the actual target library being used. IP developers can also tell the coreBuilder tool to indicate a specific technology library if desired. Example technology strings are shown below:

"15, 13, 090 - examples indicating only the feature size

"tsmc_045lp - example showing a specific vendor library

"15-low-power - generic reference to a low power library of a given feature size.

Using "low-power" instead of a vendor specific low power library suffix prevents association with a specific library vendor.

The syntax of the new command is shown below:

```
set_area_estimate -technology <tech string> \
  -configuration <config string> \
  [-label <config label>] \
  <area estimate>
```

- <tech string> is any string as described above
- <config string> is a list of parameter/value sub-lists
- <config label> is an optional description of the configuration
- <area estimate> is the actual area estimation

In addition, a new attribute AreaWeight is defined for parameters to indicate the importance of a parameter with respect to area estimates. This is an integer attribute whose default value is '1'.

The best area estimate for a given configuration is determined by finding the area estimate with the highest 'match weight', where the 'match weight' is calculated by summing the weights of each parameter whose value matches the value within the area estimate.

9.9.1 Area Estimation Example

Consider the following specified parameter weights and area estimates:

```
set_parameter_attribute P1 AreaWeight 2
set_parameter_attribute P2 AreaWeight 0
set_area_estimate 45.0 -tech 090 -config {P1 2}
set_area_estimate 55.0 -tech 090 -config {P1 0 P3 7}
```

This leads to the following "match weights" based on configuration.

Table 9-15 Area Estimation “match weights”

Configuration	Match Weight (1st estimate)	Match Weight (2nd estimate)	Area estimate
P1 = 2 P2 = 4 P3 = 7	2 + 0 + 0	0 + 0 + 1	45.0
P1 = 0 P2 = 4 P3 = 7	0 + 0 + 0	2 + 0 + 1	55.0

In the coreConsultant and coreAssembler tools the Area Estimation report is available for generation on-demand, after component configuration. A report example is shown below.

Table 9-16 Area Estimation Report (coreConsultant)

Area	Technology	Match Weight Percentage	Matching Parameters
10072	090	18.9	0/1
11111	090	17.0	0/2
8000	090	15.4	0/1
20001	090	10.2	0/3
40000	090	5.6	0/4

The rows in the report are sorted by weight. In a real example, there will most likely be different technology labels. The “Matching Parameters” column provides links to separate tables showing the pertinent parameter details for that estimation, as in the table below.

Table 9-17 Area: 10072, Technology: 090, Match Percentage: 18.9

Parameter	Weight	Estimation Value	Actual Value
P1	2	0	1
P3	1	7	7
P2	0		2

A row is shown in the report for each parameter specified in the area estimate. Rows are sorted by weight. Rows with no “Estimation Value” indicate that a parameter was excluded from the estimate. Rows shown in gray indicate parameters that are included in the estimate but which had a mismatch between the user and area estimate in terms of configuration value.

This report is available in the coreBuilder, coreConsultant, and coreAssembler tools. In addition, in the coreBuilder tool, a subsystem level report is generated if there is at least one technology label that is common across all instantiated components. This report includes links to the component level reports and also contain a table with subsystem-level area estimates for each of the technology labels that is defined for every component instance.

10

Checking Your Work in Progress

The coreBuilder tool provides the Create Integration Workspaces activity, which enables you to build and verify the coreKit in its current state. Create Integration Workspaces creates a coreConsultant workspace so that you can check your work up to the current time, even in cases where you have not completed the entire list of coreKit packaging activities.

You can check the results of all activities that you have completed up to this point, including the specification of configuration intent and any coreConsultant plugins you intend to ship as part of the coreKit.

This chapter describes how to use the Create Integration Workspaces activity to check the current the current state of your coreKit.

This chapter contains the following sections:

- [“Create Integration Workspace”](#) on page 148
- [“Test the coreKit in coreConsultant”](#) on page 150

10.1 Create Integration Workspace

The Create Integration Workspaces activity packages the coreKit in its current state, installs it, then creates one or more coreConsultant workspaces where you can validate your work up to this point.



Note

If you plan to test the use of coreConsultant Design Compiler Tcl Mode (dc_shell-t) and/or FPGA synthesis (Synplify) strategies with your core, you must enable the corresponding tools before you create the integration workspace. To do so, set the corresponding options on the Build coreKit activity, see [“Building the coreKit”](#) on page 190.

To create one or more integration workspaces:

1. Select the Create Integration Workspaces activity from the Packaging section of the coreBuilder Activity List.
2. In the Create Integration Workspaces dialog which appears in the Activity View pane, specify values for the options listed in [Table 10-1](#) and click Apply.
3. To verify your preliminary coreKit using the coreConsultant tool, go on to the next section, [Test the coreKit in coreConsultant](#) on page 150.

Table 10-1 Create Integration Workspaces Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Clean Up Existing Root Directory (CleanupRootDir)	Clean out the selected root directory before creating new workspaces If you select Clean Up Existing Root Directory, the coreBuilder tool removes the existing root directory and all of its contents, then re-creates an empty root directory under the same name. Use this option if you want to overwrite existing workspaces when you repeat the Create Integration Workspaces activity. Clean Up Existing Root Directory is selected by default.
Root Directory (RootDir)	Enter the path to the root directory where you want to install the coreKit and create coreConsultant workspaces. The coreBuilder tool creates a separate subdirectory under the root directory for each workspace. You can either enter the path to the directory in the text entry field or use the directory dialog to navigate to an existing directory and then select it. The default root directory is test in your current coreBuilder workspace.
Start GUI On Workspace (StartGUIOnWorkspace)	Create a single workspace and invoke the coreConsultant tool automatically in GUI mode on that workspace The coreBuilder tool ignores this option if you are creating more than one workspace. Start GUI On Workspace is not selected by default.
Workspace Names (WorkspaceNames)	Enter a space-separated list of names to use for one or more coreConsultant workspaces After packaging the coreKit, the coreBuilder tool invokes the coreConsultant tool to install the coreKit and create the specified workspaces.

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Exclude Filegroups (ExcludeFilegroups)	Enter a space-separated list of user-defined filegroups to be excluded from the Filegroup Names box in the test coreKit You cannot exclude a pre-defined Synopsys filegroup (for example, Synopsys/Hdl).

To complete the Create Integration Workspaces activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands as shown in the following example:

```
set_activity_incomplete CreateIntegrationWorkspaces
set_activity_parameter CreateIntegrationWorkspaces RootDir \
  /usr/cores/MyCore/Builder1/test
set_activity_parameter CreateIntegrationWorkspaces CleanupRootDir true
set_activity_parameter CreateIntegrationWorkspaces WorkspaceNames \
  {MyConfig1 MyConfig2}
set_activity_parameter CreateIntegrationWorkspaces StartGUIOnWorkspace \
  false
set_activity_parameter CreateIntegrationWorkspaces ExcludeFilegroups \
  Documentation
autocomplete_activity CreateIntegrationWorkspaces
```

10.2 Test the coreKit in coreConsultant

To verify your preliminary coreKit, invoke the coreConsultant tool (if not automatically invoked) and load one of the workspaces that you created through the Create Integration Workspaces activity. For example, to invoke the coreConsultant tool and load the workspace MyConfig1 in the root directory test, use the following commands:

```
% cd <path>/test
```

```
% coreConsultant MyConfig1
```

The set of coreConsultant activities that you need to perform depends on which type of synthesis methodology specifications you want to verify: configuration intent, synthesis intent, or customizations (plugins).

For information and procedures to execute the coreConsultant activities you need to execute to check your work, the *coreConsultant User Guide*.

11

Setting Up Configurable Filegroups

You can set up your coreKit so that the coreConsultant tool writes custom files to the core integrator's workspace depending on the core integrator's selection of parameter values. To do so, you must set up configurable filegroups in the coreKit. Configurable filegroups are most commonly used in test suite regression environments (TREs), as described in ["Setting Up Verification Suites"](#). This chapter describes how to set up configurable filegroups in general.

The coreBuilder tool provides special activities in the Verification activity group for setting up TREs. To set up configurable filegroups for use in a TRE, read this chapter to get familiar with the concept of configurable filegroups, then go to ["Setting Up Verification Suites"](#) for specific procedures to set up a TRE.

This chapter contains the following sections:

- ["coreBuilder Filegroups"](#) on page 152
- ["Configurable Filegroups"](#) on page 153
- ["Selective Installation of Files"](#) on page 154
- ["Configuring Contents of Files"](#) on page 157
- ["Setting Up Post-Configuration Commands"](#) on page 162

11.1 coreBuilder Filegroups

A coreBuilder filegroup is a collection of files that serve a common purpose. You include files in a coreKit by grouping the files into filegroups and adding filegroups to the Bill of Materials for the coreKit. When you add a filegroup to the coreKit Bill of Materials, you can also apply attributes to the filegroup and its files that specify when and how the coreConsultant tool installs the files into the core integrator's workspace.

11.1.1 Types of Filegroups

Filegroups are of the following general types:

- Simple filegroup – A filegroup to which you explicitly add files, either through the GUI or by executing the `add_files_to_filegroup` command. A filegroup is either simple or autoloading, as indicated by the `AutoLoadFilegroup` attribute (`false` = simple filegroup).
- Autoloading filegroup – A filegroup to which coreBuilder automatically adds files according to the value of the `DefaultLoadPath` attribute on the filegroup. The coreBuilder tool automatically sets `DefaultLoadPath` according to options you select when you create the autoloading file group. `AutoLoadFilegroup` = `true` for an autoloading filegroup.
- Hierarchical filegroup – A filegroup that contains other filegroups. For example, Synopsys is a hierarchical filegroup; it includes Synopsys/Hdl and other automatically created filegroups.
- Configurable filegroup – A filegroup that is configurable depending on user-specified parameter values. The Hdl filegroup, which the coreBuilder tool automatically includes in the Bill of Materials, is a configurable filegroup. You can also specify other filegroups as configurable. A configurable filegroup can be either a simple filegroup or an autoloading filegroup.

11.1.2 How Filegroups Are Created

Creating and populating filegroups is the focus of the Create Bill of Materials activity, which is described in detail in [“Building the coreKit or IP-XACT Component”](#). For the purpose of the discussion of configurable filegroups, it is helpful to know that you can create a filegroup in any of the following ways:

- Through the BoM Template – the coreBuilder tool automatically creates filegroups that are defined in the BoM Template.
- Through the Create of Bill Materials activity in GUI mode.
- By executing either of the following commands:
 - `add_files_to_filegroup` – Creates a simple filegroup if the specified filegroup does not already exist.
 - `create_autoloading_filegroup` – Creates an autoloading filegroup.

Configurable filegroups are a special case because, in addition to creating and populating the filegroup, you must also specify the manner in which the filegroup is configured in the core integrator's workspace. The remainder of this chapter describes how and why to set up configurable filegroups.

11.2 Configurable Filegroups

Filegroups can be configurable in either of the following ways:

- Selective installation – The coreConsultant tool installs or does not install the filegroup depending on the value of a parameter expression.
- Text substitution – The coreConsultant tool substitutes text (customizes the file contents) for files in the filegroup depending on the values of parameter expressions.

In either case, the parameters involved may be design parameters, parameters associated with an existing activity, or new parameters that you create specifically to configure the filegroup. For filegroup configuration parameters, the coreConsultant tool automatically creates an activity to prompt the user to select values for the parameters.

[Table 11-1](#) summarizes the attributes for configurable filegroups. You can set the attributes in [Table 11-1](#), as well as other filegroup attributes, through the command line interface or through the Create Bill of Materials GUI.

The following sections describe how to use the attributes in [Table 11-1](#) to set up filegroups for selective installation and text substitution. For more detailed descriptions of individual attributes, see the coreBuilder online reference.

Table 11-1 Attributes for Configurable Filegroups

Filegroup Configuration Based on Existing Parameters	Filegroup Configuration Based on Filegroup Configuration Parameters
Required: ■ Configurable = true ■ ConfigActivity = Name of activity associated with the parameters used to configure the filegroup ■ Install = Parameter expression to evaluate to decide whether to install the filegroup, needed only for selective installation of filegroups	Required: ■ Configurable = true ■ ConfigActivity = Name of activity associated with the parameters used to configure the filegroup (if it does not already exist, it is created) ■ ConfigIntentSearchPath = Path to an intent file that creates filegroup configuration parameters When this parameter is specified as a relative path, the non-absolute path is expanded relative to the location of the file for which intent is being specified.
Optional: ■ ConfigDependsOnActivities = Activities that must be completed before configuring the filegroup ■ ConfigDependsOnGroup = Filegroups that must be configured before configuring the filegroup ■ PostPromptCmd = Tcl command to execute after configuring the filegroup	Optional: ■ ActivityGroup = Group into which to place the new filegroup configuration activity (ConfigActivity) ■ ActivitySequence = Sequence number for ConfigActivity. ■ Label = GUI label ConfigActivity ■ ConfigDependsOnActivities = Activities that must be completed before configuring the filegroup ■ ConfigDependsOnGroup = Filegroups that must be configured before configuring the filegroup ■ PostPromptCmd = Tcl command to execute after configuring the filegroup

11.3 Selective Installation of Files

Selective installation of files occurs on a filegroup basis. The coreConsultant tool either installs or does not install the filegroup, depending on the specified condition. The specified condition can involve design parameters, parameters associated with an existing activity, or parameters associated with a new activity to configure the filegroup.

The general procedure is the same in all three cases:

1. Create the filegroup, either a simple filegroup or autoload filegroup.
2. Specify filegroup configuration intent by setting attributes on the filegroup.

You can perform the above steps either through the Create Bill of Materials GUI dialog or by using the following commands:

- `add_files_to_filegroup` or `create_autoload_filegroup`
- `set_filegroup_attribute`

11.3.1 Example – Selective Filegroup Installation Based on an Existing Parameter

To selectively install a filegroup based on the value of a design configuration parameter or existing activity parameter, set the attributes listed in the left column of [Table 11-1](#) on page 153 on the filegroup.

[Example 11-1](#) shows a sequence of commands that creates and populates two filegroups (ROM_group and RAM_group), and sets them up for selective installation based on the user-specified value for design parameter `mem_type`. If `mem_type = 0`, the coreConsultant tool installs the files in RAM_group into the user's workspace/example directory. If `mem_type = 1`, the coreConsultant tool installs the files in ROM_group into the user's workspace/example directory.

You can achieve the same result by creating the filegroups and setting the attributes through the Create Bill of Materials GUI, as described in “[Building the coreKit or IP-XACT Component](#)” on page 181.

Example 11-1 Selective File Installation Based on a Design Parameter

```
create_autoload_filegroup RAM_Group -patterns example/*RAM*.vhd \
-install ./example
load_autoload_filegroup RAM_Group
set_filegroup_attribute RAM_Group Configurable 1
set_filegroup_attribute RAM_Group ConfigActivity SpecifyConfiguration
set_filegroup_attribute RAM_Group Install {=eval_param !@mem_type}
set_filegroup_attribute RAM_Group InstallWhen custom

create_autoload_filegroup ROM_Group -patterns example/*ROM*.vhd \
-install ./example
load_autoload_filegroup ROM_Group
set_filegroup_attribute ROM_Group Configurable 1
set_filegroup_attribute ROM_Group ConfigActivity SpecifyConfiguration
set_filegroup_attribute ROM_Group Install {=eval_param @mem_type}
set_filegroup_attribute ROM_Group InstallWhen custom
```

11.3.2 Example – Creating New Activity for Selective File Installation

To automatically create a new activity in which the core integrator specifies parameter values for selective file installation, set the attributes listed in the right column of [Table 11-1](#) on page 153 on the filegroup.

To create and describe the parameters for the new activity, use the following commands in a configuration intent file for the filegroup:

- `create_configuration_parameter` – Create the configuration parameter and associate it with a filegroup.
- `set_parameter_attribute` – Set attributes on the configuration parameter. Attributes are the same as those used for design parameters and custom activity parameters.

Name the configuration intent file `filename.tcl`, where `filename` is the base name of a file in the configurable filegroup. For example, to set configuration intent for file `example_type1.vhd`, name the configuration intent file `example_type1.tcl`. Set the value of the `ConfigIntentSearchPath` attribute on the filegroup to the directory that contains the configuration intent file.

[Example 11-2](#) on page 155 shows a configuration intent file that creates a configuration parameter (`periph_type`) for selective installation of filegroup `Example/type1_example`.

[Example 11-3](#) shows an example coreBuilder command script to create two filegroups and set them up for selective installation based on the user-specified value for filegroup parameter `periph_type`. The coreConsultant tool automatically creates the activity `GenerateExample` to select a value for `periph_type`. If `periph_type = 0`, the coreConsultant tool installs the files in filegroup `Example/type1_example` into the user's workspace/`example` directory. If `periph_type = 1`, the coreConsultant tool installs the files in filegroup `Example/type2_example` into the user's workspace/`example` directory.

You can achieve the same result by creating the filegroups and setting the attributes through the Create Bill of Materials GUI, as described in [“Building the coreKit or IP-XACT Component”](#) on page 181.

Example 11-2 Configuration Intent File for Selective Installation Example

```
create_configuration_parameter \
  -type boolean \
  -label {Select Peripheral Type} \
  -description {Use type 1 or type 2 peripherals} \
  -default 0 \
  -group Example/type1_example \
  periph_type

set_parameter_attribute periph_type SymbolicNames {"Type 1" "Type 2"}
```

Example 11-3 Selective File Installation Based on a Filegroup Parameter

```
add_files_to_filegroup Example/type2_example \
  -files /usr/cores/MyCore/example/diagram_type2.gif \
  -install ./example/
add_files_to_filegroup Example/type2_example \
  -files /usr/cores/MyCore/example/periph1_type2.vhd \
  -install ./example/
add_files_to_filegroup Example/type2_example \
  -files /usr/cores/MyCore/example/periph2_type2.vhd \
  -install ./example/
add_files_to_filegroup Example/type2_example \
  -files /usr/cores/MyCore/example/example_type2.vhd \
  -install ./example/
```

```

set_filegroup_attribute Example/type2_example Configurable 1
set_filegroup_attribute Example/type2_example ConfigActivity \
GenerateExample
set_filegroup_attribute Example/type2_example ActivityGroup Example
set_filegroup_attribute Example/type2_example InstallWhen custom
set_filegroup_attribute Example/type2_example ConfigIntentSearchPath \
/usr/cores/MyCore/example
add_files_to_filegroup Example/type1_example \
  -files /usr/cores/MyCore/example/example_type1.vhd \
  -install ./example/
add_files_to_filegroup Example/type1_example \
  -files /usr/cores/MyCore/example/diagram_type1.gif \
  -install ./example/
add_files_to_filegroup Example/type1_example \
  -files /usr/cores/MyCore/example/periph1_type1.vhd \
  -install ./example/
add_files_to_filegroup Example/type1_example \
  -files /usr/cores/MyCore/example/periph2_type1.vhd \
  -install ./example/

set_filegroup_attribute Example/type1_example Configurable 1
set_filegroup_attribute Example/type1_example ConfigActivity \
GenerateExample
set_filegroup_attribute Example/type1_example ActivityGroup Example
set_filegroup_attribute Example/type1_example InstallWhen custom
set_filegroup_attribute Example/type1_example ConfigIntentSearchPath \
/usr/cores/MyCore/example

set_filegroup_attribute Example Configurable 1
set_filegroup_attribute Example ConfigActivity GenerateExample
set_filegroup_attribute Example Label {Generate Example}
set_filegroup_attribute Example ConfigDependsOnActivities \
SpecifyConfiguration
set_filegroup_attribute Example ActivityGroup Example
set_filegroup_attribute Example ActivitySequence 900
set_filegroup_attribute Example InstallWhen custom
set_filegroup_attribute Example ConfigIntentSearchPath \
/usr/cores/MyCore/example

analyze_filegroup Example/type1_example
set_filegroup_attribute Example/type1_example \
  Install {=eval_param !@periph_type}
set_filegroup_attribute Example/type2_example \
  Install {=eval_param @periph_type}

autocomplete_activity CreateBillOfMaterials

```

11.4 Configuring Contents of Files

You can set up configurable filegroups so that the coreConsultant tool writes out custom versions of files based on parameter values. For example, you can set up the coreConsultant tool to write out custom code for an example design that instantiates the example peripheral components selected by the user.

To set up files with custom contents, you need to:

1. Enable text substitution in the files by inserting coreBuilder reuse-pragmas into the files.
2. Create a configurable filegroup for the files and set configuration attributes on the filegroup.

For the Synopsys/Hdl filegroup, the coreBuilder tool automatically inserts text substitution reuse-pragmas and sets the required filegroup parameters to allow design configuration based on user-selected values of design configuration parameters.

11.4.1 Enabling Text Substitution in Files

To enable text substitution in a file, you must insert coreBuilder text-substitution pragmas into the file where the text substitution is to occur. You can use three basic syntaxes to enable text substitution in a file:

- To replace the current line with the results of a Tcl command, use:

```
-- reuse-pragma startSub [<Tcl Command>] endSub
```

- To replace the current line and the line below it, use:

```
-- reuse-pragma startSub [<Tcl Command>]
<Line of text to be replaced>
```

- To replace a keyed block of lines, use:

```
-- reuse-pragma startSub MyKey [<Tcl Command>]
<Line of text to be replaced>
<Next line of text to be replaced>
-- reuse-pragma endSub MyKey
```

In the above examples, *<Tcl Command>* can be either of the coreBuilder text-substitution commands listed below (or any other user-defined Tcl command):

- IncludeIf – Include text in a file if the specified parameter expression evaluates to true.
- ReplaceFormatParam – Replace a parameter with the user-specified value in the specified format. ReplaceFormatParam is most commonly used with TRE configuration parameters.

11.4.1.1 IncludeIf Example – Simple

[Example 11-4](#) on page 158 shows a simple example of how to use IncludeIf to conditionally exclude file header information from an HDL source file. The parameter expression for IncludeIf is '0', which means that the conditional text is never be included when the file is written to disk.

Files in the Synopsys/Hdl filegroup are configurable by default. The ConfigActivity is Specify Configuration. Therefore, you do not have to set up a configurable filegroup to implement the file header exclusion in [Example 11-4](#) on page 158. Whenever the coreConsultant tool writes out the file to a core integrator workspace, the file header is not be included.

Example 11-4 Conditionally Including Text in an HDL File

```
--File: TOP.vhd
--Content: entity and architecture for top-level design
--reuse-pragma startSub FileHeader [IncludeIf 0 %subText]
--Designer: Joe Jones
--Revision History:
-- 1.0 1/12/01 Initial version.
-- 1.1 2/13/01 Added test_mode port.
-- 1.2 3/2/001 Added reuse-pragmas.
-- 1.3 4/18/01 Fixed configuration bug.
--reuse-pragma endSub FileHeader
```

11.4.1.2 Includelf Example – Advanced

[Example 11-5](#) shows a more advanced example of how to use `Includelf` with `ElseIncludeIf` to choose which text to include in a text file according to user-specified parameter values. The text file is a configurable file that describes the user's selected configuration of a core.

The parameter expressions in [Example 11-5](#) involve design configuration parameters. For example, `@addr_width==16` evaluates to true if the user sets design configuration parameter `addr_width` to 16. To identify the file as configurable, you must include it in a configurable filegroup.

Example 11-5 Including Parameter-Dependent Text in a Text File

You have selected the following design configuration:

```
reuse-pragma startSub addrbuswidth [IncludeIf @addr_width==8 %subText]
- 8-bit address bus
[ElseIncludeIf @addr_width==16]
- 16-bit address bus
[ElseIncludeIf @addr_width==32]
- 32-bit address bus
reuse-pragma endSub addrbuswidth

reuse-pragma startSub ramsize [IncludeIf @ram_size==0 %subText]
- 256-byte internal RAM type
[ElseIncludeIf @ram_size==1]
- 512-byte internal RAM type
reuse-pragma endSub ramsize

reuse-pragma startSub idlemode [IncludeIf @idle_mode_supp==0 %subText]
- Idle mode is not supported.
[ElseIncludeIf @idle_mode_supp==1]
- Idle mode is supported.
reuse-pragma endSub idlemode
```

[Example 11-6](#) shows how the coreConsultant tool writes out the configurable file in [Example 11-5](#) if the core integrator selects the following parameter values: `addr_width = 16`, `ram_size = 0`, `idle_mode_supp = 1`.

Example 11-6 Text File After Configuration

You have selected the following design configuration:

- 16-bit address bus
- 256-byte internal RAM type
- Idle mode is supported.

11.4.1.3 Determining Design Names

In some cases, particularly in configurable files for TREs, text to be substituted may include design names. In the coreConsultant tool, the core integrator can specify a design name prefix, which the coreConsultant tool then automatically prepends to all design names as part of the Specify Configuration activity. Use the following commands in configurable files to determine the current design names:

- `get_top_design_name` – Get the name of the top-level design
- `get_design_prefix` – Get the design name prefix specified by the core integrator

11.4.2 Configuring a Filegroup for Conditional Text Inclusion

For a situation such as that in [Example 11-5](#), you have to place the file in a configurable filegroup to cue the coreConsultant tool to analyze the file for text substitution pragmas and perform the text substitution.

11.4.2.1 Example – File Configuration Through Existing Parameters

If the parameter expressions in the text substitution pragmas use parameters from an existing coreConsultant activity, you must set the filegroup attributes listed in the left column of [Table 11-1](#).

[Example 11-7](#) shows an example command sequence that:

- Adds the file `YourConfiguration.txt` to the `Custom_Doc` filegroup.
- Sets up the `Custom_Doc` filegroup to be configurable based on parameters from the Specify Configuration activity (design parameters).

Example 11-7 Configurable Filegroup Script – Existing Activity Parameters

```
add_files_to_filegroup Custom_Doc \
-files /usr/cores/MyCore/doc/YourConfiguration.txt \
-install ./doc/

set_filegroup_attribute Custom_Doc Configurable 1
set_filegroup_attribute Custom_Doc ConfigActivity SpecifyConfiguration

autocomplete_activity CreateBillofMaterials
```

11.4.2.2 Example – File Configuration Through Custom Filegroup Parameters

You can also create new parameters to configure a filegroup. If you do, the coreConsultant tool creates a custom activity to prompt the user to select values for the parameters. For example, you could include a configurable filegroup that writes out the code for an example implementation of your core based on the user's selection of peripherals to be connected to the core. To implement this type of configurable filegroup, you have to:

1. Insert the text substitution pragmas in the HDL for the example design, as shown in [Example 11-8](#).
2. Create a configuration intent file for the filegroup that creates the configuration parameters and sets attributes on them. [Example 11-9](#) shows an example configuration intent file that creates the parameters referenced in [Example 11-8](#).

Name the configuration intent file filename.tcl, where filename is the base name of one of the files in the filegroup. For example, to set configuration intent for file example_top.vhd, name the configuration intent file example_top.tcl. Set the filegroup's ConfigIntentSearchPath attribute to the directory that contains the configuration intent file.

3. Set the filegroup attributes listed in right column of [Table 11-1](#). [Example 11-10](#) shows an example script that creates an autoload filegroup and sets the filegroup attributes. Alternatively, you can use the Create Bill of Materials GUI to achieve the same result.

The parameters for the custom activity use the same set of attributes as design parameters. For more examples of tuning the parameter specifications, see [Specifying Configuration Intent](#) on page 50.

Example 11-8 Text Substitution Pragmas for Custom Design Example

```
--example_top.vhd
--Top level connections for example design

<TOP instantiation>
<signal, constant, and variable definitions>

--instantiate peripheral 1
--reuse-pragma startSub Periph1 [IncludeIf @periph1_type==0 %subText]
<Instantiate type A peripheral 1>
--[ElseIncludeIf @periph1_type==1]
<Instantiate type B peripheral 1>
--reuse-pragma endSub Periph1

--instantiate peripheral 2
--reuse-pragma startSub Periph2 [IncludeIf @periph2_type==0 %subText]
<Instantiate type A peripheral 2>
--[ElseIncludeIf @periph2_type==1]
<Instantiate type B peripheral 2>
--reuse-pragma endSub Periph2
```

Example 11-9 Configuration Intent File for Custom Design Example

```
#example_top.tcl
create_configuration_parameter \
    -type boolean \
    -label {Select Peripheral 1 Type} \
```

```
-description {Use type A or type B for peripheral 1} \
-default 0 \
-group Example \
periph1_type
```

```
set_parameter_attribute periph1_type SymbolicNames {"Type A" "Type B"}
```

```
create_configuration_parameter \
-type boolean \
-label {Select Peripheral 2 Type} \
-description {Use type A or type B for peripheral 2} \
-default 0 \
-group Example \
periph2_type
```

```
set_parameter_attribute periph2_type SymbolicNames {"Type A" "Type B"}
```

Example 11-10 Command Script Custom Design Example

```
create_autoload_filegroup Example \
-patterns example/*.vhd \
-install ./example/

set_filegroup_attribute Example Label {Create Example}
set_filegroup_attribute Example Configurable 1
set_filegroup_attribute Example ConfigIntentSearchPath \
/usr/cores/MyCore/example
set_filegroup_attribute Example ConfigActivity CreateExample
set_filegroup_attribute Example ActivityGroup Example
set_filegroup_attribute Example ActivitySequence 900
set_filegroup_attribute Example ConfigDependsOnActivities \
SpecifyConfiguration

autocomplete_activity CreateBillOfMaterials
```

11.5 Setting Up Post-Configuration Commands

If you want the coreConsultant tool to execute a command after configuring and writing out the filegroup, set the PostPromptCmd attribute on the filegroup. Set the value of PostPromptCmd to the name of the Tcl command you want to execute. The Tcl command can be an existing coreConsultant command or a custom Tcl command procedure that you create.

If you want to create a custom command procedure to execute, you can define the custom command procedure directly in a configuration intent command file. Alternatively, you could create a plugin for the command procedure as described in “[Advanced Topics](#)” on page 255.

To define a custom command procedure in a configuration intent file, use the coreBuilder plugin_proc command. plugin_proc is similar to the Tcl proc command, but has more command line options to define attributes for the proc.

[Example 11-11](#) shows a configuration intent command file that creates a procedure to build a text file that describes the user’s selected design example, then displays the resulting text file in the user’s web browser.

The directory/filename rules for the intent command file are the same as for the intent command files that define configuration parameters (in fact, they can be the same file):

- Name the intent command file filename.tcl, where filename is the base name of one of the files in the filegroup.
- Set the filegroup’s ConfigIntentSearchPath attribute to the directory that contains the intent command file.

When the coreBuilder tool builds the coreKit, it analyzes the configurable filegroups and executes any intent command (.tcl) file in a directory specified by ConfigIntentSearchPath that has the same base name as one of the files in the configurable filegroup.

To apply the PostPromptCmd (ShowConfig) defined in [Example 11-11](#) to the previous examples:

- Append the text in [Example 11-11](#) to the example_top.tcl file in [Example 11-9](#).
- Add the following command before autocomplete_activity CreateBillOfMaterials in [Example 11-10](#):
set_filegroup_attribute Example PostPromptCmd ShowConfig

Example 11-11 Configuration Intent Command File for PostPromptCmd

```
#Filename = <any file in configurable filegroup>.tcl
#Building a plugin_proc to use as PostPromptCmd
plugin_proc ShowConfig {} {
    set Periph1Type [get_filegroup_parameter Example periph1_type]
    set Periph2Type [get_filegroup_parameter Example periph2_type]
    set ExampleConfigFile \
        [open [get_logical_dir]/example/Description.txt w 0744]
    puts $ExampleConfigFile "The file example/example_top.vhd in your \
workspace contains the top-level \nconnections for your selected \
example configuration."
    puts $ExampleConfigFile "It instantiates TOP with 2 peripheral devices
connected."
    if {$Periph1Type==0} {
        puts $ExampleConfigFile "Peripheral 1 is a type A peripheral."
    } elseif {$Periph1Type==1} {
        puts $ExampleConfigFile "Peripheral 1 is a type B peripheral."
    }
}
```

```
    }  
    if {$Periph2Type==0} {  
        puts $ExampleConfigFile "Peripheral 2 is a type A peripheral."  
    } elseif {$Periph2Type==1} {  
        puts $ExampleConfigFile "Peripheral 2 is a type B peripheral."  
    }  
    close $ExampleConfigFile  
    show_url [get_logical_dir]/example/Description.txt  
}
```


12

Setting Up Verification Suites

A coreKit typically includes a verification environment, also referred to as test suite regression environment (TRE), in which the core integrator can verify the core as a standalone block. A TRE typically includes a configurable testbench, input command files, expected results files, and scripts to run the simulation and check the results. The coreBuilder tool provides an automated way to package a TRE so that a core integrator can configure and simulate the TRE directly through the coreConsultant tool by executing a Verification activity in a manner similar to other coreConsultant activities.

This chapter describes how to use coreBuilder to package a configurable TRE and include it in your coreKit.

This chapter contains the following sections:

- [“Basic TRE Setup Procedure”](#) on page 166
- [“TRE Requirements”](#) on page 167
- [“Example TRE Structure”](#) on page 168
- [“Setting Up TRE Filegroups”](#) on page 169
- [“Setting Up a Verification Subprocess”](#) on page 171
- [“TRE Packaging Example”](#) on page 174
- [“Complex Verification Environments”](#) on page 179
- [“Creating an ATPG Vector Generation Script”](#) on page 180

12.1 Basic TRE Setup Procedure

To package a TRE so that the core integrators can automatically configure the TRE and execute simulations through the coreConsultant tool, you must perform the following steps:

1. Build a TRE that meets the requirements described in [TRE Configuration Intent File](#).
2. Set up configurable filegroups for the TRE.
3. Set up Verification activities for the TRE by which the core integrator selects values for test suite configuration parameters, specifies simulation execution options, invokes the simulation tool to execute the simulation, and analyzes the results.



Note

The coreConsultant tool does not provide simulation functionality. The selected simulation tool must be available in the core integrator's environment. As a core developer, you define which of the commonly used simulation tools to support for your coreKit.

12.2 TRE Requirements

The coreBuilder tool provides utilities to package an existing verification environment as a TRE for use with the coreConsultant tool. The coreBuilder and coreConsultant tools do not have built-in support for setting up and running simulation tools. Therefore, your verification environment must have the following properties:

- Batch mode invocation – The TRE must be set up to be invoked in batch mode; once invoked, it can be an interactive process. The coreConsultant tool does not interact directly with simulation tools.
- Self-checking – The TRE should write the simulation results to an output file, which the coreConsultant tool then displays for the user. The coreConsultant tool does not analyze or interpret the simulation results.

In the core integrator's environment, the coreConsultant tool interacts with the packaged TRE as follows:

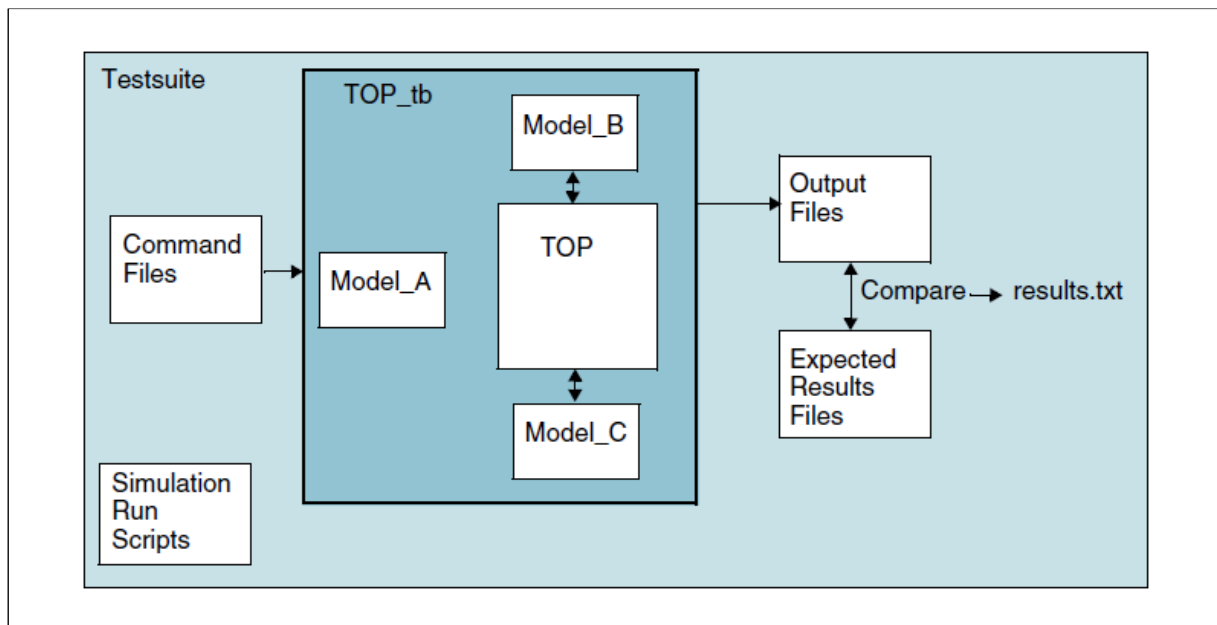
1. Writes out configurable filegroups for TRE according to core integrator's selected parameter values.
2. Invokes packaged simulation script to run simulation as a coreConsultant sub-process.
3. Monitors status of simulation sub-process.
4. Displays contents of results file when simulation is complete.

12.3 Example TRE Structure

Figure 12-1 shows the structure of an example TRE for a core. It includes:

- A testbench that instantiates the core as the device under test and functional models to simulate a typical chip environment for the core. The testbench may be coded in HDL or in a high-level verification language such as VERA. You can use configurable filegroups to implement the configurable portions of the testbench.
- A command file to drive the simulation, usually part of a configurable filegroup.
- Scripts to run the simulation on each of the supported simulators. Scripts are typically implemented as makefiles, UNIX shell scripts, or Perl programs, and may be configurable.
- A top-level UNIX script that invokes the required tool-specific simulation script.
- Expected results files (“golden simulation”) for the various tests and configurations. The testbench typically includes a mechanism for comparing the actual results to the expected results to determine pass/fail status.
- An analyzed simulation results file for the core integrator to read to check the pass/fail status and other information about the simulation. The coreConsultant tool displays this file to the core integrator when the simulation is complete.

Figure 12-1 Example TRE Architecture



12.4 Setting Up TRE Filegroups

Configurable cores typically use testbenches and testsuites that are configurable in the following ways:

- Configuration to match the core configuration. This can involve configuration of testbench components as well as inclusion or exclusion of tests that only apply to certain design configurations. You can set up configurable TRE filegroups for automatic text substitution based on values of the core's design configuration parameters.
- Configuration based on TRE-specific activity parameters. For example, you can set up a TRE activity through which the core integrator explicitly selects a testbench configuration and selects which tests to execute.

To set up configurable filegroups for the TRE:

1. Insert text substitution pragmas in the files where the text substitution is to occur. See [“Configuring Contents of Files”](#) on page 157 for details about text substitution pragmas.
2. Create and populate filegroups for the TRE. You can do this through the Verification – Testsuite Set-up activity GUI or by using the following commands as described in [“Setting Up Configurable Filegroups”](#):
 - `create_autoload_filegroup`
 - `add_files_to_filegroup`
3. Set the filegroup attributes listed in [Table 12-1](#) on the TRE filegroups. You can set the filegroup attributes through the Verification – Testsuite Set-up activity GUI or by using the `set_filegroup_attribute` command as described in [“Setting Up Configurable Filegroups”](#).

Table 12-1 Attributes for Configurable TRE Filegroup

Attribute	Description
Configurable	Set to 1 (true) to make the filegroup configurable
ConfigIntentSearchPath	One or more directories that contain configuration intent files for the filegroup When this parameter is specified as a relative path, the non-absolute path is expanded relative to the location of the file for which intent is being specified.
ConfigActivity	Name of the activity that configures the filegroup If the specified activity does not already exist, the coreConsultant tool creates the activity.
ConfigDependsOnActivities	One or more activities that must be completed before configuring the filegroup
ConfigDependsOnGroup	One or more filegroups that must be configured before configuring the filegroup
Label	GUI label for ConfigActivity
ActivitySequence	Sequence number for ConfigActivity

Attribute	Description
ActivityGroup	Group for ConfigActivity (for example, specify Verification to create a Verification tab that contains ConfigActivity)
PostPromptCmd	Tcl command to execute after configuring and writing out the filegroup For details about how to set up simulation execution as a coreConsultant subprocess, see “Setting Up a Verification Subprocess” on page 171
DeliverableType	Set to TRE for a TRE filegroup For a complex TRE with multiple configurable filegroups, set DeliverableType on the top-level filegroup that contains the configurable filegroups.

12.5 Setting Up a Verification Subprocess

The next part of setting up a TRE is to set up the subprocess that executes the simulation when the required TRE filegroups have been configured and written to disk. The coreBuilder tool provides a generic method to set up subprocesses for execution. The method involves two commands:

- `define_activity_subproc_params` – Enables a filegroup configuration activity to launch a subprocess and defines launch parameters for the subprocess.
- `launch_activity_subproc` – Launches the subprocess according to the specified parameters. Insert the `launch_activity_subproc` command in the Tcl command procedure specified by the `PostPromptCmd` attribute on the configurable filegroup.

The following sections describe how to use the above commands to set up and launch a simulation as a coreConsultant subprocess.

12.5.1 Defining the Subprocess Parameters

There are two groups of parameters for the activity subprocess:

- [Table 12-2](#) lists the parameters that the core integrator sets when executing the activity that configures the filegroup. When you define a subprocess, the coreConsultant tool automatically creates the parameters listed in [Table 12-2](#) to give the user control over how the coreConsultant tool launches the subprocess and how the coreConsultant tool notifies the user when the subprocess is complete.
- [Table 12-3](#) on page 172 lists the parameters that you set when you define the subprocess. To define the subprocess parameters in [Table 12-3](#) on page 172, insert the `define_activity_subproc_params` in a configuration intent (.tcl) file for the configurable filegroup as shown in [Example 12-1](#). The `define_activity_subproc_params` also triggers the coreConsultant tool to launch the simulation as a subprocess.

Table 12-2 Subprocess Parameters Set by Core Integrator

Parameter	Description
ScriptsOnly	If set, the coreConsultant tool generates, but does not execute, a run script for the subprocess. The core integrator can execute the script at a later time.
RunStyle	If ScriptsOnly is 0, then this selects how to run the command. The command can be run locally, through either a load-sharing interface. Valid values are "local", "lsf", and "grd".
RunOptions	If the RunStyle is grd or lsf, this parameter provides any special options to pass to the corresponding job launch command.
EmailAddress	Specifies a user to send email to when the subprocess completes or is ended. The default is the current user.

Table 12-3 Subprocess Parameters Set by Core Developer

Parameter	Description
WorkDirectory	The directory in which to place guard files and the subprocess run script The subprocess also executes from WorkDirectory. The value of WorkDirectory must be a relative path from the user's workspace and must not match the WorkDirectory value for any other activity. You set the value of WorkDirectory with the -workdir option to the define_activity_subproc_params command.
ResultsFile	The name of the results file for the subprocess, relative to WorkDirectory The coreConsultant tool displays ResultsFile in the user's web browser when the subprocess is complete. You set the value of ResultsFile with the -resultsfile option to the define_activity_subproc_params command.
MailFile	The name of a file containing information that is appended to the generic email message sent to the user specified by EmailAddress The MailFile path is relative to WorkDirectory. You set the value of MailFile with the -mailfile option to the define_activity_subproc_params command.

Example 12-1 Defining Subprocess Parameters

```
define_activity_subproc_params -workdir sim \
  -resultsfile results.txt \
  -mailfile .summary \
  Verification_Environment/FuncTest
```

12.5.2 Defining the Subprocess

To launch a specified command as a coreConsultant subprocess:

1. Create a Tcl command procedure that includes the launch_activity_subproc command. You can either build a coreConsultant plugin that defines the command procedure by executing the Load Consultant Plugins activity or, more simply, use the plugin_proc command as shown in [Example 12-2](#). The plugin_proc command is simpler to use because you can include the plugin_proc command in an intent command file for a configurable filegroup instead of building a separate plugin.
2. Specify the name of the Tcl command procedure as the value of the PostPromptCmd attribute on the configurable filegroup.

The syntax for the launch_activity_subproc command is:

```
launch_activity_subproc activity cmdstr [scriptname]
```

activity is the name of the activity that is to launch the subprocess.

cmdstr is a command, or set of commands, separated by semicolons or newlines, to execute as a subprocess.

scriptname is the name of the subprocess shell script. The default is run.scr.

Example 12-2 Defining the Subprocess in a Command Procedure

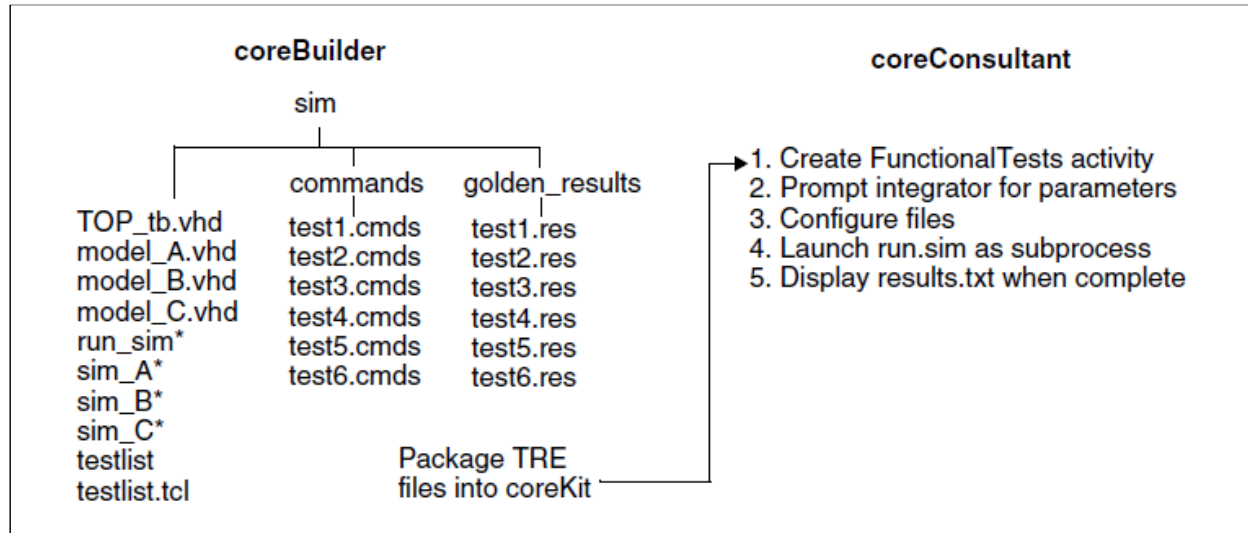
```
# Create LaunchTest command procedure
```

```
plugin_proc LaunchTests {act} {  
  set cmd "./run_sim"  
  launch_activity_subproc $act $cmd  
}
```

12.6 TRE Packaging Example

The following sections show an example set of configurable files, coreBuilder commands, and a configuration intent command file that accomplish the objectives shown in [Figure 12-2](#).

Figure 12-2 Example of Desired TRE Packaging and Execution



12.6.1 Configurable Files

The example testsuite uses two configurable files:

- **run_sim** ([Example 12-3](#)) – After configuration, contains commands to clean up the sim directory, plus a single command to invoke a simulator-specific run script (sim_A, sim_B, or sim_C)

Each script simulates each of the selected tests, compares the simulation output to the golden_results, and writes the results of the comparison to results.txt.

- **testlist** ([Example 12-4](#)) – After configuration, contains a list of tests to simulate

A TRE may include other configurable files, including the testbench itself. To automatically configure the testbench or other TRE files based on values of design parameters, see the design parameters in the text substitution pragmas. Parameters in the text substitution pragmas can be filegroup configuration parameters or design configuration parameters.

Example 12-3 Configurable Run Script (run_sim)

```
#!/bin/sh
<commands to clean up sim directory>

# reuse-pragma startSub pickScript [IncludeIf @sim_tool=="Scirocco" %subText]
./sim_A
# [ElseIncludeIf @sim_tool=="Simulator_B"]
./sim_B
# [ElseIncludeIf @sim_tool=="Simulator_C"]
./sim_C
# reuse-pragma endSub pickScript
```

Example 12-4 Configurable Test List File (testlist)

```
# reuse-pragma startSub [IncludeIf @test1 %subText]
test1

# reuse-pragma startSub [IncludeIf @test2 %subText]
test2

# reuse-pragma startSub [IncludeIf @test3 %subText]
test3

# reuse-pragma startSub [IncludeIf @test4 %subText]
test4

# reuse-pragma startSub [IncludeIf @test5 %subText]
test5

# reuse-pragma startSub [IncludeIf @test6 %subText]
test6
```

12.6.2 Loading the TRE Filegroups

[Example 12-5](#) shows the commands to load the TRE files into the Verification_Environment/FuncTest filegroup. The add_files_to_filegroup command creates the Verification_Environment/FuncTest filegroup and loads the specified files into it. (The top-level Verification_Environment filegroup is defined in the BoM template named Default Template.)

Alternatively, you could load the files and specify the filegroup parameters through the Testsuite Set-up GUI. The Testsuite Set-up GUI is similar to the Create Bill of Materials GUI described in [Building the coreKit or IP-XACT Component](#) on page 181, except that it only lists filegroups that have DeliverableType set to TRE.

Example 12-5 Commands to Load TRE Files

```
add_files_to_filegroup Verification_Environment/FuncTest \
-files {./MyCore/sim/TOP_tb.vhd \
./MyCore/sim/model_A.vhd \
./MyCore/sim/model_B.vhd \
./MyCore/sim/model_C.vhd \
./MyCore/sim/sim_A \
./MyCore/sim/sim_B \
./MyCore/sim/sim_C \
./MyCore/sim/testlist \
./MyCore/sim/run_sim \
./MyCore/sim/commands \
./MyCore/sim/golden_results } \
-install ./sim/
```

12.6.3 Setting the Filegroup Attributes

[Example 12-6](#) shows the commands to set the Verification_Environment/FuncTest to be configurable and defines the configuration activity. Comments in [Example 12-6](#) indicate the function of the individual commands.

Example 12-6 Commands to Set TRE Filegroup Attributes

```
set grp Verification_Environment/FuncTest

#Make filegroup configurable and set intent search path.
set_filegroup_attribute $grp Configurable 1
set_filegroup_attribute $grp \
ConfigIntentSearchPath ./MyCore/sim/

#Set up configuration activity for filegroup.
set_filegroup_attribute $grp \
  ConfigActivity FunctionalTests
set_filegroup_attribute $grp \
  ConfigDependsOnActivities SpecifyConfiguration
set_filegroup_attribute $grp ActivitySequence 901
set_filegroup_attribute $grp Label {Functional Tests}
set_filegroup_attribute $grp ActivityGroup Verification

#Use tree-style dialog for parameters.

set_filegroup_attribute $grp ParameterInfo \
  {{ContainerType tree}}

#Set up command to launch simulation.
set_filegroup_attribute $grp PostPromptCmd \
  {LaunchTests %item}
```

12.6.4 TRE Configuration Intent File

[Example 12-7](#) shows the configuration intent file (testlist.tcl), which performs the following functions when the coreBuilder tool analyzes the Verification_Environment/FuncTest filegroup:

- Creates the configuration parameters for the Verification_Environment/FuncTest filegroup.
- Creates the command procedure LaunchTests, which launches the configured run_sim script as a subprocess.

Example 12-7 TRE Configuration Intent File

```
#Create filegroup configuration parameters and set parameter attributes.

set tre_group Verification_Environment/FuncTest

#Select simulator.
create_configuration_parameter -group $tre_group \
  -type string \
  -default Scirocco \
```

```
-label "Select Simulator" \  
-description "Select simulation tool" \  
sim_tool  
  
set_parameter_attribute sim_tool GroupName Setup  
set_parameter_attribute sim_tool \  
EnumValues {Scirocco Simulator_B Simulator_C}  
  
#Enable test selection.  
create_configuration_parameter -group $tre_group \  
-type boolean \  
-default 1 \  
-label "Test 1" \  
-description "Run Test 1" \  
test1  
  
set_parameter_attribute test1 GroupName Tests  
  
create_configuration_parameter -group $tre_group \  
-type boolean \  
-default 1 \  
-label "Test 2" \  
-description "Run Test 2" \  
test2  
  
set_parameter_attribute test2 GroupName Tests  
  
create_configuration_parameter -group $tre_group \  
-type boolean \  
-default 1 \  
-label "Test 3" \  
-description "Run Test 3" \  
test3  
  
set_parameter_attribute test3 GroupName Tests  
  
create_configuration_parameter -group $tre_group \  
-type boolean \  
-default 1 \  
-label "Test 4" \  
-description "Run Test 4" \  
test4  
  
set_parameter_attribute test4 GroupName Tests  
  
create_configuration_parameter -group $tre_group \  
-type boolean \  
-default 1 \  
-label "Test 5" \  

```

```

    -description "Run Test 5" \
    test5

set_parameter_attribute test5 GroupName Tests

create_configuration_parameter -group $tre_group \
    -type boolean \
    -default 1 \
    -label "Test 6" \
    -description "Run Test 6" \
    test6

set_parameter_attribute test6 GroupName Tests

#Set up LaunchTests to run simulation as sub-process.
define_activity_subproc_params -workdir sim \
    -resultsfile results.txt -mailfile .summary $tre_group

plugin_proc LaunchTests {act} {
    set cmd "./run_sim"
    launch_activity_subproc $act $cmd
}

```

12.6.5 Completing the Testsuite Setup Activity

After you have set up the configurable filegroups for the TRE, complete the Testsuite Setup activity. In GUI mode, review your TRE filegroup specifications in the Testsuite Setup dialog, then click Apply.

In command line mode use the following command:

```
autocomplete_activity TestsuiteSetup
```

12.6.6 Resulting coreConsultant Verification Activities

Use the Create Integration Workspaces activity to create a coreConsultant workspace in which to check that your TRE works as intended.

There are three pages of the parameter dialog for the FunctionalTests activity. The core integrator selects parameter groups from the list at the left, sets parameters within each group, then clicks Apply.

The simulation subprocess status appears when the core integrator clicks the Results button. When the core integrator clicks View Results, the coreConsultant tool displays the file results.txt in the core integrator's web browser. (This example specifies results.txt as the create_activity_subproc_params -resultsfile option).

12.7 Complex Verification Environments

For cores that have complex verification environments, such as multiple testbenches and testsuites, you can create multiple verification activities. The methodology is similar to the previously discussed example, with the following exceptions:

- Create a separate configurable filegroup for each activity (for example, for each testbench and its associated testsuites).
- Make each of the above filegroups a filegroup member of a hierarchical filegroup. Set DeliverableType to TRE on the top-level filegroup. For example, to create an autoload filegroup named sim1 as a member of the existing Verification_Environment filegroup, use the following command:

```
create_autoload_filegroup -patterns autoload_patterns \  
-install install_dir Verification_Environment/sim1
```

12.7.1 Testing a Slave within coreAssembler

There are two pieces of information currently required to automatically test a simple slave component in the coreAssembler testbench.

- Tie-off values must be defined for any port which must be tied to value other than 0 when it is unconnected in the testbench.
- A valid memory map on which at least one register field has the “SideEffects” attribute set to “None.” This information defines the register size, register offset, the name of the interface providing access to the register, and the suggested values to write (or read if there are no writable registers).

In the Y-2006.03-SP1 release, the SimTieOff attribute was defined to enable moving the required information out of the testbench generator and into the component. This parameter allows component control over the “Simulate Subsystem” testing.

12.7.1.1 SimTieOff attribute

Description: Indicates how a pin should be tied off in the testbench if left unconnected. Default is to tie unconnected busBit/pin/port to 0. If other than “0” is required, this attribute will tie off to “value”.

Type: string

Valid on: busBit, pin, port

12.8 Creating an ATPG Vector Generation Script

Testability is an important issue for all IC designs. The Generate Test Vectors activity allows integration of the test strategy for the subsystem into the overall synthesis flow of your chip. With this activity, you can create a TestMax supported ATPG test script that is run by the user to generate test vectors to be used in the subsystem simulation.

The following ATPG attribute is provided for this activity:

Table 12-4 Generate Test Vectors – Options

Option	Description
Attribute Name: AtpgScript Attribute Value: Path to ATPG script within your component Knowledge Base.	Points to a vector generation script that is provided with a component As core developer, you can create this script that is run by the core integrator to generate test vectors for a simulation run. Default = get_design_attribute AtpgScript.

The *TestMax User's Guide* provides information on how to write scripts to generate test vectors for your subsystem. It is not the intent of this document to duplicate that information.

13

Building the coreKit or IP-XACT Component

This chapter describes the final set of tasks required to build your coreKit for a core that does not require interface definitions. If you are going to define interfaces for use with coreAssembler, go to “[Defining Interfaces for coreAssembler](#)” first to define the interfaces, then return here to build the coreKit.

This chapter contains the following sections:

- “[Creating the Bill of Materials](#)” on page [182](#)
- “[Setting Up the Help Menu](#)” on page [189](#)
- “[Building the coreKit](#)” on page [190](#)
- “[Building the IP-XACT Component](#)” on page [193](#)
- “[Verifying coreKit Contents](#)” on page [194](#)
- “[Working with BoM Templates](#)” on page [195](#)

13.1 Creating the Bill of Materials

The Create Bill of Materials activity is where you define exactly what files to include in the coreKit and how the files are to be organized and installed in the core integrator's workspace. In the coreKit Bill of Materials (BoM), files are grouped into filegroups according to their purposes. You can define your coreKit BoM by using a BoM Template, explicitly adding files and filegroups through the Create Bill of Materials activity, or a combination of both. Whatever method you use, you must complete the Create Bill of Materials activity before you can build the coreKit.

In GUI mode, you can use the Create Bill of Materials activity to perform the following tasks:

- Check existing filegroups and set attributes on them
- Add filegroups
- Edit filegroups (add or delete files)
- Set attributes on filegroups and files
- Set up configurable filegroups
- Verify the BoM

To invoke the Create Bill of Materials activity in GUI mode, select the Create Bill of Materials activity from the Packaging section of the coreBuilder Activity List. The coreBuilder tool displays the Create Bill of Materials dialog in the Activity View pane.

13.1.1 Viewing Existing Filegroups

The Create Bill of Materials dialog lists the currently defined filegroups, including:

- Synopsys filegroups – the coreBuilder tool automatically creates and populates the following mandatory filegroups:
 - Synopsys/Hdl – Created by default and populated when you execute the Load Designs activity. You cannot exclude Synopsys/Hdl or any of its members (files) from the coreKit.
 - Synopsys/DesignKbs – Created by default and populated when you execute the Load Designs activity. You cannot exclude Synopsys/DesignKbs or any of its members (files) from the coreKit.

The coreBuilder tool automatically creates additional Synopsys/* filegroups for optional data that you specify through coreBuilder activities. For example, if you create any core-specific plugins by executing the Load Consultant Plugins activity, the coreBuilder tool automatically creates the Synopsys/Plugin filegroup and adds it to the Bill of Materials.

- Filegroups defined in the BoM Template – You select a BoM Template when you create your coreBuilder workspace. The coreBuilder tool automatically creates the filegroups that are defined in your selected BoM Template.
- Any other filegroups that you have previously defined either through the Create Bill of Materials GUI, Verification (TRE) activities, or through the command line interface.

Each filegroup is a row of a spreadsheet that displays and enables you to edit values for filegroup attributes. See [“Setting Filegroup Attributes”](#) on page 184.

13.1.2 Adding a Filegroup

To add a filegroup, click the Filegroup editing – Add button to display the Add Filegroup dialog. In the Add Filegroup dialog, select values for the filegroup options listed in [Table 13-1](#).

Table 13-1 Add Filegroup Options

Option	Description
filegroupGroup or filegroup	filegroupGroup – Creates a hierarchical filegroup (filegroupGroup), which can contain filegroups Synopsys is an example of a filegroupGroup that contains filegroups Hdl and DesignKbs. filegroup – Creates a filegroup, which can contain files and directories.
Group Name	Name for the filegroup
Group Type	SimpleFilegroup – Creates a simple filegroup to which you explicitly add individual files or directories AutoloadFilegroup – Creates an autoload filegroup to which the coreBuilder tool automatically adds files that match the specified Autoload Glob Patterns
Autoload Glob Patterns	Enter one or more pattern matching expressions to automatically populate the filegroup. For example, if you specify /usr/cores/MyCore/doc/*.html, coreBuilder automatically adds all .html files in the /usr/cores/MyCore/doc directory to the autoload filegroup when you build the coreKit. See the DefaultLoadPath man page in the coreBuilder online reference for more information.
Installation Directory	The coreConsultant workspace directory where to install the filegroup For example, if you specify doc, the filegroup is installed into the core integrator's workspace/doc directory.

To add a filegroup in command line mode, use either of the following commands:

- `create_autoload_filegroup` – Create an autoload filegroup.
- `add_files_to_filegroup` – Add files to a filegroup; create the specified filegroup if it does not already exist.

13.1.3 Editing Members of a Filegroup

To add or delete files in a filegroup, select the filegroup in the File Groups spreadsheet section of the Create Bill of Materials dialog, then click the Files button to display the Specify Bill of Materials dialog. The Specify Bill of Materials dialog has separate tabs for simple and autoload filegroups.

13.1.3.1 Manipulating Simple Filegroup Members

To manipulate the members (files and/or directories) of a simple filegroup, select the filegroup in the Simple Filegroup tab of the Specify Bill of Materials dialog and use the controls to add, delete, or modify members as needed.

In command line mode, use the following commands to manipulate members of a simple filegroup:

- `add_files_to_filegroup` – Add members (files or directories) to a filegroup.
- `remove_item` – Delete member from a filegroup.

13.1.3.2 Manipulating Autoload Filegroup Members

To manipulate the members of an autoload filegroup, select the filegroup in the Autoload Filegroup tab of the Specify Bill of Materials dialog and use the controls to modify the list of glob-style matching patterns and/or installation directory for the filegroup.

In command line mode, use the `set_filegroup_attribute` command to modify the values of the following filegroup attributes:

- `DefaultLoadPath` – List of glob-style matching patterns.
- `DefaultInstallDir` – Installation directory for filegroup.

13.1.3.3 Loading an Autoload Filegroup

The coreBuilder tool automatically loads (populates) autoload filegroups when you execute the Build coreKit activity. In general, you do not need to explicitly load the contents of an autoload filegroup. The exception is if you want to set attributes on individual members of the filegroup. In this case, you must load the autoload filegroup to create the individual member objects so that you can set attributes on them.

To load and unload members of an autoload filegroup, select the filegroup in the Create Bill of Materials dialog and use the Autoload Filegroup – Load Contents and Unload Contents buttons to load and unload the filegroup contents.

In command line mode, use the following commands:

- `load_autoload_filegroup` – Load the members of an autoload filegroup.
- `unload_autoload_filegroup` – Unload the members of an autoload filegroup.

13.1.4 Deleting a Filegroup

Deleting a filegroup removes the filegroup and its members from the BoM. To delete a filegroup, select the filegroup in the Create Bill of Materials dialog and click the Delete button.

To delete a filegroup in command line mode, use the `remove_item` command. For example:

```
coreBuilder> remove_item [find_item -type filegroup MyFileGroup]
```

13.1.5 Setting Filegroup Attributes

Table 13-2 lists the filegroup attributes; see the coreBuilder online reference for detailed descriptions and usage examples.

You can view and edit the values of filegroup attributes directly in the Create Bill of Materials dialog by editing the contents of the corresponding cell in the File Groups spreadsheet.

To set/get filegroup attributes in command line mode use the following commands:

- `set_filegroup_attribute` – Set the value of a filegroup attribute
- `get_filegroup_attribute` – Get the value of a filegroup attribute

Table 13-2 Filegroup Attributes

Attribute	Short Description
Name	Name of the filegroup
Importance	Indicates whether the members of the filegroup are required or optional

Attribute	Short Description
Exists	Read-only attribute indicating whether there are any files in the filegroup
TypeOfFg	Read-only attribute that indicates the type of the filegroup ☐ S = Simple ☐ A = Autoload ☐ C = Configurable An autoload, configurable filegroup is type AC.
DeliverableType	Type of information stored in the filegroup RTL and TRE have special meaning in the coreBuilder tool; otherwise you can use any string to identify the deliverable type.
DefaultInstallDir	Installation directory for the filegroup (relative to the root directory of the coreConsultant workspace)
OneRequiredGroup	Specifies that at least one file from this filegroup or at least one file from another filegroup with same value of OneRequiredGroup must be included in the coreKit
FileContentType	The content type of the filegroup
FilePerms	File permissions to assign when installing the filegroup
Label	GUI label for the filegroup
InstallWhen	Specifies when to install the filegroup ☐ installation = During coreKit installation ☐ custom = As part of custom installation procedure
InstallUserWorkDir	Specifies how to install the filegroup in a core integrator workspace ☐ link = Link coreKit installation version of filegroup ☐ copy = Create unique copy of filegroup in workspace ☐ ignore = Do not install filegroup in user workspace.
DefaultLoadPath	Tcl list of glob-style patterns that specifies the paths from which to load this autoload filegroup Relative paths are assumed to start at the root of the workspace for the core. DefaultLoadPath does not appear in the File Groups attributes spreadsheet. To access DefaultLoadPath for an autoload filegroup, select the filegroup and click the Filegroup > Files button.
Install	Specifies the conditions under which to install the filegroup.

13.1.6 Setting File Attributes

Table 13-3 lists the file attributes; for detailed descriptions and usage examples, see the coreBuilder online reference. Files inherit several attribute values from the parent filegroup. You can set additional file attributes on individual files as well as override the inherited values for individual files.

You can view and edit the values of file attributes for simple filegroups directly in the Specify Bill of Materials dialog, which appears when you select a simple filegroup and click the Filegroup editing – Files button.

For an autoload filegroup, you must load the members of the autoload filegroup before you can set attributes on individual files. To load the members of an autoload filegroup, select the filegroup in the Create Bill of Materials dialog, then click the Autoload Filegroup – Load Contents button. In command line mode, use the `load_autoload_filegroup` command. After loading an autoload filegroup, use the following commands to set/get attributes on individual files:

- `set_attribute` – Set the value of an attribute on a specified item
- `get_attribute` – Get the value of an attribute on a specified item

You can also use the above commands for individual files in simple filegroups.

Table 13-3 File Attributes

Attribute	Short Description
Name	Name of the file
SourceDir	Source directory for the file (read-only attribute)
DefaultInstallDir	Installation directory for the file (relative to the root directory of the coreConsultant workspace)
FileContentType	The content type of the file
FilePerms	File permissions to assign when installing the file
EncryptText	Determines whether to encrypt text in the file
InstallUserWorkDir	Determines how to install the file into a core integrator workspace ■ <code>link</code> = Link coreKit installation version of file ■ <code>copy</code> = Create unique copy of file in workspace ■ <code>ignore</code> = Do not install file in user workspace.

13.1.7 Setting Configurable Filegroups

Configurable filegroups are special filegroups that you set up to be configurable in the core integrator workspace. Configurable filegroups can be conditionally installed or can include files that contain conditional text, as described in “[Setting Up Configurable Filegroups](#)” on page 151.

The preferred method to set up configurable filegroups is to use the command line method described in “[Setting Up Configurable Filegroups](#)” on page 151, then use the Create Bill of Materials GUI to check the settings. To check and/or set configuration attributes on a filegroup through the Create Bill of Materials GUI:

1. Select the filegroup in the File Groups list.
2. Click the Set Configurable button to display the Set Configurable Filegroup dialog.
3. Enter values for the options listed in [Table 13-4](#) in the Analyze tab of the Set Configurable Filegroup dialog.

4. To specify an activity to configure the filegroup, enter the information listed in [Table 13-5](#) in the Activity tab of the Set Configurable Filegroup dialog.
5. Click Apply to complete setting the configurable filegroup.
coreBuilder analyzes the configurable filegroup and creates any filegroup parameters defined in configuration intent files found in ConfigIntentSearchPath.
6. If you are creating a new activity to configure the filegroup, click the Configurable Filegroups – Configuration button to set attributes on the filegroup parameters and preview the activity dialog for the configurable filegroup. The configuration parameter dialog for filegroup parameters is the same as the configuration parameter dialog for design parameters, as described in [“Specifying Configuration Intent”](#) on page 50.

Table 13-4 Set Configurable Filegroup – Analyze Tab Options

Option	Description	Filegroup Attribute
Files Are Configurable	Sets the filegroup to be configurable	Configurable
Tcl Intent Search Path	Enter the path to the files that contain configuration intent for the filegroup Configuration intent files contain commands to create filegroup configuration parameters. For details, see “Setting Up Configurable Filegroups” on page 151.	ConfigIntentSearchPath
Pre Prompt Command	Command to execute before posting the configuration dialog for the filegroup	PrePromptCmd
Post Prompt Command	Command to execute after configuring the filegroup	PostPromptCmd

Table 13-5 Set Configurable Filegroup – Activity Tab Options

Option	Description	Filegroup Attribute
Configuration Activity	Enter the name of a new or existing activity to configure the filegroup	ConfigActivity
Define New Activity	Click to enable definition of new activity to configure the filegroup	None
Activity Label	Label for the new activity	Label
Activity Group	Activity group in which to place the new activity (either an existing group or a new group to be created)	ActivityGroup
Activity Sequence	Sequence of the activity, determines where the activity occurs relative to other activities in the group	Sequence

Option	Description	Filegroup Attribute
Depends on Filegroups	If configuration of the filegroup depends on previous configuration of another filegroup, select that filegroup name	ConfigDependsOnGroup
Depends on Activities	If configuration of the filegroup depends on previous completion of one or more activities, enter the activity names	ConfigDependsOnActivities

13.1.8 Checking the BoM

There are two checks you can perform on the BoM directly from the Create Bill of Materials dialog:

- Check BoM – Checks that all required filegroups have files and that the contents are up to date. To perform Check BoM in command line mode, use the `check_bom` command.
- Report BoM – Reports the current values of a default set of attributes on all filegroups, which indicate whether the filegroup exists, is configurable, and is up to date. You can generate more specific BoM reports by using the `report_bom` command in command line mode. See the `report_bom` man page for details.

13.2 Setting Up the Help Menu

You can link core-specific documentation to the coreConsultant console Help menu by executing the optional Set Up Help Menu activity.

To perform the Set Up Help Menu activity:

1. Make sure that the documentation file that you want to link is included in the BoM.
2. Select the Set Up Help Menu activity in the Activity List.
The coreBuilder tool displays a spreadsheet in the Activity View pane listing the current design-specific help menu items and their attributes.
3. Click Add... to add a menu item.
4. In the Add Help Menu Item dialog:
 - a. Enter values for MenuText and Description attributes for the new help menu item (see [Table 13-6](#)).
 - b. Select the HelpUrl for the new menu item by choosing a filegroup and a file within that file group.
 - c. Click OK to add the menu item.
5. Repeat the steps for all additional menu items.
6. Verify your help menu selections in the Set Up Help Menu spreadsheet, add or delete items, or change the MenuText or Description as needed, then click OK to complete the Set Up Help Menu activity.

Table 13-6 Help Menu Item Options

Option	Description
MenuText	Text that appears in the coreConsultant console Help menu
Description	Text that appears in the status bar of the coreConsultant console when the Help menu item is highlighted
HelpUrl	Destination file for the Help menu link In GUI mode, you specify the HelpUrl by selecting a filegroup and a file within that filegroup. To specify a non-coreKit file as the HelpUrl, use the add_help_menu_item command to add the Help menu item in command line mode as shown below.

To add a help menu item in command line mode, use the add_help_menu_item command. For example, the following command adds a Help menu item that points to a file on a website:

```
add_help_menu_item {Developer Tools} \  
{http://www.mywebsite.com/tools.html}\  
{List of software developer tools}
```

To complete the Set Up Help Menu activity in command line mode, use the following command:

```
autocomplete_activity SetUpHelpMenu
```

13.3 Building the coreKit

The final step in building the coreKit is to perform the Build coreKit activity. Before doing so, you must complete all other coreBuilder activities, including optional activities. If an activity does not apply for your core or does not require any user data, just use `autocomplete_activity` to complete the activity. For example, if the core does not require any interface definitions for the coreAssembler tool, complete the SoC Assembly Intent activities without actually defining interfaces.

To complete the Build coreKit activity:

1. Select the Build coreKit activity in the coreBuilder Activity List.
2. In the Basic tab of the Build coreKit dialog which appears in the Activity View pane, enter data for the options listed in [Table 13-7](#).
3. In the Implementation Tools tab of the Build coreKit dialog, enter data for the options listed in [Table 13-8](#). By allowing an implementation tool, you give the core integrator the choice to use that tool and the associated synthesis strategies. The coreConsultant tool prompts the core integrator to choose (enable) the selected implementation tool for the core from the set of allowed tools.
4. Click Apply to build the coreKit.

The coreBuilder tool builds the coreKit (name.coreKit) and places it in the install directory of your coreBuilder workspace.

Table 13-7 Build coreKit – Basic Tab Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
coreKit Name (coreKitName)	The name of your coreKit Usually, this is the name of your core. The coreBuilder tool automatically adds the filename extension “.coreKit” to whatever filename you specify. This is also used to sort components in the coreAssembler Add Component dialog box.
coreKit Version (coreKitVersion)	The version associated with this build of your coreKit The version string can be in any format. Default: 1.0
coreKit Vendor (KitVendor)	The vendor name for your coreKit The value of this attribute can be used in the coreAssembler Add Component dialog box to sort components when components are listed in vendor/library/name/version (VLNV) format. Default: Unknown
coreKit Library (KitLibrary)	The library associated with your coreKit The value of this attribute can be used in the coreAssembler Add Component dialog box to sort components when components are listed in vendor/library/name/version (VLNV) format. Default: Unknown

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Support File (SupportFile)	The name of a file containing contact information for whoever is responsible for supplying end-user support for the packaged core
Show License Agreement (LicenseFile)	Shows license agreement from file If you wish to display a license agreement to the end user, check the box and type the name of the file containing the license agreement in the accompanying data field.
Invoke Post Install Script (ScriptName)	Runs a script immediately after the coreKit is installed If you wish to run a script immediately after the coreKit is installed, check the box and specify the name of the script in the data field.
Complete with BoM Check Error (CompleteOnError)	Check this box to enable completion of the Build coreKit activity even if there are BOM checking errors
Prevent Design Prefix (PreventDesignPrefix)	Prevents coreTools users from setting a design prefix Default: 0 (False) if any configuration is done using Verilog `defines or VHDL constants, otherwise 1 (True)
Prevent SV Prefix (PreventSVPrefix)	Prevent the coreConsultant tool from inserting a prefix for SystemVerilog interfaces and packages
Prevent File Prefix (PreventFilePrefix)	Prevents coreTools users from setting a file prefix Must be set to '0' to allow filename prefixing in the coreAssembler or coreConsultant tools. Default: 1 (True); file prefixing is disabled by default for backward compatibility.

Table 13-8 Build coreKit – Implementation Tools Tab Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
dc_shell Implementation Allowed (dc_shellImplementationAllowed)	Allow dc_shell (Design Compiler) as a valid implementation tool for the coreKit Default: 1 (True)
fc_shell Implementation Allowed (fc_shellImplementationAllowed)	Allow fc_shell (Fusion Compiler) as a valid implementation tool for the coreKit Default: 0 (False)
fm_shell Implementation Allowed (fm_shellImplementationAllowed)	Allow fm_shell (the Formal Verification activity) as a verification tool for the core in the coreConsultant and coreAssembler tools Default: 1 (True)

To complete the Build coreKit activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands as shown in the following example:

```
set_activity_parameter BuildcoreKit coreKitName MyCore
set_activity_parameter BuildcoreKit dc_shellImplementationAllowed true
autocomplete_activity BuildcoreKit
```

13.4 Building the IP-XACT Component

13.4.1 Defining Component Views

The 'Define Component Views' activity lets you create multiple views for downstream tools. This activity enables the definition of hierarchical views. An entry field is available for indicating the VLNV of the hierarchical component to be referenced when the view is created.

13.4.2 Build IP-XACT Component

The 'Build IP-XACT Component' activity lets you build an IP-XACT component for the current cB workspace to the "version" specified in **Edit > Preferences > IP-XACT** using data specified in previous activities. For more about preferences, see "[Editing Tool Setup and User Preferences](#)" on page [273](#).

13.5 Verifying coreKit Contents

To verify the coreKit:

1. Invoke the coreConsultant tool and load the coreKit:

```
% cd workspace/install
```

```
% coreConsultant name.coreKit
```

The coreConsultant tool installs the coreKit in your selected installation directory.

2. Use **File > New Workspace** in the coreConsultant tool to create a workspace for the installed coreKit.
3. In the coreConsultant workspace, check that all features of the coreKit work as intended.

13.6 Working with BoM Templates

BoM templates provide a means to implement a standard set of deliverables for coreKits. You can either use one of the BoM templates included with the coreBuilder software or define and use your own. In either case, you can still add files/filegroups that are not defined in the BoM template by using the BoM editing procedures in the previous sections of this chapter.

By defining your own BoM template, you can:

- Automatically create and populate filegroups
- Control coreKit contents, ensuring that all required data is present
- Set attributes on filegroups, including configurable filegroup attributes

You select a BoM template at the beginning of the coreKit development process when you create your coreBuilder workspace.

13.6.1 Synopsys BoM Templates

Synopsys provides two BoM templates with the coreBuilder software. You can use either of the Synopsys BoM templates to build a coreKit, or use them as examples of how to create your own custom BoM template. Both of the Synopsys BoM templates are available by default for selection when you create your coreBuilder workspace:

- Default_Template
- RMM_Template

Both of the above BoM templates reside in the directory `coreBuilder_root/libraries/dware/BOM_Templates` (Default_Template.tcl and RMM_Template.tcl). You can view the filegroups defined in a BoM template by creating a coreBuilder workspace with that template and then viewing the filegroups through Create Bill of Materials dialog.

13.6.2 Creating and Using Custom BoM Templates

The coreBuilder tool searches the directories listed in your coreBuilder `rt_bom_template_dirs` variable for BoM Template files. Any file with a .tcl extension that coreBuilder finds in the specified directories is available for selection as a BoM Template when you create a coreBuilder workspace. The default value for `rt_bom_template_dirs` is `coreBuilder_root/libraries/dware/BOM_Templates`, so the Synopsys-supplied BoM Templates are available by default.

To make additional BoM Template files available for selection:

1. Create the BoM Template file as described in the following sub-sections. Use the Synopsys BoM template files as examples.
2. Add the directory that contains your custom BoM templates to the `rt_bom_template_dirs` variable definition:

```
set rt_bom_template_dirs "$rt_bom_template_dirs <MyTemplatedirectory>"
```

To make this a persistent setting across coreBuilder sessions, add the above line to your setup file at `$HOME/.synopsys_rt.setup` file. For more about setup files, see [“Editing Tool Setup and User Preferences”](#) on page 273

13.6.2.1 BoM Template Syntax

A BoM template file is essentially a Tcl command file that creates the BoM template by defining three types of objects:

- BOMTemplate – The template itself
- Group – A Group defined in the BOMTemplate creates a filegroupGroup (a hierarchical filegroup); a Group can contain one or more Deliverables.
- Deliverable – A Deliverable defined in the BOMTemplate creates a filegroup

The BoM template file also sets attributes on each of the above object types. Files are then added to Deliverable objects (filegroups) either automatically or manually. [Example 13-1](#) on page 196 shows the basic structure of a BoM template file.

[Table 13-9](#) on page 197 lists the attributes you can set on the BOMTemplate object. The attributes for Groups (filegroupGroups) and Deliverables (filegroups) are the same as those previously described for the Create Bill of Materials activity.

You can override attribute values specified in the BoM template. However, you cannot delete filegroups that are defined in the BoM template. (If Importance = optional, a filegroup can contain zero files).

[Example 13-2](#) on page 197 shows the BoM template file that creates the Synopsys BoM Template Default_Template.

Example 13-1 BoM Template File Syntax

```
set BOMTemplate {
BOMTemplate #Creates a BOMTemplate
{attr value}
{attr value}
{attr value}
{Deliverable #Creates a filegroup
{attr value}
{attr value}
{attr value}
}
{Group #Creates a filegroupGroup
{attr value}
{attr value}
{attr value}
}
{Deliverable #Creates filegroups in filegroupGroup
{attr value}
{attr value}
{attr value}
}
{Deliverable #Creates filegroups in filegroupGroup
{attr value}
{attr value}
{attr value}
}
}
```

Table 13-9 BOMTemplate Attributes

Attribute	Description
Name	Name of the BoM template.
coreBuilderBOMTemplateVersion	Version of the coreBuilder BoM template syntax used to create this BoM template Determines which template reader the coreBuilder tool invokes to read the template.
Version	Version of this BoM Template

Example 13-2 Default_Template.tcl

```
# The default BOM Template

set BOMTemplate {
    BOMTemplate
        {Name {Synopsys Default BOM Template}}
        {coreBuilderBOMTemplateVersion 1.0}
        {Version 1.0}

    {Deliverable
        {Name "Verification Environment"}
        {DeliverableType TRE}
    }

    {Deliverable
        {Name "Documentation"}
    }
}
```

13.6.2.2 Custom BoM Template Example

[Example 13-3](#) shows a custom BoM Template based on the Default_Template, with the following additional features:

- Adds attributes to the existing Documentation filegroup to make it a required, autoload filegroup.
- Creates an additional filegroupGroup (Example_Design) with two filegroups (RAM_Example and ROM_Example).
- The RAM_Example and ROM_Example filegroups are autoload filegroups.
- When the coreKit is built, either RAM_Example or ROM_Example must have files in it (due to the OneRequiredGroup attribute).

Example 13-3 Custom BoM Template

```
# Custom BoM Template to create Example filegroups

set BOMTemplate {
    BOMTemplate
```

```
{Name {IP with Implementation Example}}
{coreBuilderBOMTemplateVersion 1.0}
{Version 1.0}

{Deliverable
  {Name "Verification Environment"}
  {DeliverableType TRE}
}

{Deliverable
  {Name "Documentation"}
  {AutoloadFilegroup true}
  {DefaultLoadPath doc/*.pdf}
  {DefaultInstallDir ./doc}
  {Importance Required}
}

{Group
  {Name "Example Design"}
  {Deliverable
    {Name "RAM Example"}
    {AutoloadFilegroup true}
    {DefaultLoadPath example/RAM/*.vhd}
    {DefaultInstallDir ./example/RAM}
    {OneRequiredGroup Impl_Example}
  }
  {Deliverable
    {Name "ROM Example"}
    {AutoloadFilegroup true}
    {DefaultLoadPath example/ROM/*.vhd}
    {DefaultInstallDir ./example/ROM}
    {OneRequiredGroup Impl_Example}
  }
}
}
```

14

Defining Interfaces for coreAssembler

For cores that have similar interfaces, you can create interface definitions that enable coreAssembler users to automatically build, connect, and synthesize subsystems that use those cores. This chapter describes how to define your core's interfaces for use in coreAssembler.

This chapter contains the following sections:

- [“Subsystem Assembly with coreAssembler”](#) on page 200
- [“Terminology”](#) on page 202
- [“General Procedure for Creating Interfaces”](#) on page 203
- [“Interface Setup Example”](#) on page 217
- [“coreAssembler Customization Hooks”](#) on page 239

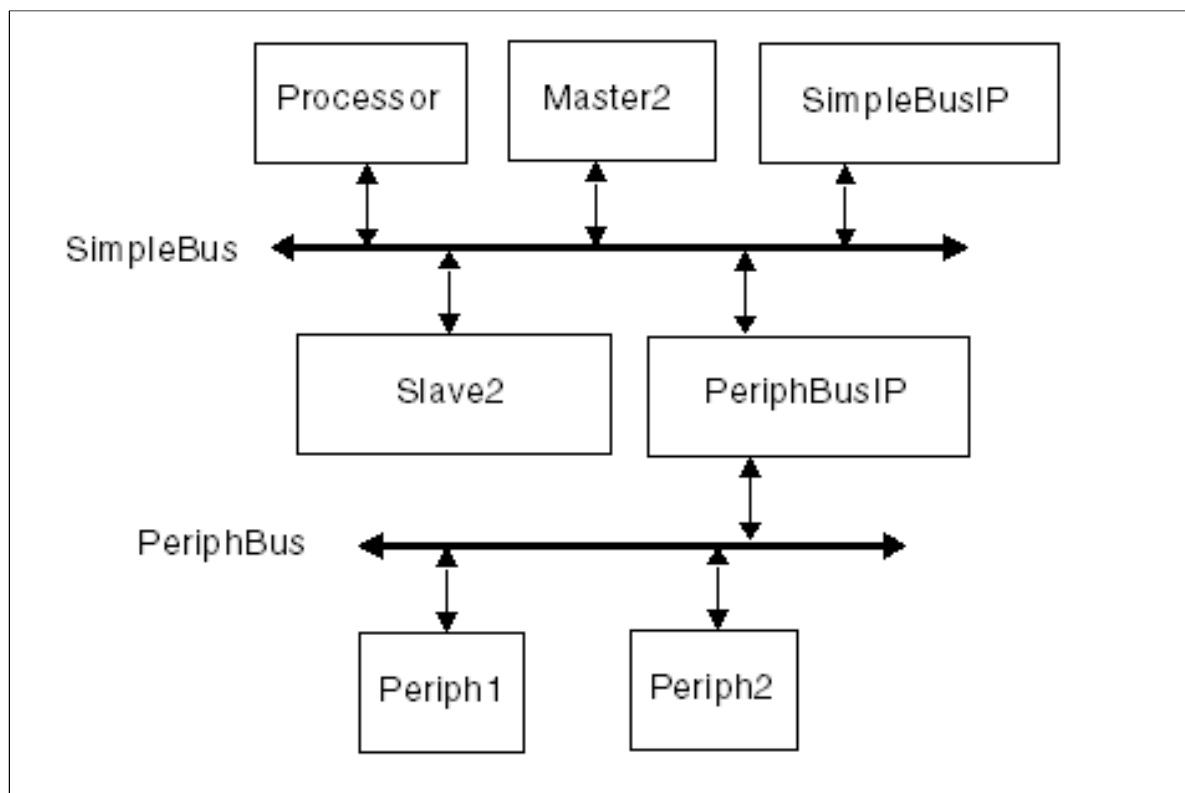
14.1 Subsystem Assembly with coreAssembler

The coreAssembler tool enables core integrators to automatically build subsystems from components with common interfaces. The interfaces can be industry-standard interfaces such as common buses or may IP-specific interfaces common to your own group of IP components.

For example, consider the subsystem shown in [Figure 14-1](#), in which the main elements are:

- SimpleBusIP – A bus IP component that provides arbitration and other protocol-related tasks on the SimpleBus
- Processor – A CPU (bus master device) attached to the SimpleBus
- Master2 – Another bus master device attached to the SimpleBus
- PeriphBusIP – A bus bridge IP block that bridges SimpleBus to PeriphBus
- PeriphBusIP is a slave device on SimpleBus and the only master on the PeriphBus.
- Slave2 – Another slave device on the SimpleBus
- Periph1 and Periph2 – Peripheral components (slave devices) attached to PeriphBus

Figure 14-1 Example Subsystem



If you define the interfaces for the components (by executing the coreBuilder SoC Assembly Intent activities) and package each component into a coreKit, you enable the coreAssembler user to:

- Build a subsystem such as that shown in [Figure 14-1](#) by adding or removing components as needed.
- Automatically connect the interfaces of components to each other. The coreAssembler tool automatically connects components that have the same interfaces.
- Manually connect and disconnect ports as needed.
- Generate the top-level HDL for the subsystem.

In addition to building the subsystem, the core integrator can also use the coreAssembler tool to:

- Configure each of the components in the subsystem, in a manner similar to the coreConsultant tool.
- Configure and execute the individual regression environment for each component.
- Export interfaces to the top level of the subsystem for connection at the next level of design hierarchy.
- Synthesize the subsystem, just as a core integrator uses the coreConsultant tool to synthesize the individual components.

The remainder of this chapter describes how to define interfaces in the coreBuilder tool for use in the coreAssembler tool. For more information about how to use the coreAssembler tool to build and synthesize subsystems, see the *coreAssembler User Guide*.

14.2 Terminology

The coreBuilder commands and attributes for defining interfaces use the following terminology:

- **Subsystem** – A system of interconnected components that a coreAssembler user can assemble from a set of components with pre-defined interfaces

The coreAssembler user selects components from a set of installed coreKits.

- **Interface** – A set of ports and parameters that define a connection point to a component

The actual connection points are the components' top-level ports. The interface definition contains the information that the coreAssembler tool needs to correctly connect ports of components (packaged as coreKits) to each other.

- **Interface provider** – A component that provides an interface for use by other components

For example, a bus IP component provides one or more interfaces for use by other components attached to the bus.

- **Interface consumer** – A component that uses an interface provided by an interface provider

For example, a bus IP component may provide a master interface which can support up to 16 bus masters. A bus master component, such as a processor, is a consumer of the interface.

- **Connectivity domain** – A region of a design, defined by the core integrator, in which all like-named interfaces should be connected

The coreAssembler tool automatically connects all like-named interfaces within a specified connectivity domain.

14.3 General Procedure for Creating Interfaces

As a core developer, you may be developing any or all of the IP blocks (cores) for use in the example subsystem shown in [Figure 14-1](#). For any core, the general procedure to define the core's interfaces for use in the coreAssembler tool is:

1. Define the interfaces in one or more Tcl command files (interface definition files).
2. Load the interface definitions into the coreBuilder tool by executing the Load Interface Definitions activity.
3. Instantiate the interfaces for a core. You can do so through the Add Interfaces activity GUI or by executing the `create_interface_instance` command in the workspace for the core.
4. Set instance-specific attributes on the interfaces and the interface ports and parameters. In particular, for interface ports and parameters that do not use the same names as the design ports and parameters, use the `InterfaceLink` attribute to explicitly make the required associations between the interface's ports/parameters and the core's ports/parameters.

You can set instance-specific attributes, such as `InterfaceLink`, either through the Add Interfaces activity GUI or by executing the following commands in the workspace:

- ☐ `set_interface_parameter_attribute`
 - ☐ `set_interface_port_attribute`
 - ☐ `set_interface_attribute`
5. After you perform the above steps for at least two coreKits that use instances of the same interface definition, test your work in the coreAssembler tool.

The following sections describe how to perform each step in the above procedure. For a complete example, see "[Interface Setup Example](#)" on page [217](#).

14.3.1 Step 1: Define Interface - GUI Method

The coreAssembler tool provides both a GUI method and a command method for creating interface definition TCL command file. The GUI-based method is useful for creating new interfaces or for major modification of existing interfaces. The command method is useful for quick instantiating of existing interfaces, or minor interface modification. Both methods create the same TCL interface definitions file, and are described here.

Interface definitions are not specific to any particular core. That is, you can instantiate the same interface definition for multiple cores. When you instantiate the interface for a core, you apply additional attributes to that instance of the interface.

14.3.1.1 GUI Method for Defining Interfaces

You access the GUI dialog in the Edit > Create TCL Interface Definition menu item. This dialog supports creation of new interface definitions TCL file as well as editing of existing interface definition TCL file. Import/edit/export of interface definition is supported in TCL format. You can also import an IP-XACT bus/abstraction definition and write a bus or abstraction definition. You cannot read a TCL interface definition and write as an IP-XACT bus or abstraction definition.

The Create TCL Interface Definition dialog has the following controls and fields:

- Interface Definition (VLNV) - The Vendor, Library, Name and Version for the interface (used by IP-XACT definitions)
- Direct Connection Allowed - Indicates that a direct master-to-slave connection is allowed
- Minimum/Maximum # of Consumers - Determines how many consumer connections can/must be made to the provider for the interface
- Interface Definition Ports - A spreadsheet to allow you to specify the ports that are included in the interface

The following can be entered for each port:

- Name - Auto-filled name from the Add Port dialog
- Direction - “fromProvider” | “fromConsumer” - Signal direction: fromProvider, fromConsumer, or bidirectional (blank)
- Default - Constant value to use when a fromConsumer port is open on the provider side:
 - none (blank) - Leave unconnected (the default)
 - zero - Connect to constant zero
 - one - Connect to constant one
 - dc - Don't care, allow Design Compiler to connect to zero or one during optimization
- Alignment - How to align data when a design does not use all of the bits of the interface port
 - full - Design port must use all bits (the default)
 - left - Align bits to left
 - right - Align bits to right
 - slice - Use bits specified by port (such as data[12:9])
- Separate - “false” (default) | “true” - Port has separate connections from each consumer to the provider (default for -direction fromConsumer)
- Optional - “false” (default) | “true” - Interface port is optional for consumers
- Description - Brief help message for the interface port
- Add Port - Add ports to the spreadsheet

When you add a name, a new row is created in the Interface Definition Ports spreadsheet, with some of the fields containing default data. You can fill in fields as you add ports, or provide details later.

- Remove Port - Remove ports from the spreadsheet

First, select a row in the Interface Definition Ports spreadsheet by clicking on the port name. Then click “Remove Port” to remove the port row from the spreadsheet.

- Interface Definition Parameters - A spreadsheet to allow you to specify the parameters included in the interface

Specify the following fields for each parameter:

- Name - The name is auto-filled from the Add Parameter dialog
- Type - “integer” | “bitfield” | “float” | “string” | “boolean” - Parameter type

- ❑ SpecifyOn - “provider” | “consumer” - The instance type that requires this parameter: provider or consumer
- ❑ Default - Default parameter value
- ❑ Label - Parameter field label (this label is used for the coreAssembler GUI)
- ❑ IsControlValue - “false” | “true”
- ❑ Description - Text description for the parameter; used for the “Whats this?” help field
- Add Parameter - Add parameters to the spreadsheet

When you add a parameter name, a new row is created in the Interface Definition Parameters spreadsheet, with some of the fields containing default data. You can fill in fields as you add ports, or provide details later.
- Remove Parameter - Removes parameters from the spreadsheet

First, select a row in the Interface Definition Parameters spreadsheet by clicking on the parameter name. Then click “Remove Parameter” to remove the parameter row from the spreadsheet.

14.3.1.2 GUI Method for Updating Interfaces

If you have an existing Interface Definitions file, you can use the GUI method to update this file, as follows:

- Choose Edit > Edit TCL Interface Definition

The “Interface Definition to Be Edited” dialog appears, which allows you to choose a TCL file to edit.
- Navigate and select the file to edit, then click “Open”

The Create TCL Interface Definition dialog is loaded with the values from the TCL file that you can add to or modify.
- After all of the additions/changes have been made, click OK to save the TCL file.



Attention

Interfaces can be common to many components. Be aware that changes to an interface definition may require updating all of these components with the new information.

14.3.2 Step 1 (alternate): Define Interfaces - Command Method

If you prefer, you can create or modify Interface Definition files using command line method. The following are required and optional commands for this purpose.

14.3.2.1 Required Interface Definition Commands

To define an interface, use the following commands in a Tcl interface definition file:

- create_interface - Create the interface and set the interface attributes (see [Table 14-1](#) on page 206).
- create_interface_parameter - Create parameters for the interface and set the interface parameter attributes listed in [Table 14-2](#) on page 206. The interface parameters may or may correspond to design parameters.

- `create_interface_port` – Create the ports for the interface and set the interface port attributes listed in [Table 14-3](#) on page 207.
- `complete_interface_definition` – Complete the interface definition. After executing this command, the interface definition cannot be modified. However, you can set instance-specific attributes on instances of the interface its ports and parameters.

14.3.2.2 Optional Interface Definition Commands

You can use the following optional commands in your interface definition command file (before the `complete_interface_definition` command). However, these commands are most commonly used to set attributes on instances of the interface, as described in “[Step 4: Set Attributes on Interface Instance](#)” on page 211:

- `set_interface_attribute` – Set attributes on the interface either instead of or in addition to setting attributes with `create_interface` options
 - `set_interface_parameter_attribute` – Set additional attributes on the interface parameters
- For example, you can set attributes to define the range of legal values and to label the parameter in the interface configuration GUI, as you can for other parameters (such as design parameters).
- `set_interface_port_attribute` – Set attributes on the interface ports, either instead of or in addition to setting attributes with `create_interface_port` options

Table 14-1 Attributes Set by `create_interface` Options

Option	Interface Attribute	Description
<i>name</i>	Name	Name of the interface, set by argument to <code>create_interface</code> command
<code>-version version</code>	InterfaceVersion	Version string for this version of the interface
<code>-min minConsumers</code>	MinConsumers	Minimum number of interface consumers
<code>-max maxConsumers</code>	MaxConsumers	Maximum number of interface consumers
<code>-require interfaces</code>	RequireInterfaces	Interface definitions required by this interface
<code>-description text</code>	Description	Brief help message for the interface
<code>-url url</code>	HelpUrl	URL of detailed help text for the interface

Table 14-2 Attributes Set by `create_interface_parameter` Options

Option	Interface Parameter Attribute	Description
<i>name</i>	Name	Name of the interface parameter, set by argument to <code>create_interface_parameter</code> command

Option	Interface Parameter Attribute	Description
-specify_on <i>specifyOn</i>	SpecifyOn	Instance type that requires this parameter value be specified: ☐ provider ☐ consumer
-type <i>type</i>	ValueType	Parameter type: ☐ integer ☐ bitfield ☐ float ☐ string ☐ boolean
-interface <i>interfaceName</i>	Parent	Name of the interface that includes this parameter
-unique	none	Specifies that the value for this parameter must be unique for each consumer
-default <i>value</i>	DefaultValue	Default value for this parameter
-label <i>label</i>	Label	GUI label for this parameter
-description <i>text</i>	Description	Brief help message for the interface parameter
-url <i>url</i>	HelpUrl	URL of detailed help text for the interface parameter
-used	UsedOnInstance	Condition under which the parameter is used on its instance

Table 14-3 Attributes Set by create_interface_port Options

Option	Interface Port Attribute	Description
<i>name</i>	Name	Name of the interface port, set by argument to create_interface_port command
-interface <i>interfaceName</i>	Parent	Name of the interface that includes this port
-direction <i>direction</i>	LinkDirection	Signal direction: ☐ fromProvider ☐ fromConsumer ☐ bidirectional (blank)
-optional	Optional	Specifies that this interface port is optional for consumers

Option	Interface Port Attribute	Description
-separate	SeparateConsumerConnection (true)	This port has separate connections from each consumer to the provider (default for -direction fromConsumer)
-common	SeparateConsumerConnection (false)	Consumers share a common connection to the provider (default for -direction bidirectional)
-size <i>size</i>	InterfaceSize	Bit-width of the port
-align <i>busAlign</i>	BusAlignment	How to align data when a design does not use all of the bits of the interface port: ☐ full – Design port must use all bits (default) ☐ left – Align bits to left ☐ right – Align bits to right ☐ slice – Use bits specified by port (such as data[12:9])
-constant <i>constantValue</i>	DefaultConstantPort	Constant value to use when a fromConsumer port is open on the provider side: ☐ none – Leave unconnected (default) ☐ zero – Connect to constant zero ☐ one – Connect to constant one ☐ dc – Don't care, allow Design Compiler to connect to zero or one during optimization
-description <i>text</i>	Description	Brief help message for the interface port
-url <i>url</i>	HelpUrl	URL of detailed help text for the interface port

14.3.2.3 Interface Definition Example File

[Example 14-1](#) shows an interface definition command file that defines a simple interface with one port and one parameter. For more complete examples of interface definitions, see [Interface Setup Example](#) on page 217. For more information about the commands and attributes involved in interface definition, see the online command reference.

Example 14-1 Interface Definition File for Single-Port Interface

```
# DataIntfc.tcl - Interface definition for DataIntfc

set DataIntfc_Version 1.0

# Create the interface. DataIntfc can have 1-4 consumers.
create_interface DataIntfc \
    -version $DataIntfc_Version \
    -min 1 \
    -max 4 \
    -description \
    "Data Interface"
```

```
# Width of data port can be 16 or 32. Value is set on the interface provider.
create_interface_parameter data_width \
    -interface DataIntfc \
    -type integer \
    -specify_on provider \
    -default 16 \
    -label "Data Bus Width" \
    -description "Number of bits on the data bus"

set_interface_parameter_attribute DataIntfc data_width \
    EnumValues {16 32}

# Single data bus from provider to all consumers.
create_interface_port data \
    -interface DataIntfc \
    -direction fromProvider \
    -size @data_width \
    -common \
    -description "Data bus."

complete_interface_definition DataIntfc
```

14.3.3 Step 2: Load Interface Definitions

For each coreKit that is to use the interfaces you defined in “[Step 1: Define Interface - GUI Method](#)” on page 203, you need to load the interface definition into the coreBuilder workspace and then instantiate the interface for the core. To load the interface definition into your coreBuilder workspace, execute the Load Interface Definitions activity. To do so in command line mode, use the commands in the following example:

```
# Interface definition file is /usr/cores/interfaces/DataIntfc.tcl.
set_activity_parameter LoadInterfaceDefinitions \
    InterfaceDefinitionFileNames {/usr/cores/interfaces/DataIntfc.tcl}
autocomplete_activity LoadInterfaceDefinitions
```

To complete the Load Interface Definitions activity in GUI mode:

1. Click Packaging > SoC Assembly Intent > Load Interface Definitions coreBuilder Activity to display the Load Interface Definitions dialog in the Activity View pane.
2. Click on the “Add ‘Interface Definitions’ From Folder” button.
3. In the “Import Directory” dialog, navigate to and select the folder that contains your interface definitions.
The coreBuilder tool loads the interface definition files in that directory (for example, DataIntfc.tcl) in the Interface Definitions File Names spreadsheet.
4. You can manage items in the spreadsheet (add more folders, or remove file entries) before applying the activity.
5. Click “Apply” to load these definitions, and then move to the Add Interfaces activity.

It is possible to utilize IP-XACT bus definitions and abstraction definitions directly in this activity. This is done by specifying the names of the files containing the bus definition and abstraction definitions to be utilized. IP-XACT files and the standard TCL interface definitions can be inter-mixed, if required.

14.3.4 Step 3: Add Interfaces

To instantiate an interface that is defined in a previously loaded interface definition file, use the `create_interface_instance` command as shown in the following example:

```
# Create instance of interface DataIntfc.
create_interface_instance \
-interface DataIntfc \
-type provider \
BusIPDataIntfc
```

The `-interface` option specifies the name of the interface to be instantiated.

The `-type` option specifies the type of component on which you are instantiating the interface, as defined in [Table 14-4](#).

Table 14-4 Interface Instance Types

Value of -type Option	GUI Label in Add Interface Instance to Core Dialog	Definition
provider	Provider	The coreKit is for an interface provider, such as a bus IP block.
consumer	Consumer	The coreKit is for an interface consumer, such as bus master or slave device.
internal-consumer	Internal Consumer	The coreKit is for an interface provider that also behaves internally as a consumer of the interface. For example, if a bus IP component provides a slave interface for up to 16 slaves, but also uses one of the slave interfaces, specify -type internal-consumer.
interface-monitor	Interface Monitor	The coreKit is for a component that is to be used only for verification purposes (SimulationModel attribute = true). An interface monitor monitors all activity on the interface. It does not count as a consumer, and it does not have any output ports.
consumer-monitor	Consumer Monitor	The coreKit is for a monitor that is to be used only for verification purposes for a specific consumer. The core integrator associates the consumer monitor with a specific consumer in coreAssembler. A consumer monitor has only input ports and connects only to signals to/from the associated consumer. The coreAssembler tool automatically transfers configuration parameters from the consumer to its associated consumer monitor.

In GUI mode, you instantiate an interface through the Add Interfaces dialog. You also set instance-specific attributes through the Add Interfaces dialog as described in “[Step 4: Set Attributes on Interface Instance](#)” on page 211.

To create the interface instance in GUI mode:

1. Click Add Interfaces in the Packaging section of the coreBuilder Activity List to invoke the Add Interfaces dialog in the Activity View pane.
2. Click Add to invoke the Add Interface Instance to Core dialog.
3. Select an interface from the Interface Definition list. The Interface Definition list includes the interface definitions that you loaded in the Load Interface Definitions activity.
4. Enter an instance name (for example, BusIPDataIntfc) in the Instance Name field.
5. Select the Interface Type (see [Table 14-4](#)).
6. Click Apply to add the interface instance and return to the Add Interfaces dialog.
7. Continue to “[Step 4: Set Attributes on Interface Instance](#)” on page 211 to set attributes on the new instance of the interface.

14.3.5 Step 4: Set Attributes on Interface Instance

You need to set attributes on your instance of an interface (and its ports and parameters) for the following reasons:

- To modify values of the attributes specified in the interface definition. For example, you may need to limit the maximum number of consumers of the interface.
- To associate the interface ports and parameters with the design ports and parameters. The coreBuilder tool automatically associates interface ports/parameters with design ports/parameters that have the same names. For design ports/parameters with names that differ from the interface definition, you need to make the required associations explicitly.
- To hook up interface ports to design ports (AssociationFormat attribute). When trying to hook an interface port with the design port, the AssociationFormat value %s is replaced by the name of the interface port. The resultant string after substitution of %s is used to find a design port.

Example: Suppose AssociationFormat attribute is set to %s_%s_suffix. Let us assume that the bus1 interface has 2 ports sig1 and sig2. Let us assume that the design has 2 ports sig1_sig1_suffix and sig2_sig2_suffix that have the correct port direction. Then the sig1 interface port will be connected to sig1_sig1_suffix interface port and the sig2 design port will be connected to the sig2_sig2_design port. Basically the 2 instances of %s in the string are replaced with the name of the corresponding interface port.

14.3.5.1 Command Line Mode:

To set attributes on an instance of an interface and its ports and parameters, use the -instance option with the set_interface_attribute, set_interface_port_attribute, and set_interface_parameter_attribute commands as shown in the following example:

```
# Limit this instance to 2 consumers.
set_interface_attribute -instance BusIPDataIntfc MaxConsumers 2

# Associate interface port data with design port rdata.
```

```

set_interface_port_attribute \
-instance BusIPDataIntfc data InterfaceLink rdata

# Associate coreBuilder-generated interface parameter NumConsumers
# with design parameter num_masters.
set_interface_parameter_attribute \
-instance BusIPDataIntfc NumConsumers InterfaceLink num_masters

```

Table 14-5, Table 14-6, and Table 14-7 list the attributes you can set on instances of interfaces and their ports and parameters. The tables also include the corresponding information needed to set attribute values through the Add Interfaces activity dialogs in GUI mode.

After you set the required interface, port, and parameter attributes, complete the Add Interfaces activity:

```
autocomplete_activity AddInterfaces
```

Table 14-5 Interface Instance Attributes

GUI Label	Attribute	Description
Association Format	AssociationFormat	Enables easy hookup of interface ports to design ports When trying to hook an interface port with the design port, %s is replaced by the name of the interface port. The resultant string after substitution of %s is used to find a design port. Default value: %s
MinConsumers	MinConsumers	Minimum number of consumers The default is inherited from the create_interface -min option in the interface definition.
MaxConsumers	MaxConsumers	Maximum number of consumers The default is inherited from the create_interface -max option in the interface definition.
Unique Consumer Connections	UniqueConsumerConnections	Indicates (if true) to show unique provider-to-consumer connections in the coreAssembler schematic view
The attributes listed below are read-only attributes that appear in the Add Interfaces dialog:		
Interface	InterfaceDefinitionName	Read-only – Name of the interface of which this is an instance This is set by create_interface_instance -interface.
Type	InterfaceType	Read-only – Interface type: provider, consumer, internal consumer, interface-monitor, or consumer-monitor This is set by create_interface_instance -type.
complete	AssociationComplete	Read-only – Indicates (if true) that all of the interface's ports and parameters have been linked to the required design ports and parameters
Version	InterfaceVersion	Read-only – Interface version string

Table 14-5 Interface Instance Attributes (continued)

GUI Label	Attribute	Description
		This is set by create_interface -version option.
Memory Map	MemoryMap	Read-only – Attached to an interface instance; specifies the name of the memory map associated with a component interface Assign with the set_instance_attribute command.

Table 14-6 Interface Instance Port Attributes

GUI Label	Attribute	Description
Link to	InterfaceLink	Name of the design port that this interface port corresponds to For an interface port that does not correspond to a design port, set InterfaceLink to <open>. If the interface port and the design port have opposite polarities, this can be indicated by prefixing the InterfaceLink value with the "!" character.
The attributes listed below are read-only attributes that appear in the Associate Interface Ports and Parameters dialog:		
Direction	RequiredPortDirection	Read-only – Indicates port directionality
#Bits	InterfaceSize	Read-only – Indicates port bit-width NOTE: If an IP-XACT bus definition, the '#Bits' read-only column is replaced with four editable columns for specifying the left and right indexes of the logical and physical port mapping which corresponds directly to details in an IP-XACT component.
needs @Param	RequiresParamIndexing	Read-only – Indicates (if true) that the value of InterfaceLink depends on a parameter value
optional	OptionalAssociation	Read-only – If OptionalAssociation = false, the interface port must be associated with a design port; if true, the association is optional
complete	AssociationComplete	Read-only – Indicates (if true) that the interface port is linked to a design port

Table 14-7 Interface Instance Parameter Attributes

GUI Label	Attribute	Description
Link to	InterfaceLink	Name of the design parameter that this interface parameter corresponds to For an interface parameter that does not correspond to a design parameter, set InterfaceLink to <open>.
Value	Value	Current value of the parameter

Table 14-7 Interface Instance Parameter Attributes (continued)

GUI Label	Attribute	Description
Read-only	ReadOnlyParam	Indicates (if true) that the parameter is read-only for coreAssembler users
The attributes listed below are read-only attributes that appear in the Associate Interface Ports and Parameters dialog:		
Specify on	SpecifyOn	Read-only – Set by create_interface_parameter -specify_on option; indicates if the parameter value must be set on the provider or a consumer of the interface
needs @Param	RequiresParamIndexing	Read-only – Indicates (if true) that the value of InterfaceLink depends on a parameter value
optional	OptionalAssociation	Read-only – If OptionalAssociation = false, the interface parameter must be associated with a design parameter; if true, the association is optional
complete	AssociationComplete	Read-only – Indicates (if true) that the interface parameter is linked to a design parameter

14.3.5.2 GUI Mode:

In GUI mode, setting attributes on interface instances is part of the Add Interfaces activity. To set attributes on an interface instance and its ports and parameters in GUI mode:

1. Open the Add Interfaces dialog if not already open and select an instance from the list of interface instances.
2. Click Associate Interface to invoke the Associate Interface Ports and Parameters dialog, where you can specify how to link the interface ports and parameters to the design ports and parameters.

The Associate Interface Ports and Parameters dialog displays the current associations, which are already linked for either of the following reasons:

- ❑ The interface port/parameter names match the design port/parameter names.
- ❑ You have already explicitly specified the links.

The Associate Interface Ports and Parameters dialog also displays the current values of the interface port and parameter attributes listed in [Table 14-6](#) and [Table 14-7](#).

3. Modify the Link to data as needed to link your interface ports and parameters to the design ports and parameters, then click OK. For interface ports and parameters that do not correspond to design parameters, yet still require association, specify <open> as the value for Link to.
4. In the Add Interface dialog, click Report to generate and display a report about the current interface specifications. If the report indicates your intended interface setup, click Apply to complete the Add Interfaces activity.

14.3.6 Step 5: Test Interface

To check that your interface definition and its instances work as you intended:

1. Build at least two coreKits that include instances of the same interface; make sure one of the coreKits is for the interface provider.
2. Use the coreConsultant tool to install the coreKits.
3. Use the coreAssembler tool to build a subsystem from the components (coreKits) that you installed in step 2. Follow the procedures in the *coreAssembler User Guide*.
4. Verify that the coreAssembler tool connects the designs as intended.

14.3.7 Summary of Interface Related Commands

[Table 14-8](#) summarizes the commands related to interface definition and instantiation. For more details, see the online reference and examples in the following section [Interface Setup Example](#) on page 217.

Table 14-8 Commands for Setting Up Interfaces

Command	Short Descriptions
create_interface	Create an interface Use create_interface in a Tcl interface definition file to create an interface that can be used in one or more coreKits.
create_interface_parameter	Create a parameter for an interface Use create_interface_parameter in an interface definition file to create parameters for the interface.
create_interface_port	Create a port for an interface Use create_interface_port in an interface definition file to create ports for the interface.
complete_interface_definition	Complete the definition an interface Use complete_interface_definition in an interface definition file to complete the definition of an interface (and its ports and parameters) and make the interface definition unmodifiable.
create_interface_instance	Create an instance of a pre-defined interface Use create_interface_instance in a coreBuilder workspace to create an instance of an interface for a core.
set_interface_attribute	Set attributes on an interface or interface instance (-instance option) Use set_interface_attribute in a Tcl interface definition file to set attributes on the interface and (optionally) in a coreBuilder workspace to set instance-specific attributes on an interface instance.
set_interface_port_attribute	Set the value of an attribute on an interface port or interface instance port (-instance option)

Table 14-8 Commands for Setting Up Interfaces (continued)

Command	Short Descriptions
	Use <code>set_interface_port_attribute</code> in a Tcl interface definition file to set attributes on interface ports and (optionally) in a coreBuilder workspace to set instance-specific attributes on an interface instance port (for example, to associate an interface port with a design port).
<code>set_interface_parameter_attribute</code>	Set the value of an attribute on an interface parameter or interface instance parameter (-instance option) Use <code>set_interface_parameter_attribute</code> in a Tcl interface definition file to set attributes on interface parameters and (optionally) in a coreBuilder workspace to set instance-specific attributes on an interface instance parameter (for example, to associate an interface parameter with a design parameter).
<code>get_interface_attribute</code>	Get the current value of an attribute on an interface or interface instance (-instance option)
<code>get_interface_port_attribute</code>	Get the current value of an attribute on an interface port or interface instance port (-instance option)
<code>get_interface_parameter_attribute</code>	Get the current value of an attribute on an interface parameter or interface instance parameter (-instance option)

14.3.8 NumConsumers Parameter

The coreBuilder tool automatically creates the NumConsumers parameter to track the number of interface consumers currently implemented in a coreAssembler subsystem. The coreAssembler tool automatically increments/decrements the NumConsumers value when the core integrator adds/removes consumers in the subsystem. You can reference the value of NumConsumers (using `@NumConsumers`) in your interface definitions, as shown in the example interface definitions in the following sections.

14.4 Interface Setup Example

The following sections show how to use the previously described procedures, commands, and attributes to set up interfaces for the example subsystem shown in [Figure 14-1](#).

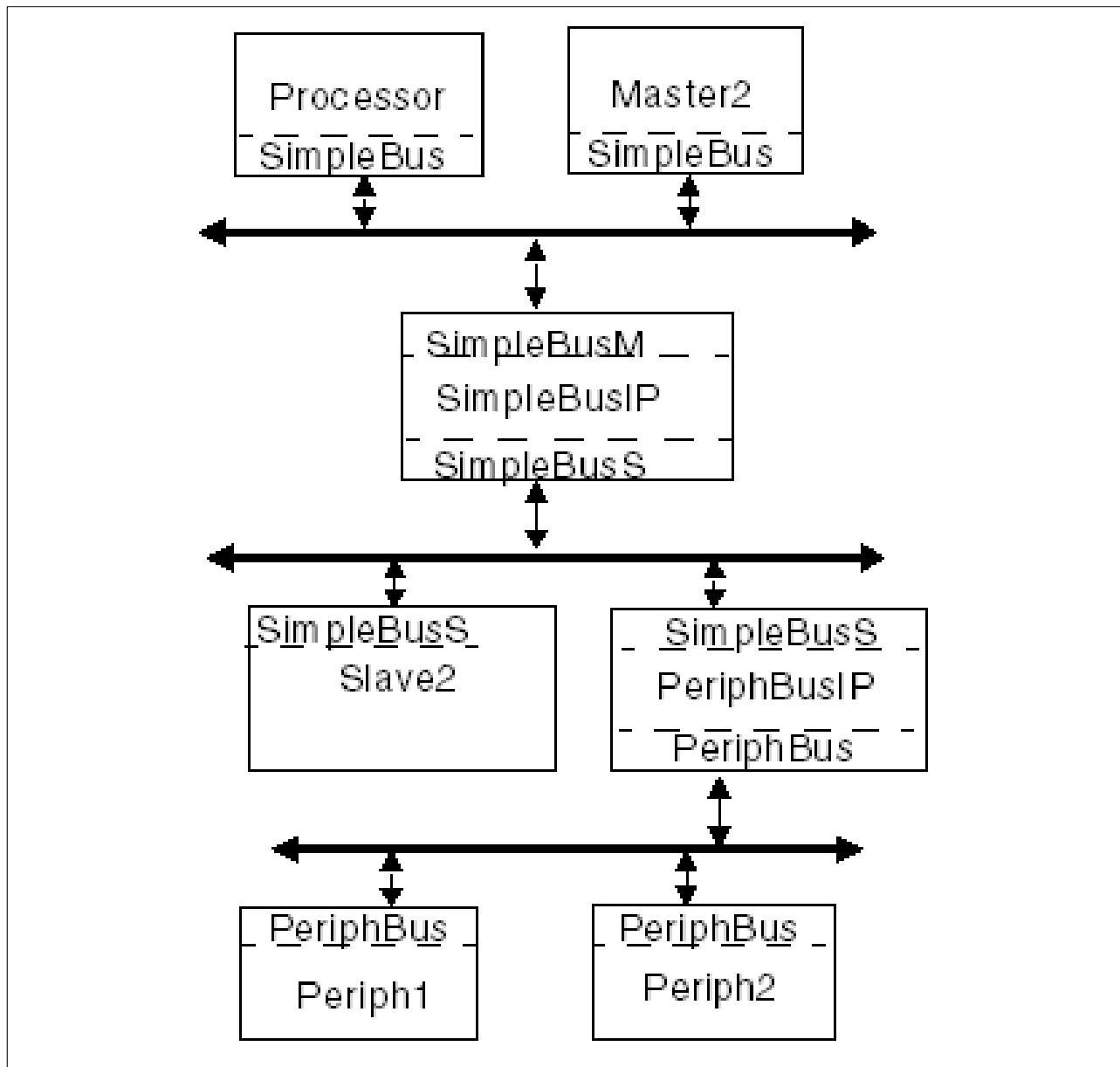
14.4.1 Required Interfaces

[Figure 14-2](#) shows the interface definitions that would be required on the various blocks in the example subsystem shown in [Figure 14-1](#) to enable automatic connection of bus components in coreAssembler. Separate interface definitions are required for master and slave devices on SimpleBus because the master and slave interfaces involve different ports and different port directionality. The interface providers and consumers are:

- SimpleBusIP is the provider of interfaces SimpleBusM and SimpleBusS.
- Processor and Master2 are consumers of SimpleBusM.
- PeriphBusIP and Slave2 are consumers of SimpleBusS.
- PeriphBusIP is the provider of interface PeriphBus.
- Periph1 and Periph2 are consumers of PeriphBus.

To create the interfaces required for automatic connection of the subsystem shown in [Figure 14-2](#), you would have to perform the following steps:

1. Define the interfaces SimpleBusM, SimpleBusS, and PeriphBus.
2. Instantiate SimpleBusM for SimpleBusIP and for each SimpleBus master (Processor and Master2).
3. Instantiate SimpleBusS for SimpleBusIP and for each SimpleBus slave (PeriphBusIP and Slave2).
4. Instantiate PeriphBus for PeriphBusIP and each PeriphBus slave (Periph1 and Periph2).

Figure 14-2 Interface Definitions for Example Subsystem

14.4.2 Defining the SimpleBusM Interface

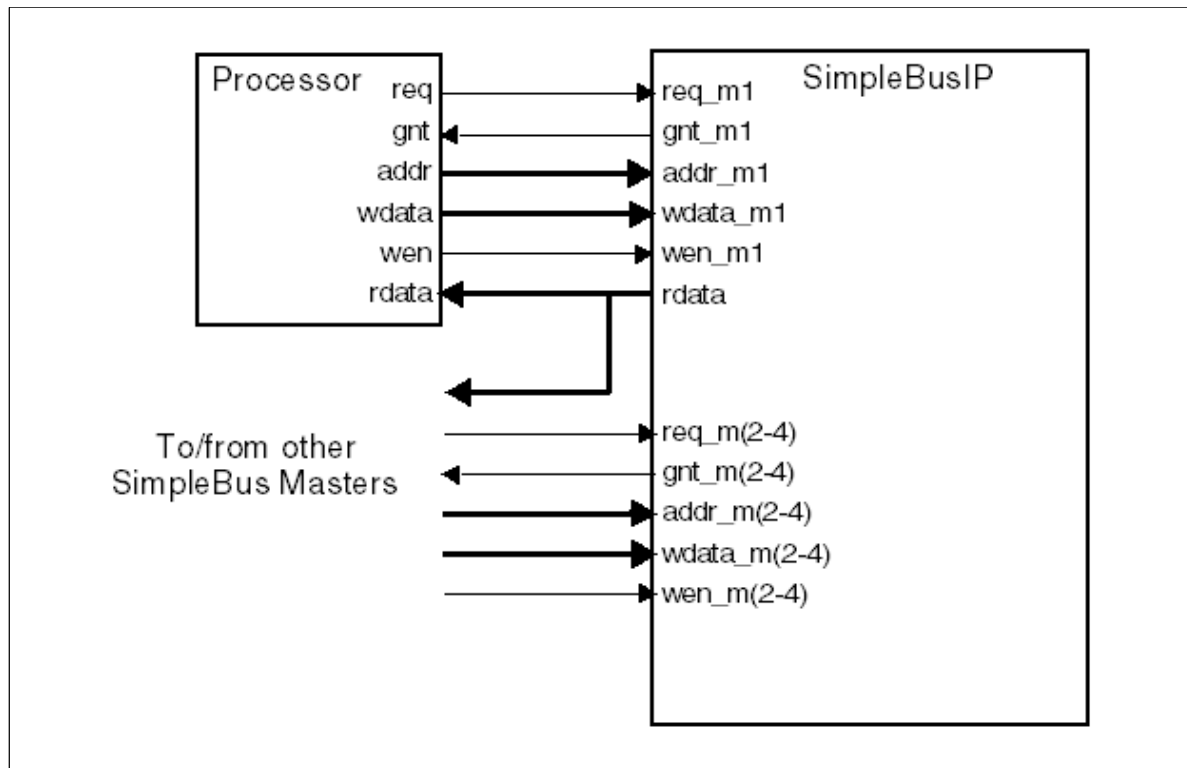
Figure 14-3 on page 219 shows the intended connection of SimpleBus master signals between the Processor and SimpleBusIP. In this example, SimpleBusIP supports connections for 1–4 SimpleBus masters. To enable automatic connection of ports as shown in Figure 21, you need to define the SimpleBusM interface, then instantiate the SimpleBusM interface for both Processor and SimpleBusIP.

Example 14-2 shows the entity declaration for the Processor (SimpleBus master) core. Example 14-3 shows the Tcl interface definition file that creates the interface SimpleBusM. The interface ports correspond the design ports. In this example, the interface ports and Processor ports have the same names; however, this is not a requirement.

Interface parameters may or may not correspond to design parameters. For example:

- The interface definition file in [Example 14-3](#) does not create interface parameters for design parameters num_alus and num_fpus, because the num_alus and num_fpus do not affect the interface configuration.
- The interface definition file in [Example 14-3](#) creates the master_slot parameter, which is not a Processor design parameter. Instead, the master_slot parameter specifies the unique master slot number occupied by Processor in a subsystem and exists because the interface provider (SimpleBusIP) can support up to 4 bus masters. SimpleBusIP uses the master_slot parameter to map interface ports to design ports according to the unique master_slot values of individual SimpleBusM consumers.

Figure 14-3 SimpleBusM Port Connections



Example 14-2 SimpleBus Ports on Example Processor Entity

```
entity Processor is
  generic (
    -- reuse-pragma attr Description Number of ALUS to implement.
    -- reuse-pragma attr Label {Number of ALUS}
    num_alus : natural range 1 to 4 := 1;
    -- reuse-pragma attr Description Number of FPUS to implement.
    -- reuse-pragma attr Label {Number of FPUS}
    num_fpus : natural range 0 to 2 := 0;
    -- reuse-pragma attr Description Size of the data bus.
    -- reuse-pragma attr EnumValues 16 32
  )
```

```

-- reuse-pragma attr Label {Data Bus Width}
data_width : natural range 16 to 32 := 16;
-- reuse-pragma beginAttr Description
-- Indicates if system is big-endian or little-endian.
-- reuse-pragma endAttr
-- reuse-pragma attr SymbolicNames big-endian little-endian
-- reuse-pragma attr Label System Endianness
big_endian : natural range 0 to 1 := 0

);

port (
    clk          : in  bit;
    rst          : in  bit;

    <other_inputs>
    <other_output>

-- SimpleBus Master Ports
    addr          : out bit_vector(15 downto 0);
    wdata         : out bit_vector(data_width-1 downto 0);
    rdata         : in  bit_vector(data_width-1 downto 0);
    req           : out bit;
    gnt           : in  bit;
    wen           : out bit
);

end Processor;

```

Example 14-3 SimpleBusM Interface Definition File

```

# SimpleBusM.tcl - Defines the SimpleBusM interface

set SimpleBus_Version 1.0

# Begin the interface definition
create_interface SimpleBusM \
    -version $SimpleBus_Version \
    -min      1 \
    -max      4 \
    -description \
    "SimpleBus Master"

# Create interface parameter to hold unique consumer slot numbers.
create_interface_parameter master_slot \
    -interface SimpleBusM \
    -type integer \
    -specify_on consumer \
    -unique \
    -label "SimpleBus Master Slot"

```

```

        -description \
"Slot number of this SimpleBus Master, range 1-4."

set_interface_parameter_attribute SimpleBusM master_slot \
    MinValue      1
set_interface_parameter_attribute SimpleBusM master_slot \
    MaxValue      @NumConsumers

# Create interface parameter to set data width for the interface.
create_interface_parameter data_width \
    -interface SimpleBusM \
    -type integer \
    -specify_on provider \
    -default 16 \
    -label "Data Bus Width" \
    -description \
"Number of bits on the SimpleBusM data bus"

set_interface_parameter_attribute SimpleBusM data_width \
    EnumValues {16 32}

# Create interface parameter to set endian mode for the interface.
create_interface_parameter big_endian \
    -interface SimpleBusM \
    -type integer \
    -specify_on provider \
    -default 0 \
    -label "Data Endianness" \
    -description \
"If set, the SimpleBusM system is big-endian."

set_interface_parameter_attribute SimpleBusM big_endian \
    MinValue      0
set_interface_parameter_attribute SimpleBusM big_endian \
    MaxValue      1
set_interface_parameter_attribute SimpleBusM big_endian \
    SymbolicNames {little-endian big-endian}

# Create 16-bit address port from each consumer.
create_interface_port addr \
    -interface SimpleBusM \
    -direction fromConsumer \
    -size 16 \
    -constant zero \
    -description \
"Address bus from each master."

# Create bus req port from each consumer.
create_interface_port req \

```

```

        -interface SimpleBusM      \
        -direction fromConsumer    \
        -constant zero             \
        -description               \
"Bus Request signal."

# Create write data port from each consumer; width = data_width.
create_interface_port wdata      \
    -interface SimpleBusM      \
    -direction fromConsumer    \
    -size @data_width         \
    -constant dc              \
    -description               \
"Write data bus."

# Create write enable port from each consumer; active high so
# tie unconnected ports low.
create_interface_port wen        \
    -interface SimpleBusM      \
    -direction fromConsumer    \
    -constant zero             \
    -description               \
"Indicates that a write transfer is taking place."

# Create unique gnt port to each consumer.
create_interface_port gnt        \
    -interface SimpleBusM      \
    -direction fromProvider    \
    -separate                  \
    -description               \
"Grant signal to bus master."

# Create single read data port to all consumers.
create_interface_port rdata      \
    -interface SimpleBusM      \
    -direction fromProvider    \
    -common                    \
    -size @data_width         \
    -description               \
"Read data from slaves."

# Complete the interface definition
complete_interface_definition SimpleBusM

```

14.4.3 Instantiating SimpleBusM for Processor

[Example 14-4](#) shows how to instantiate the SimpleBusM interface for Processor (a SimpleBusM consumer) in your coreBuilder workspace for the Processor coreKit.

For the Processor example, you do not need to specify any additional instance-specific attributes on the interface instance ports because the interface port names match the corresponding Processor port names. The interface parameters that have corresponding Processor parameters also use the same names as the corresponding design parameters. The coreBuilder tool automatically maps the interface ports and parameters to the design ports and parameters that have the same names.

However, the interface parameter `master_slot` does not have a corresponding design parameter and, therefore must have the `InterfaceLink` attribute set to `<open>`.

This completes the interface definition for the Processor block. The next step for the Processor core would be to build the Processor coreKit and test the interface connection in the coreAssembler tool.

Example 14-4 Instantiating the SimpleBusM Interface for Processor

```
# Load SimpleBusM interface definition
set_activity_parameter LoadInterfaceDefinitions \
InterfaceDefinitionFileNames /usr/cores/interfaces/SimpleBusM.tcl
autocomplete_activity LoadInterfaceDefinitions

create_interface_instance -interface SimpleBusM \
                           -type consumer \
                           SimpleBusM_CPU

#Interface parameter master_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance SimpleBusM_CPU \
    master_slot InterfaceLink <open>

autocomplete_activity AddInterfaces
```

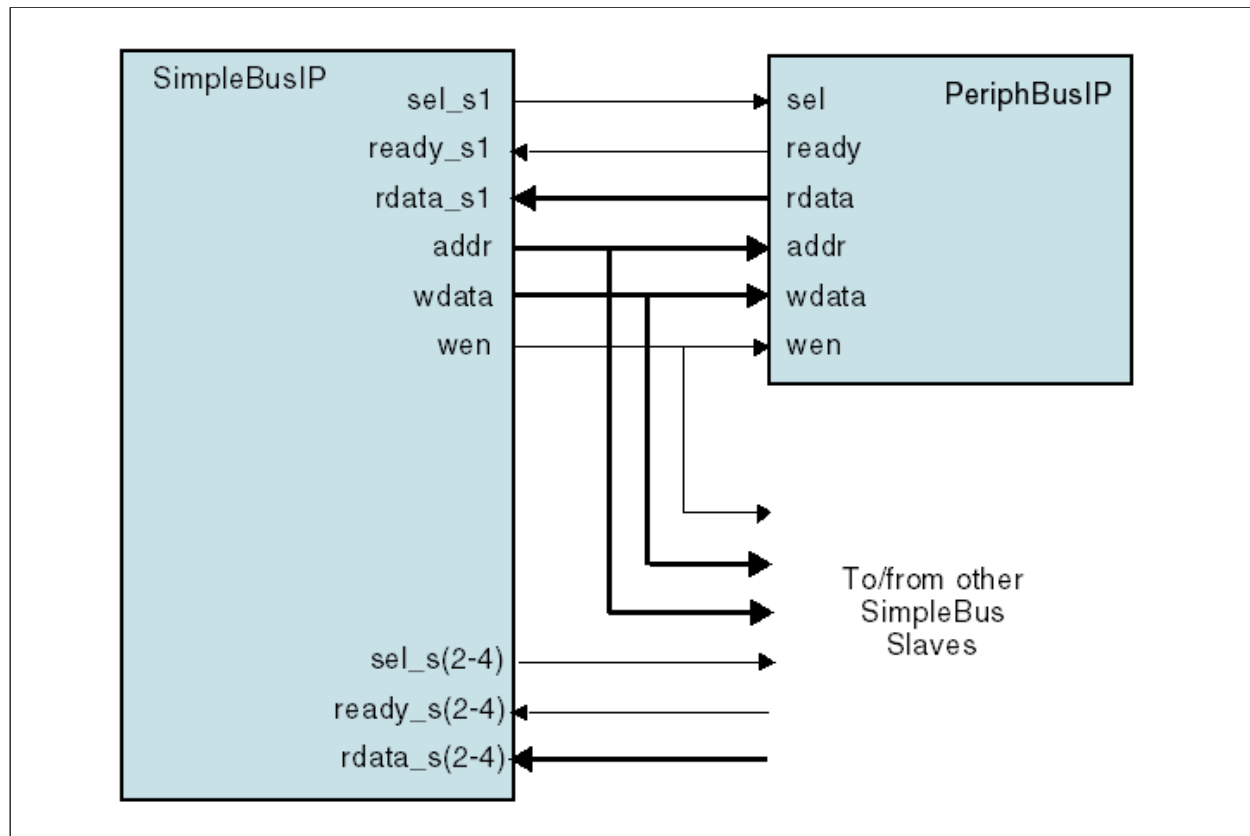
14.4.4 Defining the SimpleBusS Interface

SimpleBusIP requires instances of both SimpleBusM and SimpleBusS. Before instantiating SimpleBusM for the SimpleBusIP block, define the SimpleBusS interface so that you can load both interface definitions when you execute the Load Interface Definitions activity for SimpleBusIP.

[Figure 14-4](#) on page 224 shows the intended connections between SimpleBusIP and a SimpleBus slave device. SimpleBusIP is the provider of SimpleBusS; slaves, including the PeriphBusIP block, are consumers.

[Example 14-5](#) on page 224 shows the interface definition file that creates the SimpleBusS interface.

The following sections show how to instantiate SimpleBusS on SimpleBusIP (the provider) and PeriphBusIP (a consumer).

Figure 14-4 SimpleBusS Port Connections**Example 14-5 SimpleBusS Interface Definition File**

```

set SimpleBus_Version 1.0

# Begin the interface definition
create_interface SimpleBusS \
    -version $SimpleBus_Version \
    -min 1 \
    -max 4 \
    -description \
    "SimpleBus Slave"
# Create interface parameter to hold unique consumer slot numbers.
create_interface_parameter slave_slot \
    -interface SimpleBusS \
    -type integer \
    -specify_on consumer \
    -unique \
    -label "SimpleBus Slave Slot" \
    -description \
    "Slot number of this SimpleBus Slave, range 1-4."

set_interface_parameter_attribute SimpleBusS slave_slot \

```

```

        MinValue      1
set_interface_parameter_attribute SimpleBusS slave_slot \
        MaxValue      @NumConsumers

# Create interface parameter to set data width for the interface.
create_interface_parameter data_width \
        -interface SimpleBusS \
        -type integer \
        -specify_on provider \
        -default 16 \
        -label "Data Bus Width" \
        -description \
        "Number of bits on the SimpleBusS data bus"

set_interface_parameter_attribute SimpleBusS data_width \
        EnumValues {16 32}

# Create interface parameter to set endian mode for the interface.
create_interface_parameter big_endian \
        -interface SimpleBusS \
        -type integer \
        -specify_on provider \
        -default 0 \
        -label "Data Endianness" \
        -description \
        "If set, the SimpleBusS system is big-endian."

set_interface_parameter_attribute SimpleBusS big_endian \
        MinValue      0
set_interface_parameter_attribute SimpleBusS big_endian \
        MaxValue      1
set_interface_parameter_attribute SimpleBusS big_endian \
        SymbolicNames {little-endian big-endian}

# Create single 16-bit address port to each consumer.
create_interface_port addr \
        -interface SimpleBusS \
        -direction fromProvider \
        -size 16 \
        -common \
        -description \
        "Muxed address bus to all slaves."

# Create ready port from each consumer; active high so
# tie unconnected ports low.
create_interface_port ready \
        -interface SimpleBusS \
        -direction fromConsumer \
        -constant zero

```

```

        -description          \
"Slave ready signal."

# Create read data port from each consumer; width = data_width.
create_interface_port rdata          \
        -interface SimpleBusS      \
        -direction fromConsumer    \
        -size      @data_width     \
        -constant  dc              \
        -description              \
"Read data bus from slave."

# Create single write enable port to all consumers.
create_interface_port wen            \
        -interface SimpleBusS      \
        -direction fromProvider    \
        -common                    \
        -description              \
"Indicates that a write transfer is taking place."

# Create unique sel port to each consumer.
create_interface_port sel            \
        -interface SimpleBusS      \
        -direction fromProvider    \
        -separate                  \
        -description              \
"Select signal to selected slave."

# Create single write data port to all consumers.
create_interface_port wdata          \
        -interface SimpleBusS      \
        -direction fromProvider    \
        -common                    \
        -size      @data_width     \
        -description              \
"Muxed write data to all slaves."

# Complete the interface definition
complete_interface_definition SimpleBusS

```

14.4.5 Instantiating Interfaces for SimpleBusIP

SimpleBusIP is the provider of the SimpleBusM and SimpleBusS interfaces and, therefore, requires instances of both SimpleBusM and SimpleBusS. Instantiating SimpleBusM and SimpleBusS for SimpleBusIP is more complex because not all SimpleBusIP port names match their corresponding interface port names. Therefore, some interface ports require explicit association with corresponding ports on the SimpleBusIP core.

[Example 14-6](#) shows the SimpleBus master and slave port definitions in the SimpleBusIP entity declaration. The GenerateIf attributes on the optional ports determine whether those ports exist in the configured version of the SimpleBusIP design.

[Example 14-7](#) shows how to load the interface definitions for SimpleBusM and SimpleBusS, and create instances of each interface for SimpleBusIP. Alternatively, you could instantiate the interfaces through the Add Interfaces GUI.

Example 14-6 Master and Slave Ports on SimpleBusIP

```
entity SimpleBusIP is
  generic (
    -- reuse-pragma attr Description Size of the data bus.
    -- reuse-pragma attr EnumValues 16 32
    -- reuse-pragma attr Label {Data Bus Width}
    data_width : natural range 16 to 32 := 16;
    -- reuse-pragma beginAttr Description
    -- Indicates if system is big-endian or little-endian.
    -- reuse-pragma endAttr
    -- reuse-pragma attr SymbolicNames little-endian big-endian
    -- reuse-pragma attr Label System Endianness

    big_endian : natural range 0 to 1 := 0;
    -- reuse-pragma beginAttr Description
    -- Total number of masters.
    -- reuse-pragma endAttr
    -- reuse-pragma attr Label Number of Masters
    num_masters : natural range 1 to 4 := 1;
    -- reuse-pragma beginAttr Description
    -- Total number of slaves.
    -- reuse-pragma endAttr
    -- reuse-pragma attr Label Number of Slaves
    num_slaves : natural range 1 to 4 := 1
  );

  port (
    clk          : in  bit;
    rst          : in  bit;

    --Master 1 Ports
    req_m1       : in  bit;
    gnt_m1       : out bit;
    addr_m1      : in  bit_vector(15 downto 0);
    wdata_m1     : in  bit_vector(data_width-1 downto 0);
    wen_m1       : in  bit;

    --Master 2 Ports
    -- reuse-pragma attr if(Name =~.*_m2) GenerateIf @num_masters>1
    req_m2       : in  bit;
    gnt_m2       : out bit;
```

```

        addr_m2          : in  bit_vector(15 downto 0);
        wdata_m2         : in  bit_vector(data_width-1 downto 0);
        wen_m2           : in  bit;

--Master 3 Ports
-- reuse-pragma attr if(Name =~.*_m3) GenerateIf @num_masters>2
        req_m3           : in  bit;
        gnt_m3           : out bit;
        addr_m3          : in  bit_vector(15 downto 0);
        wdata_m3         : in  bit_vector(data_width-1 downto 0);
        wen_m3           : in  bit;

--Master 4 Ports
-- reuse-pragma attr if(Name =~.*_m4) GenerateIf @num_masters>3
        req_m4           : in  bit;
        gnt_m4           : out bit;
        addr_m4          : in  bit_vector(15 downto 0);
        wdata_m4         : in  bit_vector(data_width-1 downto 0);
        wen_m4           : in  bit;

-- Read data to all masters
        rdata            : out bit_vector(data_width-1 downto 0);

--Slave 1 Ports
        sel_s1           : out bit;
        ready_s1         : in  bit;
        rdata_s1         : in  bit_vector(data_width-1 downto 0);

--Slave 2 Ports
-- reuse-pragma attr if(Name =~.*_s2) GenerateIf @num_slaves>1
        sel_s2           : out bit;
        ready_s2         : in  bit;
        rdata_s2         : in  bit_vector(data_width-1 downto 0);

--Slave 3 Ports
-- reuse-pragma attr if(Name =~.*_s3) GenerateIf @num_slaves>2
        sel_s3           : out bit;
        ready_s3         : in  bit;
        rdata_s3         : in  bit_vector(data_width-1 downto 0);

--Slave 4 Ports
-- reuse-pragma attr if(Name =~.*_s4) GenerateIf @num_slaves>3
        sel_s4           : out bit;
        ready_s4         : in  bit;
        rdata_s4         : in  bit_vector(data_width-1 downto 0);

-- Muxed ports to/from all slaves
        wdata            : out bit_vector(data_width-1 downto 0);
        addr             : out bit_vector(15 downto 0);

```

```

        wen                : out bit
    );

end SimpleBusIP;

```

Example 14-7 Instantiating Interfaces for SimpleBusIP

```

# Load SimpleBusM and SimpleBusS interface definitions.
set_activity_parameter LoadInterfaceDefinitions \
    InterfaceDefinitionFileNames \
    {/usr/cores/interfaces/SimpleBusM.tcl \
    /usr/cores/interfaces/SimpleBusS.tcl}

autocomplete_activity LoadInterfaceDefinitions
set_interface_port_attribute -instance SimpleBusM_BusIP \
    wdata InterfaceLink wdata_m@master_slot

set_interface_port_attribute -instance SimpleBusM_BusIP \
    wen InterfaceLink wen_m@master_slot

# All other interface port names match design port names.

# Create instance of interface SimpleBusS.
create_interface_instance -interface SimpleBusS\
    -type provider \
    SimpleBusS_BusIP

# Create instance of SimpleBusM.
create_interface_instance -interface SimpleBusM\
    -type provider \
    SimpleBusM_BusIP

# Link NumConsumers interface parameter to num_masters design parameter.
set_interface_parameter_attribute -instance SimpleBusM_BusIP \
    NumConsumers InterfaceLink num_masters

#Interface parameter master_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance SimpleBusM_BusIP \
    master_slot InterfaceLink <open>

# Link interface ports to design ports.
set_interface_port_attribute -instance SimpleBusM_BusIP \
    addr InterfaceLink addr_m@master_slot

set_interface_port_attribute -instance SimpleBusM_BusIP \
    req InterfaceLink req_m@master_slot

set_interface_port_attribute -instance SimpleBusM_BusIP \
    gnt InterfaceLink gnt_m@master_slot

```

```
# Link NumConsumers interface parameter to num_slaves design parameter.
set_interface_parameter_attribute -instance SimpleBusS_BusIP \
    NumConsumers InterfaceLink num_slaves

#Interface parameter slave_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance SimpleBusS_BusIP \
    slave_slot InterfaceLink <open>

# Link interface ports to design ports.
set_interface_port_attribute -instance SimpleBusS_BusIP \
    rdata InterfaceLink rdata_s@slave_slot

set_interface_port_attribute -instance SimpleBusS_BusIP \
    sel InterfaceLink sel_s@slave_slot

set_interface_port_attribute -instance SimpleBusS_BusIP \
    ready InterfaceLink ready_s@slave_slot

# All other interface port names match design port names.

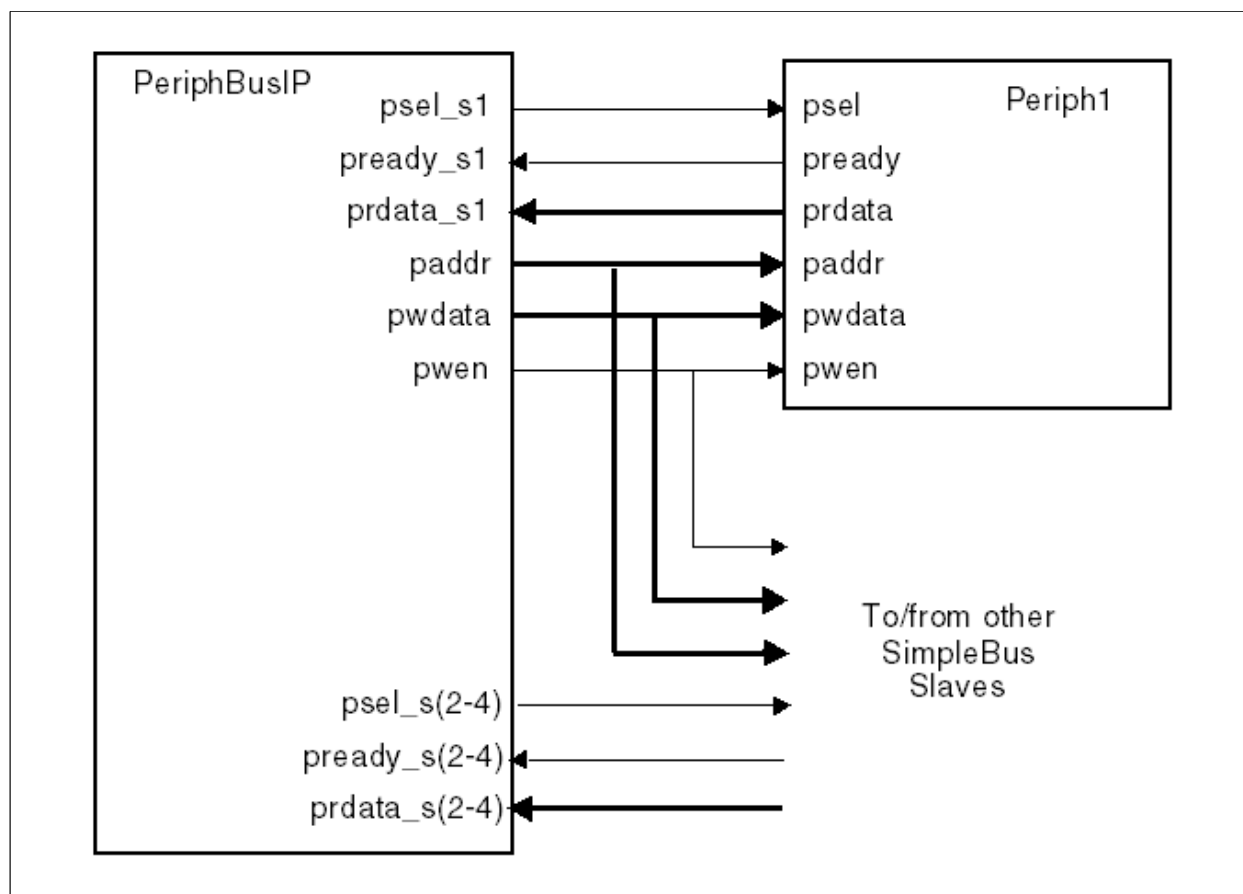
autocomplete_activity AddInterfaces
```

14.4.6 Defining the PeriphBus Interface

The next interface to define is PeriphBus because PeriphBusIP requires an instance of SimpleBusS and an instance of PeriphBus.

[Figure 14-5](#) shows the intended connections between PeriphBusIP and a PeriphBus slave device. PeriphBusIP is the provider of PeriphBus; slaves, including the Periph1 block, are consumers.

[Example 14-8](#) shows the interface definition file that creates the PeriphBus interface.

Figure 14-5 PeriphBus Port Connections**Example 14-8 PeriphBus Interface Definition File**

```

set PeriphBus_Version 1.0

# Begin PeriphBus interface definition.
create_interface PeriphBus \
    -version $PeriphBus_Version \
    -min 1 \
    -max 4 \
    -description \
    "PeriphBus Slave Interface"

# Create interface parameter to hold unique consumer slot numbers.
create_interface_parameter periph_slot \
    -interface PeriphBus \
    -type integer \
    -specify_on consumer \
    -unique \
    -label "PeriphBus Slave Slot" \
    -description \
    "Slot number of this PeriphBus Slave, range 1-4."

```

```

set_interface_parameter_attribute PeriphBus periph_slot \
    MinValue      1
set_interface_parameter_attribute PeriphBus periph_slot \
    MaxValue      @NumConsumers

# Create interface parameter to set data width for the interface.
create_interface_parameter data_width \
    -interface PeriphBus \
    -type integer \
    -specify_on provider \
    -default 16 \
    -label "Data Bus Width" \
    -description \
    "Number of bits on the PeriphBus data bus"

set_interface_parameter_attribute PeriphBus data_width \
    EnumValues {16 32}

# Create interface parameter to set endian mode for the interface.
create_interface_parameter big_endian \
    -interface PeriphBus \
    -type integer \
    -specify_on provider \
    -default 0 \
    -label "Data Endianness" \
    -description \
    "If set, the PeriphBus system is big-endian."

set_interface_parameter_attribute PeriphBus big_endian \
    MinValue      0
set_interface_parameter_attribute PeriphBus big_endian \
    MaxValue      1
set_interface_parameter_attribute PeriphBus big_endian \
    SymbolicNames {little-endian big-endian}

# Create single addr port to all consumers.
create_interface_port paddr \
    -interface PeriphBus \
    -direction fromProvider \
    -size 16 \
    -common \
    -description \
    "Address bus to all PeriphBus slaves."

# Create ready port from each consumer; active high so
# tie unconnected ports low.
create_interface_port pready \
    -interface PeriphBus

```

```

        -direction fromConsumer \
        -constant zero         \
        -description            \
"PeriphBus slave ready signal."

# Create read data port from each consumer; width = data_width.
create_interface_port prdata \
    -interface PeriphBus \
    -direction fromConsumer \
    -size @data_width \
    -constant dc \
    -description \
"Read data bus from PeriphBus slave."

# Create single write enable port to all consumers.
create_interface_port pwen \
    -interface PeriphBus \
    -direction fromProvider \
    -common \
    -description \
"Indicates that a PeriphBus write transfer is taking place."

# Create unique psel port to each consumer.
create_interface_port psel \
    -interface PeriphBus \
    -direction fromProvider \
    -separate \
    -description \
"Select signal to selected PeriphBus slave."

# Create single write data port to all consumers.
create_interface_port pwdata \
    -interface PeriphBus \
    -direction fromProvider \
    -common \
    -size @data_width \
    -description \
"Write data to all PeriphBus slaves."

# Complete PeriphBus interface definition.
complete_interface_definition PeriphBus

```

14.4.7 Instantiating Interfaces for PeriphBus IP

[Example 14-9](#) shows the VHDL entity definition for PeriphBusIP. PeriphBusIP requires instances of both SimpleBusS (defined in [Example 14-5](#) on page 224) and PeriphBus (defined in [Example 14-8](#) on page 231). [Example 14-10](#) on page 235 shows how to instantiate both SimpleBusS and PeriphBus for PeriphBusIP. Alternatively, you could instantiate the interfaces through the Add Interfaces GUI.

Example 14-9 VHDL Entity Definition for PeriphBusIP

```

entity PeriphBusIP is
  generic (
    -- reuse-pragma attr Description Size of the data bus.
    -- reuse-pragma attr EnumValues 16 32
    -- reuse-pragma attr Label {Data Bus Width}
    data_width : natural range 16 to 32 := 16;
    -- reuse-pragma beginAttr Description
    -- Indicates if system is big-endian or little-endian.
    -- reuse-pragma endAttr
    -- reuse-pragma attr SymbolicNames big-endian little-endian
    -- reuse-pragma attr Label System Endianness
    big_endian : natural range 0 to 1 := 0;
    -- reuse-pragma beginAttr Description
    -- Number of peripheral devices
    -- reuse-pragma endAttr
    -- reuse-pragma attr Label Number of PeriphBus Slaves
    num_pslaves : natural range 1 to 4 := 1
  );

  port (
    clk      : in  bit;
    rst      : in  bit;

    --SimpleBus ports
    addr      : in  bit_vector(15 downto 0);
    wdata     : in  bit_vector(data_width-1 downto 0);
    rdata     : out bit_vector(data_width-1 downto 0);
    ready     : out bit;
    sel       : in  bit;
    wen       : in  bit;

    --PeriphBus ports
    --PeriphBus Slave 1 Ports
    psel_s1   : out bit;
    pready_s1  : in  bit;
    prdata_s1  : in  bit_vector(data_width-1 downto 0);

    --PeriphBus Slave 2 Ports
    -- reuse-pragma attr if(Name =~.*_s2) GenerateIf @num_pslaves>1
    psel_s2   : out bit;
    pready_s2  : in  bit;
    prdata_s2  : in  bit_vector(data_width-1 downto 0);

    --Slave 3 Ports
    -- reuse-pragma attr if(Name =~.*_s3) GenerateIf @num_pslaves>2
    psel_s3   : out bit;
    pready_s3  : in  bit;
    prdata_s3  : in  bit_vector(data_width-1 downto 0);

```

```
--Slave 4 Ports
-- reuse-pragma attr if(Name =~.*_s4) GenerateIf @num_pslaves>3
    psel_s4          : out bit;
    pready_s4        : in  bit;
    prdata_s4        : in  bit_vector(data_width-1 downto 0);

-- Muxed ports to/from all slaves
    pwwdata          : out bit_vector(data_width-1 downto 0);
    paddr            : out bit_vector(15 downto 0);
    pwen             : out bit
);

end PeriphBusIP;
```

Example 14-10 Instantiating Interfaces for PeriphBusIP

```
# Load SimpleBusS and PeriphBus interface definitions.
set_activity_parameter LoadInterfaceDefinitions \
    InterfaceDefinitionFileNames \
    {/usr/cores/interfaces/SimpleBusS.tcl \
    /usr/cores/interfaces/PeriphBus.tcl}

autocomplete_activity LoadInterfaceDefinitions

# Create instance of SimpleBusS interface.
create_interface_instance -interface SimpleBusS \
    -type consumer \
    SimpleBusS_PBusIP

#Interface parameter slave_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance SimpleBusS_PBusIP \
    slave_slot InterfaceLink <open>

# Create instance of interface PeriphBus.
create_interface_instance -interface PeriphBus\
    -type provider \
    PeriphBus_BusIP

# Link NumConsumers interface parameter to num_pslaves design parameter.
set_interface_parameter_attribute -instance PeriphBus_BusIP \
    NumConsumers InterfaceLink num_pslaves

#Interface parameter periph_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance PeriphBus_BusIP \
    periph_slot InterfaceLink <open>

# Link interface ports to design ports.
set_interface_port_attribute -instance PeriphBus_BusIP \
    prdata InterfaceLink prdata_s@periph_slot
```

```

set_interface_port_attribute -instance PeriphBus_BusIP \
    psel InterfaceLink psel_s@periph_slot

set_interface_port_attribute -instance PeriphBus_BusIP \
    pready InterfaceLink pready_s@periph_slot

# All other interface port names match design port names.
autocomplete_activity AddInterfaces

```

14.4.8 Instantiating PeriphBus for a Consumer

[Example 14-11](#) shows the VHDL entity definition for Periph1 (a PeriphBus consumer). [Example 14-12](#) shows how to instantiate PeriphBus for Periph1.

Example 14-11 VHDL Entity Definition for Periph1

```

entity Periph1 is
  generic (
    -- reuse-pragma attr Description Size of the data bus.
    -- reuse-pragma attr EnumValues 16 32
    -- reuse-pragma attr Label {Data Bus Width}
    data_width : natural range 16 to 32 := 16;
    -- reuse-pragma beginAttr Description
    -- Indicates if system is big-endian or little-endian.
    -- reuse-pragma endAttr
    -- reuse-pragma attr SymbolicNames big-endian little-endian
    -- reuse-pragma attr Label System Endianness
    big_endian : natural range 0 to 1 := 0
  );

  port (
    clk      : in  bit;
    rst      : in  bit;

    --PeriphBus ports
    paddr     : in  bit_vector(15 downto 0);
    pwrdata   : in  bit_vector(data_width-1 downto 0);
    prdata    : out bit_vector(data_width-1 downto 0);
    pready    : out bit;
    psel      : in  bit;
    pwen      : in  bit
  );

end Periph1;

```

Example 14-12 Instantiating the PeriphBus Interface for Periph1

```

# Load PeriphBus interface definition.
set_activity_parameter LoadInterfaceDefinitions \
InterfaceDefinitionFileNames /usr/cores/interfaces/PeriphBus.tcl

```

```
autocomplete_activity LoadInterfaceDefinitions

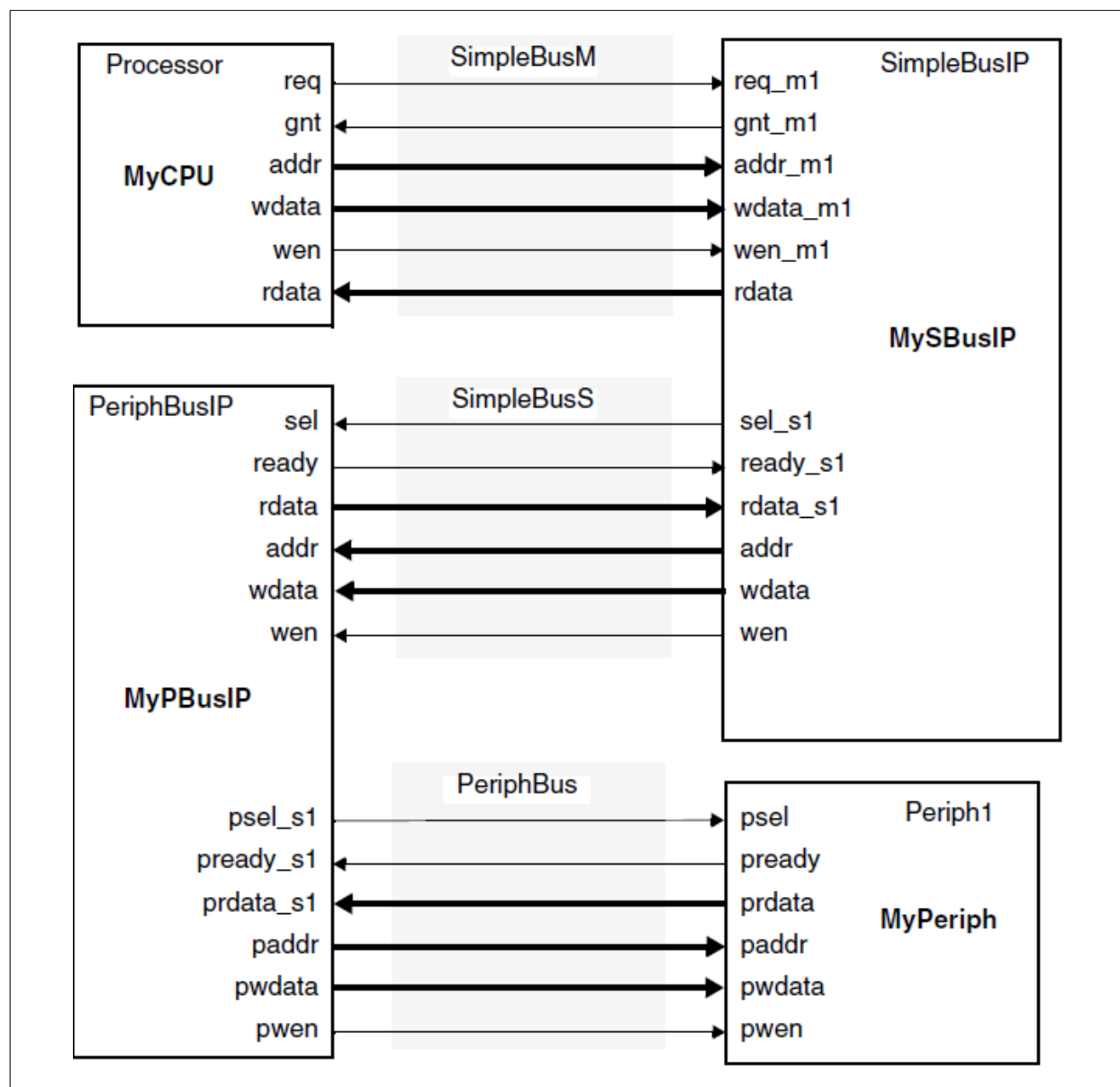
# Create instance of PeriphBus interface.
create_interface_instance -interface PeriphBus \
                        -type consumer \
                        PeriphBus_P1

#Interface parameter periph_slot is not linked to a design parameter.
set_interface_parameter_attribute -instance PeriphBus_P1 \
    periph_slot InterfaceLink <open>
autocomplete_activity AddInterfaces
```

14.4.9 Example coreAssembler Connections

[Figure 14-6](#) on [page 238](#) shows the set of port connections that the coreAssembler tool would automatically generate if a core integrator created a subsystem with the following component instances:

- SimpleBusIP instantiated as MySBusIP.
- One Processor block instantiated as MyCPU.
- PeriphBusIP instantiated as MyPBusIP.
- One Periph1 block instantiated as MyPeriph.

Figure 14-6 Example Subsystem Port Connections

14.5 coreAssembler Customization Hooks

14.5.1 Custom Component Actions (Hook)

You can add customization hooks to direct the coreAssembler tool to execute a Tcl command each time a given component is added to a subsystem. This is useful in defining code which can perform the following actions:

- Add another component based on the content of the subsystem, for example adding a bridge component to enable a particular part to connect to a bus already in the subsystem
- Set configuration values based on the contents of the subsystem
- Set the IconPath attribute on the top design to a .bmp file which is used to represent instances of this core in the subsystem tree (the corresponding file should be included in the BOM for the core and the value of IconPath should be relative to the installation directory)

To use a customization hook, you add the command:

```
add_instantiate_component_hook [-connect] <code>
```

into the plugin object file, where <code> is the name of the plugin procedure to execute as the hook. This command directs the coreAssembler tool to execute the Tcl procedure after a component has been instantiated.

If the -connect option is specified, then <code> is executed just prior to automatic interface connection in the coreAssembler tool. Without the -connect option, <code> is executed after automatic interface connections are made (the default).

For syntax details and additional examples, see the coreTools Command Reference.

14.5.2 Custom Component Dialogs

You can create your own custom dialog box to query for parameter values, or other custom uses. Within the coreAssembler tool, these custom dialogs can be used to enhance the ease-of-use of components which do not typically connect automatically to the proper components. For example, if you have one AHB in a subsystem and insert the DW_ahb_eh2h, the bridge is connected to both sides of the single AHB. With this feature a dialog can be included with the bridge coreKit which asks the user for the name of the 'master' and 'slave' AHB components. Connections to these components would then be made as specified.

The following show the user parameter dialog to build and return the values of a set of user defined parameters:

```
show_user_parameter_dialog
```

To view the syntax of this command and the returned values, enter in the coreTools command line:

```
man show_user_parameter_dialog
```

The custom dialog appears automatically just after the component is instantiated in coreAssembler but before any automatic connections are made.

15

Batch Mode

The coreBuilder GUI mode is generally convenient for new users because there is no need to learn the coreBuilder command syntax. For advanced users, batch modes provide more rapid turnaround of design and design packaging changes later in the design cycle. Using a batch mode script to build your coreKit enables you to make changes by simply editing a file then executing a single command. This chapter describes how to use the batch mode features.

This chapter contains the following sections:

- [“Command Line Interface”](#) on page 242
- [“Using Batch Mode”](#) on page 245

15.1 Command Line Interface

The coreBuilder tool provides a command-line interface so that you can enter commands directly from the command line, or write the commands to a file for batch-mode execution. Throughout this user guide, there are examples of how to use command line mode to perform tasks that you can also perform in GUI mode. The following sections provide some more general information on how to use command line mode. [Table 15-1](#) defines the elements of the coreBuilder command line syntax that you need to understand to use command line mode effectively. For complete lists and definitions of all commands, attributes, and item types, see the online command reference.

Table 15-1 Command Line Syntax Elements

Syntax Element	Definition
item	A design object or coreBuilder object Design items include designs, ports, clocks, and design parameters. The coreBuilder items that you generally interact with include workspaces, activities, filegroups, files, parameters, and attributes.
command	Manipulates one or more items (for example, open_workspace or set_port_attribute)
attribute	Modifier attached to an item to describe it (for example HasMuxes) You generally set attributes on design objects like clocks, ports, designs, and parameters. Each item has a predefined set of attributes.
parameter	Configures an item according to an established set of options For example, a design configuration parameter is a parameter of the associated design. Tool activities have parameters and you can create parameters for configurable filegroups and interface definitions. Parameters have attributes that determine the current value and legal range.
activity	A step in the coreBuilder work flow You set values on activity parameters to configure the activity to perform the desired operation.
strategy	A coreConsultant synthesis strategy Each strategy has a set of parameters that enable the core integrator to configure the execution of the strategy.

15.1.1 Accessing the Command Line Interface

You can invoke the command line interface in two ways:

- Enter commands at the coreBuilder> prompt in the coreBuilder console
- Invoke the coreBuilder tool in shell mode:
% \$SYNOPSIS/bin/coreBuilder -shell

For complete descriptions of all of the available commands, see the coreBuilder online command reference.

15.1.2 Reporting Activity Parameters

To list the names and current status of coreBuilder activities, use the `report_activities` command. To list the names and a brief description of the parameters associated with an activity, use the `report_activity_parameters` command. For example, see [Figure 15-1](#)

Figure 15-1 Example - Reporting Activity Parameters

```
coreBuilder> report_activity_parameters VerifyIntent
Name                               HelpText
-----
DoConsistencyCheck  Description:  Ensure Intent specifications are consistent.
                   All electrically equivalent ports must
                   have equivalent values for input/output
                   delay, driving cell, and load type.
                   Clocks has period greater than delay and
                   uncertainty.
                   DefaultValue: 1
DoCompletenessCheck Description:  Ensure constraints are complete.
                   All clocks have a period specified.
                   Ports have input/output delay
                   and driving cell set.
                   DefaultValue: 1
```

15.1.3 Setting the Value of an Activity Parameter

To set the value of a parameter on a coreBuilder activity, use the `set_activity_parameter` command. For example, to only perform completeness checks when you execute the Verify Intent activity:

```
coreBuilder> set_activity_parameter VerifyIntent DoConsistencyCheck 0
coreBuilder> set_activity_parameter VerifyIntent DoCompletenessCheck 1
```

When you use `set_activity_parameter` to change the value of an activity parameter, the status of the associated activity is set to incomplete until the activity is successfully executed.

15.1.4 Re-Executing the coreBuilder Design Flow

When you modify one or more activity parameters, you need to execute the coreBuilder design flow to implement those changes. The command below automatically completes various sets of activities, depending on the options specified:

```
autocomplete_activity <activity name> [-enable | -recur]
```

Table 15-2 Options for autocomplete_activity

Option	Description
none	Automatically complete all pre-requisite activities, then complete the specified activity (default)
-enable	Enable, but do not complete, the specified activity by automatically completing all pre-requisite activities
-recur	Automatically complete the specified activity and all activities that depend on the specified activity

For example, if you change specifications for the Verify Intent activity, use `autocomplete_activity` to re-execute Verify Intent automatically, this time using the new specifications:

```
coreBuilder> autocomplete_activity VerifyIntent
```

As previously stated, using the `set_activity_parameter` sets the state of that activity to incomplete. However, changes that you make by executing commands such as `set_design_attribute`, `set_port_attribute`, and `set_filegroup_attribute` do not cause any changes to the current state of coreBuilder activities.

For example, if you change the value of a port attribute by executing `set_port_attribute`, use the `set_activity_incomplete` command to set the state of SpecifyPortIntent to incomplete, then auto-complete SpecifyPortIntent (and, optionally selected downstream activities) to implement the change. For example, to re-execute the complete packaging flow with a new port attribute value, enter the following commands:

```
coreBuilder> set_activity_incomplete SpecifyPortIntent
coreBuilder> set_port_attribute MyPort MaxInputDelay {=percent_of_period 25}
coreBuilder> autocomplete_activity BuildcoreKit
```

15.1.5 Abbreviating Commands

You can abbreviate coreBuilder commands and options to the shortest unambiguous string. For example, you can abbreviate the `set_port_attribute` command to “`set_por`”.



Note

1. Command abbreviation is included as a convenience for command line users.
2. You cannot abbreviate built-in Tcl commands.
3. Do not use command abbreviation in batch script files.
4. You should also remember that existing command abbreviations may become ambiguous if and when future versions of coreBuilder add to or change the list of commands.

For a list of all commands that match the ambiguous abbreviation, use the help command with wildcard pattern matching:

```
coreBuilder> help cur*
current_design      # Set/get the current design
current_instance    # Set/get the instance
current_kb          # Make the named knowledge base the /
                    # current one.

coreBuilder>
```

15.2 Using Batch Mode

You can execute coreBuilder commands in batch mode by writing the required sequence of commands to a file using Tcl syntax. The coreBuilder tool supports both standard Tcl commands and coreBuilder-specific Tcl commands. Batch mode execution is useful for regression testing and experimentation with different design alternatives.

15.2.1 Batch Command File Examples

A common use for batch mode is to change values for one or more parameters for an activity, then re-execute that and all subsequent activities. This scenario assumes that you have already used the GUI to complete the design flow at least once, but this is not a strict requirement.

[Example 15-1](#) changes the value of a port attribute, then re-executes SpecifyPortIntent and all subsequent steps in the coreBuilder design flow, using the currently stored specifications for the subsequent activities.

Example 15-1 Batch File – Port Attribute Change

```
#Change parameter value
set_activity_incomplete SpecifyPortIntent
set_port_attribute MyPort MaxInputDelay {=percent_of_period 25}
#Re-execute SpecifyPortIntent and subsequent activities
autocomplete_activity BuildcoreKit
#Quit coreBuilder
quit
```

[Example 15-2](#) changes a set of parameters for an activity and re-executes it (and all subsequent activities in the coreBuilder design flow) using the currently stored specifications.

Example 15-2 Batch File – Activity Specification Change

```
#Change activity specifications
set_activity_parameter LoadConsultantPlugins PluginDirectories ./cC_Plugins
#Re-execute this activity and all subsequent activities
autocomplete_activity BuildcoreKit
#Quit coreBuilder
quit
```

For details about the commands used in the following examples, see the coreBuilder online reference.

15.2.2 Write Batch Script

You can automatically write a batch script file that contains the commands required to re-create the current state of your workspace. You can then use the resulting batch script as an example of how to write a batch script, or to create another identical workspace in batch mode, or to modify the batch file and create a new workspace with different options.

To automatically generate a batch script in command line mode, use the `write_batch_script` command. For example:

```
coreBuilder> write_batch_script my_script.tcl
```

For the complete set of `write_batch_script` options, see the online command reference.

To automatically generate a batch script in GUI mode, select **File > Write Batch Script**. The coreBuilder tool displays the Create Batch Script dialog. Select values for the Create Batch Script options listed in [Table 15-3](#), then click OK.

Table 15-3 Create Batch Script Options

Option	Description
Output File Name	File name for the resulting batch file (for example, my_script.tcl) The default directory is the current directory. Use the ellipsis button (...) to navigate to and select a specific directory.
Include create_workspace command	Includes the create_workspace command needed to create the workspace (with the same workspace name)
Include Design Constraints	Includes the commands required to specify the currently specified synthesis methodology If you do not select Include Design Constraints, the batch script does not include any set_design_attribute or set_port_attribute commands that you have executed in the workspace, either through the GUI or in command line mode.
Include configuration intent	Includes the commands required to specify the currently specified configuration intent If you do not select Include configuration intent, the batch script does not include the set_parameter_attribute commands required to implement your currently specified configuration intent.
Include SoC assembly (interfaces) intent	Includes the commands required to specify the currently specified interface intent If you do not select Include SoC assembly (interfaces) intent, the batch script does not include create_interface_instance or any other interface-related commands that you have executed in the workspace, either through the GUI or in command line mode.
Include activity completion	Includes the commands required to complete the activities that are currently completed in your workspace, with the current set of activity parameter values If you do not select Include activity completion, the batch script does not include any set_activity_parameter or autocomple_activity commands that you have executed in the workspace, either through the GUI or in command line mode.
Include BOM (filegroups)	Includes the commands required to set up the current BoM specifications If you do not select Include BOM (filegroups), the batch script does not include any filegroup-related commands that you have executed in the workspace, either through the GUI or in command line mode.
Write default parameter values	Includes all of the commands required to define the default values Produces a much larger batch script as a result.

15.2.3 Executing a Batch Command File

To invoke the coreBuilder tool and automatically execute a batch command file, specify your user workspace name and use the -f option to specify the name of the batch command file. For example:

```
% coreBuilder -shell workspace_name -f batch.tcl
```

This example performs the following sequence:

1. Starts the coreBuilder tool in shell mode.
2. Loads your user workspace (workspace_name).
3. Executes the batch mode command file batch.tcl.

If your batch script includes a command to create a workspace, omit the workspace_name. For example:

```
% coreBuilder -shell -f batch.tcl
```

You can also execute a batch command file from the command line interface by using the Tcl source command, for example:

```
coreBuilder> source batch.tcl
```


16

Using IP-XACT Components

This chapter describes the usage of IP-XACT components.

This chapter contains the following sections:

- [“IP-XACT Overview”](#) on page 250
- [“List of Features”](#) on page 251
- [“Importing and Exporting IP-XACT”](#) on page 252
- [“For More Information”](#) on page 253

16.1 IP-XACT Overview

The IP-XACT standard consists of an XML schema that defines a meta-data representation for IP and tool flows, and a set of semantics further restricting how the schema should be utilized. The schema defines a large number of XML elements, but there are only four key elements that users and tools typically interact with.

- **Abstraction definition** -- Defines the ports and properties of a particular abstraction (such as RTL or TLM) of a given bus definition.
- **Abtractor** -- A specialized form of a component, which is used to provide the details about how to translate between two interfaces when the interfaces are at different levels of abstraction.
- **Bus definition** -- Defines the signals that compose a bus (for example, AHB). This includes information such as which signals are present on a master, slave, or system interface, signal direction, width, default value, and so on.
- **Catalog** -- Defines a mapping between IP-XACT files on the disk and their corresponding IP-XACT type and VLNV.
- **Component** -- Defines a single IP. Contains information about I/O, register and address map requirements, bus interfaces (instances of bus definitions), available views, and so on. Components are elements that can be instantiated in a subsystem by interacting with a design environment. Components may be hierarchical.
- **Design** -- Defines a subsystem. Contains information indicating how a set of components is to be connected and configured. Connections are made at the interface level. Designs can be converted into hierarchical components and then instantiated in a higher-level design.
- **DesignConfiguration** -- Stores modeling information such as view selections and parameter configuration values.
- **Generator** -- Generator elements do not appear as top-level elements (in other words, at the root of an XML file), but they are a critical part of the standard. Generators define tools that are to be run to transform the design in some way. Typical examples include configuration, RTL generation, and building and running a simulation model. The generator element describes when and where a tool should be run and defines how to invoke the tool, but the actual tools are independent of the generator. In the current release of the schema all generators are separate processes that are passed information in XML format and which can pass back an ECO list (also in XML format), which the design environment applies to the design.
- **GeneratorChain** -- Defines a sequence of generators to be run at one time.

16.2 List of Features

The following IP-XACT features are supported in the coreTools applications.

16.2.1 IP-XACT Packaging Features

16.2.1.1 Support Use of IP-XACT Bus Definitions in coreBuilder

When using the Load Interface Definitions activity, the list of interface definition files can include files that contain IP-XACT bus definitions. IP-XACT bus definitions are translated into interface definitions as is currently done in the coreAssembler tool. These translated interface definitions can then be instantiated during **Add Interfaces**. The interface names are based on the VLNV of the bus definition as follows:

- V_L_N_V for a symmetric bus definition
- V_L_N_V_master for the master interface of a non-symmetric bus definition
- V_L_N_V_slave for the slave interface of a non-symmetric bus definition
- V_L_N_V_system_<group> for a system interface with the specified group name

16.2.1.2 Generating an IP-XACT Component

By default, generation of an IP-XACT component is enabled. The Build coreKit activity includes a check-box to generate the IP-XACT component.



Note

The **Edit > Preferences > IP-XACT** dialog allows you to specify whether IP-XACT output files are “automatically” written. This dialog also lets you select IP-XACT output file version. Disabling the “Write IP-XACT components” check box increases performance if you have no need for IP-XACT output. For more about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 273.

Components generated by the coreBuilder tool include a fileSet for each filegroup in the Bill-of-Materials, and parameters are set up to be configurable, unlike the case from the coreConsultant tool where the parameter values are static. The generated IP-XACT component contains vendor extensions for some key functionality not available in IP-XACT, but it is not a complete representation of the coreKit. For example, plugins are not represented, conditional I/O is not supported, and parameter checking is not available.

16.2.1.3 Support Partial Import of IP-XACT Components in coreBuilder

To import information from an existing IP-XACT file, use the following command:

```
import_ipxact_data <IP-XACT component file name>
```

This command is valid any time after the Load Designs activity has been completed. It can be used to import the following information:

- Memory maps

Memory maps in the IP-XACT file are read in and replace all pre-existing memory maps.

- Vendor extensions

Vendor extensions on ports, model parameters, component parameters, and the component itself are imported. Extensions in the 'snps' namespace are removed and all others are saved so that they can be included in any generated IP-XACT component written by the coreBuilder, coreConsultant, or coreAssembler tool.

16.3 Importing and Exporting IP-XACT

The coreBuilder tool allows you to import and export components using the IP-XACT XML schema.

- Importing IP-XACT - Use the File > Import IP-XACT Data... menu item to load an IP-XACT XML formatted component.

This menu item calls the command `import_spirit_data` which imports memory maps from an IP-XACT XML file and replaces any existing memory maps. In addition, it stores any vendor extensions on ports, model parameters, component parameters, and the component itself. This ensures that components are saved properly if you choose to generate an IP-XACT component as part of the Build coreKit activity.

- Exporting IP-XACT - Use the **Edit > Preferences > IP-XACT** dialog to specify “Write IP-XACT components” and other IP-XACT controls. For more about preferences, see “[Editing Tool Setup and User Preferences](#)” on page 273 .

16.4 For More Information

For version support and a complete list of features and usage of the IP-XACT schema in the coreTools applications, see “Using IP-XACT Components” in the *coreAssembler User Guide*.

17

Advanced Topics

This chapter describes how to use advanced features of the coreBuilder tool.

This chapter contains the following sections:

- [“Setting Up Advanced Parameter Dialogs”](#) on page 256
- [“Customizing coreTools”](#) on page 259
- [“Working with Plugins”](#) on page 265
- [“Packaging Verification IP”](#) on page 269
- [“Packaging Hard Macros”](#) on page 271
- [“Editing Tool Setup and User Preferences”](#) on page 273
- [“Table Driven Data Import/Export”](#) on page 286

17.1 Setting Up Advanced Parameter Dialogs

You can control the parameter dialog for design parameters or custom activity parameters by setting the `ParameterInfo` attribute on the associated design or activity.

For example, if your core has a large number of parameters, you can organize the parameters into groups and use a tree-style dialog for parameter specification. You can also do the same for parameters that are associated with configurable filegroups, including TRE filegroups.

The value for `ParameterInfo` is a Tcl list of statement-value pairs, as shown in [Example 17-1](#). [Table 17-1](#) on page 256 lists the valid `ParameterInfo` statements and their legal values.

Table 17-1 ParameterInfo Statements

Statement	Description	Legal Values
Group	Name of parameter group Names can be hierarchical (Group/Subgroup). Names of sibling groups must be unique.	Text string
Text	Text string to add to the group of parameters in the parameter dialog The text string can contain a sub-expression to specify the sequence for text string. The default position for the text is at the beginning of the group of parameters.	Text string
Label	GUI label for the parameter group.	Text string
ContainerType	Specifies how to organize the groups in the parameter dialog	dialog – Single-page dialog (the default). tab – Groups of parameters appear on separate tabs. tree – Groups of parameters are selectable in a navigation tree.
LayoutAs	Specifies how to group parameters	none – Each parameter group on a separate tab. Tabs within tabs force the generation of a tree container. groupbox – Each group in a separate box. spreadsheet – Group elements arranged in columns and rows. table – Arrange each group horizontally. If the group is at the top level of hierarchy, the default is none. Otherwise, the default is groupbox.
MaxPageCount	Maximum number of tabs allowed in a parameter dialog If the number of tabs exceeds <code>MaxPageCount</code> , the coreBuilder tool automatically uses a tree dialog.	Integer The default is 10.

Table 17-1 ParameterInfo Statements (continued)

Statement	Description	Legal Values
Parameters	List of parameters to be included in the group (overrides the GroupName attribute value on individual parameters) If you include a parameter in more than one group, the coreBuilder tool places the parameter in the first group in which the parameter is listed. You can include more than one Parameter statement per group. The coreBuilder tool appends the additional parameters to the parameter list.	List of parameters
BoxType	Specifies the appearance of the parameter grouping box in the parameter dialog	outline – Draw a box around the parameter group (the default). indent – Indent the parameters, but do not draw a box.

Example 17-1 Setting ParameterInfo on a Design

```

set pInfo \
{{ContainerType tree}
{Group Options
  {Label {Optional Features}}
  {Parameters {p_width timer2 serial pwr_mgmt }}}
{Group MemAddresses
  {Label {Memory Addresses}}
  {Text {Specify non-overlapping Address Ranges}}
  {Group Mem1
    {LayoutAs table}
    {Label {Memory Space 1}}
    {Parameters {mem1_start mem1_end}}}
  {Group Mem2
    {LayoutAs table}
    {Label {Memory Space 2}}
    {Parameters {mem2_start mem2_end}}}
  {Group Mem3
    {LayoutAs table}
    {Label {Memory Space 3}}
    {Parameters {mem3_start mem3_end}}}}
{Group IOAddresses
  {Label {I/O Addresses}}
  {Text {Specify non-overlapping Address Ranges}}
  {Group IO1
    {LayoutAs groupbox}
    {Label {I/O Space 1}}
    {Parameters {io1_start io1_end}}}
  {Group IO2
    {LayoutAs groupbox}

```

```
        {Label {I/O Space 2}}
        {Parameters {io2_start io2_end}}}}
    {Group IO3
        {LayoutAs groupbox}
        {Label {I/O Space 3}}
        {Parameters {io3_start io3_end}}}}
}

current_design TOP
set_design_attribute ParameterInfo $pInfo
```

17.2 Customizing coreTools

The coreTools applications provide flexible activity flows for completing core development, packaging, and integration activities. To support custom flows and design needs, you can:

- Define custom Tcl procedures to perform required tasks
- Modify the execution of existing activities
- Add new activities to the design flow
- Check the configured subsystem
- Customize workspaces

Your customizations can be applicable to:

- An entire site
- A specific product (coreBuilder, coreConsultant, or coreAssembler)
- A specific core
- A specific user

17.2.1 General Customization Procedure

In general, customizing coreTools applications involves the following steps:

1. Write the Tcl code that implements the custom functionality (for example, modifying an existing activity).
2. Build the Tcl code into a plugin. For core-specific plugins that you distribute with the coreKit, you build the plugin by executing the Load Consultant Plugins activity. Otherwise, you use the `create_plugin_kb` command.
3. Load the plugin into the tool. For core-specific plugins that you distribute with the coreKit, the coreConsultant tool automatically loads the plugin before opening the user's workspace. For other plugins, you can load the plugin in either of the following ways:
 - Place the plugin .kb file in a plugin directory that is listed in the UNIX variable `RT_*_PLUGINS` (for example, `RT_BUILDER_PLUGINS`). See [Plugin Directories](#) on page 265.
 - Read the plugin .kb file explicitly by executing the `read_kb` or `load_plugin` command.

17.2.2 Activities

Each entry in the Activity List is a unique activity that has the following properties:

- Name – Used to reference the activity in a batch Tcl script. For example, the Verify Intent activity exists in all of the coreTools applications.
- Label – Used by the GUI to represent the activity. For example, the TechSetup activity has the label “Specify Reference Technology” in the coreBuilder tool, and “Specify Target Technology” in the coreConsultant tool.
- State – The State is either complete or incomplete. This corresponds to the activity check-box, which is checked if the activity has been completed.

- Prerequisites – Other activities that must be completed before this particular activity becomes enabled. For example, the coreConsultant tool disables the SpecifyConfiguration activity until the core integrator successfully completes the TechSetup activity.
- Parameters – Objects that capture user input pertaining to the activity. For example, the TechSetup activity uses the parameters SearchPath, TargetLibrary, and LinkLibrary, among others. The activity dialogs display the parameters and their current values.

When you execute an activity in GUI mode, the following process occurs:

```
execute pre-activity command (if defined)
show dialog containing activity parameters
if { user selected Default button } {
set default parameter values
continue
}
if { user selected Okay button } {
validate parameter values
execute post-activity command (if defined)
} else {
execute cancel-activity command (if defined)
}
```

A Tcl error caused by any of these commands stops all processing for the activity. If value checking for the parameters results in an error, the parameter dialog remains posted, and focus within the window shifts to the first parameter containing an error.

17.2.3 Implementing a Custom Tcl Command Procedure

To implement a custom Tcl command procedure, independent of any activity, code the command procedure in a .tcl file and build a plugin for the command procedure as described in [“Working with Plugins”](#) on page 265.

[Example 17-2](#) shows a simple *name.tcl* file that creates a simple custom Tcl proc named TimeStamp.

[Example 17-3](#) shows a VHDL file that calls the TimeStamp proc to perform text substitution. If you build a plugin from the Tcl file in [Example 17-2](#), the coreConsultant tool writes the current system date and time into the file each time a core integrator generates a user-specified configuration of the core.

Example 17-2 Tcl File to Create Custom Command Procedure

```
proc TimeStamp { } {
    set timestamp [date]
    return "-- File written: $timestamp"
}
```

Example 17-3 Substituting Text With a Custom Command Procedure

```
-- reuse-pragma startSub [TimeStamp]
-- Mar 24, 2000 8:00 PM EST
```

17.2.4 Modifying Existing Activities

You can modify an existing activity by using the `add_activity_hook` command. The `add_activity_hook` command adds a specified Tcl command to be executed as either a pre-activity, post-activity, or cancel-activity command.

[Example 17-4](#) shows an example command sequence that adds pre-activity, post-activity, and cancel-activity hooks to the `TechSetup` activity.

Example 17-4 Use of the `add_activity_hook` command

```
add_activity_hook -before -prepend TechSetup {echo "Hello"}
add_activity_hook -after TechSetup {echo "Okay"}
add_activity_hook -cancel TechSetup {echo "Cancel"}
```

To build the above commands into a plugin:

1. Place the commands in a file with a file name that ends with `Obj.tcl` (for example, `MyPlugin_Obj.tcl`).
2. Build the plugin using either the `Load Consultant Plugins` activity (for core-specific plugins) or the `create_plugin_kb` command (for non-core-specific plugins). See [“Building a Core-Specific Plugin” on page 259](#) on page 266 and [“Building a General Purpose Plugin” on page 268](#) for more information about building plugins.

If you want to use custom command procedures as the activity hook commands, define the command procedures in a separate (.tcl) file and use the custom procedure names as the command argument to `add_activity_hook`, as shown in [Example 17-5](#).

Example 17-5 Activity Hooks with Custom Command Procedures

```
# MyPlugin_Obj.tcl
add_activity_hook -before -prepend TechSetup hello_msg
add_activity_hook -after TechSetup okay_msg
add_activity_hook -cancel TechSetup cancel_msg

# MyPlugin_Obj.tcl
proc hello_msg {} {
    echo "Hello"
}
proc cancel_msg {} {
    echo "Cancel"
}
proc okay_msg {} {
    echo "Okay"
}
```

Build plugin as described in [“Building a Core-Specific Plugin”](#) or [“Building a General Purpose Plugin”](#).

17.2.5 Adding Custom Activities

You can also create custom activities and insert them at specified points in the core developer or integrator activity flow. For example, you could create a custom `coreConsultant` activity that uses custom command procedures to perform scan chain insertion for a core.

To create a custom activity, use the following commands in a name_Obj.tcl file, as shown in [Example 17-6](#):

- **create_custom_activity** – Defines a new activity, specifies the label to be shown in the activity checklist, defines short help (for right-mouse-button help) and detailed help (for Help button), and specifies the pre-activity and post-activity command strings. Define the Tcl command procedures (for example, `verifyReadyForChain` and `addScanChain`) in a separate .tcl file in the same directory so that they are also included in the plugin. (See [Example 17-7](#) on page 263.)
- **create_custom_activity_parameter** – Defines the parameters needed to gather information from the user. In [Example 17-6](#), the first instance of this command specifies how many scan chains should be inserted. The second instance defines the insertion mode. You can set additional attributes on the parameters to refine the look of the associated parameter dialog.
- **complete_custom_activity_definition** – Defines the activity's relationship to the other activities. In [Example 17-6](#), the custom activity is not enabled until `SpecifyConfiguration` is complete, and `Synthesize` is not enabled until `ScanInsertion` is complete. In addition, a sequence number controls the activity's sequential position within the Activity List. To list the sequence number of existing activities, use the command:

```
report_attribute [find_item -type activity] -attr Sequence
```

For details on specific command line options, see the man pages for the above commands.

Example 17-6 Tcl Code for a Typical Custom Activity

```
# Scan_Obj.tcl
# Define activity for setting up RTL scan insertion.
set A [create_custom_activity \
  -name ScanInsertion \
  -type Global \
  -label "Add scan chain to core." \
  -pre "verifyReadyForChain" \
  -post "addScanChain" \
  -description "Activity for adding scan chains" \
  -url [file join docs scan help.html]]
# Indicates the number of scan chains to be inserted.
set P [create_custom_activity_parameter \
  -activity $A \
  -name NumberOfChains \
  -type integer \
  -label "Number of Scan Chains" \
  -description \
    {Specify the desired number of scan chains to be inserted.} \
  -default 2]
# Indicates the insertion mode to be used.
set P [create_custom_activity_parameter \
  -activity $A \
  -name InsertionMode \
  -type string \
  -label "Insertion Mode" \
  -description {Defines the scan chain insertion mode.}]
set_attribute $P -attr EnumValues \
  -value "Basic Advanced"
```

```

set_attribute $P -attr DefaultValue -value "Basic"
# Set up requirements and sequencing This activity
# can't be completed until after configuration, and
# must be completed before synthesis.

complete_custom_activity_definition -activity $A \
  -sequence 200 \
  -prereqs SpecifyConfiguration \
  -postreqs Synthesize

```

When invoked in the coreConsultant tool, the custom ScanInsertion activity calls the custom command procedure `verifyReadyForChain`, shown in [Example 17-7](#). If `verifyReadyForChain` returns without error, the coreConsultant tool displays the parameter dialog.

Assuming the user closes the dialog using the OK button, the ScanInsertion activity calls the custom command procedure `addScanChain`, which is also shown in [Example 17-7](#).

Example 17-7 Command Procedures for Custom Activity

```

# Scan.tcl
proc verifyReadForChain {} {
  # Make sure RTL insertion tool is available
  set result [exec which insert_scan_tool]
  if { ![file exists $result] } {
    return -code error \
      "Can't locate: insert_scan_tool."
  }
}
proc addScanChain {} {
  # Get parameter values.
  set chains [get_activity_parameter ScanInsertion \
    NumberOfChains]
  set mode [get_activity_parameter ScanInsertion \
    InsertionMode]
  # Get source code to be operated upon and then pass
  # it to the RTL scan chain insertion tool.
  set files [get_hdl_file_list]
  set rc [catch {exec insert_scan_tool -mode $mode \
    -chains $chains -files $files}]
  if {$rc} {
    return -code error "insert_scan_tool failed."
  }
}

```

The procedures in [Example 17-7](#) use several of the Tcl utility commands available for plugin development. For details about commands available for use for plugin development, see the coreBuilder online reference.

17.2.6 Custom Subsystem Checking

You can insert custom subsystem checking into a specific core by using the `add_verify_subsystem_hook` command. This command executes a Tcl command to verify a subsystem at various places in subsystem specification. It can be set up to run before or after the component is instantiated, or at the end of subsystem specification after the built-in coreAssembler `verify_subsystem` command. The hook returns any errors found during verification.

For example, to add a hook to execute the Tcl command `'verify_before_instantiation'` before instantiating a core, you would add the following command to a plugin object file associated with the core:

```
add_verify_subsystem_hook -before verify_before_instantiation
```

For syntax details and additional examples, see the coreTools online reference.

17.2.7 Customizing Workspaces

Another way to customize coreTools applications is to add a hook to a workspace to cause a specified action to occur when a workspace is created, opened, or closed. For example, if your core requires that the QoR Effort synthesis strategy parameter be set to high, you can add a workspace hook to set QoR Effort to high each time a user creates a workspace from the coreKit.

To add a workspace hook, use the `add_workspace_hook` command as shown in the following example:

```
add_workspace_hook -create {set_strategy_parameter DC_opto_strategy \  
QorEffort high}
```

The above example sets the QorEffort parameter on the DC_opto_strategy to high when a workspace is created. For more details about the `add_workspace_hook` command, see the `add_workspace_hook` man page in the coreBuilder online reference.

When used with the `-create` option, the `add_workspace_hook` must be part of a plugin and the plugin must be loaded when a user creates a workspace. The QorEffort example applies to coreConsultant workspaces. Therefore, to set QorEffort to high as the default value for all workspaces created from a coreKit:

1. Place the `add_workspace_hook` command in a file with a filename that ends with `Obj.tcl` (for example, `HighEffortObj.tcl`).
2. Specify the path to the `Obj.tcl` file when you execute the Load Consultant Plugins activity.

To build a non-core-specific plugin, use the `create_plugin_kb` command, as described in the following section, [“Working with Plugins”](#).

17.3 Working with Plugins

Plugins implement your customizations for the applicable situation (such as tool, user, or workspace). To build a plugin, you use coreTools plugin utilities. Then, you load the plugin into the applicable tool, either implicitly by placing it in the applicable plugin directory or explicitly by reading the plugin.kb file using the read_kb command or by issuing the load_plugin command.

Core-specific plugins are a special case. You create a core-specific plugin by executing the Load Consultant Plugins activity. The following sections describe how to work with plugin directories and how to build plugins.

17.3.1 Plugin Directories

A plugin is a knowledge database (.kb file) that can be stored in one of several pre-defined locations so that it gets loaded whenever the applicable tool is started. Several UNIX environment variables point to specific directories that support the automatic loading of plugin knowledge databases:

- RT_PLUGINS – Defines one or more directories that contain mode-independent plugins applicable to all of the coreTools applications. The default value is: <tool installation root>/libraries/dware/plugins.
- RT_BUILDER_PLUGINS – Defines one or more directories that contain plugins applicable only to coreTools applications. The default value is: <tool installation root>/libraries/dware/plugins/builder.
- RT_CONSULTANT_PLUGINS – Defines one or more directories that contain plugins applicable only to the coreConsultant tool. The default value is: <tool installation root>/libraries/dware/plugins/consultant.
- RT_ASSEMBLER_PLUGINS – Defines one or more directories that contain plugins applicable only to the coreAssembler tool. The default value is: <tool installation root>/libraries/dware/plugins/assembler.

17.3.2 Writing Plugins

You create a plugin by writing the Tcl code, saving it as one or more files in a single directory, and then packaging the code as a plugin .kb file. Tcl code for a plugin falls into two basic functional categories. The code categories are evaluated at different times, so be aware of the differences:

- Code that defines an object that resides in the plugin .kb file (a custom activity, for example)
Tcl code that defines or modifies objects must be evaluated only when the plugin is being built, and not when it is being loaded for use. Code of this sort is stored under a filename ending in “Obj.tcl.”
- Code that defines procedures or sets global variables that become available when the plugin is in use
Tcl code that defines procedures or sets variables during core development or integration sessions must be evaluated when the plugin is loaded. This type of code is stored under a filename ending simply in “.tcl,” which means that the code is evaluated both when the plugin is built, and again whenever the plugin is loaded for use.

The .tcl files required for a plugin, therefore, depend on the function of the plugin, as listed in [Table 17-2](#).

Table 17-2 Tcl Source Files Required for Plugins

Plugin Functionality	Required Source Files
Defines custom Tcl procs, but does not modify existing activities or add custom activities.	<i>name.tcl</i> – Defines the custom procs
Modifies (adds a hook to) one or more existing activities.	<i>name_Obj.tcl</i> – Contains the <i>add_activity_hook</i> or <i>add_workspace_hook</i> commands
Creates one or more custom activities.	<i>name.tcl</i> – Defines custom procs <i>name_Obj.tcl</i> – Creates and defines the new activities and associated activity parameters.

17.3.3 Building a Core-Specific Plugin

For core-specific plugins to be included in a coreKit, execute the Load Consultant Plugins activity to build the plugin. The coreBuilder tool automatically includes the plugin in the coreKit and the coreConsultant tool automatically loads the plugin when loading a workspace for the coreKit.

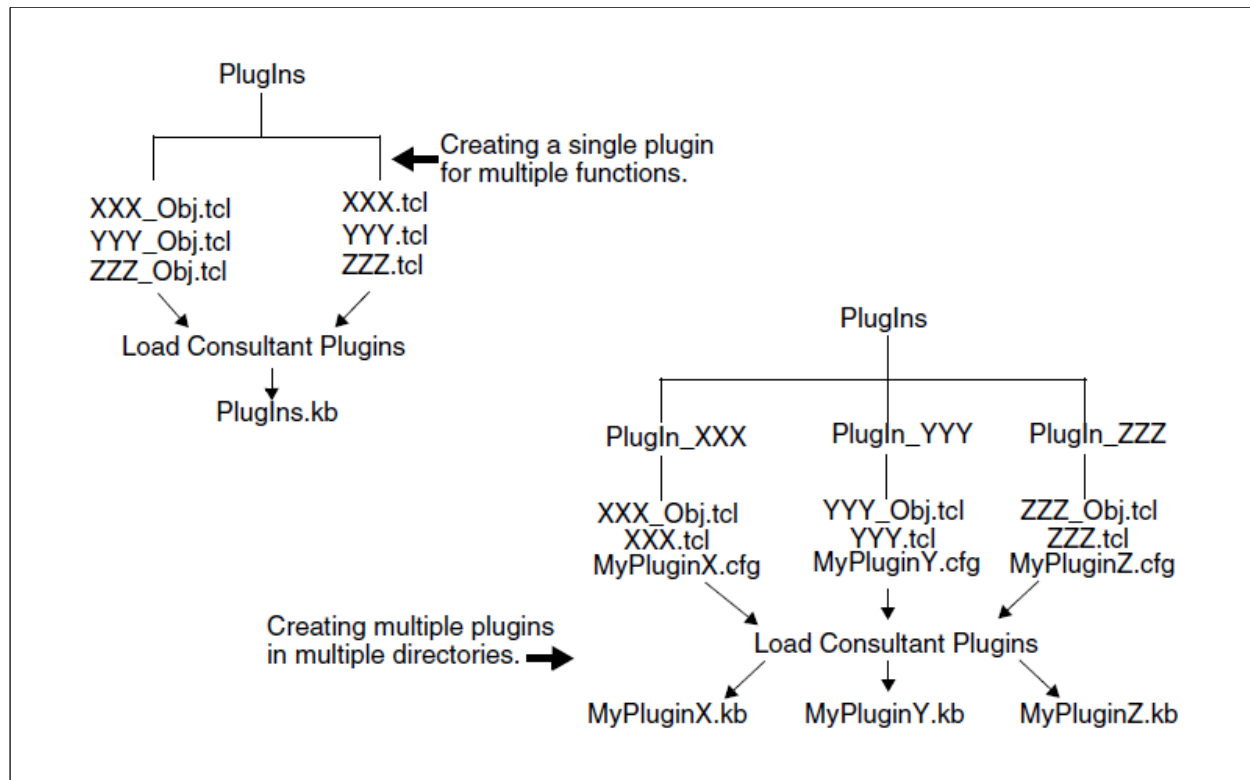
To complete the Load Consultant Plugins activity in GUI mode:

1. Create the .tcl and (if necessary) Obj.tcl files for the plugins.
You can create a single plugin .kb file that performs multiple functions or you can create separate plugin .kb files for separate functions.
2. Place the .tcl files and Obj.tcl files for the plugins in a single directory. To create more than one plugin .kb file, place the .tcl and Obj.tcl files for each plugin in separate directories, as shown in [Figure 17-1](#) on page 267.

3. Select the Load Consultant Plugins activity from the coreBuilder Activity List.

4. Specify one or more plugin directories that you created in step 2 in the Load Consultant Plugins dialog in the Activity View pane, then click Apply.

The coreBuilder tool automatically creates a plugin KB in each of the plugin directories, then adds the plugin .kb files to the BoM. The default plugin name matches that of the plugin source directory, as shown in the single directory example in [Figure 17-1](#). To use a different name for a plugin, create an empty file with the name *name.cfg* in the plugin source directory, as shown in the multiple directory example in [Figure 17-1](#).

Figure 17-1 Loading coreConsultant Plugins

To complete the Load Consultant Plugins activity in command line mode:

1. Create the .tcl and (if necessary) Obj.tcl files for the plugins and place them in one or more plugin source directories as previously described.
2. Set the PluginDirectories parameter on the LoadConsultantPlugins activity to a list of plugin source directories:

```
coreBuilder> set_activity_parameter LoadConsultantPlugins
PluginDirectories      {<list of plugin source directories>}
```

3. Complete the LoadConsultantPlugins activity:

```
coreBuilder> autocomplete_activity LoadConsultantPlugins
```

Example 17-8 shows how to complete the LoadConsultantPlugins activity for the multiple source directory example in Figure 17-1.

Example 17-8 Batch Command File to Complete LoadConsultantPlugins

```
set_activity_parameter LoadConsultantPlugins PluginDirectories \
  {/usr/fred/MyCore/PlugIns/Plugin_XXX \
  /usr/fred/MyCore/PlugIns/Plugin_YYY \
  /usr/fred/MyCore/PlugIns/Plugin_ZZZ}
autocomplete_activity LoadConsultantPlugins
```

17.3.4 Building a General Purpose Plugin

For non-core-specific plugins, use the `create_plugin_kb` command to build the plugin:

```
create_plugin_kb -name plugin_name [-mode requiredMode] [directory]
```

- *plugin_name* is the name of the plugin .kb file.
- *requiredMode* specifies which tool mode must be in use to load the plugin: builder, consultant, developer, or assembler. You can specify multiple modes (for example, "builder developer"). If you do not specify *requiredMode*, the plugin can be used in all modes.
- *directory* is the directory that contains the .tcl source files for the plugin. The default is the current directory.

For example, to invoke the coreBuilder tool in shell mode to build a coreBuilder-specific plugin named MyPlugin from the files in plugin_src, then quit, enter the following command from the UNIX prompt:

```
% coreBuilder -x "create_plugin_kb -name MyPlugin -mode builder plugin_src; quit"
```

You can add files (both configurable and non-configurable) to a plugin using the `add_files_to_filegroup` and `create_autoload_filegroup` commands. An example location for these files is as follows:

```
MyPlugin/
MyPlugin_Obj.tcl
MyPlugin_Src.tcl
DataFiles/
dataFile1.txt
dataFile2.txt
```

In this example, the file `MyPlugin_Obj.tcl` can contain the following commands for adding the files to the plugin:

```
set files { dataFiles/dataFile1.txt dataFiles/dataFile2.txt }
set fgObj1 [add_files_to_filegroup PluginFG1 -files $files -install ABC]
# Set filegroup to configurable and specify the activity.
set_attribute $fgObj1 -attr Configurable -value 1
set_attribute $fgObj1 -attr ConfigActivity -value MyPluginActivity
```

To use the plugin, you have two options:

- Place it in the proper plugin directory, as specified in "[Plugin Directories](#)" on page 265.
- Use the `load_plugin` and `unload_plugin` commands. Using `load_plugin` gives you control when the plugin is loaded. Also, it will automatically install any non-configurable files, which will not happen automatically if you simply place the plugin in the plugin directories.

17.4 Packaging Verification IP

You can use the coreBuilder tool to package a verification IP block into a coreKit for use with the coreConsultant tool. In the coreConsultant tool, the design flow for verification IP is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and synthesis related activities are irrelevant. The intent is to use coreBuilder packaging features to build a coreKit that contains all the data needed to use the verification IP.

The verification IP may be in the form of an HDL model, C model, timing model, VERA model, or some other format. The coreBuilder packaging process requires that the top-level view of the IP block is in VHDL or Verilog format. Therefore, non-HDL verification IP requires a top-level VHDL or Verilog wrapper.

The coreBuilder packaging flow for a block of verification IP is:

1. If the IP is not in VHDL or Verilog format, write a VHDL or Verilog wrapper for the IP block.
2. If the IP is configurable through HDL design parameters, insert reuse-pragmas in the HDL to specify the configuration intent for the design.

If the IP is configurable but not through HDL design parameters, set up a configurable filegroup to implement the configuration options (see [“Setting Up Configurable Filegroups”](#) on page 151).

3. Insert reuse-pragma `translate_off/translate_on` pairs around sections of unsynthesizable code in HDL files.
4. Set the `SimulationModel` attribute to true by including the following commands in an intent command file for the design:

```
current_design top_design
set_design_attribute SimulationModel true
```
5. Execute the Load Designs activity. Specify the names of the HDL design files as the value of the `InputFileNames` parameter. If the HDL files are used as a wrapper for a .db file for the actual design, specify the name of the .db file as the value of the `ExtraLinkLibs` parameter. See [“Loading Your Design”](#) on page 103.
6. Execute the Design Configuration Intent activity. If the verification IP is configurable through HDL design parameters, check the configuration intent as described in [“Refining Configuration Intent”](#) on page 97.
7. Complete the synthesis intent activities. Your design does not appear in the list of designs in the synthesis intent dialogs because the `SimulationModel` attribute is set to true. Just complete the synthesis intent activities without supplying any data. See [“Specifying Synthesis Intent”](#) on page 125.
8. Complete any optional activities needed for your design such as:
 - Load Consultant Plugins if your coreKit requires plugins. See [“Building a Core-Specific Plugin”](#) on page 259 on page 266 for details.
 - SoC Assembly Intent activities (Load Interface Definitions and Add Interfaces) if your design requires interface specifications for coreAssembler. See [“Defining Interfaces for coreAssembler”](#) on page 199 for details.
9. Use the Create Integration Workspaces activity to build a coreConsultant workspace in which to verify your packaging specifications, including any configuration intent that you have specified. Make any necessary modifications in your coreBuilder workspace.

10. Build the coreKit. Include all files needed to use the model, such as documentation and additional models.
11. Verify the coreKit in the coreConsultant and (optionally) coreAssembler tools.

17.5 Packaging Hard Macros

You can use the coreBuilder tool to package a hard macro into a coreKit for use with the coreConsultant tool. Hard macros are, by nature, not configurable and not synthesizable. Therefore, the coreConsultant design flow for a hard macro is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and synthesis related activities are irrelevant because they have already been done.

The coreBuilder packaging process requires that the top-level view of the IP block is in VHDL or Verilog format. Therefore, a hard macro requires a top-level VHDL or Verilog wrapper.

The coreBuilder packaging flow for a hard macro is:

1. Write a VHDL or Verilog wrapper for the hard macro.
2. Although the hard macro itself is not configurable, you can implement configuration options by including more than one configuration of the hard macro and using HDL parameters in the HDL wrapper to select the configuration. If you implement this type of configuration through HDL design parameters, insert reuse-pragmas in the HDL to specify the configuration intent for the design.

You can also implement configuration options by writing out different files depending on the core integrator's specified options. To do, so you must set up a configurable filegroup as described in ["Setting Up Configurable Filegroups"](#) on page 151.

3. Set the PreserveMapping attribute to true by including the following commands in an intent command file for the design:

```
current_design design
set_design_attribute PreserveMapping true
```

4. Execute the Load Designs activity. Specify the names of the HDL design files as the value of the InputFileNames parameter. Specify the name of the hard macro .db file as the value of the ExtraLinkLibs parameter. See ["Loading Your Design"](#) on page 103.
5. Execute the Design Configuration Intent activity.

If selection of hard macro configuration is through HDL design parameters, check the configuration intent as described in ["Refining Configuration Intent"](#) on page 97.

6. Complete the synthesis intent activities. You do not need to set any additional attributes on the design or its ports because core integrators do not need to synthesize the design. See ["Specifying Synthesis Intent"](#) on page 125.

7. Complete any optional activities needed for your design such as:

- Load Consultant Plugins if your coreKit requires plugins. See ["Building a Core-Specific Plugin"](#) on page 259 on page 266 for details.
- SoC Assembly Intent activities (Load Interface Definitions and Add Interfaces) if your design requires interface specifications for coreAssembler. See ["Defining Interfaces for coreAssembler"](#) on page 199 for details.

8. Use the Create Integration Workspaces activity to build a coreConsultant workspace in which to verify your packaging specifications, including any configuration intent that you have specified. Make any necessary modifications in your coreBuilder workspace.

9. Build the coreKit. Include any additional files needed to use the design, such as documentation and additional models.
10. Verify the coreKit in the coreConsultant and (optionally) coreAssembler tools.

17.6 Editing Tool Setup and User Preferences

You can control certain tool aspects that are set automatically when the tool opens. This is done using:

- “Setup Variables” on page 273
- “User Preferences” on page 274

17.6.1 Setup Variables

To establish a custom environment, you can set tool variables and define tcl commands in .synopsys_rt.setup files. When launched, the coreBuilder tool looks for three .synopsys_rt.setup files in the following order:

1. Site-wide coreBuilder variable settings in:
`<install_dir>/admin/setup/.synopsys_rt.setup`
You can view the contents of this file to see the variables that are most commonly set.
2. Personal settings in:
`$HOME/.synopsys_rt.setup`
For a short sample of a personal setup file, see `<install_dir>/admin/setup/user.synopsys_rt.setup`.
3. Project-specific settings can be placed in a .synopsys_rt.setup that resides in the directory where you invoked the coreBuilder tool.

When you invoke the coreBuilder tool, it reads and executes the setup files before executing any command line arguments.

The variable settings and tcl command definitions in the most recently read file take precedence over those in previously read files. In other words, the setup file in your \$HOME directory overrides the tool setup in `<install_dir>/admin/setup/.synopsys_rt.setup`, and a project-specific setup file overrides all others.

To define useful tcl commands in the setup file, use the ‘proc’ statement. Several examples are as follows:

```
proc fpa { args } { return [eval find_item -type param $args] }
proc fpi { args } { return [eval find_item -type pin $args] }
proc fn { args } { return [eval find_item -type net $args] }
proc fc { args } { return [eval find_item -type cell $args] }
```

 **Note**

All coreTools applications use the same setup file.

Table 17-3 describes some of the variables that you can set in .synopsys_rt.setup files. For other variables you can set, see the `<install_dir>/admin/setup/.synopsys_rt.setup` file.

Table 17-3 Setup Variables

Variable	Description	Default
component_search_path	List of directories that contain installed coreKits; used to determine available components for the coreConsultant tool	[getenv\DESIGNWARE_\HOME]/iip/latest

Table 17-3 Setup Variables (continued)

Variable	Description	Default
search_path	The list of directories to be searched when the coreConsultant tool looks for workspace-specific files	"\$synopsys_root/ \ libraries/dware/rt."
scratch_dir	The directory used as a staging area during installation of a coreKit and for the storage of temporary files	/tmp
collection_print_item_limit	The maximum number of item names to print as part of a command result	10
default_browser_command	The HTML browser to be used Note that PDF files are opened from either a direct \$PATH to acroread, or as a plugin to your default browser.	firefox / mozilla / netscape
netscape (Path)	If you use Netscape, specify the path to your installed version of Netscape.	"netscape"
netscape (LaunchArgs)	Arguments passed to Netscape when the coreConsultant tool invokes Netscape with show_url_netscape.	

17.6.2 User Preferences

The coreBuilder tool reads a set of tool interface preferences (if they exist) when it opens in either GUI or command-line mode. When set, the preferences reside in the following file:

\$HOME/.synopsys_rt_prefs.tcl

All preferences are user-specific and are used (as appropriate) for each coreTools application you run.

To set your preferences, choose Edit > Preferences from the toolbar menu. Note that the settings are saved when the coreBuilder tool is closed.



Note

All coreTools applications use the same preferences file.

Table 17-4 on page 274 describes the available preferences.

Table 17-4 Edit > Preferences Menu

Preference	Description
Appearance > Global > Fonts	
GUI: Default font Name: app_default_font_family Default: Bitstream Vera Serif, 10	Specifies display font family and size used for most application text at startup This font must be proportional.
GUI: Console font Name: app_fixed_width_font_family Default: Courier, 10	Font family and size used for console text

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Appearance > Global > Spreadsheet	
GUI: Show spreadsheet Boolean values as a checkbox Name: show_spreadsheet_bool_as_checkboxes Default: 1 (true)	Show spreadsheet Boolean values as a check box when true; otherwise, display as a combination box
GUI: Show all spreadsheet columns Name: show_all_grid_columns Default: 0 (false)	Show all attributes in spreadsheets, not just the most common set
Appearance > External Browser	
GUI: External browser launch command Name: default_browser_command Default: firefox %s	The default browser invocation command string replaces %s with the URL
GUI: External PDF viewer Name: default_pdf_viewer Default: acroread	The default viewer for PDF files The PDF file name is appended to whatever string is specified for the viewer.
Appearance > Windows	
Dialog Window GUI: Show the main application dialog tabwindow on tool startup Name: show_tabwindow_on_guistart Default: 1 (true)	Show the main application dialog tabwindow on tool startup
Console Window GUI: Initial console height (in percent) Name: initial_console_height Values: 15, ..., 50 Default: 19	Relative height of console at tool startup This value is a percentage of window space that the console should occupy. The value 0 means do not show the console, which can then be shown using View > Console Window.
Activity Window GUI: Initial activity window width (in percent) Name: initial_activity_window_width Values: 10, ..., 50 Default: 25	Relative width of activity window at tool startup This value is a percentage of window space that the activity window should occupy. The value 0 means do not show the activity window, which can then be shown using View > Activity Window.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Synopsys IP Updates	
GUI: Enable automatic Synopsys IP update checks Name: allow_automatic_update_checks Default: 1 (true)	Enable automatic update checks for used Synopsys IP If disabled, you can perform a manual IP update check anytime using the Help > Check For IP Updates menu item.
GUI: Synopsys IP update check frequency in days Name: automatic_update_check_frequency Default: -1	The frequency to check for available updates of the tool and the Synopsys IP being used The default setting of -1 causes the coreBuilder tool to automatically prompt you to set this preference when you open a workspace that contains Synopsys IP.
GUI: Inform Synopsys about opting out of automatic IP update checks Name: inform_optingout_automatic_update_checks Default: 1 (true)	Inform Synopsys about opting out of automatic IP update checks
Workspace Options (see also, " Creating a Workspace " on page 43)	
GUI: Include synthesis related activities Name: ws_options_synthesis Default: 1 (true)	Include activities in the activity list which relate to the generation of a gate-level implementation of the component or subsystem
GUI: Write batch script after each assembly-related activity Name: ws_options_batch Default: 0 (false)	Write a batch script for recreating the subsystem as each activity in the 'Create RTL' group completes The batch script is written to <workspace>/export/batch.tcl.
GUI: Include component specific unit-level test activities Name: ws_options_unittest Default: 1 (true)	Include activities associated with unit-level testing of DesignWare implementation IP This includes GTECH model generation and unit-level simulation. Note: This option applies only in the coreAssembler tool.
GUI: Show "Memory Map Specification" activity Name: ws_options_memorymapspecification Default: 1 (true)	Show the Memory Map Specification activity and its associated dialog in the workspace activity flow Note: This option applies only in the coreAssembler tool.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Document Generation	
GUI: Generated Documentation Format Name: documentation_format Default: DocBook	Generate reports/documentation in the specified format: ■ DocBook: User configurable reports are generated in DocBook XML and then automatically translated to HTML. ■ HTML: Non-configurable reports are generated directly in HTML format.
GUI: Compress Documentation Name: docbook_compress_documentation Default: 1 (true)	When set to true, items that are grouped with DocGroup are not reported individually For more about DocGroup and DocMaster, see the man pages.
GUI: Separate Files Per Parameter Group Name: docbook_file_per_param_group Default: 0 (false)	■ 0 (false): All parameters are written to a single table. ■ 1 (true): Each page from the configuration dialog has a separate table, and each table is written to a different file.
GUI: Separate Files Per Address Block Name: docbook_separate_address_block_files Default: 1 (true)	■ 0 (false): All register and register field information for an address block is written to a single file. ■ 1 (true): Information for each register is written to a file and register field information is written to a separate file.
GUI: Separate Files Per Port Group Name: docbook_separate_file_per_port_group Default: 1 (true)	■ 0 (false): All ports are written to a single file. ■ 1 (true): Each port group is written to a a separate file.
GUI: Port Documentation Attributes Name: docbook_port_attrs Default: PortDirection *DocGeneratelf *DocSynchronousTo *DocRegisteredBy *DocPowerDomain *DocDescriptionField *DocDescription	Indicates which attributes appear in DocBook documentation generated for top-level ports (signals) of a component Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. If the DocGeneratelf attribute is included, it is excluded from reports generated for configured designs.
GUI: Parameter Documentation Attributes Name: doc_param_attrs Default: Value *Name *Values *DocDefaultValue *DocEnabled *DocDescriptionField *DocDescription	Indicates which attributes appear in DocBook documentation generated for top-level parameters of a component Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. If the Value attribute is included, it is excluded from reports generated for unconfigured designs.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
	Note that the name attribute is included by default because the first column of the parameter tables uses the GUI label instead of the parameter name.
GUI: Register Documentation Attributes Name: docbook_register_attrs Default: DocGlobalAddressOffset *DocShortDescription	Indicates which attributes appear in DocBook documentation generated for registers within a memory map of a component Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset.
GUI: Register Field Documentation Attributes Name: docbook_field_attra Default: Name DocMemoryAccess *DisplayName *Reserved *DocRegisterResetValue *DocTestable *DocRegisterResetMask *DocVolatileMemory *WriteConstraint *MinWriteConstraint *MaxWriteConstraint *DocDescription *DocDescriptionField *BitRangeVariables *MemoryAccessDescription *ReadActionDescription *WriteBehaviorDescription	Indicates which attributes appear in DocBook documentation generated for fields within a register Attributes whose names are preceded by '*' all appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. Note that the Name attribute is included by default because the first column of the field tables uses the bit range as a descriptor.
Formal Verification	
GUI: Target stage netlist Name: formver_use_target_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseTargetStageNetlist
GUI: Reference stage netlist Name: formver_use_reference_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseReferenceStageNetlist
Tech Setup	
GUI: Run DC to init variables Name: run_dc_to_init_variables Default: 0 (false)	Forces coreTools applications to run Design Compiler to determine the values of synthesis setup variables
Assembly	
GUI: Default Port Name Format (via exported interface)	The format to be used for generating port names for exported interfaces

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Name: auto_port_name_format Default: %e_%p	Port name format usage is as follows: ■ %c: Component instance name ■ %p: Interface instance port name ■ %e: Exported interface name ■ %i: Interface instance name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Default Port Name Format (via manual export) Name: manual_port_name_format Default: %c_%p	The format to be used for generating port names for manually exported ports Port name format usage is as follows: ■ %c: Component instance name ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Default Signal Name Format Name: signal_name_format Default: %c_%p	The format to be used for generating signal names when writing RTL. Signal name format usage is as follows: ■ %c: Component instance name ■ %l: Use load component name instead of driver (single load only) ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Automatically apply when leaving an activity dialog or page Name: auto_apply Default: 0 (false)	Controls how the tool handles changes to dialog boxes or pages that have not been applied ■ 1 (true): Automatically apply changes in the dialog or page ■ 0 (false): Query the user whether to apply or cancel the changes
GUI: Allow incomplete subsystem Name: allow_incomplete_subsystem Default: 0 (false)	Controls activity completion and RTL generation for an incomplete subsystem
GUI: Allow undriven pins (see help) Name: allow_undriven_pins Default: 0 (false)	When true, errors related to undriven pins are downgraded to warnings

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
	Note: The resulting netlist will not include these connections because they are effectively undriven, so not including the connections is functionally equivalent to including an undriven wire.
GUI: Remove Unneeded Ports Name: remove_unneeded_ports Default: 0 (false)	When true, ports created as part of the interface-based connection process that are not needed, are removed Ports are not needed when an interface is exported from a lower-level of hierarchy and the connection on one side or the other is missing. This results in exported ports that are either 'open' or connected to a constant (if the interface definition provided a default value). Ports that are connected to open are removed. For ports that are connected to a constant, the port is removed and the constant is connected to any required pins in the lower-level hierarchy.
GUI: Use Merged Glue Logic Source Name: use_merged_glue_logic_source Default: 0 (false)	When true, the generated file, <subsystem_name>_sim.lst, does not reference individual Synopsys glue logic component source The glue logic is compiled from the shared glue logic source in <coreTools image root>/auxx/dware/glue.
Process Execution	
GUI: Run style preference Name: run_style_preference Values: local, lsf, grd Default: local	Controls how to run external jobs Jobs can be run locally, through LSF, or through GRD.
GUI: Run style options Name: run_style_options Default: N/A	Specifies additional options for a given run style (run_style_preference) For lsf and grd, this specifies options for the bsub and qsub commands.
Synthesis	
GUI: Bus naming style Name: bus_naming_style Default: N/A	Sets the bus_naming_style variable for synthesis
GUI: Show rise & fall delays separately Name: synth_show_rise_and_fall_delays Default: 0 (false)	■ 1 (true): Separate columns are shown for rise and fall delay constraints. ■ 0 (false): Constraints are shown merged with minimum value shown for minimum delays and the maximum value shown for maximum delays.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
GUI: Wait for licenses Name: syn_wait_for_licenses Default: 0 (minutes)	Sets the number of minutes to wait for Design Compiler licenses and causes Design Compiler to be invoked using: '-checkout Design-Compiler -wait <minutes>/' In Design Compiler, scripts also wait before giving up when checking out needed licenses, such as Ultra, Test-Compiler, and so on.
GUI: Alib library analysis path Name: alib_library_analysis_path Default: ./	Sets the alib_library_analysis_path variable for synthesis For more information, see the man page in Design Compiler.
GUI: Alib library analysis cache Name: alib_library_analysis_cache Default: ./	Sets the alib_library_analysis_cache variable for synthesis For more information, see the man page in Design Compiler.
GUI: Include params in top design name Name: include_params_in_top_design_name Default: 1 (true)	■ 1 (true): Parameters on the top-level design cause the design name to be modified as specified in the Design Compiler variable, template_parameter_style. ■ 0 (false): The top-level design name is unmodified by its parameter settings.
Multiple Port Nets GUI: Do not fix multiple port nets Name: fix_multiple_port_nets_default Default: 1 (true)	When true, causes "set_fix_multiple_port_nets -default" to be used in Design Compiler, so no buffers are inserted to fix multiple port nets
Multiple Port Nets GUI: Buffer feedthroughs Name: fix_multiple_port_nets_feedthroughs Default: 0 (false)	When true, causes "set_fix_multiple_port_nets -feedthroughs" to be used in Design Compiler
Multiple Port Nets GUI: Buffer pins driving multiple outputs Name: fix_multiple_port_nets_outputs Default: 0 (false)	When true, causes "set_fix_multiple_port_nets -outputs" to be used in Design Compiler
Multiple Port Nets GUI: Duplicate logic constants Name: fix_multiple_port_nets_constants Default: 0 (false)	When true, causes "set_fix_multiple_port_nets -constants" to be used in Design Compiler

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
IP-XACT	
Output Controls GUI: Schema version for IP-XACT output files Name: xml_schema_version Default: 1685-2014	Indicates SPIRIT IP-XACT schema version against which the files are written Possible values are: ■ 1.0: Uses SPIRIT 1.0 ■ 1.1: Uses SPIRIT 1.1 ■ 1.2: Uses IP-XACT 1.2 ■ 1.4: Uses IP-XACT 1.4 ■ 1.5: Uses IP-XACT 1.5 ■ 1685-2009: Uses IEEE 1685-2009 ■ 1685-2014: Uses IEEE 1685-2014
Output Controls GUI: Include component synthesis constraints Name: include_synthesis_constraints_in_components Default: 0 (false)	When false, synthesis constraints are not included in generated component files.
Output Controls GUI: Include defaulted parameters in design files Name: include_defaulted_params_in_design Default: 0 (false)	■ 1 (true): Parameters that are set to default values are written to the appropriate IP-XACT design file. ■ 0 (false): Only parameters that are set to non-default values are written to the appropriate IP-XACT design file.
Output Controls GUI: Include unused bus interfaces Name: include_unused_interfaces Default: 0 (false)	When true, unused (unconnected) bus interfaces are included in the component definition.
Output Controls GUI: Allow Broadcast Interface Connections Name: allow_broadcast_connections Default: 0 (false)	When true, allow one-to-many interface connections in the IP-XACT design files, where the pins on the one-side are all outputs.
Output Controls GUI: Include fileSet information Name: include_fileset_information Default: 0 (false)	When true, fileSet information is included in the component definition.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Output Controls GUI: Include unused signals in bus interface Name: include_unused_ports_in_interface Default: 0 (false)	When true, unused bus interface signals (signals with no corresponding component signal) are included in the signalMap of the corresponding busInterface element. These appear as signalName elements that have no value.
Output Controls GUI: Write IP-XACT components Name: write_spirit_components Default: 0 (false)	When false, IP-XACT components are not rewritten when writing a hierarchical IP-XACT component or design file. Instead, references to these components are handled through the standard VLNV (vendor, library, name, version) mechanism.
Output Controls GUI: Include accessHandle information Name: include_accesshandle_information Default: 1 (true)	When true, the accessHandle information is included in the component definition.
Output Controls GUI: Use absolute path writing file/fileSet dependencies Name: write_absolute_dependency_path Default: 0 (false)	When true, use an absolute path writing file/fileSet ipxact:dependency.
Output Controls GUI: Include Defines in IP-XACT component Name: include_defines_in_fileset Default: 0 (false)	If the value of this preference parameter is true, the defines that are set are written out in IP-XACT. The defines can be set in the design attribute 'StaticDefineExprs' for coreBuilder, coreConsultant, and coreAssembler. The defines can also be set in the LoadDesigns activity parameter 'CompileDefines' in coreBuilder.
Output Controls GUI: Include Doc attributes Name: include_doc_attrs Default: 0 (false)	The Control variable enables exporting of Documentation attributes in the component XML. The Doc attrs are only exported for Designware IP in coreConsultant and coreAssembler. ■ DocGroup: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocMaster: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocOffset: coreBuilder (register), coreConsultant (register) ■ DocGenerateIf: coreBuilder (port), coreConsultant (N/A) ■ DocRegisterResetValue: coreBuilder (register, registerField), coreConsultant (register, registerField) ■ DocDescriptionField: coreBuilder (register, registerField, port, param), coreConsultant (register, registerField, port, param)

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Output Controls GUI: Interface to Design Port Width Check Name: interface_to_design_port_width_check Default: 0 (false)	If the value of this preference parameter is true, then width of the design and Interface ports are validated for the same width before writing out the IP-XACT.
Schema Validation GUI: Input errors should be Name: xml_schema_validation Default: 1 (warnings)	Indicates how to deal with XML schema validation errors when reading XML files: ■ 0 (suppressed): All errors are suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Schema Validation GUI: Output errors should be Name: xml_schema_output_validation Default: 0 (suppressed)	Indicates how to deal with XML schema validation errors when writing XML files: ■ 0 (suppressed): Validation is suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Assembly GUI: Enable Automatic Connections for IP-XACT Buss Definitions Name: auto_connect_spirit_definitions Default: 1 (true)	When true, indicates that interfaces whose definition comes from an IP-XACT bus definition (as opposed to a coreBuilder TCL interface definition) can be connected automatically.
Assembly GUI: Use IP-XACT ad-hoc Connection Names as Signal Names Name: use_ipxact_adhoc_names_as_signal_names Default: 0 (false)	When true, indicates that signals defined through ad-hoc connections should be named based on the name specified in the ad-hoc connection element.
Assembly GUI: Treat single pin ad hoc connections as a connection to 'open' Name: ipxact_treat_single_pin_adhoc_as_open Default: 0 (false)	When true, indicates that single pin ad-hoc connections will be treated as a connection to 'open' for both reading and writing IP-XACT design files.
Assembly GUI: Treat unconnected interfaces as unused in IP-XACT designs Name: ipxact_mark_unconnected_interfaces_as_unused	When true, unconnected interfaces on an IP-XACT component will be marked as unused when reading a design file which does not make a connection to the interface.

Table 17-4 Edit > Preferences Menu (continued)

Preference	Description
Default: 0 (false)	

17.7 Table Driven Data Import/Export

The coreTools applications support import and export of data in a table-based format to enable user flows that are driven from spreadsheets. This provides both an ease-of-use advantage (no customer conversion to coreTools commands is required) as well as a performance benefit because table processing is generally faster than command processing. The table read/write commands are available in batch mode and are also available through the **File > Table...** menu.

17.7.1 General Table Formatting for All coreTools Applications

Table files can contain one or more tables. Within a table file, tables can be named or anonymous. Named tables provide two capabilities that are not available with anonymous tables:

- Named tables can contain blank lines.
- Named tables can be extracted from multi-table files and processed without processing of other named or anonymous tables.

All tables have the following general format: `<table name><header row><data>`, where the `<table name>` is optional.

- When reading a table file, a table is assumed to begin at the first non-blank row. If that row contains exactly two cells, the first of which has a value of "Table", then it is assumed to be a table name. Otherwise, it is assumed to be the row containing the column headers (in other words, the `<header row>`).
- If there is a table name, the next non-blank line is assumed to be the `<header row>`. The columns that have entries in the `<header row>` define the set of columns for the table.
- The `<data>` section of the table begins at the next non-blank line following the `<header row>`. The `<data>` section ends at the next blank line beyond its starting point, unless the table is named and the table is being read with the `-blanks` option specified.

All of the table reading commands locate and process tables in a consistent fashion, and share a number of common arguments:

- **<table file>**: The name of the table file to be processed
- **-tables <table names>**: The names of tables to be processed in the given table file

Only tables with the indicated names are processed. If this option is not specified, then all tables in the table file are processed (and must therefore be supported by the same table reading command).

- **-columns <column data>**: Defines static columns that are assumed to exist in all processed tables

The `<column data>` is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.

- **-map <column map>**: Defines column header name mapping

Different tables have different requirements regarding the names of columns in the table. This option allows you to map from the names seen in the table to the names expected by the table-reading command. This enables user-defined column names while still providing the table reader with known names. The `<column map>` is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table-reading command.

- **-separator <column separator>**: Defines the character used to separate columns in the table file; defaults to the tab character

The table writing commands also share a number of common arguments:

- **<table file>**: The file into which the table is written.
- **-name**: The name of the table file. If specified, a <table name> row is included to identify the table.
- **-append**: If specified, the table is appended to the indicated file; otherwise, the file is overwritten.
- **-map <column map>**: Defines column header name mapping.

Different tables have different requirements regarding the names of columns in the table. This option allows you to map from the names desired in the table to the names expected by the table-writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table-writing command.

- **-separator <column separator>**: Defines the character used to separate columns in the table file; defaults to the tab character .
- **-columns <column names>**: Indicates a list of columns whose values are expected to be identical across all table rows and which should, therefore, be excluded from the table. If the table writer determines that the values are not identical, the table-write operation fails.

17.7.2 Table Commands

17.7.2.1 read_attribute_table

This command provides basic attribute table read capability:

```
read_attribute_table <table filename> \
    [-tables <table names>] \
    [-columns <column name value pairs>] \
    [-map <column mapped name pairs>] \
    [-separator <column separator>] \
    [-create | -update] \
    [-loops]
    [-blanks]
    [-comment_indicator <table comment indicator>]
```

This command allows you to set attributes on objects in the coreTools data model. Attribute tables can have the following columns:

Table 17-5 Attribute Table Columns

Column Name	Optional	Description
Type	No	The object type
Name	No	The object name
<attribute name>	Yes	The name of the attribute to be set An arbitrary number of these columns is allowed.

This command is used to set the indicated attributes on the indicated objects. Objects are located using the item type and item name. When a table contains objects of multiple types, scoping is handled automatically when switching between object types that have a parent-child relationship. For example ‘port’ lines appearing after a ‘design’ line are assumed to indicate ports within that design. Scoping is supported for the following object type transitions:

- design --> port
- memoryMap --> addressBlock
- addressBlock --> register
- addressBlock --> registerArray
- registerArray --> register
- register --> registerField

As each of these transitions occurs, the context is changed by pushing the prior element onto the context stack. When the reverse transitions occur, elements are popped from the stack as appropriate. This scoping means that names specified for items can always be simple local names without context in the name itself.

If neither the -create nor -update option is specified, each line is assumed to refer to an object that already exists in the data model. If -create is specified, each object is assumed to not exist and is created before setting any attributes on it. If -update is specified, a search for it is performed and the object is created only if it does not already exist.

If the -loops option is specified, each name is scanned for loop syntax. The loop syntax is:

{<loop var>:<start index>...<end index>}

When seen, the line is replicated for each inclusive index value. Each attribute value is scanned for \$<loop var> and that string is replaced with the loop index value. When a line with loop syntax is used for scoping, all scoped objects are created within each replicated scoping object. For example, [Table 17-6](#) would result in two registers being created, each with two fields, F1 and F2.

Table 17-6 Example Creating Two Registers with Two Fields F1 and F2

Type	Name	Size	Description
register	Interrupt{i:0...1}	32	Interrupt \$i
field	F{j:0...1}	16	Field \$j in interrupt \$i

17.7.2.2 write_attribute_table

This command provides basic attribute table write capability:

```
write_attribute_table <table filename> \  
    [-name <table name>] \  
    [-append] \  
    [-separator <column separator>] \  
    [-comment_indicator <table comment indicator>]  
    [-map <column name map pairs>] \  
    [-columns <column name value pairs>] \  

```

```
-items <list of items> \  
-attributes <list of attributes> \  
-defaults
```

This command writes attribute data in table format to the indicated file. The resulting table includes a row for each specified item and a column for each specified attribute, plus columns for the object name and type (unless removed with `-columns`). If there are multiple types of objects in the list to be written, only those attributes that apply to an object type are written for a given object. If the `-defaults` option is not specified, attributes whose values match their default are not written.